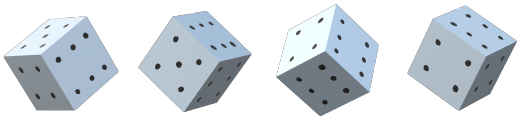
IB Data Science:
modelling and machine learning with probability

Damon Wischik, Computer Laboratory, Cambridge University



Contents

ACKNOWLEDGEMENTS

Thanks to Richard Gibbens, Jakub Perlin, Thomas Sauerwald, Alicja Chaszczewicz, L.A. Mlodzian, Raghul Parthipan, Tudor Suciu, Dimitris Los, Gemma Penson, Jino Osmani, Jamie Syiek, Bruce Mauger, Louie Pietroni, Martynas Stankevicius, Michael Lee, Lucy Robertson, Peter Hu, and Apinan Hasthanasombat for spotting bugs in earlier versions of these notes.

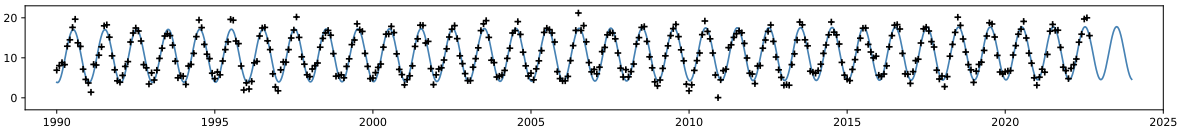
October 9, 2025 at 19:13:00

Part I
Learning with probability models

Many tools in machine learning, from simple linear regression to neural network classifiers, boil down in the end to two simple steps: (1) write out a probability model, then (2) estimate the model's parameters by using data. This is called *fitting* the model, or *training*, or *learning* the parameters.

There's one standard fitting procedure, the simplest most obvious thing it can be, and it's used all the way from classic statistics to neural networks. This procedure is based on optimization. We first use maths to figure out an expression to optimize, based on our probability model. We then run numerical optimization on a computer (or, for very simple toy problems, exact optimization using pen and paper tools from calculus). Sections 1.3 and 1.4 describe this fitting procedure, first with maths and then with simple Python. Section 3 applies it to neural networks, using PyTorch for numerical optimization.

Perhaps more important than the fitting procedure is how we invent a probability model in the first place. First, what exactly do we mean by a *probability model*? Let's look at an example, for Cambridge temperatures. The dataset[3] consists of monthly readings from January 1959 to the present, shown here with black crosses:



Our first instinct might be to model these datapoints as a sinusoid, reflecting the annual summer / winter cycle, plus a long-term warming trend, for example the line shown above, which has the formula

$$temp(t) = c + \alpha \sin\big(2\pi(t + \phi)\big) + \gamma t$$

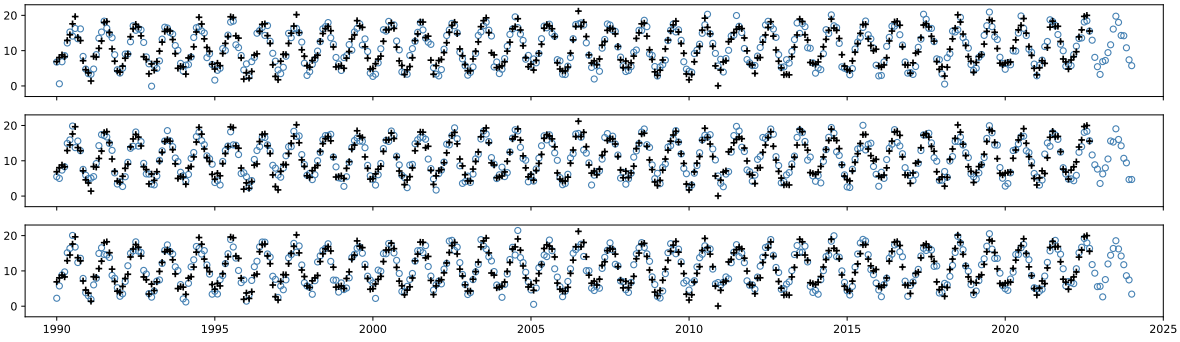
where c, α, ϕ, and γ are the model's parameters. The problem with a nice clean equation like this is that it doesn't actually describe the dataset — the points don't actually lie *on* the line. It's daft to use a model that is proved wrong by the data in front of our eyes!

fitted value for γ is 2.24°C per century

A probability model allows for noise in the dataset, and it describes noise using the language of random variation. We might for example propose the climate model

$$temp(t) = c + \alpha \sin\big(2\pi(t + \phi)\big) + \gamma t + noise$$

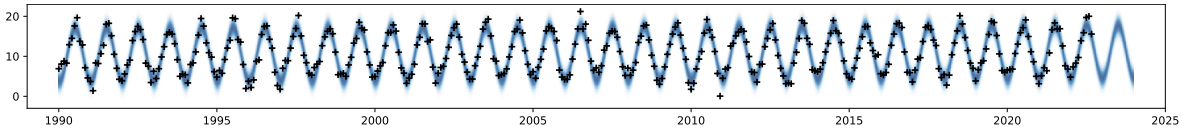
and specify that the noise term is to be generated using a Normal random variable with mean 0 and standard deviation of a few °C. Here are some sample outputs that this model produces:



No one would seriously propose our first deterministic model with the idea that it's meant to describe the data perfectly! A more nuanced interpretation is that it's only intended to capture the trend, not the noise; and that when we fit the model we can ignore errors of

[3] Historic station data from the UK Met Office, https://www.metoffice.gov.uk/research/climate/maps-and-data/historic-station-data

plus or minus a few °C. The deterministic model leaves unspoken this margin of error — while the probability model expresses it precisely, as we can see by superimposing many random outputs into a fuzzy band:



Readings that lie within the fuzzy band are consistent with our model, readings that lie far outside are inconsistent. The language of random variables *is* the natural language with which to describe noise and acceptable margins of error.

When a data scientist looks at a dataset, they don't just see the datapoints, they also see a cloud of all the counterfactual possibilities of what the datapoints *might have been*, each possibility weighted by its likelihood. It's a core skill to be able to look at the world like this, not only to see what happened but also to invent a model that describes what might have happened. A large part of this course, and of machine learning in general, is knowing enough building blocks to come up with useful probability models.

The building blocks for probability models are random variables. You have most likely come across the basic random variables like Geometric and Gaussian in an introductory course on probability, and section 1 builds them up into richer models, including models for clustering and classifying data. Section 2 dives deep into one particular class of probability model, namely linear regression. This is a flexible and interpretable class of model, and has fast routines for fitting to data. It should be your go-to method for all sorts of data science and machine learning problems, the second thing you try to get a sense of the data you're working with. (The first thing you try should be simple tabulation!)

1. Specifying and fitting models

1.1. Specifying a probability model

What is a probability model, and how do we describe it?

I grew up playing games like SimCity. Every time you play, things turn out differently: the game starts with a random map to play on, and there's randomness in where the sims build their houses and where they travel. *Any piece of code whose output is random can be seen as a probability model.* Code is great for getting a visceral feel for how a model behaves — we can run it, plot the output, and step through with a debugger if we see anything that doesn't match the world we're aiming to model.

We can also describe probability models mathematically, using random variable notation. This is more useful for reasoning about models. In particular, it's the starting point for machine learning, that is, for fitting the model's parameters.

Good modellers are bilingual, able to work with both code and maths, and to translate between them.

THE CODE VIEW

Let's look at a simple probability model, for the dataset of Cambridge temperatures depicted on page 1. Here are the first few rows. We're interested in modelling the monthly average temperature `temp` as a function of timestamp `t`.

yyyy	mm	t	tmax	tmin	temp	af	rain	sun	station
1985	1	1985.00	3.4	-2.2	0.60	23	37.3	40.7	Cambridge
1985	2	1985.08	4.9	-1.9	1.50	13	14.6	79.0	Cambridge
1985	3	1985.16	8.7	1.1	4.90	10	45.8	97.8	Cambridge

Example 1.1.1 (Probability model for climate).
The plot on page 1 suggests modelling this as a sinusoid with a period of one year, plus a long-term trend. Here's a model:

```
def rtemp(t, α=10, ϕ=−0.25, c=11, γ=0.035, σ=2):
    π = np.pi
    pred = α*np.sin(2*π*(t+ϕ)) + c + γ*t
    return np.random.normal(loc=pred, scale=σ)
```

The job of this function is to generate a random temperature for timestamp `t`.

The function also has some magic numbers for its other arguments. Four of them describe the shape of the sinusoid: its amplitude α, the phase shift ϕ, the overall average level c, and the annual rate of temperature increase γ. The final parameter σ describes how much random noise to add. The noise is `np.random.normal`, which generates a Normal random variable, also called a Gaussian. This code provides some half-plausible default values, but really they should be learnt from a dataset.

When we come to machine learning, it will be more useful to think in terms of probability models that generate an entire dataset, rather than just a single reading.

```
url = 'https://www.cl.cam.ac.uk/teaching/current/DataSci/data/climate.csv'
climate = pandas.read_csv(url)
df = climate.loc[climate.station=='Cambridge']
# Simulate a random version of the entire dataset
temps = rtemp(df.t)
```

Note that `df.t` is a numpy array with one item per row of the dataset. Thanks to numpy's vectorized syntax, `pred` will be a vector of the same length, and the last line of `rtemp` generates a different random noise for each item in the vector.

RANDOM VARIABLE NOTATION

Instead of code, a probability model can also be written out algebraically, using random variable notation. Here's the climate model, rewritten:

Example 1.1.2 (Probability model for climate, maths version).
Let the dataset consists of records $i = 1, \ldots, n$, and let t_i be the timestamp for record i. Let's propose a probability model for the temperature readings:

$$\mathsf{Temp}_i \sim \alpha \sin\big(2\pi(t_i + \phi)\big) + c + \gamma t_i + \text{Normal}(0, \sigma^2), \quad i = 1, \ldots, n.$$

We can write this model in other ways, for example

$$\mathsf{Temp}_i = \alpha \sin\big(2\pi(t + \phi)\big) + c + \gamma t_i + E_i, \quad E_i \sim \text{Normal}(0, \sigma^2), \quad i = 1, \ldots, n.$$

The maths notation is concise, and it relies on several conventions, illustrated below. You'll also need to be familiar with the standard random variables listed in the next section. Watch out for the = symbol, which in Python code means assignment but in maths means equality, and also the ∼ symbol which has a very specific meaning in random variable notation.

standard random variables: section 1.2 page 8

```
def ry():
    x = random.random()
    y = x ** 2
    return y
```

$X \sim U[0,1]$, $Y = X^2$.

A random variable corresponds to a piece of code that returns a random output. Here we've written out random variables corresponding to two parts of the code, one for the random.random() function, and one for ry().
The random.random() function generates a floating point value uniformly between zero and one; the numpy equivalent is np.random.uniform(). In maths, we say X is *generated* or *sampled* from $U[0,1]$.

```
def ri(a,b):
    x = random.random()
    i = math.floor(a*x + b)
    return i
```

$X \sim U[0,1]$, $I = \lfloor aX + b \rfloor$.

By convention, use capital letters to denote random variables and lower case for constants. This is how we can tell from the maths notation that X and I are meant to be random variables while a and b are meant to be constants. They are called *parameters* for the random variable I.

```
x = random.random()
y = x ** 2
```

$X \sim U[0,1]$, $Y = X^2$

Any piece of code has to live somewhere. The randomness might be wrapped up inside a function, or it might be at the top level of a script as in this example. From the point of view of the person running the script, every time this code is run it will produce different values, and so they'd consider X and Y to be random variables. From the point of view of line-by-line debugging, when we're stepping through the second line we'll treat x and y as values, not as a random numbers.

```
def rz():
    x₁ = random.random()
    x₂ = random.random()
    return x₁ * math.log(x₂)
```

$X_1, X_2 \sim U[0,1]$, $Z = X_1 \log X_2$.

X_1 and X_2 are generated *independently*, both of them from $U[0,1]$. Knowing the value of X_1 gives no information about the value of X_2, at least for an idealized random number generator. In maths notation, it's usually implied that random variables are meant to be generated independently, unless stated otherwise. But well-written maths will explicitly add "X_i independent".

```
(x1,x2) = [
    random.random()
    for _ in range(2)]
```

$X_i \sim U[0,1],\ i \in \{1,2\}$

Again, this code produces independent random variables.

```
def rmyrandpair():
    x1 = random.random()
    x2 = random.random()
    y,z = (x1+x2, x1*x2)
    return (y,z)
```

$(Y, Z) \sim \mathsf{Myrandpair}$

This random variable generates a pair of values. If I'm told that the value produced for Y is very small, I can deduce that the value for Z is also very small—hence they are not independent. The maths notation indicates "Y and Z are generated together and they are (potentially) not independent".

```
λ = 3
x1 = random.uniform(0,λ)
x2 = random.uniform(0,λ)
```

$X_1, X_2 \sim U[0, \lambda]$

X_1 and X_2 are sampled independently from the uniform distribution over the range $[0, \lambda]$. Technically, it's better to say "X_1 and X_2 are independent, given the parameter λ"—if we didn't know λ, then the value of X_1 would give information about λ, which would give information about X_2. Usually, when you see a probability model written out in maths notation, "independent given the parameters" is implied but not stated explicitly.

```
x = random.random()
y = 1 - x
```

$X \sim U[0,1],\ Y = 1 - X$

The special symbol $\sim$ denotes that two random variables have identical distributions. In this example, it would be correct to write $Y \sim U[0,1]$. This means "if we plot a histogram of Y, and separately plot a histogram of $U[0,1]$, then the two plots will be the same". This is *not* an imperative statement about how we generated Y, it's a mathematical statement about Y's distribution. It this example, it's also the case that $X \sim Y$.

The expression $Y = 1 - X$ likewise is not an imperative statement, it's a mathematical equation declaring that every time we run the script the equality is true. It would be just as correct to write $X = 1 - Y$.

```
x = random.random()
y = np.random.normal(loc=x,
scale=0.1)
```

$X \sim U[0,1],\ Y \sim N(X, 0.1)$

In maths notation, the random variable X appears on the right hand side of $Y \sim \cdot$, which means "given a value for X, Y has the following distribution". We don't write $Y = N(X, 0.1)$ because $N(X, 0.1)$ specifies a distribution, not a value, and $=$ is the symbol for relating values whereas $\sim$ is the symbol for relating distributions.

```
temps = rtemp(df.t)
```
from example 1.1.1

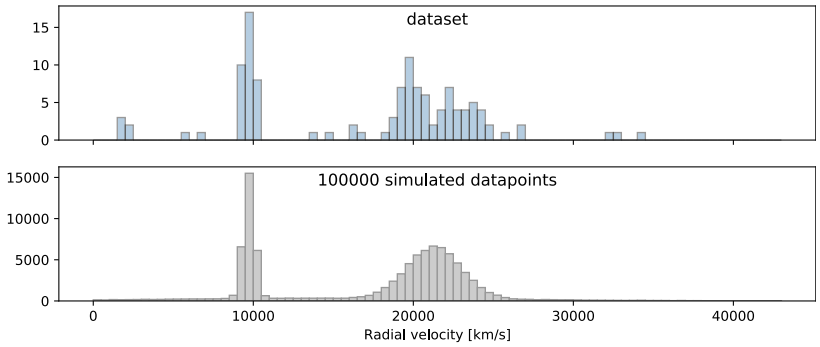
Equations for Temp_i from example 1.1.2.

We've used lower case for the unknown parameters $(\alpha, \phi, c, \gamma, \sigma)$ as well as for the known constants (t_i), and upper case for the random variable Temp_i. There is a Normal$(0, \sigma^2)$ noise term added for each i, and since it's not explicitly stated otherwise it's implied that the noise terms are generated independently for each i.

Note the difference between the $=$ and $\sim$ symbols in the second version in example 1.1.2: we use $\sim$ to make the statistical statement "E_i has the Normal distribution", and we use $=$ to make the deterministic statement "The two quantities Temp_i and E_i are related by the given equation".

MORE EXAMPLES

Exercise 1.1.3 (Gaussian mixture model).
This dataset[4] comes from an astronomy paper published in 1986 about large-scale structure in the universe. It consists of data about 120 galaxies, from a survey of the Corona Borealis area of the night sky, and it has the radial speeds at which those galaxies are moving away from us. (We know the universe is expanding, so all the galaxies are moving away from us, and the speed can be measured using redshift). If the universe were uniform, with galaxies studded through it like cherries in a fruit cake, then we'd expect to see a smooth distribution of galaxy speeds. But if the universe is made up of filaments of galaxies with vast voids between, then we'd expect to see the speeds clustered, and that's what the histogram of the data suggests. The top histogram shows the histogram of the galaxy speeds in the dataset, and the bottom shows a histogram from a simulated set of 100,000 galaxies from a 'clusters' probability model.



This probability model assumes there are two main clusters, one centered around 9,740 km/s and the other around 21,330 km/s, plus a third very broad cluster. The code first selects a cluster at random, and then it generates a random velocity using that cluster's mean and variance. Such a model is called a Gaussian mixture model.

```
url = 'https://www.cl.cam.ac.uk/teaching/current/DataSci/data/galaxies_orig.csv'
galaxies = pandas.read_csv(url)
galaxies = galaxies['velocity'].values

def rgalaxies(size, p, μ, σ): # EXERCISE: rewrite in numpy
    res = []
    for _ in range(size):
        k = np.random.choice([1,2,3], p=p)
        μi, σi = μ[k-1], σ[k-1]
        x = np.random.normal(loc=μi, scale=σi)
        res.append(x)
    return res

# parameters, estimated by 'eyeballing' the dataset
p = [0.28, 0.54, 0.18]
μ = [9740, 21300, 15000]
σ = [340, 1700, 10600]

fig,(ax1,ax2) = plt.subplots(2,1, figsize=(15,6), sharex=True)
ax1.hist(galaxies)
ax2.hist(rgalaxies(100000,p,μ,σ))
```

Write this probability model in random variable notation.

Let K be a randomly generated cluster drawn from the Categorical distribution, $K \sim \text{Cat}(p)$. In other words,

the Categorical random variable, page 9

[4] Dataset from M. Postman, J. P. Huchra, and M. J. Geller. "Probes of large-scale structures in the Corona Borealis region". In: *Astronomical Journal* (1986). A readable explanation of the astronomy behind the dataset is given by Kathryn Roeder. "Density estimation with confidence sets exemplified by supervlusters and voids in the galaxies". In: *Journal of the American Statistical Association* (1990).

$$K = \begin{cases} 1 & \textit{with prob. } p_1 \\ 2 & \textit{with prob. } p_2 \\ 3 & \textit{with prob. } p_3. \end{cases}$$

Then let the galaxy speed X be

$$X \sim \text{Normal}(\mu_K, \sigma_K^2).$$

□

1.2. Standard random variables

To build up useful probability models, it's useful to have a repertoire of standard random variables, and to know what they are used for. The tables on the next two pages list some standard random variables.

Random variables can return any type you want. We say a random variable is $\mathcal{S}$-valued if $\mathcal{S}$ is the set to which all its output values belong (synonyms: has support $\mathcal{S}$, or takes values in $\mathcal{S}$). Usually we'll build up our probability models out of discrete integer-valued and continuous real-valued random variables.

Wikipedia is a handy reference. Here are two examples, the entries for the Poisson distribution and the Normal distribution. The Poisson random variable is discrete (its support is the set of integers) so Wikipedia tells us its probability mass function; the Normal distribution is continuous (its support is the set of all real numbers) so Wikipedia tells us its probability density function.

Poisson distribution

Notation	$\text{Pois}(\lambda)$
Parameters	$\lambda > 0$ (rate)
Support	$k \in \{0, 1, \dots\}$
PMF	$\frac{\lambda^k e^{-\lambda}}{k!}$
CDF	$e^{-\lambda} \sum_{i=0}^{\lfloor k \rfloor} \frac{\lambda^i}{i!}$
Mean	λ
Variance	λ

Normal distribution

Notation	$N(\mu, \sigma^2)$ or $\text{Normal}(\mu, \sigma^2)$
Parameters	mean $\mu \in \mathbb{R}$, scale $\sigma > 0$
Support	$x \in \mathbb{R}$
PDF	$\frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x-\mu)^2/2\sigma^2}$
CDF	$\Phi\left(\frac{x-\mu}{\sigma}\right)$
Mean	μ
Variance	σ^2

For when we need to use the pdf or cdf etc. in code, there are also useful functions in scipy.stats, with a consistent naming convention, shown here for the Normal distribution. The ... are the parameters for the distribution. But watch out! numpy and scipy.stats and Wikipedia don't always use consistent parameters.

```
scipy.stats.norm.mean(...), median, var, std
scipy.stats.norm.pdf(x=x, ...)]
scipy.stats.norm.cdf(x=x, ...)]
scipy.stats.norm.ppf(q=q, ...)] # inverse of the cdf [5]
```

THE NORMAL DISTRIBUTION

The Normal distribution is so widely used in data science and machine learning that it's worth mentioning two properties here.

see IA Probability lecture 6

- If a and b are constants then

$$a + b\,\text{Normal}(0,1) \sim a + \text{Normal}(0, b^2) \sim \text{Normal}(a, b^2)$$

- When adding independent Normals, the parameters add up:

$$\text{Normal}(\mu, \sigma^2) + \text{Normal}(\nu, \rho^2) \sim \text{Normal}(\mu + \nu, \sigma^2 + \rho^2)$$

Most random variables don't behave so nicely! For example, if $Y \sim \text{Bin}(n, p)$ then $aY + b$ is not a Binomial (unless $a = 0$ and $b = 1$.)

An engineer's rule of thumb says that an arbitrary random variable can be approximated reasonably well by a Normal. This is so useful and simple that it can't possibly always be true—but what's remarkable is that it's so often nearly true. In IA Probability you learned about the Central Limit Theorem, says that we can approximate sums of random variables by a Normal, but the value of the approximation goes further.[6]

[5] The inverse of the cumulative distribution function returns x such that $\mathbb{P}(X \le x) = q$. We can use this to look up quantiles: for example ppf(q=.25) returns the lower quartile. For discrete random variables, when cdf jumps up in steps, it returns $\min\{x : \mathbb{P}(X \le x) \ge q\}$.

[6] See *Probability Theory: the Logic of Science* by E.T.Jaynes (CUP 2003) chapter 7 for why this might be.

DISCRETE RANDOM VARIABLES (integer-valued)

Binomial $X \sim \text{Bin}(n,p)$ np.random.binomial(n,p)	$\mathbb{P}(X=x) = \binom{n}{x} p^x (1-p)^{n-x}$ $x \in \{0,1,\dots,n\}$	If we toss a biased coin n times, and each coin has chance $p \in [0,1]$ of heads, the total number of heads is $\text{Bin}(n,p)$. When $n=1$, i.e. a single coin toss, it's called a Bernoulli random variable. There is a generalization to more than two outcomes, called the Multinomial.
Geometric $X \sim \text{Geom}(p)$ np.random.geometric(p)	$\mathbb{P}(X=x) = (1-p)^{x-1}p$ $x \in \{1,2,\dots\}$	If we're playing a lottery, and each week the chance of winning is $p \in [0,1]$, then the week of our first success is $\text{Geom}(p)$. Confusingly, the same name is used for the version that starts at 0, i.e. which counts the number of failures before our first success, for which $\mathbb{P}(X'=x) = (1-p)^x p$.
Poisson $X \sim \text{Pois}(\lambda)$ np.random.poisson(lam=λ)	$\mathbb{P}(X=x) = \dfrac{\lambda^x e^{-\lambda}}{x!}$ $x \in \{0,1,\dots\}$	Suppose we're counting the number of events in a fixed interval of time, for example the number of buses passing a spot on the street, and suppose the events are unrelated. If the average number of buses per hour is μ then the total number of buses in time t can be modelled as $\text{Pois}(\mu t)$.
Categorical $X \sim \text{Cat}([p_1,\dots,p_k])$ np.random.choice([1,...,k], p)	$\mathbb{P}(X=x) = p_x$ $x \in \{1,\dots,k\}$	This simply means: pick outcome x with probability p_x. It needs each p_x to be in $[0,1]$ and $p_1 + \cdots + p_k = 1$.

To sample n independent values from a distribution, use e.g. np.random.binomial(.... size=n).

For reproducible code, it's better to use numpy generators. Initialize the generator with rng=np.random.default_rng(seed=10) and then generate values with e.g. rng.binomial(n,p).

CONTINUOUS RANDOM VARIABLES (real-valued)

Uniform $X \sim U[a, b]$ np.random.uniform(low=a, high=b)	$\text{pdf}(x) = \dfrac{1}{b-a}$ $x \in [a, b]$	A floating point value, uniformly distributed in the interval $[a, b]$, for $a < b$.
Normal / Gaussian $X \sim N(\mu, \sigma^2)$ np.random.normal(loc=μ, scale=σ)	$\text{pdf}(x) = \dfrac{1}{\sqrt{2\pi\sigma^2}} e^{-(x-\mu)^2/2\sigma^2}$ $x \in \mathbb{R}$	Popular in data analysis because it's a good model for things that are the aggregate of many small pieces, for example a person's height is the aggregate of many influences in genetics and the environment. Very popular in machine learning because it's easy to do probability calculations with it. There is also a multidimensional version, called the Multivariate Normal.
Exponential $X \sim \text{Exp}(\lambda)$ np.random.exponential(scale=$1/\lambda$)	$\text{pdf}(x) = \lambda e^{-\lambda x}$ $x > 0$	Used for modelling waiting times for many natural processes, for example the time until the next bus. The parameter $\lambda > 0$ is called the *rate*, since the chance of the event happening in a short interval of time $[t, t+\delta]$ is $\approx \lambda\delta$. If the times between buses is $\text{Exp}(\lambda)$ then the number of buses in time t is $\text{Pois}(\lambda t)$.
Pareto $X \sim \text{Pareto}(\alpha)$ np.random.pareto(shape=α)	$\text{pdf}(x) = \alpha x^{-(\alpha+1)}$ $x \geq 1$	Used with $0 < \alpha < 2$ for modelling 'cascade' events such as sizes of forest fires, where there are events of wildly different sizes, and the entire outcome can hinge on one 'black swan' event[7]. When $\alpha > 2$ it is well-behaved.
Beta $X \sim \text{Beta}(\alpha, \beta)$ np.random.beta(α,β)	$\text{pdf}(x) = B_{\alpha,\beta}\, x^{\alpha-1}(1-x)^{\beta-1}$ $x \in [0, 1]$ $B_{\alpha,\beta} = \dfrac{(\alpha+\beta-1)!}{(\alpha-1)!(\beta-1)!}$	This distribution appears in Bayesian calculation, e.g. section 5.2 page 92. We use it as a *random* probability: let $P \sim \text{Beta}(\alpha, \beta)$ be the probability of heads for a possibly biased coin, and let $Y \sim \text{Bin}(n, P)$ be the outcome of n tosses. Needs $\alpha > 0$ and $\beta > 0$. If either is non-integer, the factorials are replaced by a continuized version called the Gamma function.
Gamma $X \sim \Gamma(k, \lambda)$ np.random.gamma(shape=k,scale=$1/\lambda$)	$\text{pdf}(x) = \dfrac{\lambda^k x^{k-1} e^{-\lambda x}}{(k-1)!}$ $x > 0$	Useful when we want a small positive random value, for example to use as the so-called 'precision' of a Normal, $Y \sim N(\mu, 1/X)$. It has mean k/λ and variance k/λ^2. Needs $k > 0$ and $\lambda > 0$. If k is non-integer, the factorial is replaced by the Gamma function.

[7] Nassim Nicholas Taleb. *The Black Swan: The Impact of the Highly Improbable.* 2nd ed. Random House, 2010

1.3. Maximum likelihood estimation

> *"All the impressive achievements of deep learning amount to just curve fitting," [Judea Pearl] said recently. […]*
>
> *The way you talk about curve fitting, it sounds like you're not very impressed with machine learning [remarks the interviewer]. "No, I'm very impressed, because we did not expect that so many problems could be solved by pure curve fitting. It turns out they can."*
>
> An interview with Judea Pearl[8]

Many probability models have unknown parameters which we'd like to estimate using the observed data—this is the 'learning' in machine learning, also called 'fitting the model'. Remarkably, there is a simple fitting procedure that works well across a huge range of models, from the simple climate model on page 1 all the way to neural networks. This procedure is called *maximum likelihood estimation*, and it's curve fitting of the sort referred to by Judea Pearl. Likelihood maximization isn't just one technique among many, it's the key idea underlying a huge range of machine learning; and the rest of Part I is all about generalizations of the worked examples in this section.

Definitions. Given a probability model for the observed data, the *likelihood* is the probability of that data (if it's discrete), or the probability density (if it's continuous).

- If the observed data consists of a collection of datapoints, and if the probability model says that they're all independent, then the likelihood is the product of all the likelihoods of the individual datapoints.

A model with unknown parameters θ is said to be *parameterized by* θ. Here θ may be as simple as a single real number, or it may be a collection of values of various types. The *maximum likelihood estimator* of θ, or *mle*, is the parameter that maximizes the likelihood, and it's written as $\hat{\theta}$. Think of the $\hat{}$ as a mountain top, reminding us that we found a maximum!

- There may not be a unique maximum … until we've shown that the maximizer $\hat{\theta}$ is unique, we should only say that $\hat{\theta}$ is *a* mle, not *the* mle.
- For probability models with multiple unknown parameters, we must maximize the likelihood over all parameters simultaneously. Even if we're only interested in one of them, we have to estimate all of them and then just ignore the ones we're not interested in.
- Suppose we have a model parameterized by θ, and we want to estimate $\phi = f(\theta)$ for some given function f. If $\hat{\theta}$ is an mle for θ, then $\hat{\phi} = f(\hat{\theta})$ is an mle for ϕ, called the plug-in estimator.

Here are some worked examples of maximum likelihood parameter estimation. The probability models we'll look at here are based on simple random variables that you will hopefully be familiar with from an introductory course in probability, such as the Normal and Exponential, and the probability density formulae will hopefully be familiar to you. Section 1.5 will give a more formal explanation of where these formulae come from, and section 5.3 will explain how to derive probability density functions for custom probability models.

Pay close attention to how we go from dataset + probability model to maximization problem, because similar setups will appear again and again. Don't bother too much about the calculus behind the optimizations: it's included here for completeness (and because you might be asked for similar derivations in the exam!), but in practical data science and machine learning we leave the optimization to a computer.

[8] Judea Pearl won the ACM Turing Award, the "Nobel prize of computer science". His field is causal reasoning and artificial intelligence. Kevin Hartnett. "To Build Truly Intelligent Machines, Teach Them Cause and Effect". In: *Quanta magazine* (May 2018). URL: https://www.quantamagazine.org/to-build-truly-intelligent-machines-teach-them-cause-and-effect-20180515/

Exercise 1.3.1 (Coin tosses).
Suppose we take a biased coin, and tossed it $n = 10$ times, and observe $x = 6$ heads. Let's use the probability model $X \sim \text{Bin}(n, p)$, where p is the probability of heads. The probability mass function is

$$\mathbb{P}(\text{num. heads} = x) = \binom{n}{x} p^x (1-p)^{n-x}, \quad x \in \{0, 1, \ldots, n\}.$$

Estimate p.

$\binom{n}{x}$ is the binomial coefficient, equal to $n! \,/\, x!(n-x)!$

The likelihood is the probability of the observed data[9] $x = 6$*:*

$$\text{lik} = \binom{n}{x} p^x (1-p)^{n-x}$$

differentiate as we're trying to find maximum

We want to find the value of p that maximizes this. To find it, solve

$$\frac{d}{dp}\text{lik} = \binom{n}{x}\left(xp^{x-1}(1-p)^{n-x} - (n-x)p^x(1-p)^{n-x-1}\right) = 0$$

which has the solution

$$\hat{p} = \frac{x}{n}.$$

It's often easier to maximize $\log(\text{lik}(\cdot))$ *rather than* $\text{lik}(\cdot)$*: it must give us the same solution, because* log *is an increasing function. In this case,*

$$\log \text{lik} = \kappa + x \log p + (n-x)\log(1-p)$$

where $\kappa = \log \binom{n}{x}$*. Note that κ doesn't depend on p—so as far as likelihood is concerned, κ is a constant. When we solve*

$$\frac{d}{dp}\log \text{lik} = \frac{x}{p} - \frac{n-x}{1-p} = 0$$

we also get the solution $\hat{p} = x/n$*.*
□

Exercise 1.3.2 (Exponential sample).
Let the dataset be a list of real numbers, $x_1, \ldots, x_n$, all > 0. Let's use the probability model that says they are all independent $\text{Exp}(\lambda)$ random variables, where λ is unknown. This is called the Exponential distribution and it has probability density function

$$\text{pdf}(x) = \lambda e^{-\lambda x}.$$

Estimate λ.

This is a case where the observed data consists of a collection of datapoints $x_1, ..., x_n$, all independent. So the likelihood is the product of the likelihoods of each of the individual datapoints. The Exponential is a continuous random variable, so the likelihood is given by the probability density function. Thus

$$\begin{aligned}\text{lik} &= \lambda e^{-\lambda x_1} \times \cdots \times \lambda e^{-\lambda x_n} \\ &= \lambda^n e^{-\lambda \sum_{i=1}^n x_i}.\end{aligned}$$

(It's a surprisingly common mistake to just copy out the probability density function and leave x in the likelihood formula, rather than replacing it with x_i. Don't just copy! Think about the dataset whose likelihood you're calculating!)

[9] Why did we take the observed data to be $x = 6$, rather than the pair $(x, n) = (6, 10)$? In general, we take the observed data to be *whatever might be different if we reran the experiment.* The probability model specified in the question describes a coin-tosser who decided on $n = 10$ in advance, and then tossed the coin, and happened to see $x = 6$, i.e. it treats x as the observed data and $n = 10$ as a fixed parameter. But other interpretations of the first sentence are possible—for example, perhaps the coin-tosser kept tossing coins until she saw $x = 6$ heads, and the observed data is n, the number of tosses needed. If we believed this to be the mechanism, we'd have chosen a different probability model.

In the real world, no one ever tells you the probability model or the mechanism behind a dataset, and you have to invent one yourself.

The maximum likelihood estimator for λ is the value that maximizes the likelihood. To find it, we'll use the log trick:

$$\log \text{lik} = n \log \lambda - \lambda \sum_i x_i$$
$$\frac{d}{d\lambda} \log \text{lik} = \frac{n}{\lambda} - \sum_i x_i$$

and solving $d \log \text{lik} / d\lambda = 0$ gives

$$\hat{\lambda} = \frac{n}{\sum_i x_i}.$$

□

Exercise 1.3.3 (Normal sample).
Let the dataset be a list of real numbers, $x_1, \ldots, x_n$. Let's use the probability model that says they are all independent Normal(μ, σ^2) random variables, where $\mu \in \mathbb{R}$ and $\sigma > 0$ are unknown. Estimate these parameters.

This is a case with multiple unknown parameters, so we must maximize the likelihood over all parameters simultaneously.

The Normal is a continuous random variable with probability density function

$$\text{pdf}(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x-\mu)^2/2\sigma^2}.$$

Our probability model says that the datapoints are all independent, so the likelihood is the product of the likelihoods of individual datapoints,

$$\text{lik} = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x_i-\mu)^2/2\sigma^2},$$

so the log likelihood is a sum,

$$\begin{aligned} \log \text{lik} &= \sum_i \Big\{ -\frac{1}{2} \log(2\pi\sigma^2) - \frac{(x_i - \mu)^2}{2\sigma^2} \Big\} \\ &= -\frac{n}{2} \log(2\pi) - n \log \sigma - \frac{\sum_i (x_i - \mu)^2}{2\sigma^2}. \end{aligned}$$

There are two unknown parameters here, μ and σ, so we must find the maximum over both of them simultaneously. We can do this by solving a pair of simultaneous equations:

$$\frac{\partial}{\partial \mu} \log \text{lik} = \frac{\sum_i 2(x_i - \mu)}{2\sigma^2} = 0$$
$$\frac{\partial}{\partial \sigma} \log \text{lik} = -\frac{n}{\sigma} + \frac{\sum_i (x_i - \mu)^2}{\sigma^3} = 0.$$

After a bit of algebra, the solution is

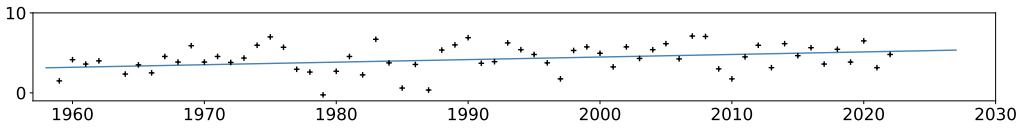
$$\hat{\mu} = \frac{\sum_i x_i}{n}, \qquad \hat{\sigma} = \sqrt{\frac{1}{n} \sum_i (x_i - \hat{\mu})^2}.$$

□

Exercise 1.3.4 (Curve fitting / nuisance parameters).
For the climate dataset from example 1.1.1 page 3, considering just the temperatures in January, let t_i be the year and let y_i be the temperature for record $i = 1, \ldots, n$. Using the probability model

$$Y_i \sim \alpha + \gamma t_i + \text{Normal}(0, \sigma^2)$$

estimate γ, the annual rate of temperature change. In other words, fit a straight-line 'curve' $y = \alpha + \gamma t$, and report the slope. This plot shows January temperatures in Cambridge with black crosses, and also the fitted line:



(The take-away from this exercise is how we deal with nuisance parameters, in this case α and σ.)

Before answering this, some remarks.

- The question carefully distinguishes between the actual observed temperature in the dataset, y_i, and the distribution from the proposed probability model, Y_i. As usual, our convention is to use lower case for constants and parameters, and upper case for random variables.
- The question asks us to consider just the January temperatures. This is daft! — it's a sin to throw away data. But to make use of the full dataset we'd need a better model, such as the sinusoidal model on page 4, and to find maximum likelihood estimates from such a model we need better tools. Such tools will be the topic of sections 1.4 and 2.

The question implies that the temperatures are independent random variables, since the probability model doesn't explicitly say otherwise, so the likelihood of the dataset is the product of individual likelihoods. For a single temperature Y_i, the algebra of the Normal distribution tells us that

algebra of Normal: page 8

$$Y_i \sim \text{Normal}\big(\alpha + \gamma t_i, \sigma^2\big)$$

which has probability density function

$$\text{pdf}_i(y) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(y-\alpha-\gamma t_i)^2/2\sigma^2}.$$

The likelihood is thus

$$\text{lik} = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(y_i-\alpha-\gamma t_i)^2/2\sigma^2}$$

(where we've been careful to replace the generic y by the appropriate values y_i from the dataset), and the log likelihood is

$$\log \text{lik} = -\frac{n}{2}\log(2\pi) - n\log\sigma - \frac{1}{2\sigma^2}\sum_{i=1}^{n}\big(y_i - \alpha - \gamma t_i\big)^2.$$

We're only asked to estimate γ, but there are actually three parameters in this model, α, γ, and σ. We haven't been told anything about α or σ, so we'll treat all three parameters as unknown. Since there are multiple unknown parameters, we must find the maximum over all of them simultaneously. We can do this by solving three simultaneous equations:

$$\frac{\partial}{\partial\alpha}\log\text{lik} = 0, \qquad \frac{\partial}{\partial\gamma}\log\text{lik} = 0, \qquad \frac{\partial}{\partial\sigma}\log\text{lik} = 0.$$

After a little algebra the solution for γ is

$$\hat{\gamma} = \frac{(\sum_i t_i)(\sum_i y_i) - n\sum_i t_i y_i}{(\sum_i t_i)^2 - n\sum_i t_i^2}$$

```
url = 'https://www.cl.cam.ac.uk/teaching/current/DataSci/data/climate.csv'
climate = pandas.read_csv(url)
df = climate.loc[(climate.station=='Cambridge') & (climate.mm==1)]
(t, y, n) = df.yyyy, df.temp, len(df)
γ̂ = (np.sum(t)*np.sum(y) - n*np.sum(y*t)) / (np.sum(t)**2 - n*np.sum(t**2))
```

The result is $\hat{\gamma} = 0.032604$ °C/year. The question only asked us to estimate γ, so we needn't bother reporting $\hat{\alpha}$ or $\hat{\sigma}$.
□

Exercise 1.3.5 (Categorical sample / constrained optimization).
Suppose we ask 100 people their views on Brexit, and let $x_i \in \{L, R, U\}$ be the choice of person i, L = Leave, R = Remain, U = Undecided. Find the maximum likelihood estimator for p_L from the probability model

$$\mathbb{P}(X_i = \text{Leave}) = p_L, \quad \mathbb{P}(X_i = \text{Remain}) = p_R, \quad \mathbb{P}(X_i = \text{Undecided}) = p_U.$$

(It's implied by the question that the unknown parameters are $p_L \geq 0$, $p_R \geq 0$, and $p_U \geq 0$, and that they form a legitimate probability distribution i.e. they satisfy the constraint $p_L + p_R + p_U = 1$. The take-away from this exercise is how we handle constraints on the parameters.)

This is called a Categorical random variable. We could also write it as $X_i \sim \text{Cat}([p_L, p_R, p_U])$.

It's implied that the views of different people are to be modelled as independent, since the probability model doesn't explicitly say otherwise, so the likelihood of the entire dataset is the product of the individual likelihoods,

$$\text{lik} = \prod_{i=1}^{n} \mathbb{P}(X_i = x_i).$$

Let n_L be the total number of people for whom $x_i = L$, and n_R and n_U likewise. Then we can write the product more simply:

$$\begin{aligned} \text{lik} &= p_L^{n_L} p_R^{n_R} p_U^{n_U} \\ \log \text{lik} &= n_L \log p_L + n_R \log p_R + n_U \log p_U. \end{aligned}$$

The question asks for the maximum likelihood estimator for p_L. But the general rule for maximum likelihood estimation says that we have to maximize the likelihood over all parameters simultaneously, even if we're only interested in one of them. Here we have to maximize over p_L, p_R, and p_U, subject to the constraint that $p_L + p_R + p_U = 1$. It's easiest to deal with this constraint by substitution, i.e., we'll do the maximization over p_L and p_R, and substitute in $p_U = 1 - p_L - p_R$. Rewriting the log likelihood in terms of these variables,

$$\log \text{lik} = n_L \log p_L + n_R \log p_R + n_U \log(1 - p_L - p_R).$$

This is a function of two variables. To find the maximum, we need to solve two equations simultaneously:

$$\frac{\partial}{\partial p_L} \log \text{lik} = 0 \qquad \textit{and} \qquad \frac{\partial}{\partial p_R} \log \text{lik} = 0.$$

Doing the differentiation,

$$\frac{n_L}{\hat{p}_L} - \frac{n_U}{1 - \hat{p}_L - \hat{p}_R} = 0 \qquad \textit{and} \qquad \frac{n_r}{\hat{p}_R} - \frac{n_U}{1 - \hat{p}_L - \hat{p}_R} = 0$$

which after some alegebra gives

$$\hat{p}_L = \frac{n_L}{n_L + n_R + n_U} \qquad \textit{and} \qquad \hat{p}_R = \frac{n_R}{n_L + n_R + n_U}.$$

□

The general rule says that we have to maximize the likelihood over all unknown parameters, even if we're only interested in one of them. Let's see how things would go wrong if we only maximized over p_L. If we had INCORRECTLY solved

$$\frac{d}{dp_L} \log \text{lik} = 0$$

we'd get the solution

$$\hat{p}_L = (1 - p_R)\frac{x_L}{n - x_R}.$$

This is NO USE as an estimator for p_L because the right hand side depends on p_R which is unknown.

Exercise 1.3.6 (Using indicator functions to handle if/then clauses).
We throw a k-sided die, and get the answer 10. It's a fancy die with lots of sides, tricky to count k by eye. Estimate k, using the probability model

$$\mathbb{P}(\text{throw } x) = \frac{1}{k}, \qquad x \in \{1, \ldots, k\}.$$

(The take-away from this exercise is how to handle problems where the unknown parameter appears in the range of the random variable. Or, more generally, how to deal with problems where the likelihood has an if/then clause.)

First, here's the wrong approach. Write out the likelihood, which is just the probability of the observed data:

$$\text{lik} = \frac{1}{k}$$

and try to maximize it. The solution is $k = 1$ (is there even such a thing as a one-sided die???) which is nonsense because it ignores x.

The better approach is to write out the likelihood function in full, incorporating the restriction $x \in \{1, \ldots, k\}$ into the likelihood function itself:

$$\text{lik} = \begin{cases} 1/k & \text{if } x \geq 1 \text{ and } x \leq k \\ 0 & \text{otherwise.} \end{cases}$$

(We'll leave it implicit that k is an integer, rather than writing it into the likelihood function. When I'm ready to do the optimization we'll make sure to remember to optimize over integer values of k.) Next, we'll rewrite these cases using indicator functions. Indicator functions are just a notational trick that lets us turn if/then conditions into algebra. The indicator function is defined to be

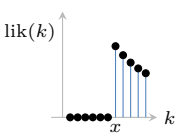
$$1_A = \begin{cases} 1 & \text{if statement } A \text{ is true} \\ 0 & \text{if statement } A \text{ is false.} \end{cases}$$

So the likelihood function can be written as

$$\begin{aligned} \text{lik} &= \frac{1}{k} 1_{x \geq 1 \text{ and } x \leq k} \\ &= \frac{1}{k} 1_{x \geq 1} 1_{x \leq k}. \end{aligned}$$

Note how the indicator notation turns 'and' into 'times': the product $1_{x \geq 1} 1_{x \leq k}$ is only equal to 1 when both factors are 1 i.e. when both conditions are true. Now for the second trick with indicator functions. We can rewrite the likelihood as

$$\text{lik} = \frac{1}{k} 1_{x \geq 1} 1_{k \geq x}.$$



I flipped the condition $x \leq k$ into $k \geq x$, to emphasize that I'm interested in this as a function of k. The $1_{x \geq 1}$ term can be ignored, because it doesn't involve k. Now it's easy to sketch the likelihood, and the maximum is clearly at $\hat{k} = x$.
□

Exercise 1.3.7 (Plug-in estimation).
Gamblers like to specify probabilities in terms of *odds*: when they say the odds of winning are 1-to-h, they mean we're likely to get 1 win for every h losses, i.e. the probability of winning is $p = 1/(1 + h)$.

Consider the biased coin from example 1.3.1. Estimate the odds of heads.

In that earlier exercise, we fitted the model $X \sim \text{Bin}(n, p)$, where X is the number of heads from n coin tosses. Inverting the formula for p in terms of h, we see that $h = (1-p)/p$. This is a function of p, and the plug-in principle (the last bullet point of the definitions at the beginning of this section) tells us that the mle for h is simply $\hat{h} = (1-\hat{p})/\hat{p} = (n-x)/x$.
□

Exercise 1.3.8 (Plug-in estimation).
Consider a dataset consisting of two collections of real numbers, $x_1, \ldots, x_m$ and $y_1, \ldots, y_n$. Model the first collection as Normal(μ, σ^2) random variables, and the second collection as Normal(ν, σ^2) random variables, all $m+n$ variables independent, where μ, ν, and σ are unknown. Estimate $\mu - \nu$.

Our probability model says that the datapoints are all independent, so the likelihood is the product of the likelihoods of all $m+n$ individual datapoints,

$$\text{lik} = \left(\prod_{i=1}^{m} \frac{1}{\sqrt{2\phi\sigma^2}} e^{-(x_i-\mu)/2\sigma^2} \right) \left(\prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(y_i-\nu)/2\sigma^2} \right).$$

(Don't be tempted to write out separate probability models for the two collections of observations! Our observed data is both collections, and it's a good habit to always write out a single probability model for the entirety of the observed data.)

This model has three unknown parameters, and we can fit it in the usual way by solving three simultaneous equations:

$$\frac{\partial}{\partial \mu} \log \text{lik} = 0, \qquad \frac{\partial}{\partial \nu} \log \text{lik} = 0, \qquad \frac{\partial}{\partial \sigma} \log \text{lik} = 0.$$

The algebra simplifies massively, and the answer is

$$\hat{\mu} = \frac{1}{m} \sum_i x_i, \qquad \hat{\nu} = \frac{1}{n} \sum_i y_i, \qquad \hat{\sigma} = \textit{more fiddly.}$$

We've been asked to estimate $f(\mu, \nu, \sigma) = \mu - \nu$. The plug-in principle (the last bullet point from the definitions at the beginning of this section) tells us that the mle for $f(\mu, \nu, \sigma)$ is given by $f(\hat{\mu}, \hat{\nu}, \hat{\sigma}) = \hat{\mu} - \hat{\nu}$.
□

SANITY CHECKS

We have used the term 'maximum likelihood estimat*or*'. This emphasizes that the mle is a *function*, which takes the observed data as its input, and returns an estimated value as its output. When you derive an estimator, scan through your formula and double-check that it doesn't have any unknown parameters. The incorrect solution at the end of exercise 1.3.5, where we tried to find the mle by maximizing over just a single parameter, fails this sanity check.

Another useful sanity check is to think whether your mle depends on the data in the way you'd expect. The incorrect solution to exercise 1.3.6 fails this check.

It's also worth checking that the 'observed data' for which you found the likelihood includes absolutely all the data that has any bearing on the unknown parameter or parameters. It's mathematically legitimate to find the mle using only some of the available data, but it's a venial sin to throw data away when you could use it to learn.

1.4. Numerical optimization with scipy

Learning from data requires optimization. The workhorse of machine learning is numerical optimization, since mathematical solutions aren't known except for the most basic of probability models. There is much advice to be found about numerical optimization algorithms, but not much that sheds light on the concepts behind data science and machine learning, so we won't say much here.

Seeing as numerical optimization is such an important workhorse, there are many specialized algorithms. Indeed, in any useful branch of machine learning, chances are there's some specialized library for efficient numerical optimization within that branch. In this section we'll look at a simple general-purpose tool that will be adequate for most of the small problems in this course. (For bigger problems, see section 3.3.)

Optimization with scipy. To find the minimum of a function $f : \mathbb{R}^K \to \mathbb{R}$,

```
import scipy.optimize

# The function to minimize. Input x is a length-K list, output is a real number
def f(x):
    return ....

x0 = [...] # where to start the search, a length-K list
x̂ = scipy.optimize.fmin(f, x0)
```

There is no scipy.optimize.fmax, so to maximize $f(x)$ we should find the minimum of $-f(x)$.

The optimization routine isn't omniscient. It will find a local minimum, not necessarily a global minimum. It might fail to find even a local minimum, if the function isn't well-behaved. To make it happy, pick a sensible x_0, based on your understanding of roughly what the answer is likely to be.

Sometimes we want to find a minimum over a constrained domain, for example to find $\min_{x>0} f(x)$. The problem with just calling scipy's fmin(f,...) is that it will range over any values for x that it wants, hunting for the minimum, and perhaps our function throws an exception or gives nonsense answers for $x \leq 0$. In such cases it's best to transform parameters into an unconstrained domain. For example,

```
scipy.optimize.fmin(lambda lx: f(np.exp(lx)), np.log(x0))
```

which is shorthand syntax for defining a helper function $f'(y) = f(e^y)$ and calling fmin on f'. Finding $y \in \mathbb{R}$ that minimizes $f(e^y)$ does the same thing as finding $x > 0$ that minimizes $f(x)$. Exercise 1.4.1 illustrates. Another common transformation is the so-called softmax function, illustrated in exercise 1.4.2.

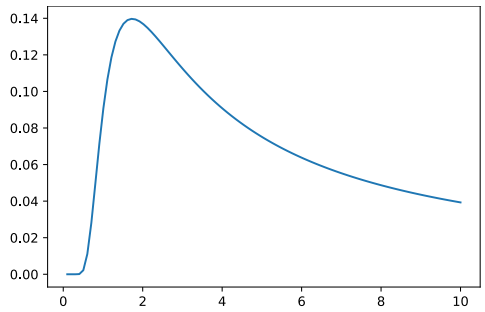
Exercise 1.4.1. Find the maximum over $\sigma > 0$ of

$$f(\sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-3/2\sigma^2}$$

Let's optimize over $\tau \in \mathbb{R}$, with the transform $\sigma = e^\tau$. This ensures $\sigma > 0$.

```
def f(σ):
    return np.exp(-3/2/σ**2) / np.sqrt(2*π*σ**2)

# From the plot, the maximum is somewhere around σ = 2
# I implemented f using numpy maths functions, so that it's automatically vectorized,
# which is handy for plotting.
fig,ax = plt.subplots()
σ = np.linspace(0,10,100)[1:] # remove σ = 0, where the function doesn't work
ax.plot(σ, f(σ))
```

```
plt.show()
```




```
# Optimize in terms of τ = log σ, τ ∈ ℝ
# We want to maximize f, i.e. minimize -f
# fmin returns a list of length 1; unpack it with (τ̂,)=...
(τ̂,) = scipy.optimize.fmin(lambda τ: -f(np.exp(τ)), np.log(2))
σ̂ = np.exp(τ̂)
```

□

Exercise 1.4.2 (Softmax transformation).
Find the maximum of

$$f(p_1, p_2, p_3) = 0.2 \log p_1 + 0.5 \log p_2 + 0.3 \log p_3$$

over $p_1, p_2, p_3 \in [0, 1]$ such that $p_1 + p_2 + p_3 = 1$.

Let's optimize over $(s_1, s_2, s_3) \in \mathbb{R}^3$, with the transform

$$p_1 = \frac{e^{s_1}}{e^{s_1} + e^{s_2} + e^{s_3}}, \quad p_2 = \frac{e^{s_2}}{e^{s_1} + e^{s_2} + e^{s_3}}, \quad p_3 = \frac{e^{s_3}}{e^{s_1} + e^{s_2} + e^{s_3}}.$$

The exponentiation makes sure we get positive values, even for negative s_i. The normalization by $e^{s_1} + e^{s_2} + e^{s_3}$ ensures we get $p_1 + p_2 + p_3 = 1$. Since the p_i are positive and sum to one, they must be in the range $[0, 1]$. And every valid (p_1, p_2, p_3) corresponds to some (s_1, s_2, s_3), so we're not missing anything by this transform.

```
def f(p):
    (p₁,p₂,p₃) = p
    return 0.2*np.log(p₁) + 0.5*np.log(p₂) + 0.3*np.log(p₃)

def softmax(s):
    p = np.exp(s)
    return p / np.sum(p)

# Initial guess s₁ = s₂ = s₃ = 0, i.e. p₁ = p₂ = p₃ = 1/3
ŝ = scipy.optimize.fmin(lambda s: -f(softmax(s)), [0,0,0])
p̂ = softmax(ŝ) # answer: [0.2, 0.5, 0.3]
```

We don't actually need three parameters because we know $p_3 = 1 - p_1 - p_2$ so there are only two free variables in this problem; another way to see this is to note that for example $(s_1, s_2, s_3) = (1, 5, 2)$ gives exactly the same p_i as $(s_1, s_2, s_3) = (1 + c, 5 + c, 2 + c)$ for any c, so we might as well fix one of the s_i at 0. When we're optimizing using calculus and algebra we have to use this sort of trick; but when we're optimizing numerically the redundancy doesn't matter, and scipy.optimize *just returns an arbitrary choice among all optimal s.*
□

There's nothing special about the transform we used; we could use any transform we like, as long as it produces (p_1, p_2, p_3) that satisfy the constraint. This particular choice is a version of the so-called *softmax* transform, widely used in machine learning.

the softmax transform used for neural network classification: section 3.2 page 60

It's overkill to use a computer to solve this particular optimization problem. It's exactly the problem from exercise 1.3.5 on page 15, and we could have derived the optimum using maths.

1.5. Likelihood notation

The two fundamental steps of machine learning are (1) invent a probability model and (2) fit it using maximum likelihood estimation. For the second step, we need to be able to write out a formula for the likelihood.

Likelihood is such a central concept in machine learning that it's useful to have better notation than what we've used so far. Good notation allows us to express ourselves more precisely, using algebra rather than waffly sentences. This section outlines the notation we'll be using throughout this book.

Likelihood. The *likelihood function*[10] for a random variable X, written $\Pr_X$, is defined as

$$\Pr_X(x) = \mathbb{P}(X = x) \qquad \text{in the case where } X \text{ is a discrete r.v.}$$

and as

$$\Pr_X(x) = \text{pdf}(x) \qquad \text{in the case where } X \text{ is a continuous r.v.}$$

where pdf is the probability density function for X.

The job of this likelihood notation is to keep track of two separate things: the random variable X, which is a function that can give different answers; and a value x which is one possible output value of the function. Read $\Pr_X(x)$ as "the likelihood **for** X **of** x". This separation is particularly helpful for more complex probability models. As any mathematician will tell you, the way to deal with complexity is to invent good notation!

(Our $\Pr_X(x)$ notation is not by any means universal. In engineering, it's common to write for example $p(x)p(y)$ and expect the reader to intuit that there are two different random variables being described, what these notes would write as $\Pr_X(x)\Pr_Y(y)$. Another common engineering notation is $\mathcal{N}(x; \mu, \sigma^2)$ to refer to the density of the Normal distribution; this at least is unambiguous, but it doesn't help if we want to refer to custom random variables, for example the density of $|N(\mu, \sigma^2)|$. In mathematics, the style is to be absolutely precise but a little long-winded: "Let $X \sim N(\mu, \sigma^2)$, let $Y = |X|$, and let $g(y)$ be the probability density function of Y." The notation in these notes is, in the author's opinion, superior to both styles.)

Here are some of the conventions and common cases, covering everything we need for working with simple probability models and standard random variables. Section 5.3 illustrates how to find the likelihood for custom probability models.

Parameterized random variables: $\Pr_X(x\,;\,\theta)$	For a random variable which takes parameters, it's sometimes useful to emphasize those parameters, especially when they are parameters we want to learn from data. We can write $\Pr_X(x\,;\,\theta)$ to emphasize that the distribution of X depends on θ. Example: $$X \sim U[0,\theta] \quad \Leftrightarrow \quad \Pr_X(x\,;\,\theta) = \frac{1}{\theta} \text{ for } x \in [0,\theta]$$ It is a matter of style and taste whether or not you include the parameters in Pr.
Transformed random variables, e.g. $\Pr_{X+Y}(z)$ or $\Pr_{X^2}(z)$	We'll build up probability models by taking the basic building blocks (discrete and continuous random variables) and doing things to them. The Pr notation helps us keep track. Instead of writing out "Given two discrete random variables X and Y, let $Z = X + Y$, and then $\mathbb{P}(Z = z) = \ldots$" we just write "$\Pr_{X+Y}(z) = \ldots$"

[10] It's common especially in statistics to reserve the word 'likelihood' for parameters and hypotheses, for example "the likelihood that $\theta > 0$ given the data". In machine learning, it's more common to read "the likelihood of the data". In an earlier version of these notes I gave these two concepts different names, but students couldn't see the point—and I agree with them.

Pairs of random variables: $\Pr_{X,Y}(x, y)$

If we have a pair of random variables, then it's useful to be able to reason about them jointly. For a pair of discrete random variables[11]

$$\Pr_{X,Y}(x, y) = \mathbb{P}(X = x \text{ and } Y = y).$$

For continuous random variables there's a somewhat trickier definition which we'll come to in section 5. What matters for now is the intuition (that it's an 'and' sort of quantity), and the rules for the following two special cases, independence and sequential generation.

Independent r.v.: $\Pr_{X,Y}(x, y) = \Pr_X(x) \Pr_Y(y)$

If two random variables X and Y are independent, then the likelihood of the pair is the product of likelihoods. For discrete random variables this follows immediately from the definition of independence; for continuous random variables it takes a bit more technical work.

In fact the converse is also true: if the joint likelihood $\Pr_{X,Y}(x, y)$ factorizes into a term with x and a term with y, then X and Y are independent.

I.i.d. sample: $\Pr(x_1, \ldots, x_n) = \Pr_X(x_1) \times \cdots \times \Pr_X(x_n)$

Many machine learning problems involve a sample of independent observations from a single distribution, also known as an 'independent identically-distributed' or i.i.d. sample. The equation for $\Pr(x_1, \ldots, x_n)$ is the likelihood of an i.i.d. sample of n values drawn from a common distribution X.

Sequential generation of X then Y: $\Pr_{X,Y}(x, y) = \Pr_X(x) \Pr_Y(y\,;\, x)$

Suppose we first generate X and next generate Y using X, for example

```
x = np.random.uniform(-1,1)
y = np.random.normal(loc=x**2, scale=σ)
```

or equivalently $X \sim U[-1, 1]$, $Y \sim N(X^2, \sigma^2)$. This says that the distribution of Y is parameterized by the value x taken by X, in this case

$$\Pr_Y(y\,;\, x, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(y-x^2)/2\sigma^2}$$

and the joint likelihood is a product, this likelihood for Y times the likelihood for X.

Guess! $\Pr(x, y, z)$ $\Pr_{X,Y,Z}(u)$

When building up probability models, it's often fairly obvious from the context which random variables we are referring to. For example, if we see $\Pr(x, y, z)$, we know it's referring to a tuple of three random variables, probably called (X, Y, Z). Or if we see $\Pr_{X,Y,Z}(u)$, we know that we're describing a tuple random variable (X, Y, Z), and therefore the output u must be a tuple of length 3. Think of this as type inference for maths notation!

The power of good notation

There are many results in machine learning that have two versions, one for discrete and another for continuous, and it's a real nuisance to have to keep saying every sentence twice, especially when the formulae work pretty much the same in both cases. The power of the $\Pr_X(x)$ notation is that it lets us simply write out one formula and say "Oh, and it just works for both discrete and continuous". Here are two examples.

[11] Formally, the pair $Z = (X, Y)$ is itself a discrete random variable, and $\Pr_{X,Y}(x, y)$ is nothing other than the probability mass function of Z, namely $\mathbb{P}\big(Z = (x, y)\big)$.

Maximum likelihood estimation. Section 1.3 described maximum likelihood estimation, and it used waffly wording to deal with all the different cases (discrete, continuous, single datapoint, many datapoints). With the $\Pr_X(x)$ notation, we can describe it much more succinctly:

Suppose we have a probability model where the possible outcomes are described by a random variable X whose distribution depends on an unknown parameter θ, and suppose we have observed the outcome x. Then the maximum likelihood estimator for θ is that value that maximizes $\Pr_X(x\,;\,\theta)$,

$$\hat{\theta} = \arg\max_{\theta} \Pr_X(x\,;\,\theta).$$

Here X could describe a single observation, a tuple, an i.i.d. sample, or any other type of random variable at all. And θ could be a single value, or a tuple of several values.

Bayes's rule for continuous random variables. We know what Bayes's rule means for discrete random variables:

$$\mathbb{P}(X = x \mid Y = y) = \frac{\mathbb{P}(X = x)\ \mathbb{P}(Y = y \mid X = x)}{\mathbb{P}(Y = y)} \quad \text{when } \mathbb{P}(Y = y) > 0.$$

But what if Y is a continuous random variable? Then $\mathbb{P}(Y = y) = 0$ for every y, so it looks like Bayes's rule is no use. But it turns out that there is version of Bayes's rule that does work, and it can be written

$$\Pr_X(x \mid Y = y) = \frac{\Pr_X(x)\ \Pr_Y(y \mid X = x)}{\Pr_Y(y)} \quad \text{when } \Pr_Y(y) > 0$$

and this is more useful because even though the probability $\mathbb{P}(Y = y)$ is zero for all y, the probability density function $\Pr_Y(y)$ isn't. We will come back to this equation, and explain the meanings of the terms, in Part II. For now, the thing to remember is: $\Pr_X(x)$ is just $\mathbb{P}(X = x)$ for a discrete random variable, and for a continuous random variable it behaves how we'd like it to behave.

* * *

We've worked with three ways of describing a probability model: code, random variable notation, and likelihood. None of these is more definitive than the others, they're just different views of the same thing. When someone asks you for a probability model, you can give your answer in any of these three ways.

I like the code view best for intuition: I can run the code and plot the output, and see if it matches the real-world objects I'm trying to model. The likelihood view is what we need for machine learning, because it's the likelihood formula that we feed into our optimizer. The random variable notation is (arguably) the cleanest and most concise way to describe a probability model, and more importantly it's the bridge between code and likelihood.

1.6. Types of model

There are three broad tasks for which we use models, prediction, description, and generation; and two broad ways to train models, supervised and unsupervised.

These labels are fuzzy, overlapping, and incomplete, and as you gain experience in modelling you'll begin to wonder why people bother trying to separate out things that are really all just variations on "write out a probability model and fit it". Nonetheless it's useful to know the vocabulary, if only as a shorthand when you want to explain what sort of modelling you're doing.

- *Prediction* describes simple tasks such as "predict the temperature on a given date". There's an output variable called the *response* or *label* (temperature), and one or more input variables called the *predictors* or *features* or *covariates* (date). Where the response is a real number, such as temperature, it's called *regression*.
- Image classification also fits under the 'prediction' heading. Here the predictor variables are the pixels of the image, and the label is the class of the image. This sort of prediction, in which the response has a fixed number of possible values, is called *classification*.
- *Description* is when we're interested in the internals of the fitted model. In example 1.3.4 we fitted a model for temperatures in Cambridge, and used it to estimate the annual rate of increase — this is descriptive modelling.
- *Generation* is when we use the model to synthesize new values. Take the simple random climate model from example 1.1.1: we tell it a date t and it synthesizes a random temperature for that date. If we call it over and over again, we'll see a distribution of temperatures for that date.
- 'Generative AI', technically speaking, refers to any probability model, since probability models allow us to generate new values. Our simple random climate model could be called generative AI! The opposite of generative AI is deterministic modelling, for example the no-noise model for temperature on page 1, or simulators based on differential equations.
- *Supervised learning* means training a predictive model. (Some people use the word 'prediction' only for deterministic models, and some use it for both deterministic and probabilistic. 'Supervised learning' is used for both.) Our dataset consists of pairs (x_i, y_i) where x_i is the predictor variable or variables for record i, and y_i the response, also called the *ground truth*. The goal is to train the model so that its predicted responses match the ground truth.
- *Unsupervised learning* is a general term for any machine learning method that works with unlabelled data, i.e. our dataset consists simply of a collection of values x_i. Often there's an emphasis on clustering algorithms. In my opinion, all the cool stuff is *generative unsupervised learning*, in which the goal is to find a probability distribution that matches the distribution of the datapoints. An elementary example is fitting a Normal distribution to a collection of real numbers, example 1.3.3. A much more interesting example is Large Language Models, discussed below.

Example 1.6.1 (MNIST digits / types of model).
The famous MNIST dataset[12] consists of hand-written digits stored as 28×28 greyscale images, annotated with the digit they represent. Let $x_i \in \mathbb{R}^{28\times 28}$ be the ith image in the dataset, and let $y_i \in \{0, 1, \ldots, 9\}$ be its label.

What might we use predictive models for, in the context of this dataset? What about generative? How might these models be trained? (There are many examples of supervised and unsupervised learning in section 1.7. You might like to read ahead before answering this part.)

- *Consider a function f that takes an image x as input, and returns as output a label y,*

Training this sort of model was discussed in IA MLRD

[12] Yann LeCun, Corinna Cortes, and Christopher J. C. Burges. *The MNIST Database of handwritten digits*. 1998. URL: http://yann.lecun.com/exdb/mnist/

and has extra parameters θ. This is a predictive *model, and fitting it is called* supervised learning. *We might train it by choosing the parameters θ so as to maximize the accuracy,*

$$\sum_i 1_{f(x_i;\theta)=y_i}.$$

- *Modify f so that it returns a distribution Y over labels, indicating how likely each label is. This is technically both a* generative *model and a* predictive *model, though some people might shy away from calling it generative.*
  Fitting this model is again called supervised learning*, since for each predictor x_i the ground truth label y_i is given. Writing $\Pr_Y(y;x)$ for the probability that our function gives to label y when fed image x, we might train the model by maximizing the log likelihood of the dataset,*
  $$\sum_i \log \Pr_Y(y_i\ ;\ x_i).$$
- *Consider a 'handwriting generator' that takes a label y as input, and generates a random image X corresponding to that label. Again, this is both a* generative *and a* predictive *model, though some people might shy away from calling it predictive.*
  Fitting this model is again called supervised learning*, since for each predictor y_i a ground-truth response x_i is given. Writing $\Pr_X(x;y)$ for the likelihood that our generator gives to image x when fed label y, we might train the model by maximizing the log likelihood*
  $$\sum_i \log \Pr_X(x_i\ ;\ y_i).$$
- *Consider a function that generates a random pair (X,Y) where X is a label and Y is a corresponding image. This is a* generative *model.*
  Training this model is called unsupervised learning *since there is only a response variable, the pair (x_i,y_i), and no input variable. Writing $\Pr_{X,Y}(x,y)$ for the joint likelihood of (X,Y), we might train the model by maximizing the log likelihood*
  $$\sum_i \log \Pr_{X,Y}(x_i,y_i).$$

□

Example 1.6.2 (Language models).
What type of model is a Large Language Model such as ChatGPT or Claude?

This is a trick question! The naive answer is that it's a predictive model, since its job is to produce a reply (the response variable) given an input prompt (the predictor variable). In fact, language models are built as generative models for strings. We'll say much more in section 11, but for now here's a very crude outline:

The training dataset consists of a collection of documents, each turned into a string. Write $x^{(i)} = [x_1^{(i)}, \ldots, x_{n_i}^{(i)}]$ for the ith string, consisting of n_i characters. (Actually they use 'tokens', but think of characters for simplicity.)

Our dataset thus consists of a collection of values $x^{(i)}$, and the goal is to find a probability model that matches the distribution of the datapoints. The probability model can be thought of as a chunk of code that produces a random string. Write $X = [X_1, \ldots, X_N]$ for a random string produced by this model, and write $\Pr_X$ for its likelihood function. Training consists simply of maximizing $\sum_i \log \Pr_X(x_i)$ — classic unsupervised learning.

When we ask the language model for a response to a prompt $p = [p_1, \ldots, p_m]$, what it does is generate a sample from X conditional on $(X_1 = p_1, \ldots, X_m = p_m)$. In other words, it generates a plausible 'completion' of the string. If you've used APIs from OpenAI or others, you'll have come across the idea of completions.

The reason it's trained in this way is that there's a gigantic training dataset of documents, consisting of pretty much every piece of text that can be scraped from the web or pirated, and with a huge dataset we have the potential to train very rich models with billions of parameters. If we had to rely only on supervised learning we'd need a dataset of prompts and ground-truth replies, and it'd be pretty much impossible to collect such a big dataset.
□

1.7. Supervised and unsupervised learning

The message of this book is that most machine learning boils down in the end to two steps: (1) invent a probability model with parameters, (2) fit the model using maximum likelihood estimation. The art is inventing models that fit the data well, and for which the optimization is tractable. It's useful to have a repertoire of patterns to draw on. Let's spell out the general procedure, and then work through examples.

Probabilistic supervised learning. Suppose we're given a dataset $(x_1, y_1), \ldots, (x_n, y_n)$ where y_i is the ground truth label for record i and x_i is the predictor variable or variables.

1. Choose a probability distribution for the label, where the distribution depends on one or more unknown parameters as well as on the predictors. Write its likelihood as
$$\Pr_Y(y\,;\,x,\theta), \qquad \theta \text{ the unknown parameter(s).}$$
2. Model the dataset as independent observations of Y drawn from this distribution, i.e. let the likelihood of the dataset be
$$\Pr(y_1,\ldots,y_n\,;\,x_1,\ldots,x_n,\theta) = \Pr_Y(y_1\,;\,x_1,\theta)\times\cdots\times\Pr_Y(y_n\,;\,x_n,\theta).$$
(It gets tedious to repeat the predictors $x_1,\ldots,x_n$ all over the place, and we're treating them as known values not as observations or parameters, so feel free to omit them.)
3. Estimate θ using maximum likelihood estimation, i.e. by solving
$$\hat\theta = \arg\max_\theta \log\Pr(y_1,\ldots,y_n\,;\,x_1,\ldots,x_n,\theta).$$
This is called *fitting* or *training* the model.

Probabilistic unsupervised learning. Suppose we're given a dataset $x_1,\ldots,x_n$.

1. Choose a distribution with tuneable parameters, call it X, and let its parameters be θ. We want $x_1,\ldots,x_n$ to look like independent samples from X.
2. Write out the likelihood of the dataset. Since we're modelling the datapoints as independent, the likelihood of the entire dataset is a product,
$$\Pr(x_1,\ldots,x_n\,;\,\theta) = \Pr_X(x_1\,;\,\theta)\times\cdots\times\Pr_X(x_n\,;\,\theta).$$
3. Fit the model using maximum likelihood estimation, i.e. solve
$$\hat\theta = \arg\max_\theta\Big\{\log\Pr(x_1,\ldots,x_n\,;\,\theta)\Big\}$$

Supervised and unsupervised modelling are almost identical. The only difference is the type of model we want to invent:

	Supervised	Unsupervised
Data	Each record i consists of (x_i, y_i)	Each record i consists of x_i
Parameters	Unknown parameters θ	Unknown parameters θ
Model	Independent responses Y_i	Independent samples X_i
Likelihood	$\Pr_Y(y_i\,;\,x_i,\theta)$	$\Pr_X(x_i\,;\,\theta)$
Fitting	Learn θ using mle	Learn θ using mle

Exercise 1.7.1 (Straight-line fit / supervised).
Given a labelled dataset $[(y_1, x_1), \ldots, (y_n, x_n)]$ consisting of pairs of real numbers, fit the model

$$Y_i = a + bx_i + \text{Normal}(0, \sigma^2),$$

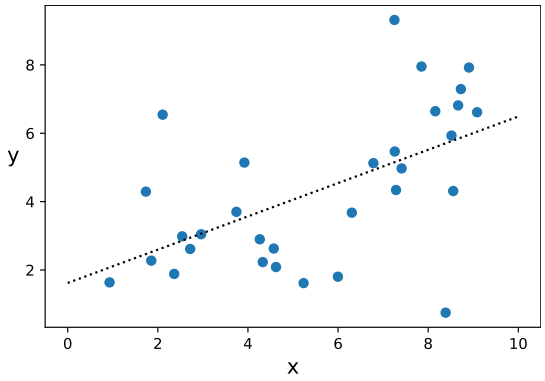
which we could alternatively write as

$$Y_i \sim \text{Normal}\big(a + bx_i, \sigma^2\big)$$

where σ is given, and a and b are parameters to be estimated.

the two ways of writing this model are equivalent: see properties of the Normal distribution, page 8, and IA Probability lecture 6

```
# get the data, and the known parameter σ
x = np.array([...])
y = np.array([...])
σ = ...

# define the likelihood, and maximize it
def loglik(y, x, θ):
    a,b = θ
    lik = scipy.stats.norm.pdf(y, loc=a+b*x, scale=σ)
    return np.log(lik)

initial_guess = [0,1]
â,b̂ = scipy.optimize.fmin(lambda θ: -np.sum(loglik(y,x,θ)),
                          initial_guess, maxiter=5000)

# Plot the line y = â + b̂x, and superimpose the dataset
xnew = numpy.linspace(-.5,1.5,100)
plt.plot(xnew, â+b̂*xnew, linestyle='dotted', color='black')
plt.scatter(x, y)
```



□

Alternatively, in this case, the equations are simple enough that we can solve them with maths rather than computation.

The likelihood function for Y_i is

$$\Pr_Y(y_i\,;\, x_i, a, b) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(y_i - a - bx_i)^2/2\sigma^2}$$

so the log likelihood of the dataset is

$$\begin{aligned}
\log \Pr(y_1, \ldots, y_n\,;\, x_1, \ldots, x_n, a, b) &= \sum_{i=1}^{n} \log \Pr_Y(y_i\,;\, x_i, a, b) \\
&= \sum_{i=1}^{n} \Big\{ -\frac{1}{2}\log(2\pi\sigma^2) - \frac{(y_i - a - bx_i)^2}{2\sigma^2} \Big\} \\
&= -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i - a - bx_i)^2.
\end{aligned}$$

To find the maximum likelihood estimator, differentiate with respect to a and b and find where the derivative is equal to zero. There are two parameters, so we have a pair of simultaneous equations to solve:

$$\frac{\partial}{\partial a}\log\text{lik} = \frac{1}{\sigma^2}\sum_{i=1}^{n}(y_i - a - bx_i) = 0$$
$$\frac{\partial}{\partial b}\log\text{lik} = \frac{1}{\sigma^2}\sum_{i=1}^{n}(y_i - a - bx_i)x_i = 0.$$

The solution is

$$\hat{b} = \frac{n\bar{x}\bar{y} - \sum_i x_i y_i}{n\bar{x}^2 - \sum_i x_i^2}, \qquad \hat{a} = \bar{y} - \hat{b}\bar{x}$$

where $\bar{x} = \sum_i x_i/n$ and $\bar{y} = \sum_i y_i/n$.
□

Exercise 1.7.2 (Binomial regression / supervised).
The UK Home Office makes available several datasets of police records, at `data.police.uk`. The stop-and-search dataset has been preprocessed to list the number of stops and the number of those that led to the police finding something suspicious, for each police force and each year.

police_force	year	stops	find
bedfordshire	2017	786	231
cambridgeshire	2016	1691	621
cambridgeshire	2017	581	264

Fit the model $Y_i \sim \text{Bin}(x_i, p)$ where Y_i is the number of 'find' incidents in a given police force and year, x_i is the number of stops, and p is the parameter to estimate.

The binomial distribution is a discrete random variable commonly used for counting the number of successes in a sequence of yes-no trials. If $X \sim \text{Bin}(n, p)$ then n is the number of trials, $0 \le p \le 1$ is the success probability, and the probability mass function is

Bin is for the Binomial random variable, page 9

$$\mathbb{P}(X = r) = \binom{n}{r} p^r (1-p)^{n-r}, \quad r \in \{0, \dots, n\}.$$

We'll assume the records in the dataset are independent, since we're not told otherwise. In maths notation,

$$\Pr(y_1, \dots, y_n \,;\, p) = \prod_{i=1}^{n} \binom{x_i}{y_i} p^{y_i}(1-p)^{x_i - y_i}.$$

(We've omitted the $x_1, \dots, x_n$ on the left hand side, because it's tedious to write it them out every single time.) The log likelihood is

$$\begin{aligned}\log\Pr(y_1, \dots, y_n \,;\, p) &= \sum_i \left(\log\binom{x_i}{y_i} + y_i \log p + (x_i - y_i)\log(1-p)\right) \\ &= \kappa + \left(\sum_i y_i\right)\log p + \left(\sum_i x_i - \sum_i y_i\right)\log(1-p)\end{aligned}$$

where κ is a constant i.e. doesn't depend on p. The maximum likelihood estimator for p solves

$$\frac{d}{dp}\log\text{lik}(p \mid y_1, \dots, y_n) = 0$$

and the solution is

$$\hat{p} = \frac{\sum_i y_i}{\sum_i x_i}.$$

□
This is not a very interesting model. The only slightly non-obvious thing it's told us is "don't estimate p separately for each police force and year, then average these estimates; instead estimate p from the whole aggregated data". The modelling exercise becomes much more interesting when we use it to investigate the influence of multiple covariates, e.g. how gender and race interact.

Example 1.7.3 (Fitting a Normal distribution / unsupervised).
Let the dataset be $x_1, \ldots, x_n$. Fit a Normal(μ, σ^2) distribution, where μ and σ are unknown.

The Normal distribution: page 10

If $X \sim \text{Normal}(\mu, \sigma^2)$ then X is a continuous random variable with likelihood

$$\Pr_X(x\,;\,\mu,\sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x-\mu)^2/2\sigma^2}$$

where $-\infty < \mu < \infty$ and $0 < \sigma < \infty$. The log likelihood function of a dataset $x_1, \ldots, x_n$ is

$$\begin{aligned}\log \Pr(x_1, \ldots, x_n\,;\,\mu,\sigma) &= \sum_{i=1}^{n} \log \Pr_X(x_i \mid \mu,\sigma) \\ &= \sum_i \Big\{ -\frac{1}{2}\log(2\pi\sigma^2) - \frac{(x_i-\mu)^2}{2\sigma^2} \Big\} \\ &= -\frac{n}{2}\log(2\pi) - n\log\sigma - \frac{\sum_i (x_i-\mu)^2}{2\sigma^2}.\end{aligned}$$

We saw in exercise 1.3.3 that the maximum is obtained at

$$\hat{\mu} = \frac{\sum_i x_i}{n}, \qquad \hat{\sigma} = \sqrt{\frac{1}{n}\sum_i (x_i - \hat{\mu})^2}.$$

□

Exercise 1.7.4 (Gaussian mixture model / unsupervised).
In a Gaussian mixture model, we suppose that each observation belongs to one of a certain number of classes, and we model each class as a Gaussian random variable with its own mean and variance. Earlier in these notes (exercise 1.1.3 page 6) we used a Gaussian mixture model for a dataset of galaxy speeds. The likelihood function for a Gaussian mixture model is

$$\Pr_X(x) = \sum_{k=1}^{m} p_k \Pr_{N(\mu_k, \sigma_k^2)}(x)$$

where m is the number of classes, p_k is the probability of class k, and μ_k and σ_k^2 are the mean and variance for class k. (We'll see later how to derive this.)

Fit a Gaussian mixture model with three components to the galaxy speed dataset.

calculating the likelihood function: section 5.3 page 95

It's easy to say what we want to achieve: we want to find parameters $p = (p_1, p_2, p_3)$, $\mu = (\mu_1, \mu_2, \mu_3)$, and $\sigma = (\sigma_1, \sigma_2, \sigma_3)$ to maximize the log likelihood of the dataset $(x_1, \ldots, x_n)$:

$$\log \Pr(x_1, \ldots, x_n) = \sum_{i=1}^{n} \log\Big[\sum_{k=1}^{3} p_k \Pr_{N(\mu_k, \sigma_k^2)}(x_i)\Big].$$

The constraints are $p_k \in [0,1]$, $p_1 + p_2 + p_3 = 1$, and $\sigma_k > 0$. This is not the sort of optimization where calculus will be of any use, so we'll use numerical optimization instead. First, as described in section 1.4, we'll rewrite in terms of unconstrained parameters. In this case, let's optimize over

$$\theta = (q_1, q_2,\ \mu_1, \mu_2, \mu_3,\ \tau_1, \tau_2, \tau_3) \in \mathbb{R}^8$$

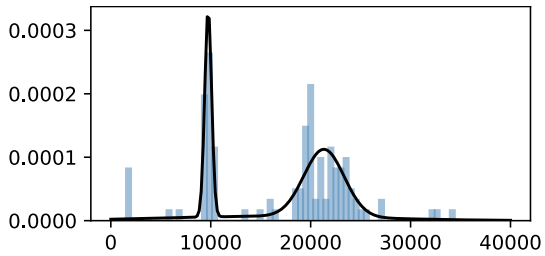
transformed into the parameters we want by

$$p_1 = \frac{e^{q_1}}{e^{q_1} + e^{q_2} + e^0}, \qquad p_2 = \frac{e^{q_2}}{e^{q_1} + e^{q_2} + e^0}, \qquad p_3 = \frac{e^0}{e^{q_1} + e^{q_2} + e^0}, \qquad \sigma_k = e^{\tau_k}.$$

We can now define the likelihood function, and maximize the log likelihood.

```
url = 'https://www.cl.cam.ac.uk/teaching/current/DataSci/data/galaxies_orig.csv'
galaxies = pandas.read_csv(url)
galaxies = galaxies['velocity'].values
```

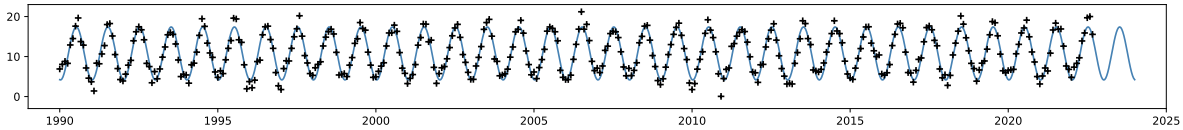
```
4
5  import scipy.stats
6  ϕ = scipy.stats.norm.pdf # likelihood for the Normal distribution
7
8  def transform_par(θ): # EXERCISE: rewrite so it works for any number of classes
9      q1,q2, μ1,μ2,μ3, τ1,τ2,τ3 = θ
10     p = np.exp([q1,q2,0])
11     p1,p2,p3 = p / np.sum(p)
12     σ1,σ2,σ3 = np.exp([τ1,τ2,τ3])
13     return (p1,p2,p3), (μ1,μ2,μ3), (σ1,σ2,σ3)
14
15 def logPr(x, θ): # EXERCISE: rewrite so it works for any number of classes
16     (p1,p2,p3), (μ1,μ2,μ3), (σ1,σ2,σ3) = transform_par(θ)
17     lik = p1*ϕ(x,loc=μ1,scale=σ1) + p2*ϕ(x,loc=μ2,scale=σ2) + p3*ϕ(x,loc=μ3,scale=σ3)
18     return np.log(lik)
19
20 # initial guess from looking at the plot on page 6
21 initial_guess = [0,0, 10000,20000,24000, math.log(1000),math.log(5000),math.log(8000)]
22 θ̂ = scipy.optimize.fmin(lambda θ: -np.sum(logPr(galaxies, θ)), initial_guess, maxiter=5000)
23
24 # Plot the fitted density, and a histogram of the actual data
25 # (Using density=True rescales the histogram to have total area
26 # equal to 1, making it directly comparable to the fitted density plot.)
27 fig,ax = plt.subplots(figsize=(6,3))
28 x = np.linspace(0,40000,200)
29 f = np.exp(logPr(x, θ̂))
30 ax.plot(x, f)
31 ax.hist(galaxies, bins = np.linspace(0,40000,80))
```



□

2. Feature spaces / linear regression

Here's a fitted model, for monthly average temperatures in Cambridge. The crosses mark the datapoints, and the grey line shows the model, which is a sinusoid whose amplitude, phase, and offset have been fitted to the data. When we see a plot like this, it's crying out for investigation: would the model fit better if we allowed for a long-term warming trend? should the trend be linear, or is warming accelerating? is the difference between winter and summer constant, or are the extremes getting more extreme?



There are so many questions we could ask, we need to be able to frame them in maths and turn them into code with barely a moment's thought. We need to spend our time thinking about the model, not messing around with compilers or optimizers.

This section introduces a class of models called *linear models*. Once we know a few standard tricks, described in section 2.2, it's easy to craft linear models to answer all sorts of modelling questions. They're also very fast to fit, using specialist routines derived from linear algebra. Linear models should be your go-to models for all sorts of data science and machine learning problems, the second thing you try (after simple tabulation!) to get a sense of the data you're working with.

It's possible to view linear models in a purely algorithmic way, called *least squares estimation*, without any probability at all. But there is also a probabilistic interpretation using Normal random variables. In fact, least squares estimation was invented by Gauss, as was the Normal distribution—another name for the Gaussian distribution. It's best to see linear models as just another example of probabilistic modelling, because least squares estimation only works for real-valued labels (like temperature) whereas the probability model can easily be extended to other types of data.

There is another more important reason for preferring the probabilistic interpretation: measuring how certain we should be in our answers. The climate data shown above shows a steady increase of 0.35378°C per decade—but how precisely do we know this? surely not to 5 decimal places! All the tools for quantifying certainty, which we'll study in Part III, are based on probability models.

2.1. Fitting a linear model

Linear modelling is a type of supervised learning. Suppose we're given a dataset $(x_1, y_1), \ldots, (x_n, y_n)$ where y_i is the label for record i and x_i is a tuple of predictor variables, called *features*,

$$x_i = \big[e_{1,i}, \ldots, e_{K,i}\big].$$

to use non-numerical predictors in a linear model, see the tricks in section 2.2

Linear modelling requires that all the variables in the dataset are numerical: $y_i \in \mathbb{R}$, and each $e_{k,i} \in \mathbb{R}$.

Linear model. A *linear model* is a model of the form

$$y_i \approx \beta_1 e_{1,i} + \cdots + \beta_K e_{K,i}$$

where $\beta_k \in \mathbb{R}$ is a parameter weighting the kth feature. It's convenient to write this in vector notation as

$$y \approx \beta_1 e_1 + \cdots + \beta_K e_K$$

where $y = [y_1, \ldots, y_n]$ is called the *response vector* and $e_k = [e_{k,1}, \ldots, e_{k,n}]$ is called a *feature vector*. The $\approx$ means that the model is not expected to be a perfect fit; we define the *residual vector* ε to be

$$\varepsilon = y - \big(\beta_1 e_1 + \cdots + \beta_K e_K\big).$$

Least squares estimation means picking the parameters β to minimize the *mean square error* $\sum_i \varepsilon_i^2/n$. Use sklearn.linear_model.LinearRegression() to do this, as described below. Once we have estimates for the parameters, call them $\hat{\beta}$, we can predict the response for a new case:

$$y_{\text{predicted}} = \hat{\beta}_1 e_{1,\text{new}} + \cdots + \hat{\beta}_K e_{K,\text{new}}.$$

Example 2.1.1 ().
The Iris dataset was collected by the botanist Edgar Anderson and popularized by Ronald Fisher[13] in 1936. The dataset consists of 50 samples from each of three species of iris, each with four measurements. The full dataset is https://www.cl.cam.ac.uk/teaching/current/DataSci/data/iris.csv.

Petal length	Petal width	Sepal length	Sepal width	species
1.0	0.2	4.6	3.6	setosa
5.0	1.9	6.3	2.5	virginica
5.8	1.6	7.2	3.0	virginica
1.7	0.5	5.1	3.3	setosa
4.2	1.2	5.7	3.0	versicolor
...				

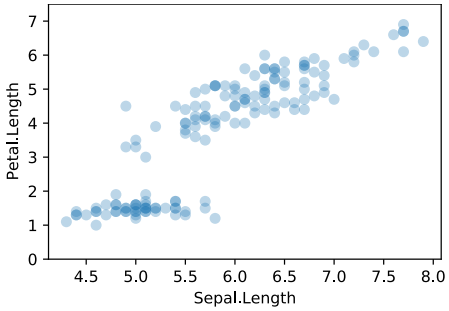
Let's investigate how petal length depends on sepal length. Here is a plot:

[13] It's tempting for computer scientists and mathematicians to think that data science is about algorithms and calculating with distributions and so on, but shared datasets are arguably more important. C.P. Scott, the former editor of *The Guardian*, said "Comment is free, but facts are sacred".

Modern advances in neural networks and deep learning were propelled by two shared datasets: the MNIST database of handwritten digits, and the ImageNet database of labelled photos. The story of ImageNet and of Fei-Fei Li, the researcher who collected it, is told in *The data that transformed AI research—and possibly the world*, https://qz.com/1034972/the-data-that-changed-the-direction-of-ai-research-and-possibly-the-world/.

In addition to shared datasets, it's also useful to have a shared challenge, what David Donoho calls a *common task framework*. See David Donoho. *50 years of Data Science*. Presentation at the Tukey centennial workshop. 2015. URL: http://courses.csail.mit.edu/18.337/2015/docs/50YearsDataScience.pdf

Ronald Fisher was described as a "genius who almost single-handedly created the foundations for modern statistical science." He was head of the Department of Eugenics at UCL, and later president of Gonville and Caius in Cambridge. The college has a stained glass window depicting his statistical work.



It suggests a curve. Let's fit a quadratic curve, using the linear model

$$\text{Petal.Length} \approx \alpha + \beta\,\text{Sepal.Length} + \gamma\,(\text{Sepal.Length})^2.$$

Linear does NOT mean 'straight line'. It refers to linear algebra—adding vectors, and multiplying vectors by scalars. In vector form, the model says

$$\begin{bmatrix}\text{Petal.Length}_1\\ \text{Petal.Length}_2\\ \vdots\end{bmatrix} \approx \alpha \begin{bmatrix}1\\1\\\vdots\end{bmatrix} + \beta \begin{bmatrix}\text{Sepal.Length}_1\\ \text{Sepal.Length}_2\\ \vdots\end{bmatrix} + \gamma \begin{bmatrix}(\text{Sepal.Length}_1)^2\\ (\text{Sepal.Length}_2)^2\\ \vdots\end{bmatrix}.$$

In scientific computing, the coding style is also in terms of vectors:

```
iris = pandas.read_csv('https://www.cl.cam.ac.uk/teaching/current/DataSci/data/iris.csv')

# A linear model with three feature vectors [one, x, x**2] and the response vector y
one, x = numpy.ones(len(iris)), iris['Sepal.Length']
y = iris['Petal.Length']
model = sklearn.linear_model.LinearRegression(fit_intercept=False)
model.fit(numpy.column_stack([one, x, x**2]), y)
(α,β,γ) = model.coef_
```

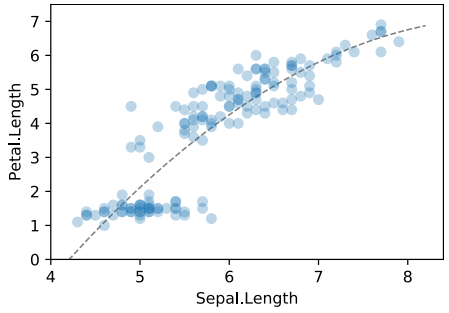
```
(-17.447,  5.392,  -0.296)
```

In fact the sklearn model fitting function always includes a one vector, unless we explicitly tell it otherwise with fit_intercept=False. Another way to write this code is

```
model2 = sklearn.linear_model.LinearRegression()
model2.fit(numpy.column_stack([x, x**2]), y)
α,(β,γ) = model2.intercept_, model2.coef_
```

What does this fit look like? We could explicitly evaluate $\alpha + \beta x + \gamma x^2$ for a range of x values and plot. Or use model.predict(), to relieve us from re-typing the model formula.

```
newx = numpy.linspace(4.2, 8.2, 20)
predy = model2.predict(numpy.column_stack([newx, newx**2]))

fig,ax = plt.subplots(figsize=(4.5,3))
ax.plot(newx, predy, color='0.5', zorder=-1, linewidth=1, linestyle='dashed')
ax.scatter(iris['Sepal.Length'], iris['Petal.Length'], alpha=.3)
ax.set_ylim(0,7.5)
ax.set_ylabel('Petal.Length')
ax.set_xlabel('Sepal.Length')
plt.show()
```



Terminology. We'd describe the quadratic linear model above as having two features, Sepal.Length, and (Sepal.Length)^2. The rows in this dataset have other attributes, and they can be transformed to create an infinite variety of features, but we'll only use the word *feature* for data attributes that are being used in a model. We call Petal.Length the *response* or *label* in this model, not a feature.

Why two features, and not one, or three? From the perspective of the person preparing the dataset, there is only one feature, Sepal.Length. From the perspective of the person computing α, β, and γ, there are two data features that have to be accounted for, and it's irrelevant that they came from the same column in the dataset. For the stickler for definitions, the definition of 'linear model' says that parameters are weighted by features, so there is really a third feature, the constant feature one with parameter α. Don't get uptight about defining the word 'feature', just write out your models explicitly, and there will be no confusion.

Notation warning. An equation like

$$\mathsf{Petal.Length} \approx \alpha + \beta\,\mathsf{Sepal.Length} + \gamma\,(\mathsf{Sepal.Length})^2.$$

can be read in two ways. One one hand it's a vector equation, where Petal.Length and Sepal.Length are vectors from the dataset. On the other hand it's a science-style equation, telling us the likely value of Petal.Length for a hypothetical new iris. When we use it for fitting, we're using it as a model for the dataset. When we use it to make predictions, we're using it as a model for the world.

* * *

The model is linear because it combines the unknown parameters α, β and γ in a linear formula. There's no reason to think this is in any way a 'true' model, and we could equally well have proposed a non-linear model e.g.

$$\mathsf{Petal.Length} \approx \alpha - \beta e^{-\gamma \mathsf{Sepal.Length}}.$$

Linear models are just easier to work with, so they're a better place to start.

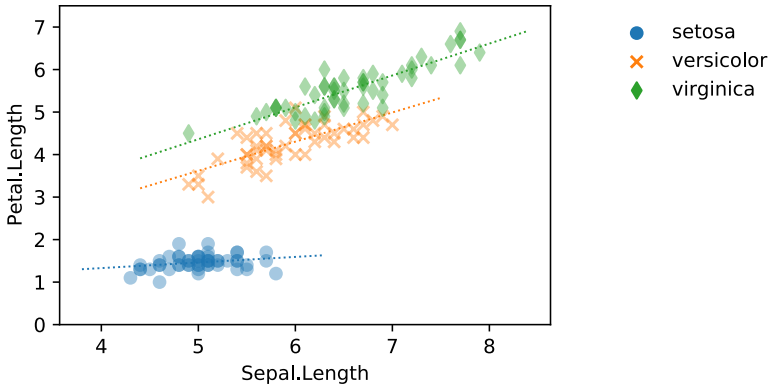
2.2. Feature design

Linear models let us spend our time thinking about what we're trying to model, not about the mechanics of making the model work …so long as we can design the right features to express what we want to model. Here is a gallery of cunning ways to use features to ask questions about a dataset.

2.2.1. ONE-HOT CODING

One-hot coding is used to turn an enum feature[14] (also called *categorical* or *factor*) into a collection of binary features, so it can be used in a linear model. Here's an example.

The Iris data is made up of three species. Maybe there's a straight-line fit between petal length and sepal length, but with different slopes and intercepts for each species.



One way to write this is

$$\text{Petal.Length} \approx \alpha_{\text{species}} + \beta_{\text{species}}\text{Sepal.Length}.$$

We can write this as a linear model. Since it has six parameters (an intercept α and a slope β for each of three species), the linear model must have six feature vectors, weighted by these six parameters. A moment's thought tells us that the feature vectors must be as follows:

$$\begin{matrix}\text{seto}\\ \text{virg}\\ \text{virg}\\ \text{seto}\\ \text{vers}\end{matrix}\begin{bmatrix}\text{PL}_1\\ \text{PL}_2\\ \text{PL}_3\\ \text{PL}_4\\ \text{PL}_5\\ \vdots\end{bmatrix} \approx \alpha_{\text{seto}}\begin{bmatrix}1\\0\\0\\1\\0\\\vdots\end{bmatrix} + \alpha_{\text{virg}}\begin{bmatrix}0\\1\\1\\0\\0\\\vdots\end{bmatrix} + \alpha_{\text{vers}}\begin{bmatrix}0\\0\\0\\0\\1\\\vdots\end{bmatrix} + \beta_{\text{seto}}\begin{bmatrix}\text{SL}_1\\0\\0\\\text{SL}_4\\0\\\vdots\end{bmatrix} + \beta_{\text{virg}}\begin{bmatrix}0\\\text{SL}_2\\\text{SL}_3\\0\\0\\\vdots\end{bmatrix} + \beta_{\text{vers}}\begin{bmatrix}0\\0\\0\\0\\\text{SL}_5\\\vdots\end{bmatrix}$$

Here's code to create these features:

```
1  species, SL, PL = iris['Species'], iris['Sepal.Length'], iris['Petal.Length']
2  species_levels = ['setosa','versicolor','virginica']
3  i1,i2,i3 = [np.where(species==k,1,0) for k in species_levels]
4  X = np.column_stack([i1, i2, i3, i1*SL, i2*SL, i3*SL])
5  model = sklearn.linear_model.LinearRegression(fit_intercept=False)
6  model.fit(X, PL)
```

The magic is line 3, which defines the binary vectors. They are called the *one-hot encoding* of the species vector. We'd write them in algebraic notation as for example $i_1 = 1[\text{species} = \text{"setosa"}]$ or $i_1 = 1_{\text{species}=\text{"setosa"}}$. Remember that species is a vector, and so i_1, i_2, i_3 are also vectors.

on line line:ewmult, i_1*SL is numpy notation for elementwise multiplication of two vectors

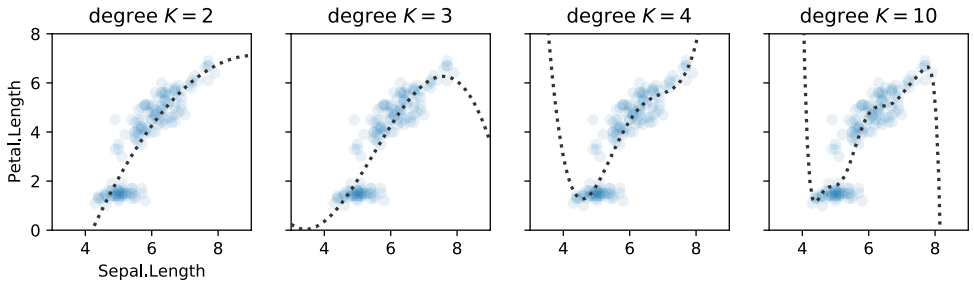
1_x also written $1[x]$ is the indicator function, $1_{\text{true}} = 1$ and $1_{\text{false}} = 0$

[14] More generally, we may wish to use rich objects like tweets or sentence fragments as predictors in a quantitative model. One-hot coding is a general purpose tool for turning enum features into numerical features, and for richer objects there will often be domain-specific tools. You'll study such quantitative models and the tools they need in Part II: distributional semantics which you will study in *Natural Language Processing*, and term frequency models for documents in *Information Retrieval*.

2.2.2. NON-LINEAR RESPONSE

We've already seen that we can use the quadratic feature $(\text{Sepal.Length})^2$ to capture smooth curves. Higher degree polynomials have more parameters to estimate, so they're more expressive and can fit the data better, but it's unwise to rely on them especially outside the range where we have data. In the Iris dataset,

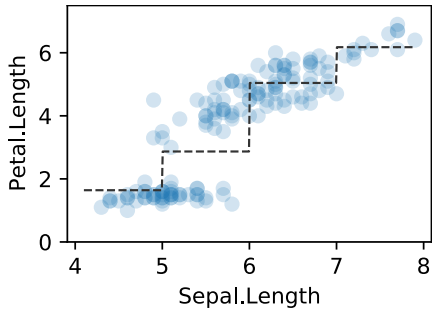
$$\text{Petal.Length} \approx \alpha + \beta_1 \,\text{Sepal.Length} + \beta_2 \,(\text{Sepal.Length})^2 + \cdots + \beta_K \,(\text{Sepal.Length})^K$$



A different approach is to use parameters for anchor points in an arbitrary curve. In this next model the arbitrary curve is a step function with fixed x-axis breaks, and least squares estimation finds the height at each step.

$\lfloor x \rfloor$ is x rounded down to the nearest integer

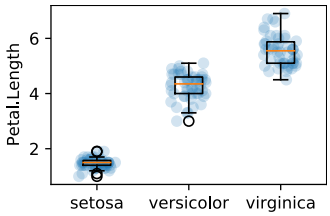
$$\text{Petal.Length} \approx \beta_4 \,1\big[\lfloor \text{Sepal.Length} \rfloor == 4\big] + \cdots + \beta_7 \,1\big[\lfloor \text{Sepal.Length} \rfloor == 7\big] \,.$$



This model is more honest than polynomials because it is upfront about being an arbitrary fit to the data, incapable of extrapolating outside the data range. This example isn't interesting (we could just as well have fitted each integer bin separately), but it's very useful when combined with other features. More guidance on curve fitting on page 39.

2.2.3. COMPARING GROUPS

We can also use one-hot coding to compare groups. Suppose we have a vector $x = [x_1, \ldots, x_m]$ from one group, and $y = [y_1, \ldots, y_n]$ from a second group, and we want to measure the difference between the groups. Obviously we could just compare the means, $\bar{y} - \bar{x}$, and there's nothing wrong with that! But it's good to see how to achieve the same thing via features.



For any sort of linear modelling question, a good starting point is to ask how we'd store the data in a spreadsheet or database table—identify the rows and the columns. Here we have two groups of measurements, so the natural way to store them is with a two-column spreadsheet, one column called group, the other called value. Now suppose we fit the linear model

$$\text{value} \approx \alpha + \beta 1[\text{group} == B].$$

The predicted value for items in group A is α, the predicted value for items in group B is $\alpha + \beta$, so β must be the difference between the two groups. (It will in fact be equal to $\bar{y} - \bar{x}$, but we need to linear algebra to prove that.)

linear algebra, section 2.5

group	value
A	x_1
$\vdots$	$\vdots$
A	x_m
B	y_1
$\vdots$	$\vdots$
B	y_n

2.2.4. PERIODIC PATTERNS

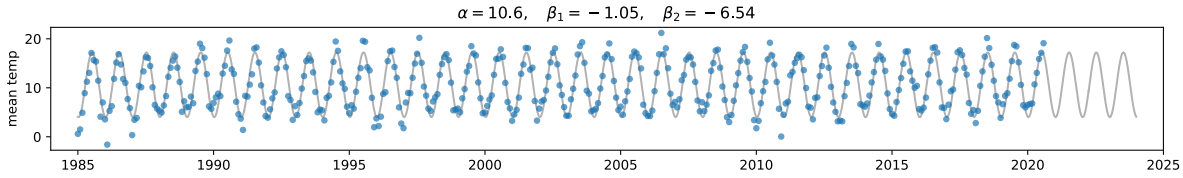
Example 2.2.1 ().
The UK Met Office makes available historic data[15] from 37 stations around the UK. Each station has monthly records for mean daily maximum temperature tmax, mean daily minimum temperature tmin, days of air frost af, total rainfall rain, and total sunshine duration sun. Coverage varies; the longest records are from Oxford and from Armagh, going back to 1853. A snapshot is available at `https://www.cl.cam.ac.uk/teaching/current/DataSci/data/climate.csv`.

month	tmax	tmin	af	rain	sun	station	lat	lng	alt_m
1963 Sep	14.7	5.9	0	126.4	127.7	Eskdalemuir	55.311	-3.206	242
1955 Aug	–	–	–	35.1	194.7	Shawbury	52.794	-2.663	72
1937 May	15.3	8.4	0	59.8	184.8	Lowestoft	52.483	1.727	18
2007 Aug	20.6	11.8	0	40.3	204.6	Waddington	53.175	-0.522	68
1925 July	21.8	12.6	0	23.2	–	Sheffield	53.381	-1.490	131
...									

The annual cycle makes it hard to compare the datapoints, for example to look for evidence of increasing temperatures. We could simply average over the 12 months of each year, and plot this average over time. This isn't ideal, because averaging is lossy i.e. we'd be throwing away data; and because a missing value for one month will cause the entire year to be missing. A cleverer solution is to use features to model the effects we're trying to capture. Let's consider the model

$$\text{temp} \approx \alpha + \beta \sin(2\pi \text{t} + \theta)$$

where t is the date in years, and α, β, and θ are unknown parameters. The plot below shows the data and the fitted model for Cambridge station (measured at the National Institute of Agricultural Botany, near the building for Artificial Intelligence and Environmental Risk). The plot shows the mean temperature temp $= (\text{tmin} + \text{tmax})/2$.



The model is linear in α and β and not in θ—but there is a cunning trick from A-level trigonometry that lets us rewrite it as a linear model. The trick is

$$\sin(A + B) = \sin A \cos B + \cos A \sin B$$

and so our model can be rewritten

$$\text{temp} \approx \alpha + \beta_1 \sin(2\pi \text{t}) + \beta_2 \cos(2\pi \text{t}).$$

```
climate = pandas.read_csv('https://www.cl.cam.ac.uk/teaching/current/DataSci/data/climate.csv')
df = climate.loc[(climate.station=='Cambridge') & (climate.yyyy>=1985)]
```

[15] https://www.metoffice.gov.uk/public/weather/climate-historic

```
t = df.yyyy + (df.mm-1)/12
temp = (df.tmin + df.tmax)/2

X = numpy.column_stack([numpy.sin(2*numpy.pi*t), numpy.cos(2*numpy.pi*t)])
model = sklearn.linear_model.LinearRegression()
model.fit(X, temp)
α,(β1,β2) = (model.intercept_, model.coef_)
```

2.2.5. SECULAR TREND

We might reasonably suspect that the periodic model

$$\text{temp} \approx \alpha + \beta_1 \sin(2\pi t) + \beta_2 \cos(2\pi t)$$

isn't a great fit to the data, because it doesn't allow for a systematic increase over time. Systematic changes over time are called "secular", as opposed to periodic patterns (which, like the divine, are eternal and unchanging). We can incorporate a secular trend with a $+\gamma t$ term:

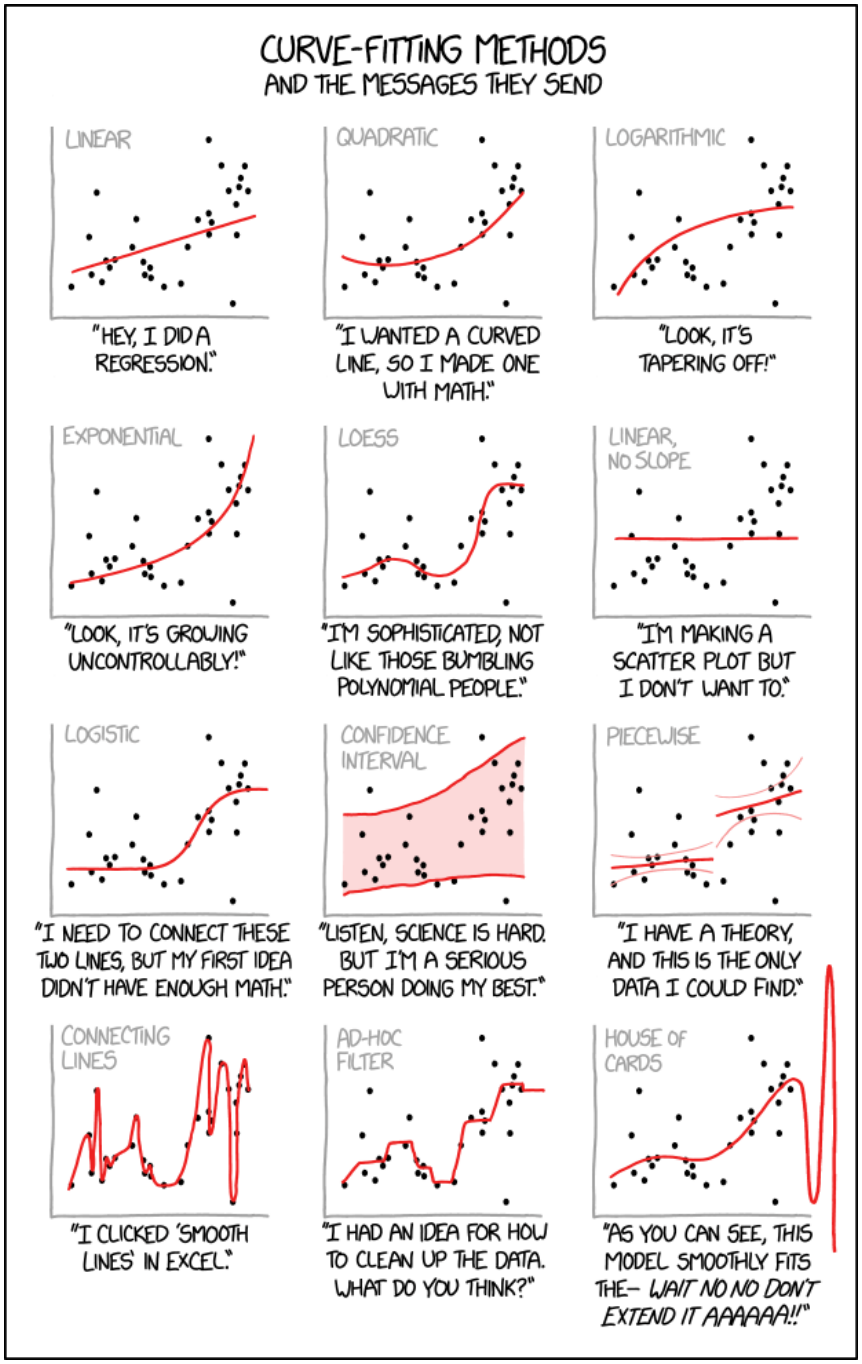
$$\text{temp} \approx \alpha + \beta_1 \sin(2\pi t) + \beta_2 \cos(2\pi t) + \gamma t$$

```
model = sklearn.linear_model.LinearRegression()
X = numpy.column_stack([numpy.sin(2*numpy.pi*t), numpy.cos(2*numpy.pi*t), t])
model.fit(X, temp)
α,(β1,β2,γ) = model.intercept_, model.coef_
```

```
(-60.458,  (-1.069, -6.5452, 0.0355))
```

Intercepts. In the purely periodic fit we got $\alpha = 10.6°\text{C}$, a reasonable figure for average annual temperatures, whereas in the model with a secular trend we get $\alpha = -60.458°\text{C}$. Why is this α so extreme? We'll return to this question in section 2.6.

xkcd by Randall Munroe, https://xkcd.com/2048/



2.3. Diagnosing a linear model

linear trend: section 2.2.5 page 38

Linear modelling makes it easy to read off summaries from a dataset, to answer questions we have asked. For example, if think there might be a linear trend in temperatures, we can just add a constant-slope feature and read off its coefficient.

Sometimes though we don't approach the dataset with a specific question in mind, and all we want is a model that fits the dataset well. Perhaps we want it for its predictions; perhaps we want it because we hope the terms in the model will give us ideas about what's behind our dataset. Then it's useful to know how to *discover* features. For example, if we lived in a bubble and didn't know anything about climate change, and someone handed us a CSV file with the climate dataset, how might we discover that it's worth adding a constant-slope term?

Discovering features. After fitting a model, compute the values that the model predicts, and use this to compute vector of residuals

$$\text{residual} = \text{observed} - \text{predicted}.$$

Plot this, to find out if we have missed any features worth incorporating. We should plot the residuals in every way we can, against any predictor variable we can think of, and if we see a systematic pattern then we should decide on a formula for it and add a term to our model, and then repeat.

Here's an example. For the climate data in section 2.2.4 we fitted the model

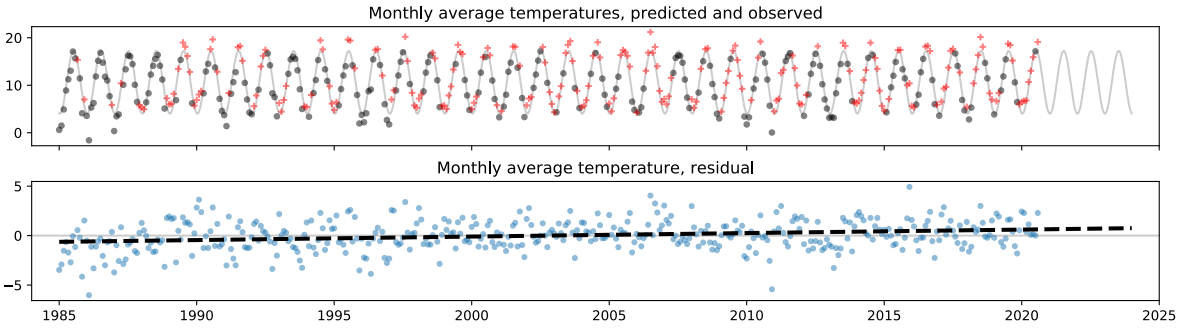
$$\mathsf{temp} \approx \alpha + \beta_1 \sin(2\pi \mathsf{t}) + \beta_2 \cos(2\pi \mathsf{t}).$$

Let the vector of residuals be

$$\varepsilon = \mathsf{temp} - \Big(\alpha + \beta_1 \sin(2\pi \mathsf{t}) + \beta_2 \cos(2\pi \mathsf{t})\Big).$$

Here are two plots depicting ε: the top plot shows the datapoints coded by whether $\varepsilon > 0$ or < 0, and the bottom plot shows ε directly together with a straight line fit. These plots show clearly that there's a systematic trend over time. If ε varies systematically with some feature, it's a sign that we could improve the model (make the model fit better) by incorporating that feature into the model, in this case

$$\mathsf{temp} \approx \alpha + \beta_1 \sin(2\pi \mathsf{t}) + \beta_2 \cos(2\pi \mathsf{t}) + \gamma t.$$




```
climate = pandas.read_csv('https://www.cl.cam.ac.uk/teaching/current/DataSci/data/climate.csv')
df = climate.loc[(climate.station=='Cambridge') & (climate.yyyy>=1985)]
t = df.yyyy + (df.mm - 1) / 12
temp = (df.tmin + df.tmax) / 2

# fit the model
X = numpy.column_stack([numpy.sin(2*numpy.pi*t), numpy.cos(2*numpy.pi*t)])
model = sklearn.linear_model.LinearRegression()
model.fit(X, temp)

```

```
# extract the residuals
pred = model.predict(X)
resid = temp - pred

with plt.rc_context({'figure.figsize': (15, 1.7*2.2), 'figure.subplot.hspace': 0.3}):
    fig,(ax1,ax2) = plt.subplots(nrows=2,ncols=1, sharex=True)

# Top plot: arrows and points
for (t1,pred1,resid1) in zip(t, pred, resid):
    ax1.arrow(t1, pred1, 0, resid1, alpha=0.5, color='blue' if resid1<=0 else 'red')
ax1.scatter(t, temp, s=15, alpha=0.5, c=numpy.where(resid<=0,'blue','red'))
# and a smooth line to show the fit
newt = numpy.linspace(1985, 2024, 1000)
newtemp = model.predict(numpy.column_stack([numpy.sin(2*numpy.pi*newt), numpy.cos(2*numpy.pi*newt)]))
ax1.plot(newt, newtemp, color='0.7', zorder=-1)

# Bottom plot: just the points
ax2.scatter(t, resid, s=15, alpha=0.5, c=numpy.where(resid<=0,'blue','red'))
ax2.axhline(0, color='0.6', zorder=-1)

ax1.set_xlim([1984, 2025])
ax1.set_title('Monthly average temperatures, predicted and observed')
ax2.set_title('Monthly average temperature, residual')
plt.show()
```

OVERFITTING

At this point, you're probably wondering: couldn't we just program a computer to keep on adding extra features? There are two problems with this, one pragmatic, one conceptual.

The pragmatic problem is that there can be exponentially many features to try. It may turn out that when we plot the residuals as a function of feature e then there's no systematic pattern, and when we plot them as a function of feature f there's no systematic pattern, but when we plot them as a function of e and f together then there is a pattern. To account for this, we'd need to augment the model by adding 'interaction features' that depend on both e and f.

(We've seen an example of this already in section 2.2.1 page 35. We could have used a per-species feature with three parameters, or we could have used a straight-line dependence on sepal length using two parameters, but we used the two together which took six parameters.)

So, for every predictor we have, we might want to explore how the residuals depend on that predictor crossed with any other predictor, and so on. The number of possible features grows exponentially in the number of predictor variables.

The conceptual reason not to add too many features is that it leads to bad models. We saw this before in section 2.2.2 page 36. When we add lots of polynomial terms we get a model which fits *the data that we have* very nicely, but which produces silly answers where we don't have data.

We'll return to overfitting in Part III of this course. For now, here's some general guidance. If there's a feature that you, as a data scientist, have reasonable grounds for believing might be relevant, then add it in. Limit yourself to what your background knowledge tells you is plausible. Then you're likely to be safe from overfitting.

2.4. Linear regression and least squares

tl;dr. A *linear regression* is a probabilistic model of the form

$$Y_i \sim \beta_1 e_{1,i} + \cdots + \beta_K e_{K,i} + \text{Normal}(0, \sigma^2)$$

where $e_1, \ldots, e_K$ are covariates, Y is the random response, and σ and $\beta_1, \ldots, \beta_K$ are unknown parameters. It is implicit that the Y_i are independent.

Fitting this model to a vector of observed values y is equivalent to least squares estimation for linear model

$$y = \beta_1 e_1 + \cdots + \beta_K e_K + \varepsilon.$$

Furthermore, the maximum likelihood estimator for σ is

$$\hat{\sigma} = \sqrt{\frac{1}{n}\sum_i \varepsilon_i^2}.$$

To demonstrate the link between linear regression and linear models, it's easier to work through an illustration rather than to write out abstract equations.

For the Iris dataset on page 32, we investigated how petal length depends on sepal length. Consider the linear regression model

$$\mathsf{Petal.Length}_i \sim \alpha + \beta\, \mathsf{Sepal.Length}_i + \gamma\, (\mathsf{Sepal.Length}_i)^2 + \text{Normal}(0, \sigma^2)$$

where $i \in \{1, \ldots, n\}$ indexes the rows of the dataset, and each $\mathsf{Petal.Length}_i$ is an independent random variable, and $\mathsf{Sepal.Length}_i$ is being treated as a covariate i.e. a non-random value. For brevity, let $Y_i = \mathsf{Petal.Length}_i$, let $e_i = \mathsf{Sepal.Length}_i$, and let $f_i = (\mathsf{Sepal.Length}_i)^2$, giving

$$Y_i \sim \alpha + \beta e_i + \gamma f_i + \text{Normal}(0, \sigma^2)$$

which (following the remark on page 8) can be rewritten

$$Y_i \sim \text{Normal}\big(\alpha + \beta e_i + \gamma f_i\,,\, \sigma^2\big).$$

Then the density function for a single observation y_i is

$$\Pr(y_i\,;\, \alpha, \beta, \gamma, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\left(y_i - (\alpha + \beta e_i + \gamma f_i)\right)^2 / 2\sigma^2}$$

and the log likelihood of the entire response vector y is

$$\log \Pr(y\,;\, \alpha, \beta, \gamma, \sigma) = -\frac{n}{2}\log\big(2\pi\sigma^2\big) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}\Big(y_i - (\alpha + \beta e_i + \gamma f_i)\Big)^2.$$

Let's find the maximum likelihood estimators for α, β, γ, and σ. We'll do this in two steps. The first step is to maximize the last term, i.e. find $\hat{\alpha}$, $\hat{\beta}$, and $\hat{\gamma}$ that solve

$$\min_{\alpha,\beta,\gamma} \big\|y - (\alpha 1 + \beta e + \gamma f)\big\|^2.$$

In this equation we have switched to vector notation, and 1 means the vector $[1, 1, \ldots, 1]$. This is nothing other than least squares estimation for the linear model

$$\mathsf{Petal.Length} \approx \alpha 1 + \beta \mathsf{Sepal.Length} + \gamma(\mathsf{Sepal.Length}^2).$$

The second step is to find σ to maximize what's left, i.e. to solve

$$\max_{\sigma > 0}\left\{-\frac{n}{2}\log\big(2\pi\sigma^2\big) - \frac{1}{2\sigma^2}\big\|y - (\hat{\alpha}1 + \hat{\beta}e + \hat{\gamma}f)\big\|^2\right\}.$$

This is a simple one-parameter optimization problem, once we know $\hat{\alpha}$, $\hat{\beta}$, and $\hat{\gamma}$, and the solution is

$$\hat{\sigma} = \sqrt{\frac{1}{n}\big\|y - (\hat{\alpha}1 + \hat{\beta}e + \hat{\gamma}f)\big\|^2}. \tag{1}$$

2.5. The geometry of linear models

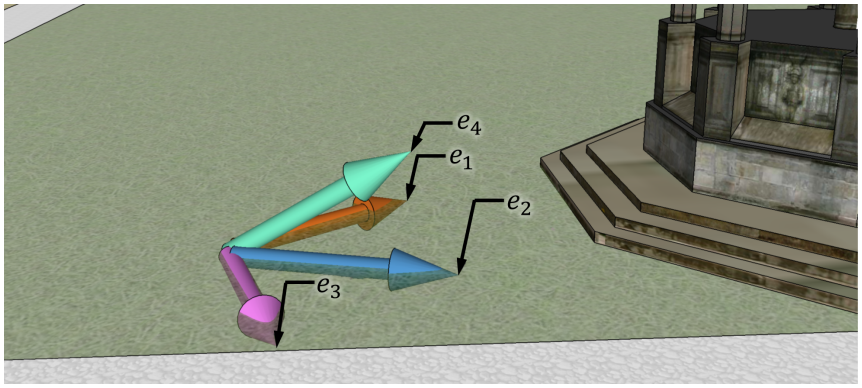
A linear model like the Iris model on page 33 is a vector equation,

$$\begin{bmatrix} \mathsf{Petal.Length}_1 \\ \mathsf{Petal.Length}_2 \\ \vdots \end{bmatrix} \approx \alpha \begin{bmatrix} 1 \\ 1 \\ \vdots \end{bmatrix} + \beta \begin{bmatrix} \mathsf{Sepal.Length}_1 \\ \mathsf{Sepal.Length}_2 \\ \vdots \end{bmatrix} + \gamma \begin{bmatrix} (\mathsf{Sepal.Length}_1)^2 \\ (\mathsf{Sepal.Length}_2)^2 \\ \vdots \end{bmatrix}.$$

In mathematics, vector equations like this come under the heading of *linear mathematics*. For data science all we need is vectors in simple Euclidean space, $\mathbb{R}^n$ where n is the number of records in the dataset—but the maths of linear algebra is abstract and can be applied to many other settings.[16] The appendix on the course website[17] presents the abstract maths.

For the purposes of this course, it's useful to study some linear algebra in order to understand how to interpret the parameters in a linear model. For some linear models, the parameter estimates we get out aren't reproducible—the fitting routine makes a seeminingly arbitrary choice—but somehow it doesn't affect our predictions. This is a symptom of so-called 'confounded features'. By studying linear algebra, especially the idea of linear independence, we'll be able to craft reliable models that tell us what we're hoping to learn.

SPANS AND LINEAR INDEPENDENCE



The *subspace spanned* by a collection of vectors $\{e_1, \ldots, e_K\}$, also called their *span*, is the set of all linear combinations:

$$\text{span} = \Big\{ \lambda_1 e_1 + \cdots + \lambda_K e_K \ : \ \lambda_k \in \mathbb{R} \ \text{ for all } k \Big\}.$$

A collection of vectors $\{e_1, \ldots, e_K\}$ is *linearly dependent* if there is some set of real numbers $(\lambda_1, \ldots, \lambda_K)$, not all equal to zero, such that

$$\lambda_1 e_1 + \cdots + \lambda_K e_K = 0.$$

If so, at least one of the e_k can be written as a linear combination of the others, i.e. it lies in the span of the others. Otherwise, they are *linearly independent*, and

$$\lambda_1 e_1 + \cdots + \lambda_K e_K = 0 \quad \Rightarrow \quad \lambda_1 = \cdots = \lambda_K = 0.$$

Computationally, to test if vectors are linearly independent, compute the rank of the matrix $[e_1, \ldots, e_K]$. If rank $= K$ they are linearly independent, otherwise rank $< K$ and they are linearly dependent.

```
np.linalg.matrix_rank(np.column_stack([e1,...,eK]))
```

[16] See Part II lecture courses on *Digital Signal Processing* and *Computer Vision* (Fourier transforms and wavelets, where vectors represent functions) and *Quantum Computing* (where vectors represent quantum states).

[17] https://www.cl.cam.ac.uk/teaching/current/DataSci/materials.html

Exercise 2.5.1. In the snapshot of Trinity College Great Court shown above, three of the arrows are embedded in the lawn and the fourth points slightly up. Are the arrows linearly independent? If not, find a linearly independent subset that spans the same space.

The three arrows $\{e_1, e_2, e_3\}$ embedded in the lawn are linearly dependent—it looks like $e_2 = e_1 + e_3$, i.e. $e_1 - e_2 + e_3 = 0$. We could discard any one of them, and we'd still be able to get anywhere on the lawn with some linear combination of the other two.

As for e_4, it sticks up in the air, so there's no way it can be made up out of the others. So, for example, $\{e_1, e_2, e_4\}$ is a linearly independent subset. The span of these three is $\mathbb{R}^3$, the same as the span of the original four arrows.
□

Exercise 2.5.2. Are the following five vectors linearly independent? If not, find a subset that is and that spans the same space.

$$\begin{aligned} e_1 &= [1,1,1,1] \\ e_2 &= [0,1,1,0] \\ e_3 &= [1,0,0,1] \\ e_4 &= [1,1,1,0] \\ e_5 &= [0,0,0,1] \end{aligned}$$

Just looking at these vectors, we can straight away see two linear relations:

$$\begin{aligned} e_2 + e_3 &= e_4 + e_5 \\ e_2 + e_3 &= e_1 \end{aligned}$$

We have a choice about what to discard. Let's discard e_5 (since $e_5 = e_2 + e_3 - e_4$, this won't reduce the span), and then discard e_3 (since $e_3 = e_1 - e_2$, this won't reduce the span) and we're left with $\{e_1, e_2, e_4\}$. Are these linearly independent? To check, suppose $\lambda_1 e_1 + \lambda_2 e_2 + \lambda_4 e_4 = 0$. Then

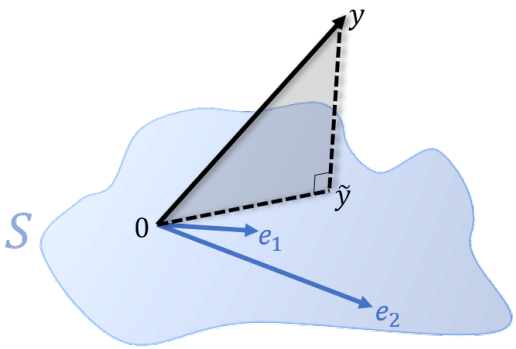
$$\lambda_1 \begin{bmatrix}1\\1\\1\\1\end{bmatrix} + \lambda_2 \begin{bmatrix}0\\1\\1\\0\end{bmatrix} + \lambda_4 \begin{bmatrix}1\\1\\1\\0\end{bmatrix} = \begin{bmatrix}0\\0\\0\\0\end{bmatrix}$$

$$\Rightarrow \quad \left.\begin{aligned} &\lambda_1 + \lambda_4 = 0 \\ &\lambda_1 + \lambda_2 + \lambda_4 = 0 \\ &\lambda_1 + \lambda_2 + \lambda_4 = 0 \\ &\lambda_1 = 0 \end{aligned}\right\}$$

$$\Rightarrow \quad \lambda_4 = 0, \quad \lambda_2 = 0.$$

So they are linearly independent.
□

PROJECTION AND LEAST SQUARES ESTIMATION



Suppose we have a subspace $\mathcal{S}$ spanned by $\{e_1, \ldots, e_K\}$, and some other vector y. Consider the question: how close can we get to y, while staying in $\mathcal{S}$? In Euclidean space, closeness is measured by Euclidean distance $\|y - s\| = \sqrt{\sum (y_i - s_i)^2}$, and so 'find the closest point' means solving

$$\min_{s \in \mathcal{S}} \|y - s\|.$$

Now let's interpret the diagram another way. Suppose we want to fit the linear model

$$y \approx \beta_1 e_1 + \cdots + \beta_K e_K.$$

What parameters should we choose? We've been using least squares estimation, which means picking the parameters so as to solve

$$\min_{\beta_1, \ldots, \beta_K} \sum_i \Big[y - (\beta_1 e_1 + \cdots + \beta_K e_K) \Big]^2.$$

These two optimizations are precisely the same thing! Searching over the span $\mathcal{S}$ *is* searching over the set of all linear combinations of feature vectors, since that's what the span is. In linear modelling, $\mathcal{S}$ is called the *feature space*.

Some terminology and facts. It can be shown that there is a unique closest point $\tilde{y} \in \mathcal{S}$. The closest point is called the *projection* of y onto $\mathcal{S}$. The residual $y - \tilde{y}$ is orthogonal to $\mathcal{S}$, i.e. $\varepsilon \cdot s = 0$ for all $s \in \mathcal{S}$.

Choosing a representation. There are two concepts that it's worth distinguishing: the feature space on one hand, and the feature vectors and parameters on the other. A given feature space might be representable in many ways, by choosing different sets of feature vectors. As long as they all give the same feature space, they must yield the same answer for the closest point $\tilde{y}$, since that depends only on $\mathcal{S}$. When we write $\tilde{y}$ as a linear combination of feature vectors,

$$\tilde{y} = \hat{\beta}_1 e_1 + \cdots + \hat{\beta}_K e_K$$

we will of course get different parameter estimates depending on our choice of feature vectors.

If the feature vectors are linearly independent, then there is a unique solution for the $\hat{\beta}_k$ coefficients. (If there were two solutions, we could subtract them and get a linear relation between the feature vectors, contradicting linear independence.) In linear modelling terms, we're guaranteed to get the same parameter estimates whenever we run least squares estimation.

If the feature vectors are linearly dependent, it's trickier. There may be multiple ways to write $\tilde{y}$ as a linear combination of the feature vectors, by using the linear relationship between them. In other words, the output of least squares estimation isn't fully determined. However, it can be shown that the model will always make the same *predictions*, whatever the parameter estimates it returns.

2.6. Interpreting parameters

A model with well-crafted features is like a beautiful equation in science, easier to understand and more subtle than endless tabulations. Choosing good features and parameters is one of the best tools we have for asking questions about a dataset.

Don't think of a model as a claim about what's *true*—any interesting dataset almost certainly has so much richness that any simple parametric model we invent is wrong—but a wrong model can still be useful. There are nevertheless *bad* models! If we pick bad features, we'll get useless parameter estimates.

Interpreting parameters. To interpret the parameters from a linear model,

1. 'Unwrap' the model by writing out the predicted response for representative datapoints. This helps us see what the parameters mean.
2. Write out the features and check if they're linearly dependent. If they are, the parameters have no intrinsic meaning; the parameters are said to be *non-identifiable*, and the features are said to be *confounded*.
3. If the features are confounded, choose a new set of features with the same feature space that isn't confounded.

If the feature vectors are confounded, then there is some feature vector that can be written in terms of the others (by definition of linear independence). We can simply discard this feature; doing so won't change the feature space. For example, if there are three features $\{e, f, g\}$ and $e = 0.2f - 0.5g$, then any linear combination

$$y = \alpha e + \beta f + \gamma g$$

can be rewritten

$$y = (\beta + 0.2\alpha)f + (\gamma - 0.5\alpha)g.$$

Thus the model with only two features $\{f, g\}$ can express anything that can be expressed with $\{e, f, g\}$.

A warning: sklearn.linear_model.LinearRegression always returns parameters for *some* linear combination, and in the non-identifiable case it will make an arbitrary choice. It will nonetheless make a valid choice—whatever parameters it chooses, the model will still make exactly the same predictions.

Example 2.6.1 ().
For the climate data on page 37, consider these two models:

$$\texttt{temp} \approx \alpha + \beta_1 \sin(2\pi \texttt{t}) + \beta_2 \cos(2\pi \texttt{t}) + \gamma \texttt{t} \qquad \text{(Model A)}$$
$$\texttt{temp} \approx \alpha' + \beta_1' \sin(2\pi \texttt{t}) + \beta_2' \cos(2\pi \texttt{t}) + \gamma'(\texttt{t} - 2000) \qquad \text{(Model B)}$$

For model A we get $\hat{\alpha} = -60.5$ and for model B we get $\hat{\alpha}' = 10.6$. Interpret these parameters.

To figure out what the parameters in a linear model mean, let's write out predictions for a representative datapoint, say $\texttt{t} = 0$ *(January in 1BC, there being no year 0AD on the calendar) or* $\texttt{t} = 2000$.

model	t	*predicted temperature*
model A	t=*0*	$\alpha + \beta_2$
model B	t=*0*	$\alpha' + \beta_2' - 2000\gamma'$
	t=*2000*	$\alpha' + \beta_2'$

So $\hat{\alpha}$ *is the predicted temperature in January 1BC (after removing the annual cycle), obtained by extrapolating a linear trend backwards from the present day. It's daft to believe such a wild extrapolation. Whereas* $\hat{\alpha}'$ *is the predicted temperature in January 2000, well within the region where we have data.*

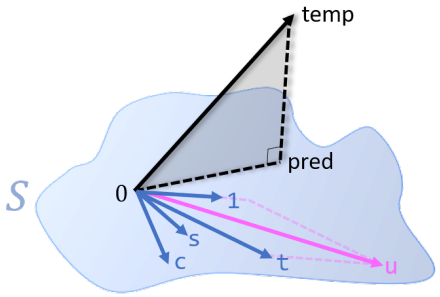
We can say something more about these two models. Let's list all the features:

$$\begin{aligned} 1 &= [1, 1, \dots] \\ t &\quad \textit{given in the dataset} \\ s &= \sin(2\pi \mathsf{t}) \\ c &= \cos(2\pi \mathsf{t}) \\ u &= \mathsf{t} - 2000 \times 1. \end{aligned}$$

The feature spaces of models A and B are identical, i.e.

$$\operatorname{span}\big(\{1, s, c, t\}\big) = \operatorname{span}\big(\{1, s, c, u\}\big).$$

Thus, when we fit the model (i.e. project temp *onto the feature space to get the model predictions), we'll obtain the same predictions in each case. The two models express exactly the same thing, they just use different coordinate systems to report their answers.*



□

Example 2.6.2 (Contrasts).
Suppose we have measurements from two groups, group A and group B, and we assemble them into a spreadsheet where each row contains a measurement, and where there are two columns, g containing the group and y containing the measurement. How would you interpret the coefficients from these linear models?

$$y \approx \alpha 1[g = A] + \beta 1[g = B] \qquad \text{(model M1)}$$
$$y \approx \alpha + \beta 1[g = B] \qquad \text{(model M2)}$$
$$y \approx \alpha + \beta 1[g = A] + \gamma 1[g = B] \qquad \text{(model M3)}$$

For model M1: the predicted response for an individual from group A is α, and the predicted response for an individual from group B is β. Saying this the other way round, α and β are the predicted responses of the two groups.

For model M2: the predicted response for an individual from group A is α, and the predicted response for an individual from group B is $\alpha + \beta$, so β is the difference between the two groups.

For model M3: the predicted response for an individual from group A is $\alpha + \beta$, and the predicted response for an individual from group B is $\alpha + \gamma$. This is a bit confusing. Let's write out some example feature vectors.

$$\begin{matrix} A \\ B \\ B \end{matrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \end{bmatrix} \approx \alpha \begin{bmatrix} 1 \\ 1 \\ 1 \\ \vdots \end{bmatrix} + \beta \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \end{bmatrix} + \gamma \begin{bmatrix} 0 \\ 1 \\ 1 \\ \vdots \end{bmatrix}$$

The three feature vectors are linearly dependent, because $1 = 1[g = A] + 1[g = B]$. Hence there is no unique way to write the projection of y onto the feature space as a linear combination of feature vectors. In other words, the parameters we end up with are arbitrary.
□

Exercise 2.6.3. Consider the simple straight-line linear regression

$$y \approx \alpha + \beta x$$

for the dataset $y = [5, 2, 1, 3]$, $x = [1, 2, 4, 5]$. Are the feature vectors linearly independent? If x or y were different, would your answer change?

The feature vectors are

$$\mathsf{one} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} \quad \textit{and} \quad \mathsf{x} = \begin{bmatrix} 1 \\ 2 \\ 4 \\ 5 \end{bmatrix}.$$

We can just look at these and see that there is no way to write one *in terms of* x*, or* x *in terms of* one*, so they are linearly independent. Alternatively, use Python to check the rank of the matrix* [one,x] *is 2:*

```
one = [1,1,1,1]
x = [1,2,4,5]
numpy.linalg.matrix_rank(numpy.column_stack([one,x]))
```

To test linear independence for an arbitrary $x = [x_1, \ldots, x_n]$, we have to ask if it's possible to solve

$$\alpha\,\mathsf{one} + \beta\,\mathsf{x} = 0$$

with non-zero coefficients. It's reasonably easy to see, in this case, that if $x_1 = \cdots = x_n$ then they are linearly dependent: either all the x_i are equal to zero in which case $(\alpha, \beta) = (0, 1)$ works, or they are nonzero in which case $(\alpha, \beta) = (x_1, 1)$ works. In other words, if all the x coordinates are equal, then we can't fit a straight line.

Alternatively, if we wanted to be formal about it, we'd write the vector equation as simultaneous equations,

$$\begin{aligned} \alpha + \beta x_1 &= 0 \\ \alpha + \beta x_2 &= 0 \\ &\vdots \end{aligned}$$

and solve them with pure algebra.

The question "are the feature vectors linearly independent?" is only about the feature vectors, so it doesn't depend on y, only on x.

□

Example 2.6.4 ().
The UK Home Office makes available several datasets of police records, at data.police.uk. The dataset police is a log of stop-and-search incidents, available as `https://www.cl.cam.ac.uk/teaching/current/DataSci/data/stop-and-search.csv`. Here is a sample of rows.

police force	operation	date-time object of search	lat	lng	gender outcome	age	ethnicity
Hampshire	NA	2014-07-31T23:20:00 controlled drugs	50.93	-1.38	Male nothing found	25–34	Asian
Hampshire	NA	2014-07-31T23:30:00 controlled drugs	50.91	-1.43	Male suspect summonsed	34+	White
Hampshire	NA	2014-07-31T23:45:00 controlled drugs	51.00	-1.49	Male nothing found	10–17	White
Hampshire	NA	2014-08-01T00:40:00 stolen goods	59.91	-1.40	Male nothing found	34+	White
Hampshire	NA	2014-08-01T02:05:00 article for use in theft	50.88	-1.32	Male nothing found	10–17	White

We wish to investigate whether there is racial bias in police decisions to stop-and-search. Consider the linear model

$$1[\mathsf{outcome} = \mathsf{find}] \approx \alpha + \beta_{\mathsf{eth}}$$

where eth is the vector of ethnicities. Write this model as a linear equation using one-hot coding. Are the parameters identifiable? If not, rewrite the model so that they are. Analyse whether this model suggests there is racial bias in policing actions.

Let's first understand how this model can describe racial bias. Write y_i for the response in record i, $y_i = 1$ if that record says the police found something, and $y_i = 0$ otherwise. The mean of y across a group is then

$$\text{mean response} = \frac{\#\text{stops where } y_i = 1}{\#\text{stops}} = \mathbb{P}(\text{police find something}).$$

Suppose for example we find that β_{Black} is low, compared to the other groups. This means that the predicted value of y for people with eth=Black is low, i.e. the probability that the police find something is low. Therefore the police must be stopping relatively more innocent people with eth=Black, which might be interpreted as saying that the police are biased against eth=Black people.

(It's a hack to treat a binary response as a real number with an implied Normal distribution. There is a more refined model called a logistic regression that uses a Bernoulli random variable for the binary response. But the hack can still give us interesting answers about the distribution of response, and it's easy!)

With one-hot coding, the model is

$$1[\mathsf{outcome{=}find}] \approx \alpha\,\mathsf{one} + \sum_{k \in ethnicities} \beta_k 1[\mathsf{eth} = k].$$

We want to know whether the parameters are identifiable, i.e. whether the feature vectors are linearly independent. We can test this by looking at the matrix rank. There are 6 features, but the matrix rank is only 5, therefore they're not linearly independent.

```
# It's a big file, so retrieve it and store locally for future use
if os.path.exists('stop-and-search.csv'):
    print("file already downloaded")
else:
    !wget "https://www.cl.cam.ac.uk/teaching/current/DataSci/data/stop-and-search.csv"
police = pandas.read_csv('stop-and-search.csv')

# Discard rows with missing ethnicity
ethnicity_levels = ['Asian','Black','Mixed','Other','White']
ok = police['officer_defined_ethnicity'].isin(ethnicity_levels)
eth = police.loc[ok, 'officer_defined_ethnicity']
```

```
12
13 # Assemble the feature matrix, one column per feature, and check its rank
14 eth_onehot = [(eth==i).astype(int) for i in ethnicity_levels]
15 X = numpy.column_stack([numpy.ones(len(eth))] + eth_onehot)
16 X.shape, numpy.linalg.matrix_rank(X)
```

```
((940998, 6), 5)
```

But this doesn't give us any insight into what's wrong with the model. For that, maths is better. First, note that the feature matrix has lots of duplicate rows, which (for the purposes of understanding identifiability) are redundant. In fact, there are only as many distinct rows as there are distinct ethnicity levels in the dataset, namely 5.

```
17 eth.value_counts()
```

```
White    532584
Black    253315
Asian    125646
Other     27809
Mixed      1644
```

So we're essentially only interested in the vectors

$$\underset{\text{one}}{\begin{bmatrix}1\\1\\1\\1\\1\end{bmatrix}} \quad \underset{1[\text{eth=Asian}]}{\begin{bmatrix}1\\0\\0\\0\\0\end{bmatrix}} \quad \underset{1[\text{eth=Black}]}{\begin{bmatrix}0\\1\\0\\0\\0\end{bmatrix}} \quad \underset{1[\text{eth=Mixed}]}{\begin{bmatrix}0\\0\\1\\0\\0\end{bmatrix}} \quad \underset{1[\text{eth=Other}]}{\begin{bmatrix}0\\0\\0\\1\\0\end{bmatrix}} \quad \underset{1[\text{eth=White}]}{\begin{bmatrix}0\\0\\0\\0\\1\end{bmatrix}}$$

Clearly the five one-hot coded vectors sum up to one*, so the vectors are linearly dependent, and we could remove any of them to end up with a linearly independent set. Let's remove* 1[eth=Asian]*, giving the model*

$$1[\text{outcome=find}] \approx \alpha\text{one} + \sum_{k \neq Asian} \beta_k 1[\text{eth} = k].$$

The parameters are identifiable, since the feature vectors are linearly independent. To interpret the parameters from this second model, let's consider some representative datapoints:

for a person with		*predicted y is*	
	eth=Asian		α
	eth=Black		$\alpha + \beta_{Black}$
	eth=Mixed		$\alpha + \beta_{Mixed}$
	eth=Other		$\alpha + \beta_{Other}$
	eth=White		$\alpha + \beta_{White}$

Thus, α is the predicted response for a person with eth=Asian*, and each β_k measures the difference in response between* eth=k *people and* eth=Asian *people. Let's fit this model.*

```
18 some_levels = [e for e in ethnicity_levels if e != 'Asian']
19 eth_onehot = [(eth==i).astype(int) for i in some_levels]
20 X = np.column_stack(eth_onehot)
21 y = np.where(police.loc[ok, 'outcome']=='find', 0, 1)
22
23 model = sklearn.linear_model.LinearRegression()
24 model.fit(X, y)
25 β = model.coef_
26 pandas.Series(β, index=some_levels)
```

```
Black   -0.021411
Mixed    0.133612
Other   -0.014554
White   -0.025912
```

This suggests that the police might be racially biased against eth=Black*,* eth=White*, and* eth=Other *people, relative to* eth=Asian *people (though this analysis doesn't tell us whether these estimates are just noise or whether they are significant—for that we need the tools of Part III).*
□

* * *

Data science is all about noise and uncertainty, whereas linear independence is a strict clean mathematical definition—so we shouldn't pay too much attention to it!

It may be that feature vectors are very close to linearly dependent but not quite, leading to parameters that are in theory identifiable but in practice not. For example, suppose the features are

$$f = [1, 1, \ldots, 1]$$
$$g = [1.00001, 1, \ldots, 1]$$

and we fit the model

$$y \approx \alpha f + \beta g.$$

Then α and β are identifiable. But when we compute confidence intervals (described in Part III), we'll likely find that we're very uncertain about α, very uncertain about β, and very certain about $\alpha + \beta$. In such a case, we'd also say (loosely) that the features are confounded.

3. Neural networks

> *In the mid-1980s two powerful new algorithms for fitting data became available: neural nets and decision trees. A new research community using these tools sprang up. Their goal was predictive accuracy. The community consisted of young computer scientists, physicists and engineers plus a few aging statisticians. They began using the new tools in working on complex prediction problems where it was obvious that data models were not applicable: speech recognition, image recognition, nonlinear time series prediction, handwriting recognition, prediction in financial markets.*
>
> Leo Breiman, *The Two Cultures*[19]

Any number of deep learning tutorials and blog posts will tell you all about prediction accuracy as the be-all and end-all of machine learning — but hardly any even mention probability or maximum likelihood estimation or any of the other tools of a classically trained statistician. Why?

I want to persuade you that neural networks should be thought of as probability models.

And that Leo Breiman is completely wrong when he says "it was obvious that data models were not applicable."

In simple supervised learning settings, we'll see that maximizing prediction accuracy *is the same thing* as maximizing likelihood, for a suitably chosen probability model. The probability modelling view is better — it's more flexible, and it uses intuitive models rather than folk wisdom and magic recipes for loss functions. But it does require a mental leap, and it doesn't give us many concrete new techniques, only a new spin on old techniques, and perhaps this is why it isn't more widely appreciated.

But it is in unsupervised learning that the elegant simplicity of probability modelling becomes clear. Here, the 'prediction accuracy' mindset has nothing to say — if I'm trying to build a neural network that can generate fake faces, for example, what would it even mean to test prediction accuracy, when there is no 'prediction'? But the likelihood maximization approach just works, as we'll see in section 3.4.

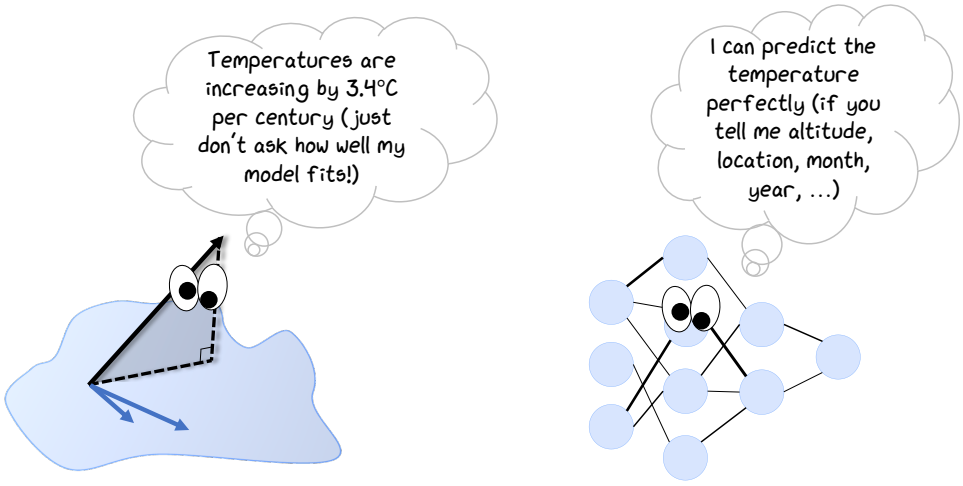
Neither the probabilistic interpretation nor the prediction accuracy mindset have anything to say about what should be *inside* the neural network. The nuts and bolts of a model — the activation functions, the number of layers, how neurons are wired together — depend entirely on the dataset that we want to fit the model to, and that is the proper place for folk wisdom. The *way* we model—the training mechanism, the evaluation metric, and so on—are what we're concerned with here.

We'll start by reviewing the 'prediction accuracy' approach to deep learning in section 3.1, and reverse engineer it to get to the corresponding probability models in section 3.2. In section 3.4 we'll look at a crude generative neural network. Better methods for generative modelling need more maths than we've covered so far, so they are left for later sections.

* * *

In section 2 on linear modelling, we hand-crafted probability models with a small number of parameters, and our aim was to interpret these parameters. In machine learning and in particular neural networks, the emphasis is on models with millions of parameters, and the aim is to build models that fit the data as well as possible. When should we look for interpretable parameters, and when should we look for a good fit?

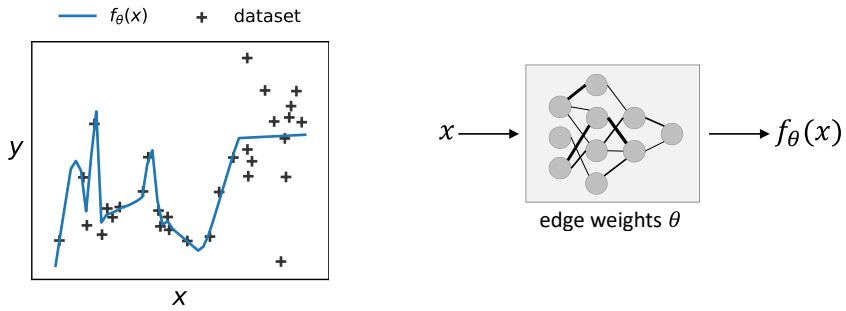
[19] Leo Breiman. "Statistical Modeling: The Two Cultures". In: *Statistical Science* (2001). URL: https://doi.org/10.1214/ss/1009213726.



In my opinion, this is a false dichotomy. Here's a thought experiment: suppose we have lots and lots of climate data, enough to build an amazingly well-fitting model, with highly accurate predictions; and suppose someone asks us "what's the average rate of temperature increase per century?" They're asking us for a *summary* of our richly detailed model—and it's a reasonable question to ask!

The features we designed in our linear models were really razor-sharp 'scalpels' for asking for very precise summaries. That's the real message of section 2.6. Unfortunately all the maths of linear modelling only works when we're asking questions about datasets—it would be great if we could ask similar questions about fitted models, e.g. neural networks, but the maths for this has not yet been developed.

3.1. Prediction accuracy



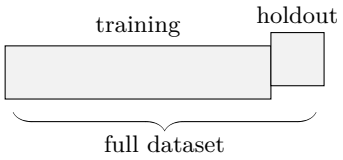
Suppose we have a labelled dataset $(x_1, y_1), \ldots, (x_n, y_n)$, where x_i is the input and y_i is the label for record i. And suppose our goal is to invent a function to predict the label, given the input. It doesn't matter at all what type of function we choose, but typically we choose a neural network with millions of edge-weight parameters. Write the prediction as $f_\theta(x)$, where θ is the set of parameters. 'Training the network' means choosing the parameters. A sensible way to measure how good the predictions are is by defining a *prediction loss function*,

$$L\big(\text{true label, prediction}\big) \qquad \text{where lower is better,}$$

(we're free to invent whatever we think is useful for our task), and then reporting the average prediction loss,

$$\text{loss} = \frac{1}{n}\sum_{i=1}^{n} L\big(y_i,\ f_\theta(x_i)\big).$$

The obvious way to train the neural network is to find θ to minimize this loss. We can do this easily by computer, using a numerical optimization routine called gradient descent, so long as the loss $L(y,\ f_\theta(x))$ is a differentiable function of θ.

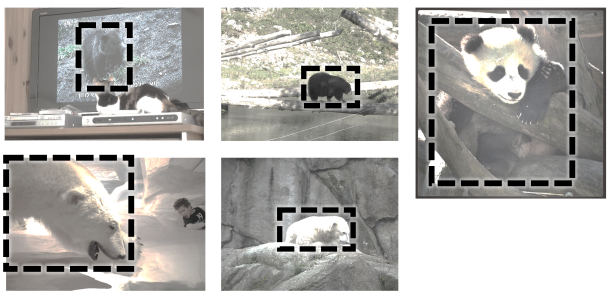


We don't want the neural network to cheat by just memorizing the dataset we give it, so we split the dataset into two parts. One part, the training data, we use to choose the θ parameters. The other part, the holdout data, we set aside until after training, and then we evaluate our neural network by measuring the loss function on the holdout data.

Evaluating models on a holdout dataset is a fantastic idea; it opens the door to many further questions, discussed in section 10

The real cleverness is in transforming an underspecified question such as "how can I identify an object in a photo?" into a prediction task with true labels, predicted labels, and a differentiable loss function. Here are some examples.

Example 3.1.1 (Regression / object detection).
The COCO dataset[20] consists of colour images of various sizes, each annotated with what it contains. The annotations contain, among other things, labels and bounding boxes. Here for example are some images that have one bear, with the bounding box shown.



One of the competitive tasks associated with this dataset is object detection — given an image, predict the coordinates of the bounding box.

In this example, the labels are values in $\mathbb{R}^4$: the coordinates of the box's center, and its width and height. A reasonable prediction loss function is squared Euclidean distance in $\mathbb{R}^4$,

$$L(label, pred) = \|label - pred\|^2.$$

In other words, our overall average loss is

$$loss = \frac{1}{n}\sum_{i=1}^{n} \|y_i - \hat{y}_i\|^2, \qquad \hat{y}_i = f_\theta(x_i).$$

This is differentiable with respect to θ, as long as $f_\theta(x)$ is made out of the standard neural network building blocks.
□

Example 3.1.2 (Image classification).
The MNIST dataset[21] consists of hand-written digits stored as 28×28 greyscale images, annotated with the digit they represent. Let $x_i \in \mathbb{R}^{28\times 28}$ be the ith image, and let $y_i \in \{0, 1, \ldots, 9\}$ be its label. The goal is to predict the label, given the image.

When there's a finite set of values that the labels can take, it's common to split the prediction into two steps. First let $f(x_i\,;\,\theta)$ output a vector of 'confidence values',

$$\vec{f}(x_i\,;\,\theta) = \big[f_1(x_i;\theta), \ldots, f_K(x_i;\theta)\big]$$

where K is the number of classes (writing $\vec{f}$ to remind ourselves that it returns a vector, and writing $\vec{f}(x_i\,;\,\theta)$ rather than $\vec{f}_\theta(x_i)$ so we can use the subscript for classes.) Then, we simply take as our prediction the class with the highest confidence. All the cleverness is wrapped up inside the function $\vec{f}(x\,;\,\theta)$. This can be as complicated a function as we like, and θ could consist of millions of unknown parameters. We'd like to make it expressive enough so that, once the best parameters have been chosen, $\vec{f}(x\,;\,\theta)$ puts a large positive weight on the best guess for image x and a large negative weight on all the others.

Let the prediction loss function just count whether or not our prediction was correct:

$$L(label,\, pred) = 1_{label\neq pred}.$$

Our overall loss can then be written as

$$loss = \frac{1}{n}\sum_{i=1}^{n} 1_{y_i \neq \arg\max_k f_k(x_i;\theta)}.$$

This loss isn't differentiable, because arg max *isn't differentiable, so we can't use it for training with gradient descent. Instead, we pluck out of thin air a differentiable loss function*

$$training\ loss = \frac{1}{n}\sum_{i=1}^{n}\left(-\sum_{k=1}^{K} 1_{y_i=k}\log p_{ik}\right), \qquad p_{ik} = \frac{e^{f_k(x_i;\theta)}}{\sum_{\ell=1}^{K} e^{f_\ell(x_i;\theta)}}$$

and cross our fingers and hope that the 'real' loss will be low when we evaluate on the holdout set.
□

This training loss function is called 'softmax cross-entropy loss with one-hot coding'. It's a waste of time trying to justify why we should use this particular loss function and not some other: the only real justification comes from ditching the 'prediction accuracy' mindset altogether and switching to the probabilistic interpretation. Nevertheless, here's some time

[20] Dataset at `https://cocodataset.org/`, described in Tsung-Yi Lin et al. "Microsoft COCO: Common Objects in Context". In: *European Conference on Computer Vision* (2014)

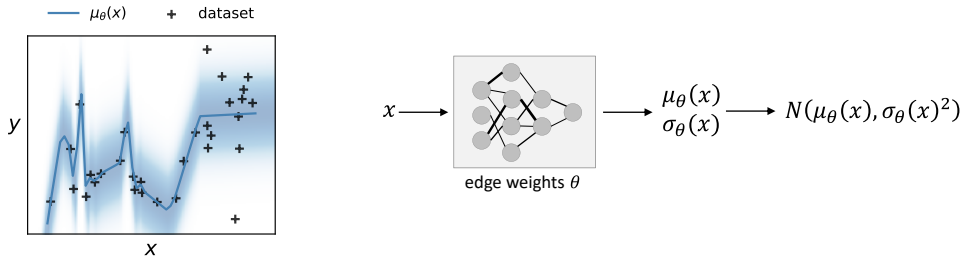
[21] Yann LeCun, Corinna Cortes, and Christopher J. C. Burges. *The MNIST Database of handwritten digits.* 1998. URL: `http://yann.lecun.com/exdb/mnist/`

wasting. This loss function has a contribution $-\log p_{iy_i}$ from record i, so to make the loss small we want p_{iy_i} to be large. To make this large, we want $f_k(x_i\,;\,\theta)$ to be larger for $k = y_i$ than for $k \neq y_i$. This makes $y_i = \arg\max_k f_k(x_i\,;\,\theta)$. In conclusion, making this training loss small should have the effect of making the 'real' loss function small.

Incidentally, the training loss function as we've written it here has a term $\sum_k 1_{y_i=k} \log p_{ik}$. This is just an algebraic trick to say "only keep the p_{ik} term where $k = y_i$", in other words it's just $\log p_{iy_i}$. When we're aiming for fast code, a lookup like p_{iy_i} isn't helpful whereas a matrix multiplication like $\sum_k 1_{y_i=k} \log p_{ij}$ is better.

3.2. Probabilistic deep learning

As per the usual supervised learning setup, suppose we have a labelled dataset $(x_1, y_1), \ldots, (x_n, y_n)$, where x_i is the input and y_i is the label for record i. The probabilistic modelling approach is to say **the labels are noisy, so let's model their distribution**. To be precise, we'll fit a parametric probability distribution $\Pr_Y(y_i\,;\, f_\theta(x_i))$ whose parameters are computed by a neural network f_θ. For example, we could use the Normal distribution, and have the neural network output a mean and a standard deviation.



The obvious way to fit a probability model is by maximum likelihood estimation. So that's what we can use for training the neural network: pick the network's parameters θ to maximize the log likelihood of the dataset. For simple probability models, we'll see that this boils down in the end to minimizing prediction loss, for a suitable loss function. Another way of saying this is: we can use all the conventional tools for training neural networks, but now we have a principled way to pick sensible loss functions.

It's good machine learning practice to split the dataset into a training set and a holdout set, and use the former for training and the latter for evaluation. Exactly the same is true for probabilistic deep learning. The metric for evaluation should again be log likelihood (this time, log likelihood of the holdout dataset), since log likelihood measures how well a probability distribution fits the dataset.

log likelihood as a measure of goodness-of-fit: section 4.3

	predictive accuracy	probability model (supervised)
data	$\{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$	.....
labels	$y_1, y_2, \ldots, y_n$	.....
task	predict the label $y_i \approx f_\theta(x_i)$	fit a probability model $\Pr_Y(y_i\,;\, f_\theta(x_i))$
training goal	invent a loss function and learn θ to minimize the prediction loss $\sum_i L(y_i,\, f_\theta(x_i))$	learn θ to maximize the log likelihood of the dataset $\sum_i \log \Pr_Y(y_i\,;\, f_\theta(x_i))$
evaluation	prediction loss on holdout dataset	log likelihood of holdout dataset

Example 3.2.1 (Regression with Gaussian errors).
Consider a regression model with Gaussian errors

$$Y_i \sim f_\theta(x_i) + N(0, \sigma^2)$$

where Y_i is a random variable modelling the label, x_i includes all relevant features, and f is a neural network with parameters θ. Suppose we've observed values $y_1, \ldots, y_n$ for the labels. Describe how to find maximum likelihood estimates $\hat{\theta}$ and $\hat{\sigma}$.

The log likelihood of the entire dataset is

$$\log \Pr(y_1, \ldots, y_n\,;\, \theta, \sigma) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}\big(y_i - f_\theta(x_i)\big)^2.$$

We want to maximize this over θ and σ. We can do the maximization over θ first: for this we need to solve

This is exactly Gauss's calculation from section 2.4

$$\textit{minimize} \quad \sum_{i=1}^{n}\big(y_i - f_\theta(x_i)\big)^2 \quad \textit{over } \theta$$

which is the same thing as training the neural network to minimize the average loss for the prediction loss function

$$L(\mathit{label}, \mathit{pred}) = (\mathit{label} - \mathit{pred})^2.$$

It's then simple calculus to find the maximum likelihood estimate for the noise parameter $\hat{\sigma}$*, and the answer is*

$$\hat{\sigma} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\big(y_i - f_\theta(x_i)\big)^2}$$

though if we're using numerical optimization to find θ *we might as well ask it to find* σ *at the same time!*
□

Example 3.2.2 (Classification).
In classification, the labels come from a finite set, call it $\{1, \dots, K\}$. Consider a probability model based on a Categorical random variable,

$$Y_i \sim \mathrm{Cat}\big(\vec{p}(x_i; \theta)\big).$$

Here $\vec{p}(x\,;\,\theta)$ is some arbitrary function that takes the predictor variables x, as well as parameters θ, and returns a probability vector

$$\vec{p}(x_i; \theta) = \big[p_1(x_i; \theta), \dots, p_K(x_i; \theta)\big]$$

and our model says $\mathbb{P}(Y_i = k) = p_k(x_i\ \theta)$. Suppose we've observed values $y_1, \dots, y_n$ for the labels. Describe how to find the maximum likelihood estimator for θ.

When trying to maximize the likelihood, it's awkward to maximize over a constrained domain—here we'd have to ensure we only pick θ *that results in a valid probability vector i.e. each probability* ≥ 0 *and all summing to one. It's easier to transform to an unconstrained space, using the softmax transform from section 1.4 page 19. Thus, let's seek a different function* $\vec{f}(x\,;\,\theta) \in \mathbb{R}^K$*, and then let*

$$p_k(x\,;\,\theta) = \frac{e^{f_k(x;\theta)}}{e^{f_1(x;\theta)} + \cdots + e^{f_K(x;\theta)}} \quad \text{for } k \in \{1, \dots, K\}.$$

Then we're free to choose any θ *at all, and we're guaranteed to end up with a probability vector* $\vec{p}$*.*

To train this model we follow the standard maximum likelihood procedure: write out the log likelihood, and maximize it. The likelihood of an individual observation y *is*

$$\Pr_Y(y\,;\,x, \theta) = p_y(x\,;\,\theta)$$

So the log likelihood of the whole dataset is

$$\log \Pr(y_1, \dots, y_n\,;\,\theta) = \sum_{i=1}^{n} \log p_{y_i}(x_i\,;\,\theta) = \sum_{i=1}^{n}\sum_{k=1}^{K} 1_{y_i=k} \log p_k(x_i\,;\,\theta).$$

We want to pick θ *to maximize this. Equivalently, sticking a minus sign in front and scaling by* n*, we want to pick* θ *to minimize*

$$\text{loss} = \frac{1}{n}\sum_{i=1}^{n} \text{crossentropy}\Big(\text{onehot}(y_i), \text{softmax}\big(\vec{f}(x_i\,;\,\theta)\big)\Big)$$

where

$$\begin{aligned}
\text{onehot}(y) &= \big[1_{y=1}, \dots, 1_{y=K}\big] \\
\text{softmax}(\vec{f}) &= \Big[\frac{e^{f_1}}{e^{f_1} + \cdots + e^{f_K}}, \dots, \frac{e^{f_K}}{e^{f_1} + \cdots + e^{f_K}}\Big] \\
\text{crossentropy}(\vec{q}, \vec{p}) &= -\big(q_1 \log p_1 + \cdots + q_K \log p_K\big).
\end{aligned}$$

□

This is known as "minimizing softmax cross-entropy loss with onehot coding". It's a sesquipedalian way of saying "I'm fitting the simplest possible probability model in the simplest possible way". It's the simplest most general model possible for a random variable with discrete outcomes because it just says "let the probability of outcome k be some arbitrary p_k". It's the bog standard way of estimating parameters, maximum likelihood estimation. And to make the optimization easy for a computer we're using the simplest possible transform, called softmax.

Interestingly, the decomposition into crossentropy and onehot and softmax makes it clear that the real prediction task here isn't "predict the label y_i", it's "predict the one-hot coded version of y_i".

* ⁕ *

The 'prediction accuracy' mindset doesn't involve any explicit probability modelling. Someone who doesn't believe in probability theory could perfectly well formulate a task as a problem of minimizing prediction loss; they might even claim that deep learning is entirely about prediction and loss functions, and doesn't need any modelling at all.

Moreover, the probability interpretation takes some mental gymnastics. The probabilist can look at a picture of a cat and say "The label for this picture is a random variable, and it takes value 'cat' with probability 92%", when any normal person would say "What do you mean, random? it's a cat, for goodness sake!"

In the author's opinion, the probabilistic interpretation is much better than the 'prediction accuracy' mindset. The real benefits will only become clear when we look at generative neural networks. But there are also benefits for supervised learning, especially if we want to model noise and uncertainty.

For example, in the context of regression, we could train a neural network model to output uncertainty in the form of a probability model $Y \sim N(\mu_\theta(x), \sigma_\theta(x)^2)$. Here $\sigma_\theta(x)$ measures how much noise the network thinks there is around x: if the training data is very noisy then training will push for a large $\sigma_\theta(x)$. The 'prediction accuracy' mindset doesn't have the tools to even begin to think about this.

The probabilistic interpretation can also help us avoid some traps of interpretation, particularly when we're trying to reason about confidence. See the "adversarial panda" on page 133 for an illustration of the kind of mistake it's easy to make. Tools for measuring confidence, which we'll study in Part III, are all based on probability models, and without a probabilistic view of neural networks we will not be able to extract confidence measures. (Confidence measures for neural networks are, however, still a topic of active research.)

3.3. Numerical optimization with PyTorch

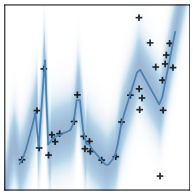
> *Gradient descent can write code better than you. I'm sorry.*
>
> *Software 1.0 is code we write. Software 2.0 is code written by the optimization based on an evaluation criterion (such as "classify this training data correctly"). It's likely that any setting where the program is not obvious but one can repeatedly evaluate the performance of it (e.g.—did you classify some images correctly? do you win games of Go?) will be subject to this transition, because the optimization can find much better code than what a human can write.*
>
> Andrej Karpathy[22]

PyTorch is a library for numerical optimization. It's oriented around the types of optimization problems that arise in neural networks—the library routines are set up to make these problems easy to implement—but it can be used for any optimization at all.

This section will show some typical PyTorch optimization code and explain its conventions, so that you can make sense of the PyTorch code snippets used in this book. For information about how PyTorch works under the hood, or for guidelines about the software engineering aspects (working with large datasets, GPUs, clusters) you should look elsewhere.

3.3.1. SPECIFYING A MODEL

To see how we specify probability models in PyTorch, here's an illustration: we'll fit a wiggle plus noise to a collection of points $(x_1, y_1), \ldots, (x_n, y_n)$ in $\mathbb{R}^2$. We'll use the model



$$Y_i \sim \text{Normal}\big(f_\theta(x_i), \sigma^2\big)$$

where $f_\theta(\cdot)$ is some simple neural network, and where both θ and σ are to be learnt.

```
1  import torch
2  import torch.optim as optim
3  import torch.nn as nn
4
5  class Wiggle(nn.Module):
6      def __init__(self):
7          super().__init__()
8          self.f = nn.Sequential(nn.Linear(1,4), nn.LeakyReLu(), ...)
9      def forward(self, x):
10         return self.f(x)
11
12 class RWiggle(nn.Module):
13     def __init__(self):
14         super().__init__()
15         self.μ = Wiggle()
16         self.σ = nn.Parameter(torch.tensor(1.0))
17     def forward(self, y, x):
18         σ = self.σ
19         return -0.5*torch.log(2*np.pi*σ²) - torch.pow(y - self.μ(x), 2) / (2*σ²)
```

Modules. The class nn.Module represents a function, typically a parameterized function. In this code there are two functions, $f_\theta(x)$ implemented in class Wiggle and $\text{Pr}_Y(y\,;\,\theta,\sigma)$ implemented in class RWiggle. A module is a regular Python class, so we're free to define whatever methods we like, but it needs to have a forward method that actually evaluates the function. When we call a module object directly, as in

```
rwiggle = RWiggle()
x,y = ...
pred = rwiggle.f(x)
loglik = rwiggle(y, x)
```

[22] Andrej Karpathy was a founding member of OpenAI. The first line is a tweet from August 2017 (karpathy), and the rest a blog post from November 2017; https://twitter.com/karpathy/status/893576281375219712 and https://karpathy.medium.com/software-2-0-a64152b37c35. Note that software 2.0 still needs a human to decide out what to optimize! I hope that this book persuades you that "log likelihood of the training dataset" is a sensible optimization goal.

then it invokes forward. For probability models, a sensible convention is to define the forward function to compute the log likelihood.

This code could just as well have been written as a single module, but it's useful to see how PyTorch lets us split a big problem into smaller reusable building blocks.

One of the building blocks that PyTorch provides is nn.Sequential, line 8, which lets us specify a pipeline of other modules applied one after the other. Exactly which modules you use will depend on the dataset you're working with;[23] this is the art of modelling, not a matter of general purpose principles, and you should be guided by common practice and folk wisdom.

Parameters. Any parameters that we'll want to optimize over, we declare in the module's __init__ with a nn.Parameter wrapper. The wrapper lets PyTorch keep track of parameters, so that when we set up the optimization loop on line 28 it's easy to specify what parameters need to be optimized. It also makes it easy to save all a model's parameters to a file, or to read them back:

```
# save parameters
torch.save(mymodel.state_dict(), 'filename.pt')

# load parameters
mymodel = MyModel()
mymodel.load_state_dict(torch.load('filename.pt'))
```

On line 15, RWiggle uses a Wiggle object, which tells PyTorch that RWiggle depends not just on its own parameter σ but also on the parameters from Wiggle, which in turn depends on all the parameters in the modules listed inside nn.Sequential on line 8. (We can also define whatever other fields we like in __init__, and if they don't have the Parameter wrapper then PyTorch will leave them alone.)

Tensors. Anything numerical we do in PyTorch, we do on torch.tensor objects. These are similar to numpy arrays. There are tensor methods that mimic many of the numpy routines: torch.zeros, torch.log, etc. Bothersomely, numpy and PyTorch methods often have different names and different arguments, so you'll need to spend a lot of time chasing the reference documentation!

PyTorch tensor operations have dual implementations, both a CPU implementation and a fast GPU implementation. In order for your code to be able to run happily on a GPU, all the operations in the functions you're optimizing should be PyTorch functions: torch.log, etc. Exactly how to make your code run on the GPU is outside the scope of this course, but there are plenty of tutorials online.

Batched functions. There are many built-in modules in torch.nn. They nearly all have the convention that they operate on batches of records, where each item in the batch is to be operated on separately. It's a good idea to follow this convention. So, for example, if we want to implement a function that operates on 2d matrices, we actually make it operate on 3d arrays where the first dimension indexes the items in the batch. In this code, Wiggle and RWiggle follow this convention: mathematically we think of them as applying to scalars, but the code insists on vectors (i.e. tensors with a single dimension) and it will act separately on each item in the vector. This is the reason for the reshaping on lines 23–24.

```
x = torch.tensor([[3.0], [4.0]]) # batch of length 2, each item a vector of length 1
y = torch.tensor([[1.0], [1.5]])
rwiggle(y, x)
```

[23] For the wiggle model, I used this:nn.Sequential(nn.Linear(1,4), nn.LeakyReLU(), nn.Linear(4,20), nn.LeakyReLU(), nn.Linear(20,20), nn.LeakyReLU(), nn.Linear(20,1)).

3.3.2. ITERATIVE OPTIMIZATION

For many optimization problems in machine learning, we iterate towards a solution, and there aren't hard and fast stopping criteria. So I like to run my optimizations interactively, inspecting the output as the optimization runs and perhaps jumping in and tweaking the settings part way through. The construction on line 30 lets me do this: I use the menu item Kernel | Interrupt, it interrupts cleanly, then I can resume the optimization afterwards. (The code for the Interruptable class is given on the next page.)

```
20 url = 'https://www.cl.cam.ac.uk/teaching/current/DataSci/data/xkcd.csv'
21 xkcd = pandas.read_csv(url)
22
23 x = torch.tensor(xkcd.x, dtype=torch.float)[:,None]
24 y = torch.tensor(xkcd.y, dtype=torch.float)[:,None]
25 rwiggle = RWiggle()
26 epoch = 0
27
28 optimizer = optim.Adam(rwiggle.parameters())
29
30 with Interruptable() as check_interrupted:
31     while True:
32         check_interrupted()
33         optimizer.zero_grad()
34         loglik = torch.mean(rwiggle(y, x))
35         (-loglik).backward() # since PyTorch optimizers find min, and we want max
36         optimizer.step()
37         epoch += 1
38         if epoch % 200 == 0:
39             print(f'epoch={epoch} loglik={loglik.item():.4} σ={rwiggle.σ.item():.4}')
```

Optimization algorithm. PyTorch comes with several iterative optimization algorithms, all of them variations on gradient descent. This code selects the Adam optimizer on line 28, and tells it to optimize over all the parameters in the model. If you find yourself needing to meddle with the optimizer then you're in the domain of engineering, not modelling, and that's outside the scope of this book.

PyTorch's numerical optimization looks for a local minimum, not global. It might get stuck in a no-good local minimum, so it's a good idea to run the optimization several times, from different randomly-chosen initial parameters. The standard modules in torch.nn initialize their parameters randomly.

The computation graph. Anything numerical we do in PyTorch, we do on torch.tensor objects. These are similar to numpy arrays. But tensors are richer than numpy arrays, because they allow PyTorch to keep track of the *computation graph* of which operations were performed on which tensors. It uses this graph for its gradient descent calculations. You don't need to know anything about computation graphs for basic PyTorch use, but you do need to know about permitted operations ...

- All the operations in the functions you're optimizing should be PyTorch functions: torch.log, etc. You're not allowed to take values out of torch tensors, feed them into some other package, get the answers, and put them back into your module. If you did that, PyTorch wouldn't know how to apply gradient descent.
- If you want to inspect a value e.g. to plot it, use x.detach().numpy(), or if the value is a scalar use x.item() as on line 39. This extracts the values from the PyTorch universe, and it will no longer be able to keep track of the computation graph for the operations you're performing. If you do want to do computations with PyTorch tensors rather than numpy, wrap them in with torch.no_grad() to tell PyTorch that this is 'dead end' code and it's not to build a computation graph.
- If you want to reset the value of a parameter, use x.data=.... Don't do it inside optimization loop between optimizer.zero_grad() and optimizer.step(), since that might mess up the computation graph.

Here's some magic Python code to support clean interrupting.

```
import signal
class Interruptable():
    class Breakout(Exception):
        pass
    def __init__(self):
        self.interrupted = False
        self.orig_handler = None
    def __enter__(self):
        self.orig_handler = signal.getsignal(signal.SIGINT)
        signal.signal(signal.SIGINT, self.handle)
        return self.check
    def __exit__(self, exc_type, exc_val, exc_tb):
        signal.signal(signal.SIGINT, self.orig_handler)
        if exc_type == Interruptable.Breakout:
            print(' stopped')
            return True
        return False
    def handle(self, signal, frame):
        if self.interrupted:
            self.orig_handler(signal, frame)
        print('Interrupting ...', end='')
        self.interrupted = True
    def check(self):
        if self.interrupted:
            raise Interruptable.Breakout
```

3.3.3. FULL EXAMPLE: MNIST DIGIT CLASSIFICATION

Example 3.3.1 (MNIST digit classification).
Consider the MNIST image dataset described on page 58, consisting of pairs (x_i, y_i) where $x_i \in \mathbb{R}^{28\times 28}$ is an image and $y_i \in \{0, 1, \ldots, 9\}$ is its label. Consider the general probability model described on page 61, in which $\vec{f} : x \mapsto \mathbb{R}^{10}$ is some neural network that outputs a vector of scores, a score for each possible digit, and we model the label as

$$Y_i \sim \mathrm{Cat}\big(\mathrm{softmax}(\vec{f}(x_i))\big).$$

Training the neural network consists in maximizing the log likelihood of the dataset,

$$\log \Pr(y_1, \ldots, y_n) = \sum_{i=1}^{n} \log\big[\mathrm{softmax}\big(\vec{f}(x_i)\big)\big]_{y_i}.$$

The dataset can be loaded with the following code, which returns a list $[(x_1, y_1), \ldots, (x_n, y_n)]$.

```
mnist = torchvision.datasets.MNIST(
    root = 'pytorch-data/',     # where to download the dataset to
    download = True,    # if files aren't here, download them
    train = True,       # whether to import the test or the train subset
    transform = torchvision.transforms.ToTensor() # convert x_i and y_i to torch.tensor
)
```

There are several things to look out for in the code below. They are all part of the craft of neural network architecture, orthogonal to probabilistic modelling concerns, so we won't do more than just mention them briefly.

- The nn.Sequential module on line 4 applies a succession of functions, one after the other. Each of the functions in the list is a module, and most have their own parameters. The modules used here have been found useful for image processing; see specialist texts about image processing to find out what they do.
- Rather than trying to optimize the entire dataset, on line 27 we're taking batches of 5 images at a time and updating the neural network weights based on the batch.
- It's been found that randomized dropout, lines 9 and 13, tends to result in better results on holdout data. (Section 10.2 explains why). The dropout module has slightly different behaviours depending on whether we're training or just taking readouts. So we need to tell our model whether it should be in training mode or not, lines 29 and 48.

```
1  class MyImageClassifier(nn.Module):
2      def __init__(self):
3          super().__init__()
4          self.f = nn.Sequential(        # input.shape [B × 1 × 28 × 28]
5              nn.Conv2d(1, 32, 3, 1),    # → [B × 32 × 26 × 26]
6              nn.ReLU(),
7              nn.Conv2d(32, 64, 3, 1),   # → [B × 64 × 24 × 24]
8              nn.MaxPool2d(2),           # → [B × 64 × 12 × 12]
9              nn.Dropout2d(0.25),
10             nn.Flatten(1),             # → [B × 9216]
11             nn.Linear(9216, 128),      # → [B × 128]
12             nn.ReLU(),
13             nn.Dropout2d(0.5),
14             nn.Linear(128, 10)         # → [B × 10]
15         )
16     # compute log likelihood for a batch of data
17     def forward(self, y, x):  # x.shape [B × 1 × 28 × 28], y.shape and output.shape [B]
18         return - nn.functional.cross_entropy(self.f(x), y, reduction='none')
19
20     # compute the class probabilities for a single image
21     def classify(self, x):             # input.shape [1 × 28 × 28]
22         q = self.f(torch.as_tensor(x)[None,...])[0]
23         return nn.functional.softmax(q, dim=0)    # output.shape [10]
24
25 model = MyImageClassifier()
```

```
26 epoch = 0
27 mnist_batched = torch.utils.data.DataLoader(mnist, batch_size=5)
28
29 model.train(mode=True)
30 optimizer = optim.Adam(model.parameters())
31
32 with Interruptable() as check_interrupted:
33     while True:
34         for batch_num, (imgs,lbls) in enumerate(mnist_batched):
35             check_interrupted()
36             optimizer.zero_grad()
37             loglik = model(lbls, imgs)
38             e = -torch.mean(loglik)
39             e.backward()
40             optimizer.step()
41
42             if batch_num % 25 == 0:
43                 IPython.display.clear_output(wait=True)
44                 print(f'epoch={epoch} batch={batch_num}/{len(mnist_batched)} loglik={-e.item()}')
45         epoch += 1
46
47 # Apply the trained network to classify a single image
48 model.train(mode=False)
49 img,_ = mnist[110]
50 model.classify(img)
```

For clean interrupting and resuming, I set up the model and a data-cycler in one cell of my notebook:

```
mymodel = ...
iter_mnist = enumerate_cycle(mnist_batched, shuffle=False)
```

and then I run the optimization in the next cell:

```
optimizer = optim.Adam(mymodel.parameters())
with Interruptable() as check_interrupted:
    for (epoch,batch_num),(imgs,lbls) in iter_mnist:
        check_interrupted()
        ... # optimization step
```

This way I can interrupt, inspect the state, and resume the optimization by rerunning the optimization cell, and it will pick up exactly where it left off. By contrast, the code on page 68 lines 32–45 will lose track of where it was in the dataset when it got interrupted. Here's the code for the data-cycler:

```
def enumerate_cycle(g):
    epoch = 0
    while True:
        for i,j in enumerate(np.random.permutation(len(g))):
            yield (epoch,i), g[j]
        epoch = epoch + 1
```

4. The goal of model-fitting

> *All models are wrong but some are useful [...] there is no need to ask the question "Is the model true?" If "truth" is to be the "whole truth" the answer must be "No". The only question of interest is "Is the model illuminating and useful?"*
>
> George Box[27]

> *One of the biggest traps for smart engineers is optimizing a thing that shouldn't exist.*
>
> Elon Musk

This book has promoted maximum likelihood estimation as the be-all and end-all of model fitting, suitable for everything from simple models to neural networks. But it's not the only approach. There are three standard ways to think about model fitting:

- the goal should be to get good estimates of the parameters in a model
- the goal should be to find a model that makes good predictions
- **the goal should be to find a model that fits i.e. describes the data**

In section 4 I will describe why I think the first two are unhelpful, why I think the third is best, and why maximum likelihood estimation is the natural way to achieve it.

The parameter-estimation mindset is this: I have a dataset, I have proposed a model for the data, and now I want to get good estimates for its parameters. You may have come across ideas such as 'unbiasedness' and 'low variance' as ways to specify what we mean by good estimates, and we'll discuss them further in section 4.5.

I think this is conceptually and practically misguided. The conceptual problem is that, as the famous statistician George Box noted, "all models are wrong". For example, if we've measured the heights of a group of students and we fit a simple Normal(μ, σ^2) model, we don't actually believe there's some fundamental law of nature that says "student heights are sampled from a Normal distribution". (That would be crazy, since a Normal distribution can take negative values!) We're just using the model to approximate and describe the data. Since no model is ever anything more than an approximation, it's futile to seek an unbiased estimator, given that the definiton of unbiasedness requires a correct model.

Models are in our head, they're not part of nature. I might have in my head Normal(μ, σ^2) and you might have in your head Normal(μ, τ), exactly the same distribution just with different parameterization; but if we're after unbiased estimates of these parameters we'll have to use different maths.[28] Model parameters are the underwear of modelling—good for personal support, but not decent to show off in public.

The practical problem with the parameter estimation mindset is that it doesn't work well in the age of neural networks. Neural networks hammer in the point that models are approximations, not Truths of Nature, in two ways. First, these models have millions of parameters, and no one is interested in the values of each individual parameter—it's how they work together that matters. Second, it's arbitrary whether one neuron or another ends up accomplishing a given task—formally, the parameters are unidentifiable. The rise of neural networks has thus come with a shift away from the parameter estimation mindset, and towards prediction accuracy.

The prediction-accuracy mindset is this: the job of a model is to make predictions, so we should judge models on the accuracy of their predictions. Innumerable machine learning tutorials and blog posts teach this way of thinking, and innumerable publications have tedious tabulations of assorted prediction-accuracy metrics such as R^2 and F_1.

I think this is dishonest and restrictive. It's dishonest because all these accuracy metrics are only there to accommodate noisy data (we don't expect a model's predictions to be 100% accurate, so we need some way to express how much inaccuracy we'll tolerate), but they try

[27] G. E. P. Box. "Robustness in the Strategy of Scientific Model Building". In: *Robustness in Statistics*. Vol. 1. May 1979, p. 40. URL: https://archive.org/details/DTIC_ADA070213.

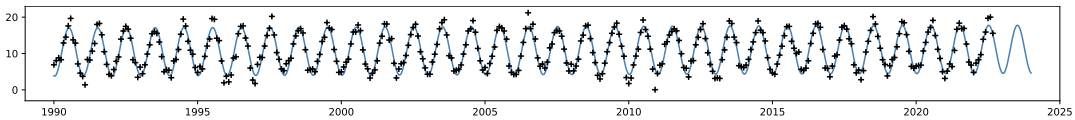
[28] If $X_1, \ldots, X_n$ are independent Normal(μ, τ) then an unbiased estimator for τ is $T = (n-1)^{-1} \sum_{i=1}^{n} (X_i - \bar{X})^2$ where $\bar{X}$ is the mean of the sample. If they are independent Normal(μ, σ^2) then an unbiased estimator for σ is $\sqrt{T}/c_4(n)$ where $c_4(n)$ has a horrible formula that can be found on Wikipedia, https://en.wikipedia.org/wiki/Unbiased_estimation_of_standard_deviation.

to do so without saying what the noise *is*. For example, when someone quotes us the R^2 score for their model, they're telling us a metric that only makes sense under the assumption of Gaussian errors (often without realizing that they're making this assumption!), and they don't tell us their estimate for the standard deviation. It's more honest to be up front and explicit what the noise model is. In section 4.4 we'll discuss prediction-accuracy metrics further.

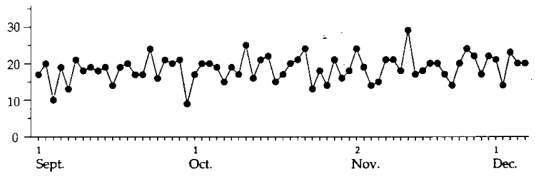
The prediction-accuracy mindset is restrictive because it only applies to models that make predictions. The prediction setup is that we have a labelled dataset $(x_1, y_1), \ldots, (x_n, y_n)$, where x_i is the input and y_i is the ground-truth label for record i, and we want to learn to predict the label given the input. But unsupervised generative modelling is at least as important. Here we have an unlabelled dataset $x_1, \ldots, x_n$, and our goal is to be able to generate new values that look like they might have come from this dataset. This is *not* a prediction task, and so the prediction-accuracy mindset simply doesn't apply. The probability-model-fitting mindset, by contrast, applies just as cleanly to generative tasks as to prediction tasks.

Predictive and generative modelling compared, sections 1.6 and 1.7. Generative neural networks, sections 3.4 and following.

The model-fitting mindset says: a model should describe the dataset in full—not just make predictions, but also describe how much variation to expect. Our very first example, the climate model from page 2, makes the point that the actual datapoints don't lie *on* the line of best fit, and so the only adequate way to account for the dataset is by modelling the variation as well.



In business-speak, the saying is "it is impossible to be properly data driven if you do not fully understand variation". Donald Wheeler[29] tells the story of the president of a shoe company with a big chart on the wall of his office, titled "Daily Percentage of Defective Pairs."



One day an astute statistician was in this office, and asked the president why he had this chart on his wall. He condescendingly replied that it was so he could tell how the plant was doing. The statistician immediately asked "How's it doing?"—and evidently no one had had the temerity to ask the president this question before, because he paused, looked at the chart, and then said simply "Well, some days are better than others!" Wheeler's point is that it's impossible to recognize when there are exceptional changes to the business process unless one knows the range of natural variability.

Section 4.1 discusses the model-fitting mindset further, and argues why likelihood is a natural way to measure fit.

[29] Donald Wheeler. *Understanding variation: the key to managing chaos.* Spc Pr, 2000, chapter 2

4.1. Likelihood measures model fit

"Model fit" means "how well a model describes the full dataset, including all the variation". We need a way to measure it. The examples in this section will I hope persuade you that **the likelihood that a model gives to a dataset** is a natural way to measure that model's fit.

This is a such a simple idea that there's surely something wrong with it! It holds a peculiar position: one one hand it's the basis of almost all successful tools in machine learning, from maximum likelihood estimation to neural network training, and on the other hand it's often not recognized, and when it is recognized it's argued against.

Maximum likelihood estimation was introduced in section 1.3 as a tool for fitting the parameters of a model to a dataset. I want you to think of it not just as a mechanical procedure for getting parameter estimates, but as the obvious consequence of the idea that likelihood measures model fit. For example, if we've decided to model a dataset as Normal(μ, σ^2) and we want to estimate μ and σ, we're really choosing a probability model from the set

$$\mathcal{M} = \left\{ \Pr_{N(\mu,\sigma^2)}(\cdot) \ : \ \mu \in \mathbb{R},\ \sigma \in \mathbb{R}_{\geq 0} \right\}$$

and maximum likelihood parameter estimation means choosing the model from $\mathcal{M}$ that has the highest likelihood. The beautiful simplicity of this perspective is that fitting parameters and choosing between models can be seen as the same thing.

Likelihood isn't the only way to measure model fit—there's no conceivable proof that it's the 'correct' measure. It's really a composite metric, a single number that summarizes all aspects of how well a model fits. For example, when fitting μ and σ in a Normal(μ, σ^2) model, a low likelihood might mean a bad choice of μ, or a bad σ, or any mixture of the two. In the author's experience, likelihood maximization is invariably the best starting point for model fitting; but subsequently it may be useful to invent other domain-specific metrics that concentrate on the parts of the model it's most important to get right.

In Natural Language Processing, likelihood is widely used for evaluation (or rather, an oddly-transformed version of it), and it's given the name *perplexity*. It's woefully underused in other areas of machine learning.

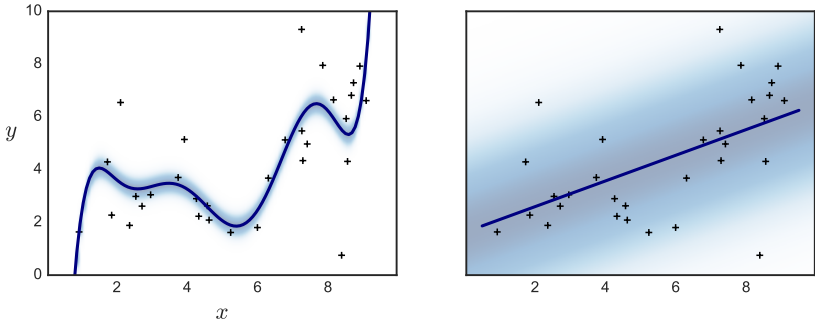
Here's a selection of examples that illustrate how likelihood is a natural way to measure model fit.

Example 4.1.1. Suppose we have a collection of datapoints $(x_1, y_1), \ldots, (x_n, y_n)$ and we want to fit a supervised model. Here are two candidate models: on the left

$$Y_i \sim -38.5 + 95.7x_i - 84.8x_i^2 + 38.3x_i^3 - 9.5x_i^4 + 1.3x_i^5 - 0.09x^6 + 0.003x_i^7 + \text{Normal}(0, 0.31^2)$$

and on the right

$$Y_i \sim 1.62 + 0.49x_i + \text{Normal}(0, 2.39^2).$$

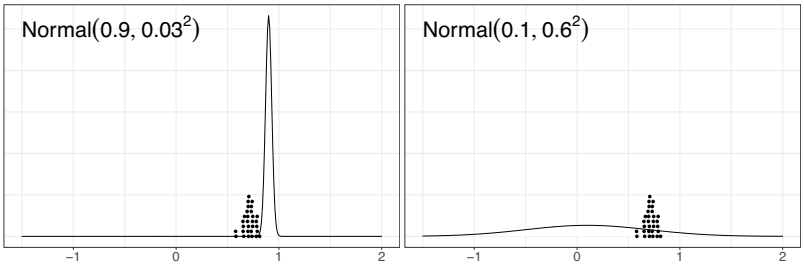


(The solid line shows the non-random part of the model. The shading indicates the spread of the noise term; at each value of x the shading spans $+/-$ two standard deviations of Y.)

The model on the left captures the shape of the dataset better, and it has smaller average error (i.e. distance between the line and the datapoints), meaning it has better prediction accuracy. But it isn't at all plausible as a *model for the dataset*, because several datapoints lie well outside the high-probability region, meaning that it's immensely unlikely that the left hand model did in fact generate the dataset. This is reflected in

the likelihood: the model on the left gives the dataset a log likelihood of -379.3, and the model on the right gives it log likelihood -64.6.

Example 4.1.2. Suppose we have a collection of datapoints $x_1, \ldots, x_n$ clustered around 0.7, and we want to fit the generative model Normal(μ, σ^2). Here are two candidate models: on the left is Normal$(0.9, 0.03^2)$, and on the right is Normal$(0.1, 0.6^2)$.



Likelihood measures how plausible it is that the model could have generated the dataset. Here, the left hand model isn't at all plausible because all its probability mass is so tightly concentrated away from the dataset. It gives the dataset a log likelihood of -570.5, versus -28.0 for the model on the right.

It's worth remarking that this is a generative modelling exercise. There's no prediction involved here, so there's no way to even talk about prediction accuracy. But the likelihood / model fit concept still makes perfect sense.

Exercise 4.1.3. Throughout October, the weather each day has been

M	T	W	T	F	S	S
				🌦	🌦	🌧
🌧	🌧	🌦	🌦	🌦	🌦	🌦
🌦	🌦	🌧	🌧	🌦	🌦	🌦
🌦	🌦	🌦	🌦	🌦	🌧	🌧
🌧	🌦	🌦	🌧	🌦		

Which of these three models fits the data best?

1. Each day is independent, $\mathbb{P}(🌧) = {}^{9}/_{29}$, $\mathbb{P}(🌦) = {}^{20}/_{29}$.
2. Each day is independent; on weekdays $\mathbb{P}(🌧) = {}^{6}/_{21}$ and $\mathbb{P}(🌦) = {}^{15}/_{21}$; on weekends $\mathbb{P}(🌧) = {}^{3}/_{8}$ and $\mathbb{P}(🌦) = {}^{5}/_{8}$.
3. For the first day, $\mathbb{P}(🌧) = {}^{9}/_{29}$ and $\mathbb{P}(🌦) = {}^{20}/_{29}$, and thereafter $\mathbb{P}(\text{same as yesterday}) = {}^{21}/_{28}$.

The data is discrete (either 🌦 or 🌧 each day), so the likelihood of the observed data is simply its probability. The calculation is elementary probability.

$$\textit{Model 1:} \quad \mathbb{P}(\textit{observed data}) = \left(\frac{9}{29}\right)^{9}\left(\frac{20}{29}\right)^{20} \approx 1.83 \times 10^{-15}$$

$$\textit{Model 2:} \quad \mathbb{P}(\textit{observed data}) = \left(\frac{6}{21}\right)^{6}\left(\frac{15}{21}\right)^{15}\left(\frac{3}{8}\right)^{3}\left(\frac{5}{8}\right)^{5} \approx 2.36 \times 10^{-9}$$

$$\textit{Model 3:} \quad \mathbb{P}(\textit{observed data}) = \frac{20}{29}\left(\frac{21}{28}\right)^{20}\left(\frac{7}{28}\right)^{8} \approx 3.34 \times 10^{-8}$$

So the third model is the best fit.
□

Example 4.1.4. Likelihood even finds its way into sports, as a way to evaluate probabilistic forecasts. Consider a sports match with a set R of possible outcomes, e.g. $R = \{\text{win}, \text{draw}, \text{lose}\}$. Suppose a sports forecasting system has produced estimated probabilities p_r for each outcome $r \in R$. A conventional way to evaluate the this forecast

is via the Brier score, defined by

$$\text{Brier}(p, y) = \sum_{r \in R} (p_r - 1_{y=r})^2$$

where y is the actual observed outcome. A persuasive case can be made that a better score is

$$\text{IGN}(p, y) = -\log_2 p_y$$

which we recognize as (negative) log likelihood. In the sports forecasting literature, this is called the "ignorance score". For details, see Edward Wheatcroft. In: *Journal of Quantitative Analysis in Sports* (2021). DOI: doi:10.1515/jqas-2019-0089.

Part II
Handling probability models

In this section we will study mathematical and computational tools for working with probability models.

Section 5 is about Bayes's rule and related maths. You have most likely learnt about the simple version of Bayes's rule and used it for calculations like "What is the probability that this patient is sick, given that the test came back with a positive test result?" I find it easiest to understand the setup by writing out an explicit probability model,

```
# Generate x (1=sick, 0=healthy), then y (1=test+ve, 0=test-ve)
x = np.random.binomial(n=1, p=0.004)  # 0.4% of the population is sick
if x == 1:
    y = np.random.binomial(n=1, p=0.94)  # if sick, test is 94% accurate
else:
    y = np.random.binomial(n=1, p=0.01)  # if healthy, test is 99% accurate
```

To work out $\mathbb{P}(\text{sick} \mid \text{test +ve})$, i.e. $\mathbb{P}(X = 1 \mid Y = 1)$, we can use Bayes's rule:

$$\mathbb{P}(X = 1 \mid Y = 1) = \frac{\mathbb{P}(X = 1)\ \mathbb{P}(Y = 1 \mid X = 1)}{\mathbb{P}(Y = 1)} = \frac{0.004 \times 0.94}{0.004 \times 0.94 + (1 - 0.004) \times 0.01}.$$

In data science and machine learning we often want to apply Bayes's rule to continuous random variables. For a continuous random variable, the probability of it taking any exact value is zero. So what does Bayes's rule mean if the test result Y is a continuous random variable, for example a blood level reading? The Bayes's equation becomes divide-by-zero! Or what if the medical condition X is a continuous random variable, for example the patient's life expectancy? The Bayes's equation involves conditioning on a zero-probability event!

There is a simple fix: use Bayes's rule written in terms of likelihood:

$$\Pr_X(x \mid Y = y) = \frac{\Pr_X(x)\ \Pr_Y(y \mid X = x)}{\Pr_Y(y)} \quad \text{when } \Pr_Y(y) > 0.$$

Section 5.1 explains what this equation means. The goal is to become comfortable with writing down equations of this general type, to have intuition and to understand the pitfalls behind what the terms mean, and to not get flummoxed by the notation. The rest of section 5 has many examples of how to do algebra with continuous random variables.

In data science and machine learning, we hardly ever use algebra to actually solve equations like Bayes's rule. We're usually interested in models that are far too rich for there to be any chance of algebraic solutions. Instead we use computers and random sampling, *computational methods*. This is the topic of section 6. All the maths examples in section 5 are great for reassuring ourselves that we understand the ideas, but the algebraic skills aren't needed to be a good data scientist.

* * *

Here is some backstory behind sections 5 and 6. Suppose we have crafted a probability model. A probability model can be expressed in mathematical equations, or it can be expressed as a simulator that uses random number generators—i.e. as a piece of code, an algorithm. Computer scientists are well used to reasoning about algorithms, for example to prove correctness or to analyse running time. Dijkstra is associated with this school of mathematical reasoning about code:[31]

> *Programming is one of the most difficult branches of applied mathematics; the poorer mathematicians had better remain pure mathematicians.*

[31] Edsger W. Dijkstra. "How do we tell truths that might hurt?" personal note EWD498. URL: http://www.cs.utexas.edu/users/EWD/ewd04xx/EWD498.PDF.

It's natural to want to be able to analyse a simulator using maths, Dijsktra-style. For example, we might have implemented a climate simulator that uses random variables, so that every time we run it we get a slightly different output, and we might want to calculate the distribution of the output. For example, we might want to know the frequency of extreme weather events. This will typically involve integrals, for calculating probabilities and expectations, and section 5 provides many examples of this sort of calculation.

There is another stance, diametrically opposed to Dijkstra's. If the maths is too hard, as it usually is, we can just run our simulator, and we'll *see* its output directly. We run it many times, and we'll see a random sample drawn from the model output's distribution. If we run it enough times, and plot a histogram of the output, we have the distribution in front of us. This is called Monte Carlo simulation, the topic of section 6. (Perhaps the integration-based approach ought to be called the "Trinity College method" in honour of Newton, its most famous son.)

The middle way, a bit of computation and a bit of maths, is powerful. There are some situations where the maths is too hard and computation is the only tool available. There are other situations, such "inverse problems" in which we know the model's output and want to deduce the likely input, where computation is too slow without some mathematical help. We might for example get a thousand-fold speedup by replacing an inner simulation loop with a deft equation.

* ∗ *

In data science, *every probability model we come up with is wrong.* We should never delude ourselves that we have a true mathematical description of the world, only that we have a reasonable approximation to the data we're given. So, given a dataset, what's the point in inventing an approximate model and then analysing that model to death, when we could just analyse the dataset itself? Section 7 looks at methods that take a dataset to be the ground truth, as opposed to sections 5 & 6 which take a model to be the ground truth. This shift in perspective is in fact the cornerstone of machine learning and especially cross-validation. It's slippery concept, more philosophy than maths, and it shines a new light on the mathematical and computational methods in sections 5 and 6.

5. The maths of random variables

5.1. Bayes's rule for random variables

The goal of this section is to take apart Bayes's rule and re-learn it, so that we understand what it really means and how to apply it to continuous random variables. This is a section for readers who want to know "but what do these equations actually *mean*?" For actual practical use of Bayes's rule, skip ahead to the next section on page 91.

Here are two examples using Bayes's rule, one discrete and the other continuous. The discrete example is trivial, but it's helpful to go through it step by step so that we can carry the ideas across to the continous example where it's decidedly non-trivial.

Example 5.1.1 (Discrete conditioning).
Someone runs this function and tells us they got $Y = 3$. What can we infer about their X?

```
def rxy_disc():
    x = numpy.random.randint(low=-5, high=6)  # from -5 to +5 inclusive
    y = numpy.random.binomial(n=6, p=(x/6)**2)
    return (x,y)
```

Example 5.1.2 (Continuous conditioning).
Someone runs this function and tells us they got $Y = 0.6$. What can we infer about their X?

```
def rxy_cts():
    x = numpy.random.uniform(-1,1)
    y = numpy.random.normal(loc=x**2, scale=0.1)
    return (x,y)
```

5.1.1. JOINT DISTRIBUTIONS AND MARGINALS

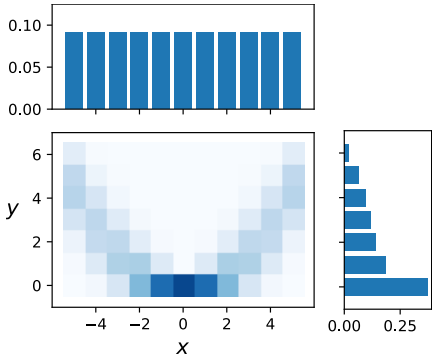
This is a refresher of IA Probability lecture 7

For the discrete example, we can immediately write out the joint distribution of X and Y:

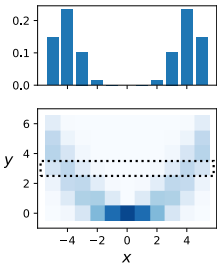
$$\begin{aligned}\Pr_{X,Y}(x,y) &= \mathbb{P}(X=x, Y=y)\\ &= \mathbb{P}(X=x)\,\mathbb{P}(Y=y \mid X=x)\\ &= \frac{1}{11}\binom{6}{y}\left((x/6)^2\right)^y\left(1-(x/6)^2\right)^{6-y}.\end{aligned}$$

The plot below shows the joint distribution, and also the marginal distributions $\mathbb{P}(X=x)$ and $\mathbb{P}(Y=y)$, which we get by using the sum rule:

$$\Pr_Y(y) = \mathbb{P}(Y=y) = \sum_x \mathbb{P}(Y=y, X=x) = \sum_x \Pr_{X,Y}(x,y).$$



We're told $Y = 3$, so we're in the $Y = 3$ row of the plot. We can just read off the values in this row, divide by the row sum so that the probabilities sum to one, and we get a distribution for X.



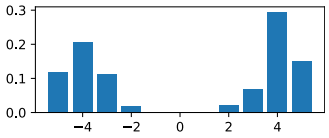
$$\mathbb{P}(X = x \mid Y = 3) = \frac{\mathbb{P}(X = x, Y = 3)}{\mathbb{P}(Y = 3)} = \frac{\Pr_{X,Y}(x, 3)}{\Pr_Y(3)}.$$

5.1.2. CONDITIONAL RANDOM VARIABLES

For a random variable X and an event C, we write $(X \mid C)$ for X *conditioned on* C. What we've just calculated is the distribution of the conditional random variable $(X \mid Y = 3)$, i.e. we've conditioned on the event $\{Y = 3\}$.

Here's code for generating conditional random variables. We can learn about the distribution of X given $Y = 3$ by simply generating lots of (X, Y) pairs, keeping those where $Y = 3$, and discarding the others. (This is called *rejection sampling*.)

```
def rx_given_y(yobs):
    while True:
        (x,y) = rxy_disc()
        if y == yobs: break
    return x

xsample = [rx_given_y(3) for _ in range(400)]
histogram(xsample)
```



This code `rx_given_y(3)` is a random number generator; it produces a different answer every time we call it. *In other words, it is a random variable.* It has everything we expect a random variable to have:

- $(X \mid Y = 3)$ has a sample space $\{-5, \ldots, 5\}$, the same sample space as X
- $(X \mid Y = y)$ is a parametric random variable, with parameter y
- $(X \mid Y = y)$ has a probability mass function, $\text{pmf}(x) = \mathbb{P}\big((X|Y = y) = x\big)$. One can prove[32], using the mathematical tools from the rest of section 5, that $\text{pmf}(x) = \mathbb{P}(X = x \mid Y = y)$.

In keeping with the $\Pr.(\cdot)$ notation from section 1.5, we ought to write the probability mass function as $\Pr_{(X \mid Y=3)}(x)$, but it looks better and is easier to read if we write it

$$\Pr_X(x \mid Y = y).$$

Now let's rewrite the equation for $\mathbb{P}(X = x \mid Y = 3)$. Writing $\Pr_{X,Y}$ for the joint distribution, $\Pr_{X,Y}(x, y) = \mathbb{P}(X = x, Y = y)$, and converting $\mathbb{P}(Y = y \mid X = x)$ to Pr notation, we get

$$\Pr_X(x \mid Y = 3) = \frac{\Pr_{X,Y}(x, 3)}{\Pr_Y(3)} = \frac{\Pr_X(x)\Pr_Y(3 \mid X = x)}{\Pr_Y(3)}.$$

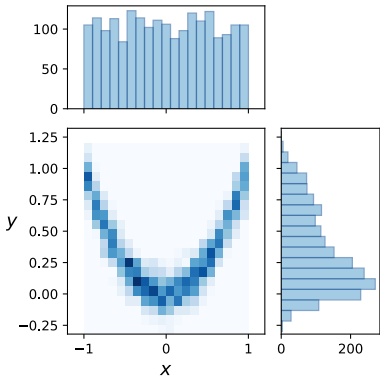
This is Bayes's rule, just written in terms of conditional random variables, and with Pr notation.

5.1.3. JOINT DISTRIBUTIONS AND MARGINALS (CONTINUOUS CASE)

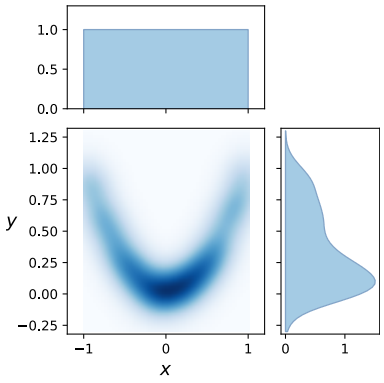
Given a random tuple such as the function `rxy_cts()`, we can generate (X, Y) samples and plot a 2d histogram. The plot on the left shows the histogram.

[32] Mathematicians prefer to go the other way: they define $(X \mid Y = y)$ via this function rather than by code, and then they'd have to work out a proof that our rejection sampling code achieves this pmf.

```
xy_sample = [rxy_cts() for _ in range(2000)]
hist2d(xy_sample)
```



histogram of samples from rxy()



the joint density function

The plot on the left also shows marginal histograms. We can get them in two ways: either (a) build a matrix of counts, which we need anyway for the 2d histogram, and then add up row and column sums, and plot bar charts, or (b) take all the x values and plot a histogram of them, and likewise the y values. These two methods are clearly identical, because they each count the same thing.

```
x_sample = [x for (x,y) in xy_sample]
y_sample = [y for (x,y) in xy_sample]
hist(x_sample)
hist(y_sample)
```

The 2d histogram is an approximation to an idealized mathematical *joint density function*, plotted on the right hand side. This is a function of two variables, x and y, and it's written $\Pr_{X,Y}(x, y)$. It's related to probabilities by

see IA Probability lecture 7

$$\mathbb{P}(x_1 \leq X \leq x_2,\ y_1 \leq Y \leq y_2) = \int_{x=x_1}^{x_2} \int_{y=y_1}^{y_2} \Pr_{X,Y}(x, y)\, dy\, dx.$$

The larger the sample, and the finer the histogram bin size, the closer the histogram gets to the joint density function.

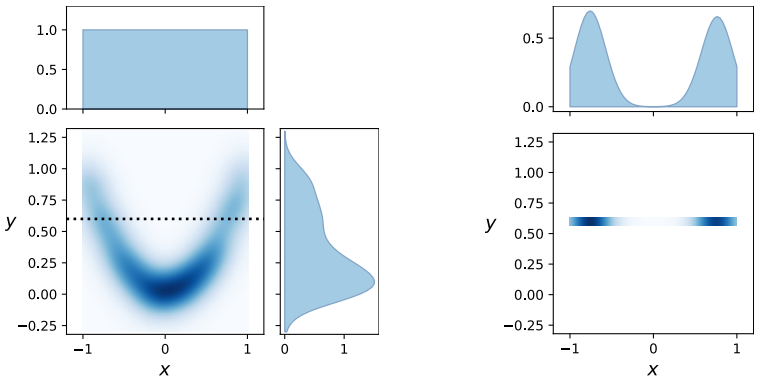
What about the probability density functions for X and Y? We'll write them as usual as $\Pr_X(x)$ and $\Pr_Y(y)$. We get them by integrating the joint density function, similar to how we summed row and column counts from the 2d histogram.

$$\Pr_X(x) = \int_y \Pr_{X,Y}(x, y)\, dy, \qquad \Pr_Y(y) = \int_x \Pr_{X,Y}(x, y)\, dx.$$

We referred to one-dimensional histograms as 'marginal histograms', for the obvious reason that we get them by writing row and column sums in the margins of the 2d table of counts. We also refer to the one-dimensional density plots as marginals. We also hear X or Y themselves, when taken in isolation, referred to as a marginal random variables.

5.1.4. CONDITIONAL RANDOM VARIABLES (CONTINUOUS CASE)

Now we can see by analogy how $(X|Y = 0.6)$ should be interpreted, in the case of example 5.1.2 where both X and Y are continuous random variables. It's what we get by taking an infinitesimal sliver of the joint density $\Pr_{X,Y}(x, y)$ at $y = 0.6$. Another way of putting it: $(X \mid Y = 0.6)$ is the random variable whose density function, call it $f(x)$, is proportional to $\Pr_{X,Y}(x, 0.6)$.



joint density of (X, Y) *slice at* $y = 0.6$

To be precise, the density is $f(x) = \Pr_{X,Y}(x, 0.6)/\Pr_Y(0.6)$. The denominator is so that $f(x)$, like any other density function, integrates to 1. The correct scaling factor is $\Pr_Y(0.6)$ because of the formula for marginal densities:

$$\Pr_Y(y) = \int_x \Pr_{X,Y}(x, y)\, dx.$$

5.1.5. BAYES'S RULE WITH LIKELIHOOD NOTATION

Given a pair of random variables (X, Y) with joint density $\Pr_{X,Y}(x, y)$, we can take a horizontal slice through the joint density plot and get

$$\Pr_X(x \mid Y = y) = \frac{\Pr_{X,Y}(x, y)}{\Pr_Y(y)}$$

or we can take a vertical slice and get

$$\Pr_Y(y \mid X = x) = \frac{\Pr_{X,Y}(x, y)}{\Pr_X(x)}.$$

Putting these two together, we get the random variable way of writing Bayes's rule:

$$\Pr_X(x \mid Y = y) = \frac{\Pr_{X,Y}(x, y)}{\Pr_Y(y)} = \frac{\Pr_X(x)\Pr_Y(y \mid X = x)}{\Pr_Y(y)}.$$

Bayes's rule is true for any pair of random variables (X, Y), but it's only useful when the question tells us $\Pr_X(x)$ and $\Pr_Y(y|X = x)$.

Sometimes Bayes's rule us written with $\Pr_Y(y)$ expanded into an integral,

$$\Pr_Y(y) = \int_x \Pr_{X,Y}(x, y)\, dx = \int_x \Pr_X(x)\Pr_Y(y \mid X = x)\, dx$$

but it's rarely useful to try to calculate $\Pr_Y(y)$, as we shall see in the next section.

5.2. Calculations with Bayes's rule

Bayes's rule says that, for a pair of random variables (X, Y),

$$\Pr_X(x \mid Y = y) = \frac{\Pr_X(x) \Pr_Y(y \mid X = x)}{\Pr_Y(y)}.$$

In this section we'll see how to use Bayes's rule in calculations. It might not seem there's anything much to say—it's a formula, we just bung in the quantities we know, and read out the answer. But there are two niceties:

- The denominator $\Pr_Y(y)$ is a real pain, and we never want to work it out explicitly if we can possibly avoid it. Indeed, for most problems, there's no closed-form solution. Typically we use Bayes's rule as

$$\Pr_X(x \mid Y = y) = \kappa \Pr_X(x) \Pr_Y(y \mid X = x) \quad \text{for some constant } \kappa.$$

- The left hand side of Bayes's rule is the likelihood function for the random variable $(X \mid Y = y)$. If we think of Bayes's rule as just a maths formula then it's easy to hit a dead end our algebra, but if we think of it as a tool for finding a likelihood function then there is a cunning trick we can use: it's a likelihood, so it must integrate to 1, so κ must be whatever is needed to make the right hand side integrate to 1 with respect to x.

In the examples in this section we'll see three different ways to deal with κ. In example 5.2.1 we just give up and leave it as an intractable integral. In example 5.2.2 we recognize that the density is that of a standard random variable so we can look up κ on Wikipedia (a rare situation, usually only seen in textbooks and exam questions!) And in example 5.2.3 the X random variable is discrete so we can compute κ by exhaustively summing over all the x-terms.

Example 5.2.1 ().
Consider the pair of random variables (X, Y) generated by

```
def rxy(σ):
    x = numpy.random.uniform(−1,1)
    y = numpy.random.normal(loc=x**2, scale=σ)
    return (x,y)
```

Or, in random variable notation, $X \sim \text{Uniform}[-1, 1]$ and $Y \sim N(X^2, \sigma^2)$.

Calculate $\Pr_X(x \mid Y = y)$.

Since $X \sim \text{Uniform}[-1, 1]$, the density is constant over that range:

$$\Pr_X(x) = \frac{1}{2} \qquad \textit{for } -1 \le x \le 1.$$

Since Y is generated from X, with a Normal distribution $N(X^2, \sigma^2)$, we can just write down its conditional density:

$$\Pr_Y(y \mid X = x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(y-x^2)^2/2\sigma^2}.$$

Bayes's rule says

$$\Pr_X(x \mid Y = y) = \kappa \Pr_X(x) \Pr_Y(y \mid X = x) = \kappa' e^{-(y-x^2)^2/2\sigma^2}.$$

Here we've gathered some more non-x terms into the constant. All that we're interested in is finding the density as a function of x. We know that densities must integrate to 1, so we can just write out what κ' must be, as an integral over x:

$$\kappa' = 1 \Big/ \int_{x=-1}^{1} e^{-(x^2-y)^2/2\sigma^2}\, dx.$$

We can't go any further without some heavy calculus, unless we use computational methods to approximate the integral—the topic of section 6.
□

Exercise 5.2.2. Consider the pair of random variables (Θ, Y) where $\Theta \sim \text{Uniform}[0,1]$ and $(Y \mid \Theta = \theta) \sim \text{Bin}(1, \theta)$. In other words, for $\theta \in [0,1]$ and $y \in \{0,1\}$,

$$\Pr_\Theta(\theta) = 1, \qquad \Pr_Y(y \mid \Theta = \theta) = \begin{cases} \theta & \text{if } y = 1 \\ 1-\theta & \text{if } y = 0 \end{cases}$$

Find the distribution of $(\Theta \mid Y = 1)$.

For $\theta \in [0,1]$,

$$\begin{aligned} \Pr_\Theta(\theta \mid Y = 1) &= \kappa \Pr_\Theta(\theta) \Pr_Y(1 \mid \Theta = \theta) \\ &= \kappa \times 1 \times \theta. \end{aligned}$$

We could calculate κ directly by integration, $\kappa = 1/\int_0^1 \theta \, d\theta$. But there's a different route, one which will serve us better for more complicated integrals:

We should have at our fingertips a collection of standard random variables. The Beta distribution $\text{Beta}(\alpha, \beta)$ *has density*

$$\Pr_{\text{Beta}(\alpha,\beta)}(x) = B_{\alpha,\beta}\, x^{\alpha-1}(1-x)^{\beta-1} \quad \text{for } x \in [0,1]$$

where $B_{\alpha,\beta}$ is a special constant given (in the case of integer α and β) by

$$B_{\alpha,\beta} = \frac{(\alpha+\beta-1)!}{(\alpha-1)!(\beta-1)!}.$$

Let's rewrite the Bayes's rule result in what looks like a horrendous overcomplication:

$$\Pr_\Theta(\theta \mid Y = 1) = \kappa \frac{1}{B_{\alpha,\beta}} B_{\alpha,\beta}\, \theta^{\alpha-1}(1-\theta)^{\beta-1} \quad \text{where} \quad \alpha = 2, \beta = 1.$$

The point of this is to put it in the form of a standard pdf:

$$= \kappa \frac{1}{B_{2,1}} \Pr_{\text{Beta}(2,1)}(\theta).$$

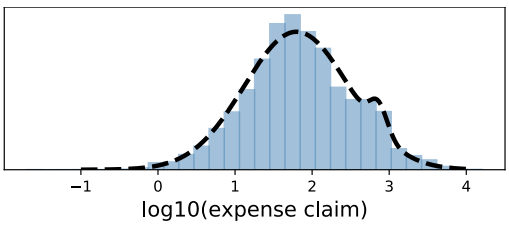
Since densities integrate to 1, the $\kappa/B_{2,1}$ term at the front must be 1, so we conclude

$$(\Theta \mid Y = 1) \sim \text{Beta}(2,1).$$

We didn't have to do any integration, we just had to remember the standard random variable density functions, and rely on the fact that someone else has already figured out the correct constant.

□

Exercise 5.2.3 (Classification).
This question concerns a dataset[33] of MP expense claims for office expenses from 2018. Let x_i be the amount claimed in record i of the dataset, and let $y_i = \log_{10} x_i$.



A histogram suggests we use a Gaussian mixture model with two components,

$$Y_i \sim \text{Normal}(\mu_K, \sigma_K^2), \qquad K \sim \text{Cat}([p, 1-p])$$

and we can estimate the parameters using maximum likelihood: with probability $p = 0.966$ a claim belongs to the main component which has mean $\mu_1 = 1.797$ and standard deviation $\sigma_1 = 0.679$, and otherwise the claim belongs to the 'hump' which has mean $\mu_2 = 2.861$ and standard deviation $\sigma_2 = 0.116$.

Find the probability that an expense claim £x belongs to the hump.

The question is asking us for $\Pr_K(2 \mid Y = \log_{10} x)$. *We'll use the tip from the start of this section: we'll look for the full likelihood function* $\Pr_K(k \mid Y = \log_{10} x)$ *for* $k \in \{1, 2\}$, *and thereby find the normalizing constant* κ. *First,*

$$\Pr_K(k) = \begin{cases} p & \text{if } k = 1 \\ 1 - p & \text{if } k = 2. \end{cases}$$

Next, the conditional distribution of Y*:*

$$\Pr_Y(y \mid K = k) = \begin{cases} \frac{1}{\sqrt{2\pi\sigma_1^2}} e^{-(y-\mu_1)^2/2\sigma_1^2} & \text{if } k = 1 \\ \frac{1}{\sqrt{2\pi\sigma_2^2}} e^{-(y-\mu_2)^2/2\sigma_2^2} & \text{if } k = 2. \end{cases}$$

Now, applying Bayes's rule,

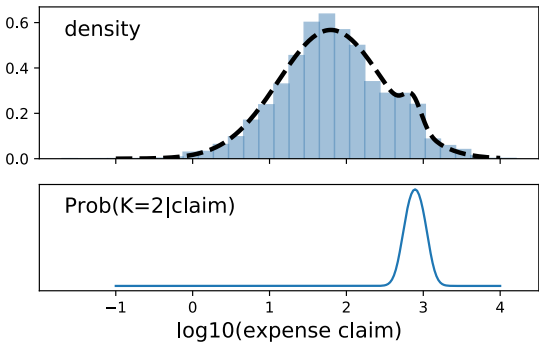
$$\begin{aligned} \Pr_K(k \mid Y = y) &= \kappa \Pr_K(k) \Pr_Y(y \mid K = k) \\ &= \begin{cases} \kappa p \frac{1}{\sqrt{2\pi\sigma_1^2}} e^{-(y-\mu_1)^2/2\sigma_1^2} & \text{if } k = 1 \\ \kappa(1-p) \frac{1}{\sqrt{2\pi\sigma_2^2}} e^{-(y-\mu_2)^2/2\sigma_2^2} & \text{if } k = 2. \end{cases} \end{aligned}$$

Let's define some variables to make this simpler:

$$= \begin{cases} \kappa q_1 & \text{if } k = 1 \\ \kappa q_2 & \text{if } k = 2. \end{cases}$$

The constant κ *is whatever it must be to make the distribution for* $(K|Y = y)$ *sum to 1, namely,* $1/(q_1 + q_2)$. *In code,*

```
ϕ = scipy.stats.norm.pdf

# Distribution of (K|Y=y)
def Pr_K_Y(k, y, p=0.966, μ1=1.797, μ2=2.861, σ1=0.679, σ2=0.116):
    q1 = p * ϕ(y, loc=μ1, scale=σ1)
    q2 = (1-p) * ϕ(y, loc=μ2, scale=σ2)
    return (q1 if k==1 else q2) / (q1 + q2)
```



□

DENSITIES SUM TO ONE

Deft use of "densities sum to one" saves lots of hassle when applying Bayes's rule. Here are some more examples outside the setting of Bayes's rule.

Exercise 5.2.4 ().
Let X be a random variable taking values $\{0, 1, \dots\}$ in with $\Pr_X(x) = \kappa r^x$ where $0 < r < 1$ is given and κ is a constant. Find κ.

Standard maths formulae: $1 + r + r^2 + \cdots = 1/(1-r)$ for $|r| < 1$, and $1 + r + \cdots + r^n = (1 - r^{k+1})/(1-r)$.

By the "densities sum to one" rule, $\sum_{x=0}^{\infty} \Pr_X(x) = 1$*, hence*

$$\sum_{x=0}^{\infty} \kappa r^x = \frac{\kappa}{1-r} = 1$$

hence $\kappa = (1-r)$*. Furthermore,*

$$F(x) = \sum_{y=0}^{x} (1-r) r^x = (1-r)\frac{1-r^{x+1}}{1-r} = 1 - r^{x+1}.$$

□

Exercise 5.2.5 (Recognizing densities).
Let X be a random variable with density

$$\Pr_X(x) = \kappa\, x(1-x)^2, \qquad x \in [0,1]$$

for some constant κ. Name the distribution of X.

It's worth knowing the density functions for certain standard random variables. One distribution, very common in exam questions about Bayesian calculations, is the Beta distribution. This describes a continous $[0,1]$*-valued random variable, with density function*

Beta distribution, page 10

$$\Pr_{\text{Beta}(\alpha,\beta)}(x) = B_{\alpha,\beta}\, x^{\alpha-1}(1-x)^{\beta-1}$$

where $B_{\alpha,\beta}$ *is a constant (i.e. doesn't depend on* x*).*

In this question, $\Pr_X(x)$ *has the same functional form (as a function of* x*) as* $\Pr_{\text{Beta}(2,3)}(x)$*; the only difference is a constant at the front. But there's no choice about the constant: it has to be whatever will make the density integrate to one. There is only one constant that will work, namely the constant* $B_{2,3}$ *at the front of the Beta distribution. Therefore* $X \sim \text{Beta}(2,3)$*.*
□

Exercise 5.2.6. Suppose Y is a $[0,1]$-valued continuous random variable with density

$$\Pr_Y(y) = \kappa\, y^2(1-y)^3\,.$$

Without using any calculus, find

$$\int_{y=0}^{1} y \Pr_Y(y)\, dy.$$

First, we should immediately recognize the density function for Y *as that of a* $\text{Beta}(\alpha = 3, \beta = 4)$ *distribution, so the normalizing constant is* $\kappa = B_{3,4} = (7-1)!\,/\,(3-1)!(4-1)! = 60$*.*

For the integral, we could just spot that it's the mean of the Beta distribution, and look it up on Wikipedia, and report the answer which is $\alpha/(\alpha+\beta) = 3/7$*.*

Alternatively, rewrite it as

$$\int_{y=0}^{1} y\Big\{B_{3,4}\, y^2(1-y)^3\Big\}\, dy = \frac{B_{3,4}}{B_{4,4}} \int_{y=0}^{1} B_{4,4}\, y^{4-1}(1-y)^{4-1}\, dy\,.$$

The point of this rewriting is so that the integrand is the density of a standard random variable, so it must integrate to 1. So the answer is $B_{3,4}/B_{4,4} = 60/140 = 3/7$*.*
□

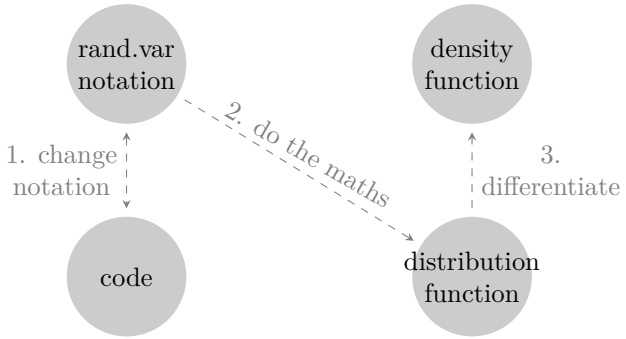
[33] Data obtained from https://www.theipsa.org.uk/mp-staffing-business-costs/your-mp. Expense claims for ≤£0 have been excluded.

5.3. Deriving the likelihood

We've seen how important the likelihood function is—it crops up all the time in machine learning, whether we're fitting probability models or applying Bayes's rule.

In Part I we got by with standard random variables, whose likelihood functions we can just look up on Wikipedia. But if we're doing creative data science and machine learning we'll be inventing our own custom probability models, and to work with them we need to be abe to derive the likelihood..

Although there is no foolproof universal method, there are some general tactics which are helpful. Typically the process goes like this, at least for continuous random variables:



1. Express the model in random variable notation, translating from code if necessary. Write the random variable we're interested in as a function of simple standard random variables. For example, the code

translating between code and random variable notation: section 1.1 page 3

$$\mathsf{x = -\ np.log(np.random.uniform())\ /\ \lambda}$$

   can be rewritten as

$$X = -\frac{1}{\lambda}\log U, \qquad U \sim \text{Uniform}[0,1].$$

2. The second step is to calculate the cumulative distribution function for the random variable we're interested in,

$$\mathbb{P}(X \leq x).$$

   First, rewrite in terms of the simple standard random variables from the first step. Then, using algebra and the general rules of probability, try to break it down into simpler parts, ideally ending up with the simple random variables standing on their own on the left hand side. It's easier to work with $\mathbb{P}(U \leq z)$ than with messy expressions like $\mathbb{P}(2U^2 - 1 < 3U)$.

independence, and the law of total probability: page viii

   If all our algebra fails, the last resort is to write $\mathbb{P}(X \leq x)$ as an integral, integrating over the density function of the simple random variables.

3. Finally, differentiate the cdf to get the probability density function, i.e. the likelihood:

$$\Pr_X(x) = \frac{d}{dx}\mathbb{P}(X \leq x).$$

For discrete random variables, it's rarely useful to go via the cdf, and we just have to use cunning algebra to find $\mathbb{P}(X = x)$ directly.

* * *

To understand the concepts behind data science and machine learning, you don't actually need to *do* any probability calculations, you can just use computational approximations instead. But if your competitor is good at probability and can derive the likelihood using maths, they'll be able to train their models faster. (Also, when it comes to the exam you won't have a computer, so you'll have to be able to do *some* calculation …)

the general-purpose computational approximation for all sorts of probability calculation: Monte Carlo simulation, section 6.1 page 103

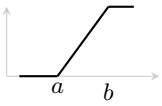
Exercise 5.3.1 (Uniform random variable).
For X generated by random.uniform(a,b), which generates a real number uniformly distributed in $[a, b]$, derive the cumulative distribution function and the density function

$$\mathbb{P}(X \le x) = \begin{cases} \frac{x-a}{b-a} & \text{for } x \in [a,b] \\ 0 & \text{for } x < a \\ 1 & \text{for } x > b \end{cases} \qquad \Pr_X(x) = \frac{1}{b-a} 1_{x \in [a,b]}$$

where the notation 1_α denotes the indicator function, which converts booleans to integers: $1_\alpha = 1$ if $\alpha =$ True and 0 if $\alpha =$ False.

For finding the cdf we'll just use common sense and think through all the cases carefully. Let's try some cases.

- *If $x < a$ then $\mathbb{P}(X \le x) = 0$ since values $< a$ are impossible*
- *If $x > b$ then $\mathbb{P}(X \le x) = 1$ since all return values are $\le b$.*
- *If $x = (a+b)/2$ then $\mathbb{P}(X \le x) = 1/2$ since all values in $[a, b]$ are equally likely, so it's a 50% chance of being in either half*



Generalizing, for any $x \in [a, b]$, $\mathbb{P}(X \le x)$ is equal to the fraction of the entire interval $[a, b]$ covered by the range $[a, x]$, i.e. $(x - a)/(b - a)$. This gives the cdf as claimed in the question.

The density is found by differentiating the cdf, and by being careful about the different cases:

$$\Pr_X(x) = \frac{d}{dx}\mathbb{P}(X \le x) = \begin{cases} \frac{1}{b-a} & \text{for } x \in [a,b] \\ 0 & \text{otherwise.} \end{cases}$$

This can be rewritten as an indicator function, as asked for. (See example 1.3.6 page 16 for a guide to using indicator functions.)
□

Exercise 5.3.2 (Exponential random variable).
For the random variable X generated by this code

```
x = - np.log(np.random.uniform())/λ   # λ > 0
```

derive the cumulative distribution function and the density function

$$\mathbb{P}(X \le x) = 1 - e^{-\lambda x} \qquad \Pr_X(x) = \lambda e^{-\lambda x}$$

This is called the Exponential random variable with rate λ.

Step 1: Translating the code directly into random variable notation,

$$X = -\frac{1}{\lambda}\log(U) \qquad \text{where} \qquad U \sim U[0,1].$$

Step 2: This is a continuous distribution, so let's find the cumulative distribution function for X, $\mathbb{P}(X \le x)$. We'll just work it out with algebra:

$$\begin{aligned} \mathbb{P}(X \le x) &= \mathbb{P}\Big(-\frac{1}{\lambda}\log U \le x\Big) && \text{using the definition of } X \\ &= \mathbb{P}\big(U \ge e^{-\lambda x}\big) && \text{by simple algebra} \\ &= 1 - \mathbb{P}(U < e^{-\lambda x}) && \text{more algebra, to let us use } U\text{'s cdf.} \end{aligned}$$

Recalling the cumulative distribution function for $U[0, 1]$ from the last exercise (or looking it up on Wikipedia),

$$\mathbb{P}\big(U[0,1] \le x\big) = \begin{cases} 0 & \text{if } x \le 0 \\ x & \text{if } x \in [0,1] \\ 1 & \text{if } x \ge 1 \end{cases}$$

thus

$$\mathbb{P}(X \le x) = 1 - e^{-\lambda x}.$$

Step 3: Differentiate the cdf for X to get the pdf for X, i.e. the likelihood.

$$\Pr_X(x) = \frac{d}{dx}\mathbb{P}(X \le x) = \frac{d}{dx}\left\{1 - e^{-\lambda x}\right\} = \lambda e^{-\lambda x}.$$

□

Exercise 5.3.3. Find the density of the random variable Y generated by this code:

```
def ry():
    x = np.random.uniform()
    return x**2
```

Step 1: Translating the code into random variable notation,

$$Y = X^2 \qquad \textit{where} \qquad X \sim U[0,1].$$

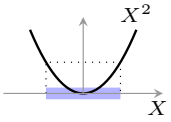
Step 2: This is a continuous random variable, so we'll work out the cdf $\mathbb{P}(Y \le y)$. It's impossible to get $Y < 0$ or $Y > 1$, so

$$\begin{aligned}\mathbb{P}(Y \le y) &= 0 \quad \textit{for } y < 0\\ \mathbb{P}(Y \le y) &= 1 \quad \textit{for } y \ge 1.\end{aligned}$$

For the general case $y \in [0,1]$, let's rewrite the probability we want in terms of simpler standard random variables, in this case in terms of $X \sim U[0,1]$:

$$\mathbb{P}(Y \le y) = \mathbb{P}(X^2 \le y).$$

We want to be able to use the cdf for X, so let's do some algebra to make it look more like "probability that X is in a certain set". It's helpful to sketch a graph of the function x^2, and read off the X-range for which $X^2 \le y$.



$$\begin{aligned}\mathbb{P}(X^2 \le y) &= \mathbb{P}\big(X \in [-\sqrt{y}, \sqrt{y}]\big)\\ &= \mathbb{P}(X \le \sqrt{y}) - \mathbb{P}(X \le -\sqrt{(}y)).\end{aligned}$$

This lets us use the standard cdf for $U[0,1]$:

$$\begin{aligned}\mathbb{P}(Y \le y) &= \sqrt{y} - 0 \quad \textit{for } y \in [0,1], \textit{ since } -\sqrt{y} < 0\\ &= \sqrt{y} \quad \textit{for } y \in [0,1].\end{aligned}$$

Step 3: Differentiate the cdf for Y to get its pdf. We have to be meticulous about the three cases $y < 0$, $y \in [0,1]$, and $y > 1$,

$$\Pr_Y(y) = \frac{d}{dy}\mathbb{P}(Y \le y) = \begin{cases} 0 & \textit{if } y < 0\\ 1/(2\sqrt{y}) & \textit{if } y \in [0,1]\\ 0 & \textit{if } y > 0.\end{cases}$$

(Don't worry about the value of $\Pr_Y$ at $y = 0$ or $y = 1$. The cumulative distribution function isn't differentiable at these points, and it doesn't matter what value we assign to the density.)
□

Exercise 5.3.4 (Minimum of independent random variables).
Let $X_1 \sim \text{Exp}(\lambda_1)$ and $X_2 \sim \text{Exp}(\lambda_2)$ be independent random variables. Let $Y = \min(X_1, X_2)$. Show that $Y \sim \text{Exp}(\lambda_1 + \lambda_2)$.

The question asks us to show that two random variables are identically distributed. We can either show that they have the same cdf, or that they have the same pdf; either is an acceptable way to show that they have the same distribution. In this question there's no obvious way to get at the pdf for Y other than by differentiating the cdf, so let's stick with proving that the

cdfs are equal. So the quantity we want to calculate is

$$\begin{aligned}
\mathbb{P}(Y \le y) &= \mathbb{P}\big(\min(X_1, X_2) \le y\big) \quad \textit{by definition of } Y \\
&= \mathbb{P}\big(X_1 \le y \textit{ or } X_2 \le y\big) \quad \textit{breaking it down as per Step 2} \\
&= 1 - \mathbb{P}\big(X_1 > y \textit{ and } X_2 > y\big) \\
&\quad \textit{rearranging so we can use independence, as per Step 2} \\
&= 1 - \mathbb{P}(X_1 > y)\,\mathbb{P}(X_2 > y) \quad \textit{by independence} \\
&= 1 - e^{-\lambda_1 y} e^{-\lambda_2 y} \quad \textit{from the cdf for } \mathrm{Exp}(\lambda_1) \textit{ and } \mathrm{Exp}(\lambda_2) \\
&= 1 - e^{-(\lambda_1+\lambda_2)y}.
\end{aligned}$$

Since the cdf for Y is exactly that of an $\mathrm{Exp}(\lambda_1 + \lambda_2)$ random variable, we conclude that $Y \sim \mathrm{Exp}(\lambda_1 + \lambda_2)$.
□

Exercise 5.3.5 (Gaussian mixture model).
Find the likelihood function of the random variable X generated by this code, where $0 \le p \le 1$:

```
def rx(p): # EXERCISE: rewrite in numpy vectorized form
    x1 = np.random.normal(loc=0.9, scale=0.3)
    x2 = np.random.normal(loc=0, scale=1.5)
    k = np.random.choice([1,2], p=[p,1-p])
    if k == 1:
        return x1
    else:
        return x2
```

(This random variable is called a Gaussian mixture model with two components. It generates $\mathrm{Normal}(0.9, 0.3^2)$ with probability p, $\mathrm{Normal}(0, 1.5^2)$ with probability $1-p$. It can obviously be generalized to have more components, or to use different distributions for the components.)

Step 1: Translate the code into random variable notation,

$$X = \begin{cases} X_1 & \textit{if } K = 1 \\ X_2 & \textit{if } K = 2 \end{cases}$$

where $X_1 \sim N(\mu_1, \sigma_1^2)$, $X_2 \sim N(\mu_2, \sigma_2^2)$, $K \sim \mathrm{Cat}([p_1, p_2])$, where the parameters of the normal distributions are as specified in the question, and where for generality we're writing $p_1 = p$ and $p_2 = 1 - p$.

Step 2: This is a continuous random variable, so we'll find the cumulative distribution function $\mathbb{P}(X \le x)$. Since X is generated differently based on the value of K, it's useful to break down the probability by conditioning on K, using the Law of Total Probability. This trick is very handy for probability models that first generate one value, and then generate a second based on the first.

Law of Total Probability: page viii

$$\mathbb{P}(X \le x) = \sum_k \mathbb{P}(X \le x \mid K = k)\,\mathbb{P}(K = k).$$

We can just fill this in directly, using the distributions from Step 1:

$$\mathbb{P}(X \le x) = \mathbb{P}\big(N(\mu_1, \sigma_1^2) \le x\big)p_1 + \mathbb{P}\big(N(\mu_2, \sigma_2^2) \le x\big)p_2.$$

This is as far as we can go with probability algebra: we've managed to rewrite the cdf for X in terms of the cdfs of standard distributions, which we could look up if we wanted to.

Step 3: Differentiate the cdf to get the pdf. This is the clever bit!

$$\mathrm{Pr}_X(x) = \frac{d}{dx}\mathbb{P}(X \le x) = p_1 \frac{d}{dx}\mathbb{P}\big(N(\mu_1, \sigma_1^2) \le x\big) + p_2 \frac{d}{dx}\mathbb{P}\big(N(\mu_2, \sigma_2^2) \le x\big).$$

We don't actually need to know the cdf of the Normal distribution, we only need to know its derivative—also known as its pdf, or likelihood. Thus

$$\Pr_X(x) = p_1 \Pr_{N(\mu_1,\sigma_1^2)}(x) + p_2 \Pr_{N(\mu_2,\sigma_2^2)}(x).$$

The pdf of the Normal distribution is something worth learning by heart. We get

$$\Pr_X(x) = \sum_{k=1}^{2} p_k \frac{1}{\sqrt{2\pi\sigma_k^2}} e^{-(x-\mu_k)^2/2\sigma_k^2}.$$

□

Exercise 5.3.6 (Generating a Geometric).
Here is code for generating a random variable:

```
1  p = ...   # a constant in the range (0,1)
2  λ = - math.log(1-p)
3  u = random.random()
4  x = - math.log(u) / λ
5  y = math.ceil(x)
```

Find the distribution of the random variable generated in line 5.

When you are asked "find the distribution", you can give your answer either as a density, or as a cumulative distribution function, or, if it's a standard distribution, by naming the distribution.

Write U, X, and Y for the three random variables. We've already seen in exercise 5.3.2 that $X \sim \text{Exp}(\lambda)$, an Exponential random variable with rate λ, which has cdf

$$\mathbb{P}(X \leq x) = 1 - e^{-\lambda x}.$$

Now for the distribution of $Y = \mathsf{math.ceil}(X)$. This is a discrete random variable, and so we could work with either the cumulative distribution function or the density. Let's try the former. For any integer k,

math.ceil rounds up to the nearest integer

$$\mathbb{P}(Y \leq k) = \mathbb{P}(\mathsf{math.ceil}(X) \leq k) = \mathbb{P}(X \leq k) = 1 - e^{-\lambda k}.$$

(This is the same formula as the cumulative distribution function for X, but X and Y are not the same: X is a continuous random variable and the formula holds for any $x > 0$, whereas Y is a discrete random variable and the formula is only correct for integer k.) Rewriting in terms of p,

$$\mathbb{P}(Y \leq k) = 1 - (e^{-\lambda})^k = 1 - (1-p)^k.$$

We might recognize this as the cumulative distribution function for a Geometric random variable and stop here. Or we could also go a step further and work out the density:

$$\mathbb{P}(Y = k) = \mathbb{P}(Y \leq k) - \mathbb{P}(Y \leq k-1) = (1-p)^{k-1} - (1-p)^k = (1-p)^{k-1}p.$$

The probabilities subtract like this because, in a Venn diagram, the event $\{Y \leq k-1\}$ is a subset of the event $\{Y \leq k\}$.

□

Exercise 5.3.7 (Equality of distributions).
Here are two pieces of code for generating random variables:

```
def rgeom(p):
    λ = - math.log(1-p)
    u = random.random()
    x = - math.log(u) / λ
    return math.ceil(x)

def rgeom2(p):
    x = 1
    while random.random() > p:
        x = x + 1
    return x
```

Show that the two pieces of code generate identically distributed random variables.

We have shown in exercise 5.3.6 that rgeom(p) *generates a* Geom(p) *random variable, for which*

$$\Pr(k) = (1-p)^{k-1}p\,, \quad \textit{for } k \in \{1, 2, \dots\}.$$

Now turn to rgeom2(p) *and the while loop. Let's work out some examples, and then try to guess the general pattern.*

- *In order for* rgeom2 *to output 1, we need the* random.random()>p *test to fail on the first pass, i.e. we need* random.random()<=p, *which happens with probability* p.
- *For* rgeom *to output 2, we need the test to succeed on the first pass then fail on the second. Let's call the first* random.random() *value* U_1 *and the second* U_2. *By the way the code is written,* U_1 *and* U_2 *are independent, so*

Why are U_1 and U_2 independent? Look back at section 1.1 page 3.

$$\mathbb{P}(X_1 > p \text{ and } X_2 \leq p) = \mathbb{P}(X_1 > p)\,\mathbb{P}(X_2 \leq p) = (1-p)p.$$

- *Generalizing, if* X *is the output of* rgeom2(p), *then*

$$\mathbb{P}(X = k) = (1-p)^{k-1}p.$$

Thus the output of rgeom2 *is a* Geom(p) *random variable, the same distribution as produced by* rgeom.
□

6. Computational methods

6.1. Monte Carlo integration

Suppose we want to find $\mathbb{P}(X \in A)$ and we have code to generate samples from X, but we haven't been able to derive the density function; or maybe we know the density but we can't solve the integral $\mathbb{P}(X \in A) = \int_{x \in A} \Pr_X(x)\, dx$. There's a blindingly obvious computational method: we just simulate X many times, and count how often it lies in A. For example,

Example 6.1.1. Let $X \sim N(\mu = 1, \sigma = 3)$. What is $\mathbb{P}(X > 5)$?

```
x = numpy.random.normal(loc=1, scale=3, size=10000)  # simulate the r.v.
i = x > 5          # i is a Boolean vector, same length as x
numpy.mean(i)      # returns the average of i, typecasting True to 1 and False to 0
```

This is a simple example of a general tool called Monte Carlo integration, which is the basis of pretty much all modern probabilistic machine learning.

To state the general case of Monte Carlo, a quick reminder about expectation. The expectation of a numerical random variable X is defined as

see IA Probability lectures 3 and 4 for the discrete case, lecture 7 for the continuous case

$$\mathbb{E}\, X = \begin{cases} \sum_x x \Pr_X(x) & \text{for a discrete random variable} \\ \int_x x \Pr_X(x)\, dx & \text{for a continuous random variable.} \end{cases}$$

For interesting probability models we'd like to work with rich complicated random variables, such as random images or random text strings, so it's more useful to work with a different version of expectation. If h is some real-valued function whose average value we want to report (we might call it a 'readout' function), then $\mathbb{E}\, h(X)$ is[35]

$$\mathbb{E}\, h(X) = \begin{cases} \sum_x h(x) \Pr_X(x) & \text{for a discrete random variable} \\ \int_x h(x) \Pr_X(x)\, dx & \text{for a continuous random variable.} \end{cases}$$

Monte Carlo approximation. We can approximate $\mathbb{E}\, h(X)$ by

$$\mathbb{E}\, h(X) \approx \frac{1}{n} \sum_{i=1}^{n} h(x_i)$$

where $x_1, \ldots, x_n$ is a sample drawn from distribution X.

The formal statement of Monte Carlo integration obscures the simplicity of the idea. To better convey the idea, we'll look at some different settings in which it's used, starting with the probability estimation with which we opened this section. Afterwards, on page 104, a note on efficient implementation in Python.

ESTIMATING PROBABILITIES

Let's tie the general definition back to the probability estimation we opened with. Suppose we want to estimate $\mathbb{P}(X \in A)$, and we have code to generate X. Define

$$h(x) = \begin{cases} 1 & \text{if } x \in A \\ 0 & \text{if } x \notin A \end{cases}$$

and let $Y = h(X)$. This is a $\{0, 1\}$-valued random variable. (This function h is called an *indicator function*, also written $h(x) = 1_{x \in A}$ or $h(x) = 1[x \in A]$. Indicator functions useful

indicator functions for dealing with boundaries: example 1.3.6 page 16

[35] There are two ways to get to $\mathbb{E}\, h(X)$: either use $\mathbb{E}\, h(X) = \int_x h(x) \Pr_X(x)\, dx$ directly, or let $Y = h(X)$, calculate $\Pr_Y(y)$, then use $\mathbb{E}\, Y = \int_y y \Pr_Y(y)\, dy$. These two methods give the same answer, a result known unkindly as 'The Law of the Unconscious Statistician', since it's easy to not even notice that there's a choice.

whenever we want to convert if/then choices into algebraic expressions that we can do maths with.) Then

$$\begin{aligned}\mathbb{E}\,Y &= 0 \times \mathbb{P}(Y=0) + 1 \times \mathbb{P}(Y=1) \quad \text{defn. of expectation}\\ &= \mathbb{P}(Y=1)\\ &= \mathbb{P}(X \in A) \quad \text{from defn. of } Y.\end{aligned}$$

Alternatively, using the Monte Carlo estimate,

$$\mathbb{E}\,Y = \mathbb{E}\,h(X) \approx \frac{1}{n}\sum_{i=1}^{n} 1_{x_i \in A}$$

where $x_1, \ldots, x_n$ is a sample from X. Thus

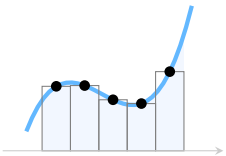
$$\mathbb{P}(X \in A) \approx \frac{1}{n}\sum_{i=1}^{n} 1_{x_i \in A}.$$

ESTIMATING AN INTEGRAL

Suppose we've been asked to find the area under a curve,

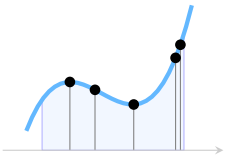
$$\int_{x=a}^{b} h(x)\,dx\,.$$

The method you might have learnt at school is to split the x range into n equally sized pieces, and approximate the function by a series of rectangles, taking the height of the rectangle to be the value of h at the midpoint.



$$\text{area} \approx \sum_{i=1}^{n} h(x_i)w, \quad \text{where } x_i = a + w\big(i - {}^1\!/_2\big)\,,\ w = \frac{b-a}{n}.$$

Let's reframe this as a probability problem. Consider a random variable X, uniformly distributed in the range $[a, b]$, and let $x_1, \ldots, x_n$ be sampled from this distribution.



The Monte Carlo approximation tells us that

$$\mathbb{E}\,h(X) \approx \frac{1}{n}\sum_{i=1}^{n} h(x_i).$$

By definition of expectation,

$$\mathbb{E}\,h(X) = \int_{x=a}^{b} h(x)\frac{1}{b-a}\,dx.$$

Putting these two together,

$$\int_{x=a}^{b} h(x)\,dx \approx \frac{b-a}{n}\sum_{i=1}^{n} h(x_i).$$

VECTORIZED COMPUTATION

To compute the Monte Carlo approximation $\sum_i h(x_i)/n$, we need to generate a large sample $(x_1, \ldots, x_n)$ and then apply h to each item. In Python, the best coding style for this is vectorized.

for vectorized numpy code, see the course material for IA Scientific Computing.

Vectorized thinking is great for conciseness. Surely no one would prefer the iterative style

```
1  tot = 0
2  for _ in range(n):
3      x = rng()
4      tot = tot + h(x)
5  tot/n
```

or even the list comprehension style

```
6  xs = [rng() for _ in range(n)]
7  sum(h(x) for x in xs) / n
```

when they can just write

```
8  x = rng(size=n)
9  numpy.mean(h(x))
```

Vectorized coding is also important from the point of view of performance. Every time the Python interpreter has to evaluate a Python expression there's a performance hit; the first two versions take this hit on every sample, whereas in the vectorized version the iteration is all done in numpy's fast C code.

We'll be using Monte Carlo a lot in this course, because it's the backbone of computational probabilistic machine learning, so we really need to be able to write fast and efficient code for it. It really shows off the strength of numpy—the whole point of numpy's vectorized calculation, and all the syntactic sugar that comes with it, is to make this sort of code easy to write and fast to run. If someone reading this code doesn't know about Monte Carlo, and isn't very familiar with numpy, they might not even spot the n on line 8, and they'd wonder what on earth the code is doing!

6.2. Bayes's rule via computation

Suppose we have a pair of random variables X and Y, and we want to find the distribution of the conditional random variable $(X \mid Y = y)$.

Bayes's rule gives us a formula for its likelihood $\Pr_X(x \mid Y = y)$. But there are two problems. First, it's often intractable to find the normalizing constant that the formula requires. Second, we may want to draw samples of $(X \mid Y = y)$, and just knowing a formula for the likelihood isn't much help.

There is a computational approach, in the same spirit as Monte Carlo approximation but more subtle. It lets us approximate probabilities of the form $\mathbb{P}(X \in A \mid Y = y)$, using a *weighed sample* of values $x_1, \ldots, x_n$ drawn from distribution X, with specially chosen weights $w_1, \ldots, w_n$:

$$\mathbb{P}(X \in A \mid Y = y) \approx \sum_{i=1}^{n} w_i 1_{x_i \in A}.$$

As with Monte Carlo approximation, it's useful to generalize this into a statement about the expected value of some real-valued readout function, $\mathbb{E}\big(h(X) \mid Y = y\big)$. Just set $h(x) = 1_{x \in A}$, and we recover $\mathbb{P}(X \in A \mid Y = y)$.

Computational Bayes. We can approximate $\mathbb{E}\big(h(X) \mid Y = y\big)$ by

$$\mathbb{E}\big(h(X) \mid Y = y\big) \approx \sum_{i=1}^{n} w_i h(x_i)$$

where $x_1, \ldots, x_n$ is a sample drawn from distribution X, and where the weights $w_1, \ldots, w_n$ are given by

$$w_i = \frac{\Pr_Y(y \mid X = x_i)}{\sum_{j=1}^{n} \Pr_Y(y \mid X = x_j)}.$$

This is a brute force way of applying Bayes's rule computationally. It's simple to use, and doesn't dependent on any advanced maths, but sometimes it takes very many samples for it to be a good approximation. Any specialist field of machine learning will have its own specialized numerical methods, and it's just the same with Bayesian machine learning—much work has been put into more sophisticated computational Bayes procedures, such as Gibbs sampling which you'll come across in later study.

Working with weighted samples. The computational approach doesn't[36] give us the likelihood function $\Pr_X(x \mid Y = y)$—but very often we can do without. The whole spirit of computational methods is to avoid doing algebra with likelihood functions, and instead to work with samples:

For approximating the variance, and for sampling, see section 7.3

- For a real-valued random variable X, if we want the conditional mean $\mathbb{E}(X \mid Y = y)$, then we can approximate it directly using a weighted sample, no need to use the integral $\int_x x \Pr_X(x \mid Y = y)\, dx$. It's also easy to approximate $\mathrm{Var}(X \mid Y = y)$.
- We can even use weighted samples to draw samples from (approximately) the conditional distribution $(X \mid Y = y)$.
- We can use weighted samples to plot (approximately) the conditional likelihood function, as we now illustrate.

For the case that X is a discrete random variable, it's easy to plot the approximate likelihood. We just want a bar chart, where the height of the bar at x is $\mathbb{P}(X = x \mid Y = y)$, which is approximately $\sum_i w_i 1_{x_i = x}$. To plot it,

```
x_, w_ = ... # the sample with associated weights
bh = pandas.DataFrame({'x':x_, 'w':w_}).groupby('x')['w'].apply(sum)
plt.bar(x=np.arange(len(bh)), height=bh, tick_label=bh.index)
```

[36] Except for the case that X is a discrete random variable. Then the likelihood function is just the probability mass function, $\Pr_X(x \mid Y = y) = \mathbb{P}(X \in \{x\} \mid Y = y)$, which the computational approach tells us.

For the case that X is a real-valued continuous random variable, there's a nice plot called the *density histogram*, which is a type of histogram that's directly comparable to a plot of the likelihood function. Split the x-axis into bins, and at each bin $x \in [\ell, u)$ plot a bar of height

$$\text{bar height for } [\ell,u) = \frac{1}{u-\ell}\,\mathbb{P}\big(X \in [\ell,u) \;\big|\; Y = y\big) \approx \frac{1}{u-\ell}\sum_{i=1}^{n} w_i 1_{\ell \le x_i < u}.$$

The denominator $(u-\ell)$ makes it so that the total *area*, summed over all bins, is $\sum_i w_i = 1$. This makes the density histogram directly comparable to a plot of the likelihood function, which must integrate to 1. Matplotlib does all of the work for us:

```
plt.hist(x_, weights=w_, density=True)
```

Exercise 6.2.1. Let $X \sim \text{Uniform}[-1,1]$ and let $Y \sim \text{Normal}(X^2, 0.1^2)$. Plot the conditional distribution of X given that $Y = 0.2$.

Here's the code:

```
xsamp = np.random.uniform(-1,1, size=1000)
w = scipy.stats.normal.pdf(0.2, loc=xsamp**2, scale=0.1)
w = w / np.sum(w)
plt.hist(xsamp, weights=w, density=True)
```

□

Exercise 6.2.2. Consider the probability model

```
def ry():
    θ = np.random.uniform(0,1)
    y = np.random.binomial(n=3, p=θ)
    return y
```

Suppose we have observed $Y = 2$ and we want to know the likely range of Θ. Plot a density histogram of $(\Theta \mid Y = 2)$.

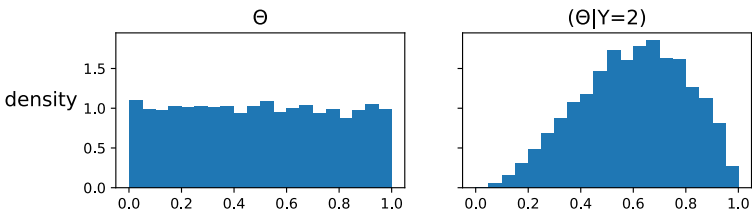
for the mathematical solution see exercise 5.2.2

The relevant distribution is

$$\Pr_Y(y \mid \Theta = \theta) = \binom{3}{y}\theta^y(1-\theta)^{3-y}$$

In the code below we're ignoring the $\binom{3}{y}$ factor, since it cancels out when we rescale the weights.

```
θsamp = np.random.uniform(low=0, high=1, size=10000)
n,y = 3,2
w = θsamp**y * (1-θsamp)**(n-y)
w = w / np.sum(w)
plt.hist(θsamp, weights=w, density=True, bins=20)
```



Quick sanity check: is it reasonable that that the distribution of $(\Theta \mid Y = 2)$ should be pushed away from uniform and towards $\theta = 2/3$? The distribution of Y is $\text{Bin}(3,\Theta)$, so the expected value of Y is 3Θ. We observed $Y = 2$, so a reasonable guess is $\Theta \approx 2/3$.

□

Example 6.2.3. Suppose we have a piece of code that can generate a random image of a face. Suppose we also have a neural network classifier, which takes in a face and guesses the mood of the person. These classifiers typically return a list of probabilities, for example "I guess this face is happy with probability 80%, sad with probability 15%, ..." How might we generate a random *happy* face?

A programmer might say "Easy, I'll just keep generating faces, feed them into the classifier, and if the 'happy' score comes out high then I'll stop and return that face." And then they're asked how high the score has to be, and they say "erm ... I'll leave it as a user-definable threshold" to disguise the fact that they have no idea. If the threshold is too high then the code will be slow, and if it's too low then it'll let through too many sad faces.

A mathematician might say "Let X be a random variable representing the face. Let $p(X)$ be the vector of probabilities returned by the classifier, and let Y be a Categorical random variable with probabilities given by $p(X)$. All we need is $\Pr_X(x \mid Y = \text{happy})$, which we can find using Bayes's rule. This solves the problem." But this answer is useless because (1) it's intractable to apply Bayes's rule, because we have no formula for $\Pr_X(x)$ and even if we did we'd have no chance of calculating the normalizing constant, and anyway (2) our task is to generate *a random happy face, not find the conditional likelihood function.*

The computational Bayes approach is a happy medium. It takes the programmer's spirit that says "hey, I have code that can generate X, let's use it!", as well as the mathematician's spirit with it's very deep and rigorous understanding of what 'conditioning' means. It's just four lines of code—but there's so much thinking behind it!

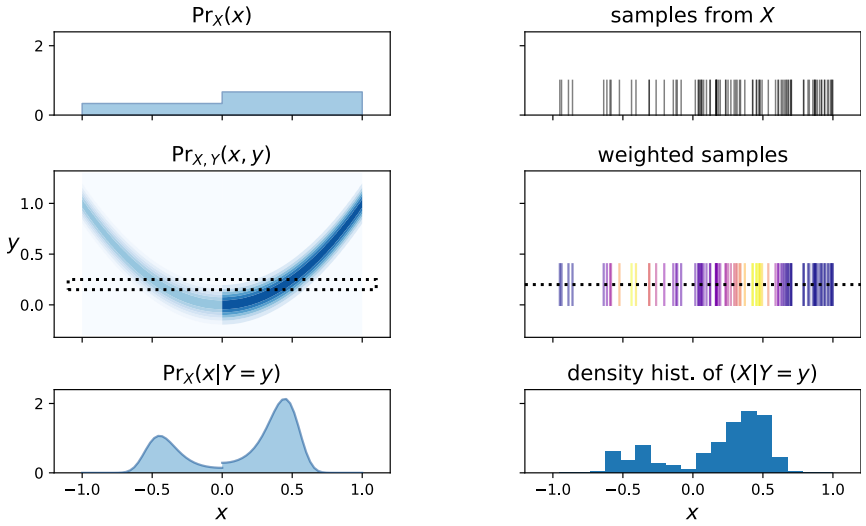
```
faces = [random.face() for _ in range(1000)]
w = [mood_classifier(x)['happy'] for x in faces]
w = w / np.sum(w)
np.random.choice(faces, p=w) # i.e. ℙ(pick face i) = w_i
```

The last line is how we draw a sample from (approximately) the conditional distribution $(X \mid Y = \text{happy})$, and is discussed in section 7.3.
□

WHY IT WORKS

To get a feel for why the computational Bayes procedure works, let's have a closer look at an example. We'll take a non-uniform distribution over the interval $[-1, 1]$ for X, and make $Y \sim \text{Normal}(X^2, 0.1^2)$. Let's find the distribution of $(X \mid Y = 0.2)$.

The left hand plots below show the mathematical interpretation: the top plot is $\Pr_X(x)$; the middle plot is the joint likelihood $\Pr_{X,Y}(x, y) = \Pr_X(x)\ \Pr_Y(y \mid X = x)$, and it highlights the horizontal slice we want to take at $y = 0.2$; and the bottom plot shows the resulting $\Pr_X(x \mid Y = 0.2)$.



On the right, the top plot shows sampled X values, call them $x_1, \ldots, x_n$. (This type of plot is called a 'rug plot' because it looks like tufts on a carpet.) As we'd expect from the plot

of $\Pr_X(x)$, there are roughly twice as many samples in $[0, 1]$ as in $[-1, 0]$, The middle plot shows these sampled values again, colour-coded by their weights $\Pr_Y(0.2 \mid X = x_i)$. The big idea is that these samples and weights mimic the horizontal slice from the joint likelihood plot: the frequency of samples is proportional to $\Pr_X(x)$, the weights are $\Pr_Y(0.2 \mid X = x)$, so frequency×weight is proportional to $\Pr_{X,Y}(x, 0.2)$. Therefore, when we sum up the weights in each bin of the bottom plot, we get the conditional density $\Pr_X(x \mid Y = 0.2)$.

DERIVATION *

You don't need to know how to derive the computational approximation to Bayes's rule, you just need to know how to use it. But it's not very hard to derive: it comes simply from writing out the expectation we want as an integral, and approximating it with Monte Carlo integration.

$$
\begin{aligned}
&\mathbb{E}\big(h(X) \mid Y = y\big) \\
&\quad = \int_x h(x) \Pr_X(x \mid Y = y)\, dx \qquad \text{by the definition of expectation} \\
&\quad = \int_x h(x)\, \kappa \Pr_X(x) \Pr_Y(y \mid X = x)\, dx \qquad \text{by Bayes's rule, where } \kappa \text{ is a constant} \\
&\quad = \int_x g(x) \Pr_X(x)\, dx \quad \text{where } g(x) = h(x)\, \kappa \Pr_Y(y \mid X = x) \\
&\quad = \mathbb{E}\, g(X) \\
&\quad \approx \frac{1}{n} \sum_{i=1}^{n} g(x_i) \quad \text{by Monte Carlo integration.}
\end{aligned}
$$

The constant κ is there to make the conditional density sum to one:

$$
\begin{aligned}
\kappa &= 1 \Big/ \int_x \Pr_X(x) \Pr_Y(y \mid X = x)\, dx \\
&= 1 \Big/ \int_x f(x) \Pr_X(x)\, dx \quad \text{where } f(x) = \Pr_Y(y \mid X = x) \\
&= 1 \,/\, \mathbb{E}\, f(X) \\
&\approx 1 \Big/ \frac{1}{n} \sum_{i=1}^{n} f(x_i) \quad \text{by Monte Carlo integration.}
\end{aligned}
$$

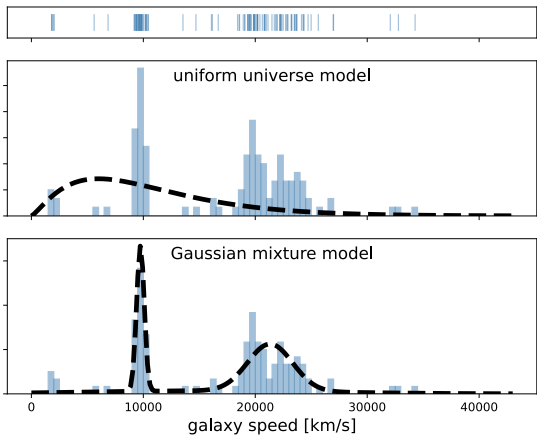
Putting these two approximations together,

$$
\mathbb{E}\big(h(X) \mid Y = y\big) \approx \frac{1}{n} \sum_{i=1}^{n} \kappa\, h(x_i) \Pr_Y(y \mid X = x_i) \approx \frac{\sum_{i=1}^{n} h(x_i) \Pr_Y(y \mid X = x_i)}{\sum_{i=1}^{n} \Pr_Y(y \mid X = x_i)}
$$

7. Empirical methods

> *All models are wrong but some are useful [...] there is no need to ask the question "Is the model true?" If "truth" is to be the "whole truth" the answer must be "No". The only question of interest is "Is the model illuminating and useful?"*
>
> George Box[41]

Students sometimes come to me asking for statistical advice. A typical question is "What distribution should I use to fit this dataset that I've collected?"



Here's a dataset, consisting of speeds at which galaxies are moving away from us. If the universe were uniform, with galaxies studded through it like cherries in a fruit cake, then we'd expect to see a smooth distribution of galaxy speeds, as per the pdf in the middle plot. It doesn't fit very well! The bottom plot shows a better-looking probability model, a Gaussian mixture model with two clusters. But is this good enough to be useful, or should we look for a better-fitting model?

Description of the galaxies dataset: exercise 1.1.3 page 6. Fitting a Gaussian mixture model: exercise 1.7.4 page 28

There is in fact a probability model that fits the dataset perfectly, called the *empirical distribution*. This model consists of "pick one of the datapoints in the dataset at random, all datapoints equally likely". If we plot its histogram, it will match perfectly the histogram of the dataset. (However, when I tell students that the best fitting model for their dataset is the dataset itself, they generally go off to look for someone who'll give them a less mystical answer.)

This section is a build-up to the empirical distribution, which we reach in section 7.3. It's an idea that needs virtually no maths or code, but it is subtle, hence the meandering buildup in sections 7.1–7.2.

* * *

The word 'empirical' means 'based on observation'. In classical Greece, there were two schools of medicine, the empiric and the rational. Rational physicians held that treatments and explanations of disease should be based on deduction from theoretical principles of how the body works. At that time the standard theory was based on the four humours, blood, phlegm, yellow bile, and black bile. Empirics on the other hand based their treatments on what they had seen to work in the past. The empiricists were considered to be inferior physicians, peddling in folk remedies ("my neighbour swears by a frog skin to cure a sore throat, worn in a pouch around the neck") without any real understanding of the disease.

In all the calculations and simulations that we've studied so far, we have been 'rational': we've worked with probability models that we invent out of thin air. But in data science, as in all science, "all models are wrong", so why try to shoehorn the data into a wrong model? The empirical approach is to simply *use the dataset itself*, since this is the best-fitting model for the probability distribution that the data might have come from.

Empirical modelling is a cornerstone of data science and machine learning. It's conceptual basis for validation on a holdout set; and it's the basis for computational methods for assessing confidence. We'll study this further in Part III.

[41] G. E. P. Box. "Robustness in the Strategy of Scientific Model Building". In: *Robustness in Statistics*. Vol. 1. May 1979, p. 40. URL: https://archive.org/details/DTIC_ADA070213. Box has been described as "one of the great statistical minds of the 20th century".

7.1. The empirical cumulative distribution function

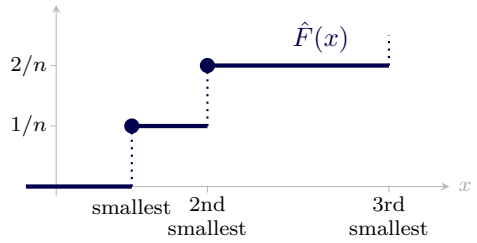
Given a dataset of n numerical values $(x_1, x_2, \ldots, x_n)$, the *empirical cumulative distribution function* of the dataset is

$$\hat{F}(x) = \frac{1}{n}\big(\text{how many items there are } \leq x\big).$$

This parallels the cumulative distribution function for a random variable X,

$$F(x) = \mathbb{P}(X \leq x).$$

It's easy to plot the empirical cumulative distribution function: just sort the data and put it on the x-axis.



In Python, using numpy and matplotlib,

```
x = [...]  # the dataset, stored as a list
ef = np.arange(1,len(x)+1)/len(x)
plt.plot(np.sort(x), ef, drawstyle='steps-post')
```

(If the dataset has duplicates, for example if the smallest and 2nd smallest datapoints are the same value, then $\hat{F}(x)$ jumps up in steps that are multiples of $1/n$. The code snippet still does the right thing.)

What's nice about the ecdf, as opposed to a histogram, is that it shows every single datapoint and it doesn't rely on an arbitrary choice of bin size. Also, if you want to show more detail for example by using a log scale, you don't need to mess around with bothersome details like "do I take the log before or after binning?"

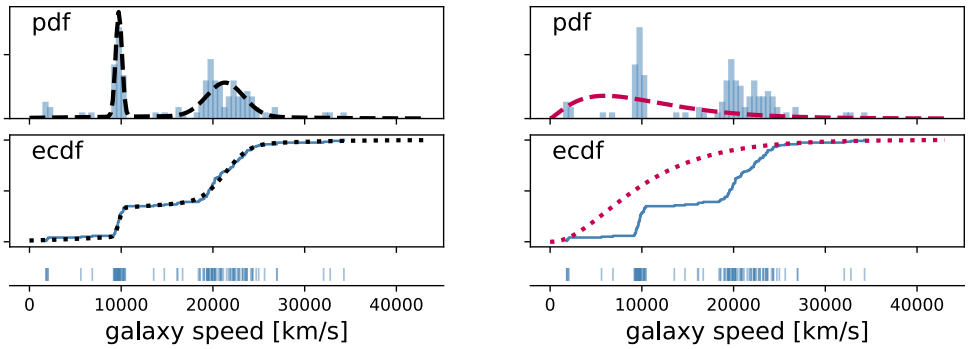
WHAT A GOOD FIT LOOKS LIKE

When we fit a probability distribution to a dataset using maximum likelihood, we'd expect that the fitted distribution's cdf should be reasonably close to the ecdf of the dataset.

Here's an illustration, the galaxy speed dataset from exercise 1.1.3 page 6. We're showing here two different models, on the left the fitted Gaussian mixture model with three components from exercise 1.7.4 page 28, and on the right the model we'd expect for a uniform universe, which has

$$\text{pdf}(x) = \text{const} \times x^{\alpha} e^{-\alpha x/x_0}, \quad \alpha = 1.3,\ x_0 = 6000.$$

The Gaussian mixture model is a much better fit, and three clusters looks reasonable.



```
url = 'https://www.cl.cam.ac.uk/teaching/current/DataSci/data/galaxies_orig.csv'
galaxies = pandas.read_csv(url)['velocity'].values
fitted_pdf, fitted_cdf = ...

fig,(ax1,ax2,ax3) = plt.subplots(3,1, sharex=True, gridspec_kw={'height_ratios':[1,1,.2]})

# Plot the density histogram and the fitted pdf
# (In a density histogram, the bar heights are scaled so that the
# total area is 1. This makes it comparable to a density plot.)
ax1.hist(galaxies, bins=30, density=True, fc='blue', alpha=.3)
ax1.plot(x, fitted_pdf(x), lw=3, color='black', linestyle='dashed')

# Plot the ecdf and the fitted cdf
ef = np.arange(1, len(galaxies)+1)/len(galaxies)
ax2.plot(np.sort(galaxies), ef, drawstyle='steps-post', color='blue')
x = np.linspace(9300, 34000, 200)
ax2.plot(x, fitted_cdf(x), color='black', lw=3, linestyle='dashed')

# Plot the raw data (a "rug plot")
for x in galaxies:
    ax3.plot([x,x],[0,1], color='blue')
```

The maximum likelihood procedure is one way to fit a parametric probability model to a dataset—but its goal is *not* to make the fitted cdf as close as possible to the dataset's ecdf. If we wanted this we'd have to first decide what we mean by 'close' (e.g. we could measure the area between the cdf curve and the ecdf), and then run an appropriate optimization; and there's no reason why this optimization should match up with the maximum likelihood optimization. Nonetheless, a maximum likelihood fit typically produces a cdf that's reasonably close to the ecdf, and if it's not close then we should investigate whether we need to pick a richer probability model.

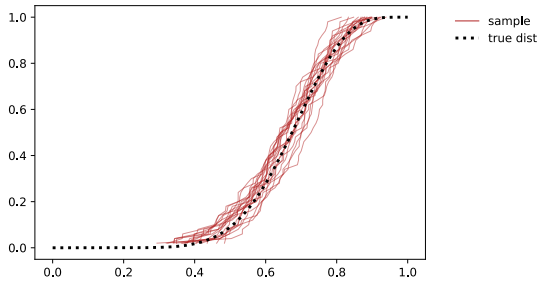
CONVERGENCE OF ECDF TO CDF

If we have a sample $x_1, \dots, x_n$ drawn from a random variable X,

$$\mathbb{P}(X \le x) = \mathbb{E}\, 1_{X \le x} \approx \frac{1}{n} \sum_{i=1}^{n} 1_{x_i \le x} \quad \text{by Monte Carlo approx.}$$

i.e. $\text{cdf}(x) \approx \text{ecdf}(x)$. Here's an illustration, 20 random samples each of size $n = 50$ drawn from the Beta$(10, 5)$ distribution. For each sample we plot its empirical distribution.

Beta distribution, page 10



7.2. Custom distributions

In generative modelling, we have a dataset $x_1, \ldots, x_n$ and we want to find a distribution that might have generated it. We typically start from an off-the-shelf model, such as a Gaussian or Exponential random variable. What if none of these standard models fit our dataset well?

We could try to invent a custom model from scratch, by plotting the histogram of the dataset then sketching a pdf that matches it. But densities must integrate to 1, and this is hard to enforce when we're sketching a custom pdf: if we make it taller in one place, we have to make it smaller everywhere else, and this gets very fiddly.

A much simpler idea is to sketch a custom cdf to match the dataset. It's easy to draw a legitimate cdf, since any function that starts at 0 and increases to 1 is valid. Then we just say "let this be my random variable". We can differentiate the cdf to get the pdf (useful if the function we drew has parameters that we want to fit with maximum likelihood estimation), and we can translate the cdf into code (useful if we want to use Monte Carlo methods). Some examples are shown below, to build up intuition for the link between custom cdfs and code.

Since we can design a completely custom random variable with whatever cdf we like, it's trivially easy to make it match the dataset perfectly: we just plot the ecdf of the dataset, and let *that* be our custom cdf. The code for this random variable is embarrassingly simple:

Custom random variable to match an ecdf.

Given a dataset of real numbers $(x_1, \ldots, x_n)$, let $\hat{X}$ be the random variable obtained by picking one of the x_i at random. This is a discrete random variable taking values in $\{x_1, \ldots, x_n\}$. Its cumulative distribution function is

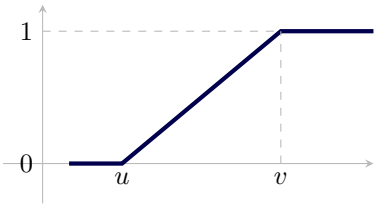
$$\mathbb{P}(\hat{X} \leq x) = \frac{1}{n}\Big(\text{how many items in the dataset are } \leq x\Big).$$

In other words, the cdf of $\hat{X}$ is the ecdf of the dataset.

ILLUSTRATIONS OF CUSTOM CDF

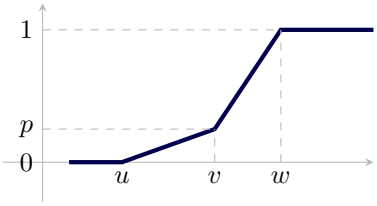
To understand where $\hat{X}$ comes from, here are some illustrations of custom cdfs and the corresponding code.

The Uniform$[u, v]$ random variable is a basic building block, and we need to be able to recognize its shape.



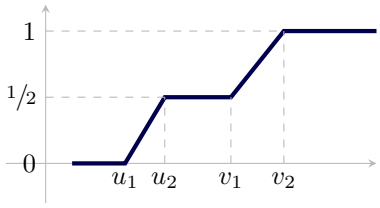

```
def rx(u,v):
    return np.random.uniform(u,v)
```

In the next plot, the cdf goes through (v, p), hence we want a random variable X with $\mathbb{P}(X \leq v) = p$ and $\mathbb{P}(X > v) = 1 - p$. We can achieve this by `np.random.choice` and an `if` statement. On each side of v the cdf is a straight line, and the pdf is the derivative of the cdf, so the pdf is constant over the interval (u, v) and also over (v, w). A constant pdf corresponds to a Uniform random variable.



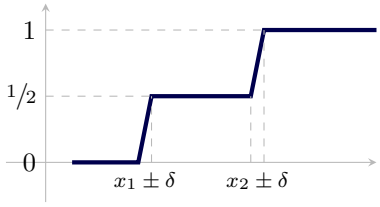

```
def rx(u,v,w,p):
    lbls = ['leq_v', 'gt_v']
    k = np.random.choice(lbls, [p, 1-p])
    if k == 'leq_v':
        return np.random.uniform(u,v)
    else:
        return np.random.uniform(v,w)
```

The next two are minor variations.



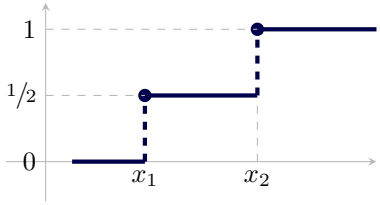

```
def rx(u1,u2,v1,v2):
    k = np.random.choice(['low','high'])
    if k == 'low':
        return np.random.uniform(u1,u2)
    else:
        return np.random.uniform(v1,v2)
```

np.random.choice(xlist) returns a random element of xlist, each element equally likely




```
def rx(x1,x2,δ):
    k = np.random.choice(['low','high'])
    if k == 'low':
        return np.random.uniform(x1–δ,x1+δ)
    else:
        return np.random.uniform(x2–δ,x2+δ)
```

In the limit as $\delta \to 0$, a Uniform$[x_1 - \delta, x_1 + \delta]$ collapses to just x_1. Thus




```
def rx(x1,x2):
    k = np.random.choice(['low','high'])
    if k == 'low':
        return x1
    else:
        return x2
```

This last example was a random variable whose cdf matches the ecdf of the length-2 dataset $\{x_1, x_2\}$. The code can be reduced a one-liner:

```
def rx(x1,x2): return np.random.choice([x1,x2])
```

When we generalize to an arbitrary list we get $\hat{X}$ described above.

7.3. The empirical distribution

> *God forbid that we should give out a dream of our own imagination for a pattern of the world.*
>
> Francis Bacon, *The Great Instauration* (1620)

Francis Bacon (1561–1626) was one of the leading thinkers at the dawn of the modern scientific era. Before him, 'science' meant logical deductive reasoning based on axioms laid down Aristotle and other authorities. Bacon said instead that science should start with neutral observations of nature, and only once there is sufficient evidence should it be summarized into laws. Bacon would have vehemently rejected maximum-likelihood model fitting, which starts by assuming the data comes from some pre-specified distribution.

The *empirical distribution* is the purest expression of Bacon's philosophy. It gives us a way to specify a probability model to match an arbitrary dataset, without our having to dream up any particular family of distributions.

Empirical distribution. Given a dataset $x_1, \ldots, x_n$, let $\hat{X}$ be the random variable obtained by picking one of the x_i at random, each i equally likely. $\hat{X}$ is a discrete random variable taking values in $\{x_1, \ldots, x_n\}$. This distribution is called the *empirical distribution of the dataset*. In Python, to generate m independent values from $\hat{X}$,

```
np.random.choice(xlist, replace=True, size=m)  # where xlist = [x_1, ..., x_n]
```

For any real-valued readout function h, the expected value of $h(\hat{X})$ is

$$\mathbb{E}\, h(\hat{X}) = \frac{1}{n}\sum_{i=1}^{n} h(x_i).$$

This definition makes sense whatever the type of values in the dataset, whether it's text strings or images or anything else, whereas the idea from section 7.2 of matching the cumulative distribution function only works for numerical datasets.

As we saw with Monte Carlo estimation and with computational Bayes, it's useful to have an equation that tells us $\mathbb{E}\, h(\hat{X})$ for an arbitrary real-valued readout function h. If $\hat{X}$ is a simple real-valued random variable, we can for example get its mean value $\mathbb{E}\,\hat{X}$ by using $h(x) = x$. If it has some exotic type, and it doesn't make sense to average $\hat{X}$, we can still get the probability of an event, $\mathbb{P}(\hat{X} \in A)$, by using $h(x) = 1_{x \in A}$.

The Zen of data science is in seeing datasets and random variables as two sides of the same coin. Here are three illustrations.

MONTE CARLO REINTERPRETED

When we're working with a probability model whose true distribution is intractable, we can simply swap out the true distribution and swap in the empirical distribution of a random sample. Here's why.

Monte Carlo is a procedure for approximating an expectation i.e. an integral, using a random sample. The approximation is

$$\mathbb{E}\, h(X) \approx \frac{1}{n}\sum_{i=1}^{n} h(x_i)$$

where $x_1, \ldots, x_n$ is a sample from X, and h is some arbitrary real-valued readout function.

Here's another interpretation. Let $\hat{X}$ have the empirical distribution of the sample, and consider $\mathbb{E}\, h(\hat{X})$: it is

$$\mathbb{E}\, h(\hat{X}) = \frac{1}{n}\sum_{i=1}^{n} h(x_i).$$

This approximation works for any readout function h, so that's not the important part here. So, don't read Monte Carlo as a way to approximate an expectation, read it instead as saying that we can approximate the *distribution* of X by the distribution of $\hat{X}$.

STATISTICS OF A DATASET

Whatever tools we have for reasoning about random variables, we can apply them just as well to datasets. Here's an illustration.

Words like 'mean' and 'variance' comes from probability theory, and should strictly speaking only be applied to random variables. But we often hear for example 'the mean and variance of a dataset' used to refer to the quantities

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i \qquad \sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2 .$$

There's a good reason for this: these are actually the mean and variance of the empirical distribution:

$$\mu = \mathbb{E}\,\hat{X} \qquad \sigma^2 = \operatorname{Var}\hat{X}$$

where $\hat{X}$ is the empirical distribution of the dataset $\{x_1, \ldots, x_n\}$.

The maths is easy. Let's write $\hat{X}$ as $\hat{X} = g(I)$ where I is a discrete uniform random variable on $\{1, 2, \ldots, n\}$ and $g(i)$ picks out the ith item in the dataset. Then, from the Law of the Unconscious Statistician,

law of the unconscious statistician, page 103

$$\mathbb{E}\,\hat{X} = \mathbb{E}\,g(I) = \sum_{i=1}^{n} g(i)\Pr_I(i) = \frac{1}{n}\sum_{i=1}^{n} x_i.$$

The result for variance is similar.

WEIGHTED SAMPLES AND COMPUTATIONAL BAYES

There is a generalized version of the empirical distribution that allows non-uniform weights. Given a dataset $x_1, \ldots, x_n$ with associated weights $w_1, \ldots, w_n$, where each weight is non-negative and the weights sum to 1, define $\hat{X}$ to be the random variable obtained by picking one of the x_i at random, where the probability of picking the ith item is w_i. For any real-valued readout function h, the law of the unconscious statistician tells us that

$$\mathbb{E}\,h(\hat{X}) = \sum_{i=1}^{n} w_i h(x_i).$$

We used this sort of empirical distribution in computational Bayes, where we were interested in finding the conditional distribution $(X \mid Y = y)$ for a pair of random variables X and Y. The basic approximation formula we used was

computational Bayes: section 6.2 page 106

$$\mathbb{E}\big(h(X) \mid Y = y\big) \approx \sum_{i=1}^{n} w_i h(x_i)$$

where $x_1, \ldots, x_n$ are sampled from X and the weight w_i is proportional to $\Pr_Y(y \mid X = x_i)$. In other words, $\mathbb{E}\big(h(X) \mid Y = y\big) = \mathbb{E}\,h(\hat{X})$, for any readout function h.

Don't read the computational Bayes formula as a way to approximate an expectation. Read it instead as saying that we can approximate the *distribution* $(X \mid Y = y)$ by the weighted empirical distribution $\hat{X}$. Thus,

- The mean and variance of $(X \mid Y = y)$ can be approximated by the mean and variance of $\hat{X}$,

  $$\mathbb{E}(X \mid Y = y) \approx \sum_{i=1}^{n} w_i x_i \qquad \operatorname{Var}(X \mid Y = y) \approx \sum_{i=1}^{n} w_i (x_i - \mu)^2.$$

  These can be derived tediously from the formula for $\mathbb{E}\big(h(X) \mid Y = y\big)$, with suitable readout functions h, but it's nicer to simply say "let's approximate $(X \mid Y = y)$ by $\hat{X}$" and then to just write down the mean and variance of the latter.

- We can sample from (approximately) the conditional distribution $(X|Y = y)$ by sampling instead from $\hat{X}$. We could justify this by noting that $\mathbb{P}(X \in A \mid Y = y) \approx \mathbb{P}(\hat{X} \in A)$ for any set A, but it's nicer to simply observe that $(X \mid Y = y)$ is approximately $\hat{X}$, so of course that's how we sample.

Part III
Inference

Induction is the glory of Science and the scandal of Philosophy.

C. D. Broad[42]

Suppose we've seen 9 swans, of which 8 are white. In the language of probability modelling, we might propose the model "a swan is white with probability p" and find the maximum likelihood estimate $\hat{p} = 0.889$ — but then what?

- It would be foolish to pronounce the general rule "the actual true probability of a swan's being white *is* 0.889", since the model is probabilistic and so other values of p are consistent with the data.
- It would be technically correct but completely vacuous to say "all we know for certain is that $0 < p < 1$".
- We could limit ourselves to the mealy-mouthed observation "my fitting procedure produced the answer $\hat{p} = 0.889$". This is undeniably true, but it's not very interesting because it doesn't carry us forwards to general laws or new predictions. As the great Cambridge physicist Rutherford is said to have said, "All science is either physics or stamp collecting"; this sort of answer is stamp collecting.

Philosophers distinguish between two types of inference:[43] deductive reasoning, and inductive reasoning. Deductive reasoning is pure mathematical logic ("all swans are white; this is a swan, therefore it is white"). Inductive reasoning is about making claims about general laws, and about making new predictions. **Induction is the leap from "this is the data I see" to "here is what I infer about the real world that produced this data".**

Induction is much harder to pin down than deductive reasoning. It's hard enough when all the data directly supports the general law ("all swans I have seen are white, therefore all swans are white"), even harder when the data is noisy.

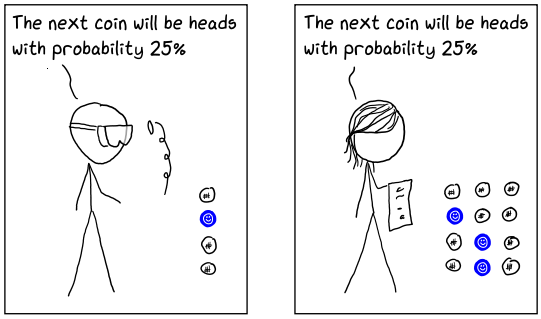
Statisticians have spent the better part of the last century thinking about ways to tackle induction, and their work has broadly speaking fallen into two schools of thought, Bayesianism and frequenticism. Machine learning has taken a different tack, which we might call empiricism. These three schools are the topics of sections 8–10. In section 10, I also give my own opinions on their strengths and weaknesses.

The central challenge of induction is finding a language to express what data tells us about the world, in a way that's neither foolishly incorrect nor vacuously true. Once we've figured out what sort of inductive claims we'd like to make, the maths is usually fairly simple.

Throughout Part III we'll mainly be looking at two ways to formulate inductive claims: confidence, and model choice. To illustrate, here's a thought experiment:

[42] C. D. Broad. *Ethics and the History of Philosophy.* Routledge, 1952. From remarks at Senate House in Cambridge in October 1926, marking the 300th anniversary of Francis Bacon's death. Francis Bacon was the "father of the scientific method". The full quotation: "May we venture to hope that when Bacon's next centenary is celebrated the great work which he set going will be completed; and that Inductive Reasoning, which has long been the glory of Science, will have ceased to be the scandal of Philosophy?" With only a few years left, I venture that Broad would be disappointed.

[43] Some philosophers like to distinguish between two types of non-deductive reasoning: inductive reasoning which is based on statistical evidence and makes predictions ("95% of swans I have seen are white, so the next swan I see has 95% chance of being white") versus abductive reasoning which is based on one-shot evidence and seeks to find an explanation ("this swan is black, so someone must have dyed it"). In data science, we work with models that combine general laws, predictions about future datapoints, and inferences about individual cases; but there are many more than two points on the spectrum, so I do not think the philosophers' distinction is useful.



Confidence. Suppose there is a possibly biased coin, and two scientists run experiments to measure its bias. Scientist A tosses it 4 times and gets 1 head, scientist B tosses it 12 times and gets 3 heads. They both produce the same fitted probability, namely that each coin is heads with probability 25% — but clearly there is a difference between the two: scientist A should be less confident about this 25%, scientist B should be more confident.

We might think of confidence as an interval. For example, perhaps scientist A concludes that the probability of heads is in the range $(25 \pm 10)\%$, and scientist B concludes it is in the range $(25 \pm 1)\%$. But how should be obtain these confidence intervals? i.e. how should we *measure* confidence?

Model choice. Another way to tackle the challenge of induction is in terms of model choice. In the coin-tossing example, suppose our two scientists started out with the belief that the coin is unbiased, but were willing to entertain the possibility of bias. In the light of the data, should either of them switch to the 'bias' model?

If the evidence in favour of 'biased' is sufficiently strong compared to the evidence in favour of 'unbiased', they might switch the 'bias' model. One simple procedure is via confidence intervals: they might first find a confidence interval for the probability of heads, and then accept the 'unbiased' model if this interval includes $p = 0.5$. More generally, it's useful to be able to measure *strength of evidence* in favour of a model, and to have rules for choosing between models based on strength of evidence.

INDUCTION IS HARD

Induction is the most perplexing part of data science. It's not tricky maths that makes it hard, it's core epistemological problems of what we mean by probability and confidence and evidence. The vocabulary we use to talk about these ideas is a conceptual minefield.

For example, suppose a weather forecaster tells us that the probability of rain tomorrow is 60%. As data scientists we understand that there's a difference between probability and confidence (see the coin tossing example), so we ask the forecaster how confident they are. They'll probably reply, witheringly, "I told you, 60%!" There are actually two types of uncertainty here (uncertainty about the probability of rain, and uncertainty about whether or not it will actually rain), and everyday language doesn't capture the subtle difference between them.

The machine learning community has by and large struggled to grapple with confidence,[44] perhaps because of this minefield of vocabulary. In section 3.2 we discussed neural networks for classification, and we saw that they produce probability estimates: for example, given an input image x, we might get an output vector

$$\hat{p}(x) = \big[\hat{p}_{\mathsf{panda}}(x), \hat{p}_{\mathsf{gibbon}}(x), \dots\big].$$

If we feed some particular image into our network and report

$$\mathbb{P}\big(\text{image label} = \mathsf{panda}\big) = 0.577$$

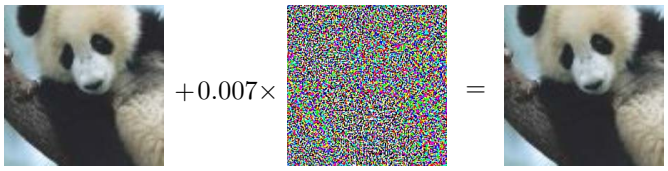
we're reporting our estimated probability value, and this is *not the same* thing as confidence. A confidence statement might say for example

$$\mathbb{P}\big(\text{image label} = \mathsf{panda}\big) = 0.577 \pm 0.15.$$

[44] There are exceptions. For example, there is a strand of work at Cambridge bringing Bayesianism into deep learning: Yarin Gal and Zoubin Ghahramani. "Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning". In: *ICML*. 2016. URL: https://arxiv.org/abs/1506.02142.

The example below, taken from an otherwise ingenious paper, demonstrates the terminology muddle.

The adversarial panda[45]



`panda`, 57.7% confidence

`nematode`, 8.2% confidence

`gibbon`, 99.3% confidence

A neural network was trained to classify images. When shown the leftmost image, it reports "`panda, 57.7% confidence`". The center image is carefully chosen noise. By blending the original panda with noise at 0.7% opacity, we obtain the rightmost image, which the neural network interprets as "`gibbon, 99.3% confidence`".

[45] I. J. Goodfellow, J. Shlens, and C. Szegedy. "Explaining and Harnessing Adversarial Examples". In: *ArXiv e-prints* (Dec. 2014). arXiv: `1412.6572 [stat.ML]`

8. Bayesianism

Bayesianism is one of the three great schools of induction. This section describes what we might call puritanical Bayesianism, a strict and mathematically coherent view of modelling. Practical data scientists tend to be pragmatic Bayesians, combining elements of Bayesian and frequentist and empirical thinking.

There are two ways into Bayesianism. If you prefer to start with the big picture, read sections 8.1–8.2 first for a philosopher's introduction. If you like HOWTOs, if you like to start with actual calculations to see how things are done so you have a grounding to ask why, then read sections 8.3–8.4 first. And then go back to read sections 8.1–8.2, because it's very easy to slip into doing calculations without really understanding what the results mean.

8.1. Bayesianist epistemology

Bayes's rule (Rev'd Thomas Bayes, 1701–1761).

$$\Pr_X(x \mid Y = y) = \frac{\Pr_X(x)\Pr_Y(y \mid X = x)}{\Pr_Y(y)}$$

Bayesianism. Thou shalt measure the strength of thy beliefs using numbers, and these numbers shall be as probabilities.

Representing belief. Bayesianism says that if there's a belief we might hold about the world, but we're not certain about that belief, then we should represent the strength of our belief by a number, and that number should behave like a probability.

For example, suppose we have a coin which we think might be biased, and let θ be the probability of heads. Suppose we're uncertain about the value of θ, but we have some prior belief. For example, we might say to ourselves "I think it's pretty likely that it's a normal fair coin, but I've recently been to a magician's fair and I might have been given a stage coin that has 95% chance of coming up heads." As Bayesianists, we'll insist on attaching a probability to each of these possibilities: we might believe for example that $\theta = 0.5$ with probability 90%, and $\theta = 0.95$ with probability 10%.

It's cleanest to represent our beliefs using a random variable. For the coin toss example above, let Θ be a random variable, with $\mathbb{P}(\Theta = 0.5) = 0.9$ and $\mathbb{P}(\Theta = 0.95) = 0.1$; random variables are convenient notation to record both the possible values and their likelihoods. If we had different beliefs, we'd use a different distribution: for example we could express complete equanimity between all possible values by using $\Theta \sim \text{Uniform}[0, 1]$; or we could use $\Theta \sim \text{Beta}(5, 5)$ to express the belief that θ is likely to be close to 0.5 but that other values in $[0, 1]$ is still possible. Whatever distribution we choose, whether discrete or continuous, the likelihood function $\Pr_\Theta(\theta)$ is what measures the strength of our belief in a particular possibility $\Theta = \theta$.

Summary of the Bayesianist creed

1. When we have a model with unknown parameters, we should represent those unknown parameters by random variables, so as to capture our degree-of-belief in each of the possible values that it might take.
2. We need to state a distribution for these random variables, before we see a dataset. This is called our *prior distribution*. It is subjective; it's meaningless to ask whether or not a prior distribution is sensible. All that matters is that we update it in a rational way.

Updating beliefs. It's rational to update our beliefs in the light of data. Bayesianists use the term *prior belief* for what they believe before seeing the data, and *posterior belief* for what the believe after.

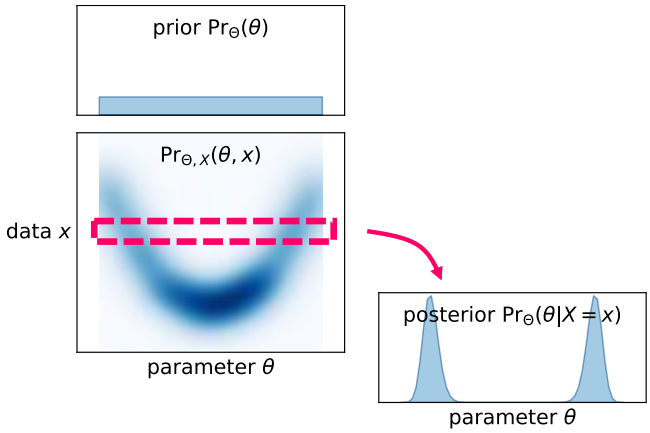
For example, suppose I toss a coin and it comes up tails, and my prior belief was that it's 90% likely to be a fair coin ($\theta = 0.5$) and 10% likely to be a magician's stage coin ($\theta = 0.95$). Intuitively, since the magician's stage coin is very unlikely to come up tails, this observed data tends to support the idea that the coin is fair. A reasonable posterior belief is that it's 99% likely to be fair, and 1% likely to be a stage coin.

What exactly is a rational basis for calculating posterior beliefs? The big idea is that the random variable for the unknown parameter (call it Θ) and the random variable for the data (call it X) are *jointly random*, i.e. they have a joint distribution. The joint likelihood can be written as

$$\Pr_{\Theta,X}(\theta, x) = \Pr_{\Theta}(\theta)\ \Pr_{X}(x \mid \Theta = \theta)$$

where $\Pr_\Theta(\theta)$ is our prior belief, and $\Pr_X(x \mid \Theta = \theta)$ is our model for how the data depends on the unknown parameter.[46] In my opinion, this is an astonishing idea: that we can use not only the same maths, but also the same probability space, to talk about both 'uncertainty about parameters of a model' and 'random outcomes of an experiment'.

In the sketch picture below, the horizontal axis is possible parameter values θ, the vertical axis is possible datapoints x, and the plot shows the joint likelihood $\Pr_{\Theta,X}(\theta, x)$. Once we've observed a particular data value x, then the only natural belief about θ is the horizontal slice through the joint likelihood plot, in other words, the conditional distribution $(\Theta \mid X = x)$. That's what Bayes's rule calculates for us.



Bayes did not invent Bayesianism. Bayesianism is a philosophy of evidence and belief that has been around since the dawn of probability in the 1660s; Laplace and de Finetti are notable figures in its development. (See section 8.6 example 8.6.2 for Laplace's Bayesianist answer to the question "will the sun rise tomorrow?", so-called Laplace smoothing.) The philosophy is called Bayesianism because it makes heavy use of Bayes's rule.

Summary of the Bayesianist creed (ctd)

4. It's rational to update our beliefs in light of evidence. We start with a *prior belief*, we see some data, and we update our beliefs to get our *posterior belief*. We do this using Bayes's rule.
5. We must have a prior belief in the first place, since without a clearly stated prior belief there is no rational basis for updating our beliefs. The prior belief might come from preconceptions, or from prior experience. (An exam question will tell you which prior to use. Real life doesn't, though Bayesian data scientists of course have common practices and rules of thumb.)

[46] For a Bayesianist, Θ is a random variable and so the hypothesis $\{\Theta = \theta\}$ is an event, so it makes sense to write the probability model as a conditional probability using the $\mid$ symbol. This is different to Part I where we wrote it as $\Pr_X(x\,;\,\theta)$ because we were simply treating θ as an unknown parameter, an extra argument to the likelihood function, not taking on the whole epistemological framework of Bayesianism.

Intellectual space versus physical space. The heart of Bayesianist thinking is that we're using a random variable to encode our beliefs, *not* to encode some property of the real world.

In every piece of modelling work, whether it's Bayesian or simple maximum likelihood estimation, we're working with a model, and this model lives inside our heads, in what we might call "intellectual space". The physical world does its own thing, heedless of our model.

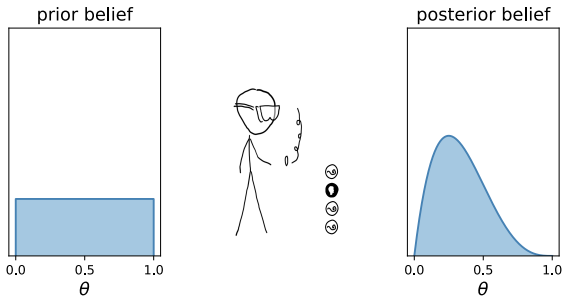
As a thought experiment, suppose that the underlying physical truth is that there's a coin and that every coin toss is independent $\text{Bin}(1, \theta_0)$ for some specific value θ_0 which Mother Nature knows but we do not. As Bayesianist modellers, perhaps we might model a sequence of coin tosses $x_1, x_2, \ldots, x_n$ as $\Theta \sim \text{Uniform}[0, 1]$, $X_i \sim \text{Bin}(1, \Theta)$. *This model lives entirely inside our intellectual space,* and it's sheer luck that both Mother Nature and we as modellers are even using the same binomial model. The random variable Θ is there as an accounting trick to encode the uncertainty we have about the parameter. Out in the physical world, there's no random parameter at all, there's only Mother Nature's fixed parameter θ_0.

The only link between physical space and intellectual space is via data: the true physical process is what generates data, and the data is what governs how we as Bayesianists update our beliefs. Hopefully, the more data we have, the more our beliefs will be nudged towards the truth. But this is *not* guaranteed, as illustrated below.

ILLUSTRATIONS

The Bayesian update. Consider a (possibly biased) coin, which has been tossed 4 times, resulting in 1 head and 3 tails. Let's model the number of heads as $X \sim \text{Bin}(4, \theta)$, where θ is an unknown parameter that we'd like to estimate from the data. Since θ is unknown, we'll treat it as a random variable Θ.

Example 8.1.1. This person happens to have chosen $\Theta \sim \text{Uniform}[0, 1]$ as their prior belief, shown as the histogram on the left.

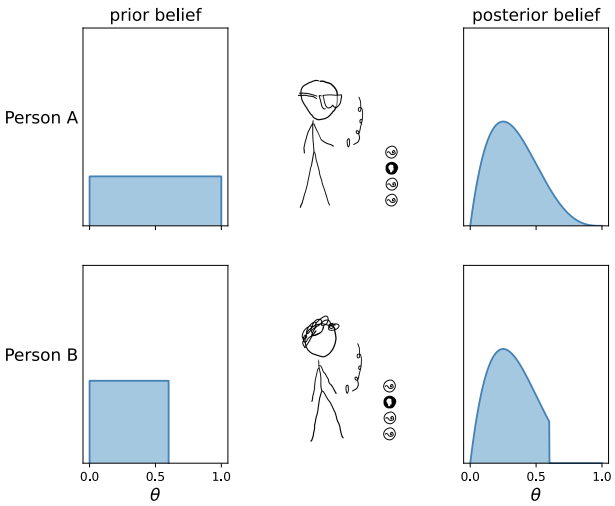


In the light of the data they update their beliefs, show as the histogram on the right. Their posterior belief puts more weight on values around $\theta = 1/4$ and less weight on values further away.

Subjectivity of prior belief. One article of faith for Bayesianists is that prior beliefs are subjective, and that they should be settled prior to seeing the data in question, based for example on prior experience or preconceptions. The following illustration shows how different prior beliefs lead to two different posterior beliefs.

Consider a (possibly biased) coin and the same data as above, 1 head and 3 tails. These tosses have been observed by two Bayesianists, both of whom are uncertain about the probability of heads, call it θ, and who consequently represent their uncertainty using a random variable Θ. Our two Bayesianists have chosen different prior distributions, and so they end up with different posterior distributions:

Example 8.1.2. Person A chooses $\Theta \sim \text{Uniform}[0, 1]$ as their prior belief: they believe that any value is equally likely. Person B is absolutely convinced that $\theta \leq 0.6$, and chooses $\Theta \sim \text{Uniform}[0, .6]$ as their prior.

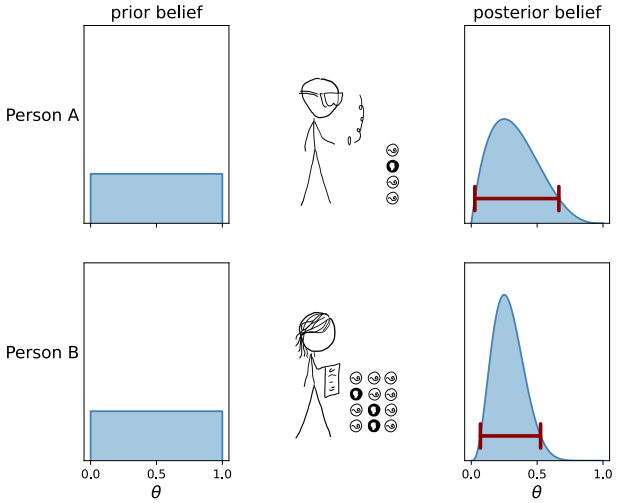


Both of them update their beliefs to put more weight on values of θ near to $1/4$. But because they started out with different prior beliefs, they end up with different posterior beliefs. It's *impossible* for person B to end up believing that $\Theta > 0.6$, no matter how many heads they see, since their prior belief said that that was impossible.

Posterior belief and confidence. In Bayesianism, we represent our beliefs about an unknown parameter by using a random variable. In particular, the posterior distribution of this unknown parameter encodes absolutely everything we've learnt from the data—and this includes uncertainty.

Consider a (possibly biased) coin being experimented upon by two Bayesianists, who both have the same prior belief about the probability of heads, but who have collected different amounts of data. They both happen to have seen the same proportion of heads, so they'd both give the same maximum likelihood estimate (25% chance of heads)—but person B has more data than person A, and this results in B's posterior distribution being "tighter" than A's. We can interpret this tightness as a measure of confidence.

Example 8.1.3. Person A tosses the coin 4 times and gets 1 head, and person B tosses it 12 times and gets 3 heads. They both start out believing that all values for θ are equally likely, so they both use a random variable $\Theta \sim \text{Uniform}[0, 1]$ to represent their prior belief.



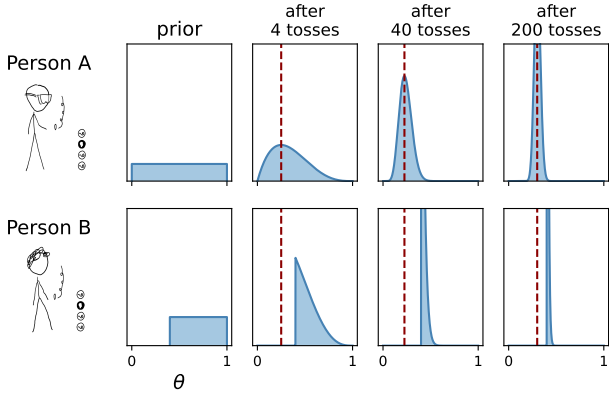
Person B has a "tighter" posterior distribution, meaning that they're more confident about the value of θ. The bars shown on the right hand plots are a simple visual way to indicate tightness; section 8.4 gives the maths for how exactly we might measure confidence.

Eventual consistency? Will our intellectual beliefs eventually agree with physical reality, as we get more and more data? Not necessarily.

In this next illustration, the physical reality is that coins tosses are generated independently, each toss coming up heads with probability 25%. Two hundred coin tosses $x_1, \ldots, x_{200}$ are observed by two Bayesianists, each believing the model $X_i \sim \text{Bin}(1, \Theta)$, and each with different prior distributions for Θ. Person A's distribution ends up more and more closely aligned with physical reality the more data they see, but person B's does not.

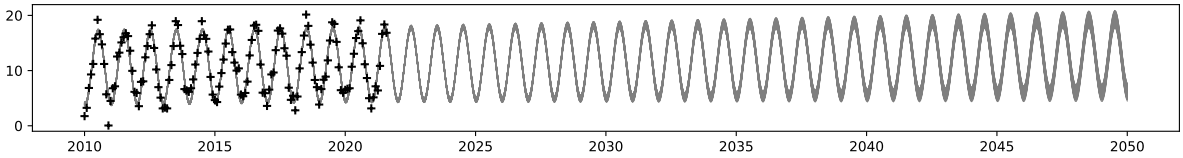
However, bearing in mind Box's famous aphorism that "all models are wrong", we shouldn't even have asked the question in the first place! Our two Bayesianists have both proposed as their model of the physical world that coin tosses are independent binomial random variables, and in the plots below that coincides exactly with the true data-generating model—but it's sheer modeller's luck that they coincide. For example, if the coin tosses were generated in batches of 4 where each batch is a random permutation of $[0, 0, 0, 1]$, but our two Bayesianists stick with their independent binomial model, then the outcome would look almost identical to the below—but neither A nor B would ever be correct.

Example 8.1.4. Person A chooses $\Theta \sim \text{Uniform}[0, 1]$ as their prior belief. Person B is absolutely convinced that $\theta \geq 0.4$, and chooses $\Theta \sim \text{Uniform}[.4, 1]$ as their prior.



Person A's posterior distribution for Θ gets closer and closer to the truth ($\theta_0 = 0.25$) as the amount of data increases. Person B's posterior distribution does not: their eventual belief is the closest fit to the data *within the space of models they think are possible*.

8.2. Asking the right question



The challenge of induction is to find language to express what we have learnt from data, in a way that is neither so precise that it's incorrect ("a swan is white with probability $p = 0.889$") nor so broad that it's vacuously true ("a swan is white with probability $p \in (0, 1)$"). The Bayesianist answer to this challenge is clean and simple:

How Bayesianists formulate questions

Q. What don't we know?

Q. How do we represent it?
Answer: As a random variable.

Q. What do we report?
Anwer: Its posterior distribution.

Q. How do we find this?
Answer: Bayes's rule.

Bayesian model-crafting is all about building probability models with the right random variables. Once we've got this right, the problem almost solves itself. Let's see how this works, in the setting of our climate model for Cambridge temperatures,

$$\mathsf{Temp} \sim \alpha \sin\big(2\pi(t + \phi)\big) + c + \gamma(t - 2000) + \text{Normal}(0, \sigma^2)$$

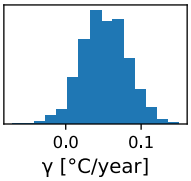
where Temp is the average temperature for month t, and t is measured in years, and the parameters α, ϕ, c, γ, and σ are unknown. In Part I we learnt how to fit the model, which answers the most basic question:

Q0. What is the estimated annual rate of temperature increase $\hat{\gamma}$?

but there are other scientific-style inference questions that we'd also like to answer:

Q1. How confident are we about our estimate for γ?
Q2. How hot might it be in August 2050?
Q3. Is there enough evidence here to reject the belief $\gamma = 0$?

Q1 (parameter uncertainty). *Parameter uncertainty can be analysed by taking the model's parameters to be random variables.* If the posterior distribution comes out to be tightly concentrated, it indicates we're very confident about its value. If it's widely spread, we're not so confident.



In the climate model above, we would choose a joint prior distribution for all the unknown parameters $(\alpha, \phi, c, \gamma, \sigma)$, then find the joint posterior distribution. If we want to report uncertainty about a particular parameter, say γ, we would find its marginal distribution from the joint posterior; a histogram for the posterior distribution of γ is shown in the margin.

Section 8.3 goes into details about finding posteriors, and section 8.4 suggests how to summarize the posterior histogram as an easily-understood confidence interval. We've already seen an illustration of this in the setting of coin tosses, example 8.1.3 page 138.

Q2 (predictions). What will be the temperature at t^* = August 2050? Our model at the top of the page proposes that the expected temperature will be

$$\mathsf{pred}(t^*) = \alpha \sin\big(2\pi(t^* + \phi)\big) + c + \gamma(t^* - 2000).$$

We are uncertain about the parameters, and *parameter uncertainty can be handled by taking the model's parameters to be random variables*, as described under Q1. This means that

$\mathsf{pred}(t^*)$ is itself a random variable, since it's simply a function of other random variables, and we can report its uncertainty for example by plotting a histogram for a specific t^*. Another way to show prediction uncertainty is as in the plot at the top of the page. There are 100 projected lines superimposed in this plot, each of them with slightly different values of the unknown parameters, sampled from their joint posterior distribution, and this shows itself as a spreading-out of the projections in later years. Section 8.5 gives more examples.

Our model doesn't just give a formula for expected temperature, it proposes that the actual observed temperature will be

$$\mathsf{Temp}(t^*) = \mathsf{pred}(t^*) + \sigma\,\mathrm{Normal}(0,1).$$

There are two sources of uncertainty here: uncertainty about the parameters, and explicit 'noise' uncertainty from the $\mathrm{Normal}(0,1)$ term. *Both sources of uncertainty should be treated as random variables.* Then it's possible to compute the combined uncertainty, for example via a histogram of $\mathsf{Temp}(t^*)$. Section 8.6 takes this idea further.

Q3 (model choice). Suppose we want to decide between two possible explanations of the data, for example the model at the top of the page *versus* the skeptic's model:

$$\mathsf{Temp} \sim \alpha \sin\big(2\pi(t+\phi)\big) + c + \mathrm{Normal}(0,\sigma^2)$$

We're uncertain about the parameter values for each of the two models, so as usual we should take them to be random variables. But there's one more thing that's unknown: which is correct out of the two models.

Model choice can be reasoned about by treating 'which model is correct' as a random variable. This will be a binary random variable, for example $M = \text{stable}$ to denote that the skeptic's model is correct and $M = \text{change}$ to denote that climate change model is correct. In other words, we're proposing the combined model

$$\mathsf{Temp} = \begin{cases} \alpha \sin\big(2\pi(t+\phi)\big) + c + \gamma(t-2000) + E, \quad E \sim N(0,\sigma^2) & \text{if } M = \text{change} \\ \alpha' \sin\big(2\pi(t+\phi')\big) + c' + E', \quad E' \sim N(0,\sigma'^2) & \text{if } M = \text{stable.} \end{cases}$$

In this model, all the parameters $(\alpha, \phi, c, \gamma, \sigma, \alpha', \phi', c', \sigma', M)$ are all random variables, the first five drawn from the climate scientist's prior, the next four from the skeptic's prior, and M drawn from the prior of whoever is doing the comparison.

After applying the Bayes update as usual we get the joint posterior distribution, from which we can extract the posterior distribution of M, i.e. the two probabilities $\mathbb{P}(M = \text{stable} \mid \text{data})$ and $\mathbb{P}(M = \text{change} \mid \text{data})$. Thus, the posterior distribution for M tells us how strongly we should believe in each of the two models in the light of data.

Section 8.7 goes into details. The algebra is involved, but the only cleverness is the insight that 'which model is correct' is an unknown parameter.

8.3. Finding the posterior

Prior and posterior distributions. Here are the concrete steps that Bayesianists follow. As in any modelling task, we start by proposing a probability model for the dataset. Let the likelihood be $\Pr_X(x \mid theta)$ where x is the data and θ is the unknown parameter or parameters.

1. Propose a random variable, call it Θ, for the unknown parameter or parameters. In other words, $\Pr_\Theta(\theta)$ measures our prior degree-of-belief that the true value is θ. This is called the *prior distribution*.

2. Use Bayes's rule to find the distribution of $(\Theta \mid X = x)$. This step is called *applying the Bayes update*, and the resulting distribution is called the *posterior distribution*.

   When you apply Bayes's rule, include all the unknowns in Θ, even if you only want to learn about one of them. (This is pragmatic guidance, not a mathematical stricture. Bayes's rule still works if you don't include all the unknowns—but students very often write down $\Pr_X(x \mid \Theta = \theta)$ incorrectly unless they include all the unknowns.) The data x should include absolutely all data that has any bearing on Θ, since it's a venial sin to discard evidence.

3. Any conclusions we want to make about the world should be expressed as statements about the distribution of $(\Theta \mid X = x)$. For example, we could plot a histogram. Other standard readouts are described in the next section.

We can apply the Bayes update either mathematically or computationally. The maths is described in section 5.2, and the computation in section 6.2. Here's a side-by-side review of both.

COMPUTATIONAL	MATHEMATICAL
(1) Take a sample $\theta_1, \ldots, \theta_m$ from Θ. (2) Compute weights $$w_i = \frac{\Pr_X(x \mid \Theta = \theta_i)}{\sum_{j=1}^m \Pr_X(x \mid \Theta = \theta_j)}$$ and (3) reason about the posterior $(\Theta \mid X = x)$ via approximate posterior probabilities $$\mathbb{P}(\Theta \in A \mid X = x) \approx \sum_{i=1}^m w_i 1_{\theta_i \in A}.$$	(1) Write down the prior likelihood $\Pr_\Theta(\theta)$. (2) Write down the posterior likelihood $$\Pr_\Theta(\theta \mid X = x) = \kappa \Pr_\Theta(\theta) \Pr_X(x \mid \Theta = \theta)$$ and find the constant κ by "densities integrate to one". Then (3) reason about the posterior $(\Theta \mid X = x)$ via its likelihood.

We'll work through two examples, showing the computational and mathematical approaches side by side. Then we'll look at some wrinkles: watching out for underflow in computational answers, and keeping the maths tractable in mathematical answers.

Exercise 8.3.1 ().
I have a coin, which might be biased. I toss it $n = 10$ times and get $x = 9$ heads. Let Θ be the probability of heads, an unknown parameter. Using the prior distribution $\Theta \sim U[0, 1]$, find the posterior distribution.

We can model the coin tosses as $X \sim \text{Bin}(n, \theta)$. So the likelihood of the observed data is

$$\Pr_X(x \mid \Theta = \theta) = \binom{n}{x} \theta^x (1 - \theta)^{n-x}, \quad x \in \{0, 1, \ldots, n\}.$$

Computational solution. *First, take a sample of values from the prior distribution:*

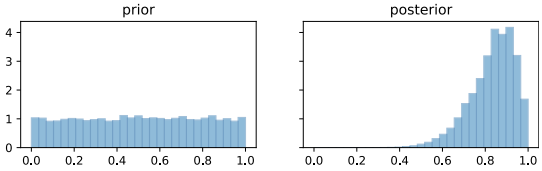
```
θ_samp = np.random.uniform(size=10000)
```

Use this to compute a weight for each sampled θ. There's no point including the constant term $\binom{n}{x}$ in prx *since it'll cancel out when we normalize the weights.*

```
n,x = 10,9
prx = θ_samp**x * (1-θ_samp)**(n-x)
w = prx / np.sum(prx)
```

Finally, report the posterior distribution. Let's report it with a density histogram, which splits the θ-axis into bins, and for each bin plots a bar of height $\mathbb{P}(\Theta \in bin)$—which is just the sum of the weights for that bin.

```
plt.hist(θ_samp, density=True, bins=np.linspace(0,1,30)) # prior
plt.hist(θ_samp, weights=w, density=True, bins=np.linspace(0,1,30)) # post.
```



□

Mathematical solution. *The prior distribution is $\Theta \sim \text{Uniform}[0,1]$ which has density*

$$\Pr_\Theta(\theta) = 1.$$

The likelihood of the data is

$$\Pr_X(x \mid \Theta = \theta) = \binom{n}{x}\theta^x(1-\theta)^{n-x}.$$

Applying Bayes's rule, the posterior density is

$$\begin{aligned}\Pr_\Theta(\theta \mid X = x) &= \kappa \times 1 \times \binom{n}{x}\theta^x(1-\theta)^{n-x} \\ &= \kappa'\theta^x(1-\theta)^{n-x}.\end{aligned}$$

There's no point keeping track of the constant term $\binom{n}{x}$; we might as well amalgamate it with the κ that comes in when we apply Bayes's rule. Hopefully this reminds you of the Beta distribution—if $Z \sim \text{Beta}(\alpha,\beta)$ then it has density

Beta distribution, page 10

$$\Pr_Z(z) = B_{\alpha,\beta}\, z^{\alpha-1}(1-z)^{\beta-1}$$

where $B_{\alpha,\beta}$ is a special constant; and so, using "densities sum to one", we conclude that $(\Theta \mid X = x) \sim \text{Beta}(x+1, n-x+1)$.
□

Exercise 8.3.2 (Multiple unknowns).
We have a random sample $x = (x_1, \ldots, x_n)$ drawn from $X \sim \text{Uniform}[a, a+b]$ where a and b are unknown parameters. Using $A \sim \text{Exp}(\lambda_0)$ and $B \sim \text{Exp}(\mu_0)$ as prior distributions, where $\lambda_0 = 0.2$ and $\mu_0 = 0.1$, find the posterior distribution of B, for the data

```
x = [2, 3, 2.1, 2.4, 3.14, 1.8]
```

The question tells us the probability model for a single observation,

$$\Pr_X(x \mid A = a, B = b) = \frac{1}{b}1_{a \le x \le a+b}, \quad x \in \mathbb{R}.$$

using indicator functions to keep track of bounds: exercise 1.3.6 page 16

(In any question where the bounds of a random variable are themselves being estimated, it's helpful to use indicator functions. That way, the algebra of indicator functions forces you to keep explicit track of bounds.) The likelihood of the entire dataset is

$$\begin{aligned}\Pr(x_1, \ldots, x_n \mid A = a, B = b) &= \frac{1}{b^n} 1_{a \le x_1 \le a+b} \times \cdots \times 1_{a \le x_n \le b} \\ &= \frac{1}{b^n} 1_{a \le \min_i x_i} 1_{\max_i x_i \le a+b}.\end{aligned}$$

Computational solution. *There are two unknown parameters here. To apply Bayes's rule, our prior distribution needs to specify all unknown parameters, since* $\Pr_X(data \mid params)$ *depends on them. Therefore we generate a random sample of* (A, B) *values. We'll assume* A *and* B *are independent. (If a question tells you about random variable distributions and doesn't explicitly say otherwise, you should take them to be independent, and state that you are doing so.)*

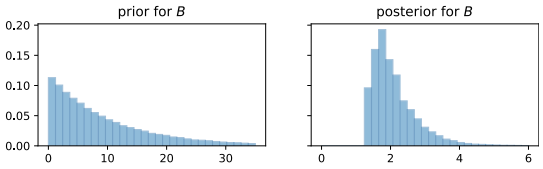
```
λ0,μ0 = 0.2, 0.1
samples = 100000
a_samp = np.random.exponential(scale=1/λ0, size=samples)
b_samp = np.random.exponential(scale=1/μ0, size=samples)
```

For every prior sample (a_i, b_i)*, we need to compute* $\Pr(dataset \mid A = a_i, B = b_i)$*. The code below uses numpy vectorized syntax: the* a_samp *and* b_samp *vectors line up, so we get a vector of* prx *values, one for each pair* (a_i, b_i)*.*

```
x = [2, 3, 2.1, 2.4, 3.14, 1.8]
n = len(x)
minx,maxx = min(x), max(x)
prx = 1/b_samp**n * np.where((a_samp <= minx) & (maxx <= a_samp+b_samp), 1, 0)
w = prx / sum(prx)
```

This question asks for the posterior distribution of B*. Our code has actually produced a weighted posterior sample of* (A, B)*. If all we want to know about is* B*, just ignore* A*. (Formally, this is finding the marginal distribution of* B*—see section 5.1.) Here is the posterior histogram of* B*, together with the prior.*

```
plt.hist(b_samp, weights=w, density=True, bins=np.linspace(0,6,30))
```



□

Mathematical solution. *There are two unknown parameters here. To apply Bayes's rule, our prior distribution needs to specify all unknown parameters, since* $\Pr_X(data \mid params)$ *depends on them. Therefore we need to write down a joint density for the pair* (A, B)*:*

$$\begin{aligned}\Pr_A(a) &= \lambda_0 e^{-\lambda_0 a} \\ \Pr_B(b) &= \mu_0 e^{-\mu_0 b} \\ \Pr_{A,B}(a, b) &= \lambda_0 \mu_0 e^{-\lambda_0 a - \mu_0 b} \quad \textit{assuming independence.}\end{aligned}$$

As before, the density function for the entire dataset is

$$\Pr(data \mid a, b) = \frac{1}{b^n} 1_{a \le \min_i x_i} 1_{\max_i x_i \le a+b}.$$

Now use Bayes's rule to find the posterior density of (A, B) *given the data:*

$$\Pr_{A,B}(a, b \mid data) = \kappa \frac{1}{b^n} e^{-\lambda_0 a - \mu_0 b} 1_{a \le \min_i x_i} 1_{a+b \ge \max_i x_i}.$$

We have gathered terms that don't involve (a, b) *into the constant* κ*, and we've flipped the indicator to emphasize that we're interested in this as a function of* a *and* b*.*

The question asks for the posterior distribution of B. We've just worked out the posterior joint distribution of (A, B), so the next step is to find the marginal for B by integrating out A.

$$\begin{aligned}\Pr_B(b \mid \mathit{data}) &= \int_a \Pr_{A,B}(a, b \mid \mathit{data})\, da \\ &= \kappa \frac{e^{-\mu_0 b}}{b^n} \int_a e^{-\lambda_0 a} 1_{a \le \min_i x_i} 1_{a \ge \max_i x_i - b}\, da \\ &= \kappa \frac{e^{-\mu_0 b}}{\lambda_0 b^n} \left(e^{-\lambda_0 \max(\max_i x_i - b, 0)} - e^{-\lambda_0 \min_i x_i} \right) 1_{b \ge \max_i x_i - \min_i x_i}.\end{aligned}$$

The last step involved some heroic integration, and even so we're stuck—this isn't any standard distribution, and it looks like it'll be intractable to find the normalizing constant. (Problems with intractable integrals like this are where computational Bayes shows its strength.)
□

Exercise 8.3.3 (Bayesian classification).
There are two types of expense claims, legitimate and fraudulent. A legitimate claim amount has distribution $X \sim \text{Exp}(\lambda_L)$, and a fraudulent claim amount has distribution $X \sim \text{Exp}(\lambda_F)$, where λ_L and λ_F are known. My prior experience tells me that 99% of claims are legitimate. A new claim comes in, for an amount x. Is it likely to be fraudulent?

We're uncertain about whether this new claim is fraudulent, and a Bayesian would say we should represent our uncertainty by a random variable. In this case we're uncertain about which of two possibilities is the case—either the claim is fraudulent or it's not—so let $\Theta \in \{\mathit{legit}, \mathit{fraud}\}$ be the random parameter. The prior distribution, stated in the question, is

$$\Pr_\Theta(\mathit{legit}) = 0.99, \qquad \Pr_\Theta(\mathit{fraud}) = 0.01.$$

Given the value of this parameter, the likelihood of the data $\Pr_X(x \mid \Theta = \theta)$ is as stated in the question. Now we can just go ahead and solve for the posterior distribution of $(\Theta \mid X = x)$. The calculations are the same as in exercise 5.2.3, and the conclusion is

$$\mathbb{P}(\Theta = \mathit{fraud} \mid X = x) = \frac{(1-p)\lambda_F e^{-\lambda_F x}}{p\lambda_L e^{-\lambda_L x} + (1-p)\lambda_F e^{-\lambda_F x}}.$$

The maths is easy; what's harder is recognizing the random parameter in the first place.
□

THINGS TO WATCH OUT FOR

There's no free lunch. In the computational approach, we usually need to put in some thought to avoid underflow, since $\Pr(\text{dataset} \mid \Theta = \theta)$ will be infinitesimally small when the dataset is large. In the mathematical approach, as we've just seen, we have to make deft choices about distributions if we want our answers to be tractable.

Exercise 8.3.4 (Avoiding underflow).
I toss a coin, which might be biased. I toss it $n = 10$ times and get $x_1 = 9$ heads. I do four more repeats of this experiment and get $[x_2, x_3, x_4, x_5] = [9, 10, 8, 10]$ heads. Let Θ be the probability of heads, an unknown parameter. Using the prior distribution $\Theta \sim \text{Uniform}[0, 1]$, find the posterior distribution.

This is nearly the same as exercise 8.3.1, except that the likelihood function is slightly different. We want the likelihood of the entire dataset $x_1, \ldots, x_5$, which is

$$\begin{aligned}\Pr(x_1, \ldots, x_5 \mid \Theta = \theta) &= \Pr(x_1 \mid \Theta = \theta) \times \cdots \Pr(x_5 \mid \Theta = \theta) \\ &\qquad \textit{assuming successive trials are independent} \\ &= \textit{const} \times \theta^s (1-\theta)^{5n-s} \qquad \textit{where } s = \sum_i x_i.\end{aligned}$$

In the computational approach we take samples $\theta_1, \ldots, \theta_m$ from the prior distribution, compute weights, then normalize them to sum to 1:

$$w_i = \theta_i^s (1-\theta_i)^{5n-s}, \qquad w_i' = \frac{w_i}{\sum_j w_j}.$$

The trouble is that these pre-normalization weights can be tiny, when n or s is large, and summing tiny values can lead to problems with numerical accuracy. A better approach is to use the so-called 'log-sum-exp' trick: instead of computing $\sum_j w_j$ directly,

$$w_i' = \frac{w_i}{\sum_j w_j} = \frac{e^{\log w_i}}{\sum_j e^{\log w_j}} = \frac{e^{\log w_i - M}}{\sum_j e^{\log w_j - M}} \quad \textit{where} \quad M = \max_j \log w_j.$$

The equation is true for any value of M, but this particular choice ensures that the largest term in the sum is 1, so the sum is numerically well-behaved. In code,

```
θ_samp = np.random.uniform(size=10000)
n = 10
sx = np.sum([9,9,10,8,10])
logprx = sx * np.log(θ_samp) + (5*n-sx) * np.log(1-θ_samp)
w = np.exp(logprx - max(logprx))
w = w / np.sum(w)
```

Then we can plot a density histogram as before.
□

Exercise 8.3.5 (Conjugate prior).
Here is an generalisation of exercise 8.3.1. That exercise used a uniform prior distribution, but now we will use a more expressive distribution.

I have a coin, which might be biased. I toss it n times and get x heads. Let θ be the probability of heads. Using the prior distribution $\Theta \sim \text{Beta}(\alpha_0, \beta_0)$ for given constants $\alpha_0 > 0$ and $\beta_0 > 0$, show that the posterior distribution is

$$(\Theta \mid x) \sim \text{Beta}(\alpha_0 + x, \beta_0 + n - x).$$

(The prior distribution was Beta, and we find that the posterior is another Beta but with different parameters. When the probability model for X and the prior distribution for Θ match up nicely, the posterior ends up in the same family as the prior but with different parameters. We refer to this as a *conjugate prior*.)

* * *

In well-chosen models, it shouldn't matter too much what the prior is. For example, in exercise 8.3.5, if n is very large then α_0 and β_0 have little influence. If we have too little data then the prior distribution will have a big impact on our answer. Bayesianism makes it easy to crank a handle and get out answers—but you should always stop and reflect whether your answers really reflect the data or whether they just reflect the assumptions you put in. This is usually easy to see from a maths formula, much harder to see in the output of a computation.

8.4. Readouts from posterior distributions

You're working on a dataset. You've invented a probability model to describe the data, with some unknown parameters that are crucial to your company's business. With just a few qualms you invent a reasonable prior distribution for the parameters, and you write code to sample from the posterior distribution.

Your boss excitedly asks you "So, what values did you find for these parameters?"

"That's a category error" you reply sanctimoniously. "As a Bayesian, I consider parameters to be random variables. They are described with probability distributions. We can't pick out a single value to describe an entire distribution."

"Tell me a value, or you're fired," your boss says.

There are several standard summaries that Bayesians can report, to describe their posterior distributions.

POSTERIOR POINT ESTIMATES

If we really truly have to report a single value, here are some choices. Let $(\Theta \mid \text{data})$ be the posterior distribution that we want to describe.

- Report the value of θ that maximizes $\Pr_\Theta(\theta \mid \text{data})$. This is called the *maximum a posteriori* (MAP) estimate—a fancy name to give this estimate the façade of rigour. It's easy to see that if the prior distribution is uniform, then the MAP estimate is exactly the same as the maximum likelihood estimate.
- If Θ is a simple $\mathbb{R}$-valued random variable, we could just report a straightforward summary of $(\Theta \mid \text{data})$ such as its mean $\mathbb{E}(\Theta \mid \text{data})$ or its median.
- In some problems there is a natural *loss function* $L(\phi, \theta)$, which measures the price you pay if you report estimate ϕ and the true value is θ. Then you should report the estimate $\hat{\theta}$ that minimizes the *expected posterior loss*,

$$\hat{\theta} = \arg\min_{\phi} \mathbb{E}\big(L(\phi, \Theta) \mid \text{data}\big)$$

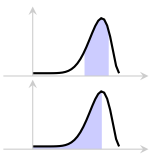
  where the expectation is over the random variable Θ. With some reasonably simple calculus one can show that for $L(\phi, \theta) = (\phi - \theta)^2$ the solution is to report the posterior mean, and for $L(\phi, \theta) = |\phi - \theta|$ the solution is to report the posterior median.

Computationally, if we have a vector of samples θ_samp with an associated vector of posterior weights w, then

- the MAP estimate is easy, θ_samp[np.argmax(w)]
- the posterior mean is easy, np.sum(θ_samp * w)
- the posterior median can be found by

```
1  i = np.argsort(θ_samp)
2  θ_samp,w = θ_samp[i], w[i]          # sort so θ is ascending
3  F = np.cumsum(w)                    # cumulative distribution
4  post_median = θ_samp[F>=0.5][0]     # pick smallest θ such that F ≥ 0.5
```

POSTERIOR CONFIDENCE INTERVALS



A 95% confidence interval for a random variable Y is an interval $[\mathsf{lo}, \mathsf{hi}]$ such that $\mathbb{P}(Y \in [\mathsf{lo}, \mathsf{hi}]) = 0.95$. A good way to express our uncertainty about the unknown parameter is by giving a confidence interval for it, also called a *posterior confidence interval*: report an interval $[\mathsf{lo}, \mathsf{hi}]$ such that

$$\mathbb{P}\big(\Theta \in [\mathsf{lo}, \mathsf{hi}] \;\big|\; \text{data}\big) = 0.95.$$

A common choice is to report a two-sided interval, choosing lo and hi so that

$$\mathbb{P}(\Theta < \mathsf{lo}) = 0.025 \quad \text{and} \quad \mathbb{P}(\Theta > \mathsf{hi}) = 0.025.$$

But other choices are just as legitimate, for example the one-sided confidence interval where lo is the smallest possible value of Θ and hi is chosen so that $\mathbb{P}(\Theta > \text{hi}) = 0.05$.

Computationally, if we have a collection of sample values θ_samp with posterior weights w,

```
i = numpy.argsort(θ_samp)
θ_samp, w = θ_samp[i], w[i]
F = numpy.cumsum(w)
# two-sided 95% confidence interval
(lo, hi) = (θ_samp[F<0.025][−1], θ_samp[F>0.975][0])
```

Exercise 8.4.1. For the two scientists tossing coins on page 131, assuming they each have a uniform prior distribution for the probability of heads, find 95% confidence intervals.

According to the maths for exercise 8.3.1 on page 143, the first scientist's posterior distribution is $\text{Beta}(2, 4)$*, and the second's is* $\text{Beta}(4, 10)$*. We can find two-sided confidence intervals using the inverse cdf for the Beta distribution:*

ppf function: see page 8

```
# First scientist – 1 head, 3 tails
postA = scipy.stats.beta(a=2, b=4)
(lo, hi) = postA.ppf(.025), postA.ppf(.975)    # (lo, hi) = (.05, .72)

# Second scientist – 3 heads, 9 tails
postB = scipy.stats.beta(a=4, b=10)
(lo, hi) = postB.ppf(.025), postB.pdf(.975)    # (lo, hi) = (.09, .54)
```

□

8.5. Bayesian regression

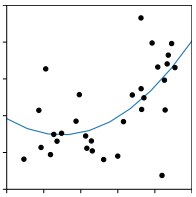
Suppose we have a collection of datapoints (x_i, y_i) and we want to predict y as a function of x. The natural starting point is to propose a probability model for the data. For example, we might propose a simple model based on a quadratic curve,

$$Y \sim \alpha + \beta x + \gamma x^2 + N(0, \sigma^2).$$

We'd then like to estimate the response function, in this case $\mu(x) = \alpha + \beta x + \gamma x^2$.

The obvious thing to do is to find the maximum likelihood estimates $\hat{\alpha}$, $\hat{\beta}$ and $\hat{\gamma}$, and to simply plot $\hat{\mu}(x) = \hat{\alpha} + \hat{\beta}x + \hat{\gamma}x^2$.

MLE



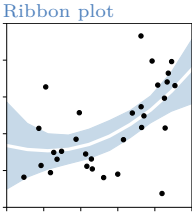
But what about uncertainty? It's crazy to think that $\hat{\mu}(x)$ is the exact true response function, since it's a single estimate from noisy data. How might we compute uncertainty, and how might we plot our fitted function to communicate this? The Bayesianist approach tells us how to start:

1. The fundamental tenet of Bayesianism is that we should represent uncertainty by treating our unknowns as random variables. Here, the unknowns are the four parameters, $\Theta = (\alpha, \beta, \gamma, \sigma)$.
2. We must specify a prior distribution for these parameters, and this prior distribution should be specified prior to seeing the dataset we're trying to fit. It may seem preposterous to have a prior belief about something as unmoored as a toy dataset in a textbook—but that's what Bayesianism demands. We just have to invent something, and hope there's enough data that our answer isn't too sensitive to our prior belief.
3. The posterior distribution for these parameters can be found using the standard Bayes update.

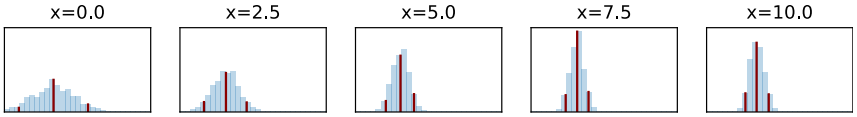
Next, how should we communicate our posterior uncertainty? According to the underpants theory of modelling , it's not decent to present model parameters in public—the parameters and indeed the entire probability model are fictions inside the head of the modeller. Our audience is likely to understand us better if we tell them about things in their domain: in particular, they are likely to be interested in the predictions that the model makes. In other words, we need to depict the uncertainty in $\mu(x)$.

Underpants theory: page 73

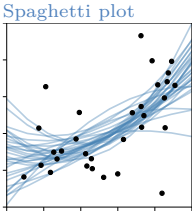
Ribbon plot



One depiction is called the *ribbon plot*. At any particular x value, we must think of $\mu(x)$ as a *random variable*. It's random because it's a function of α and β and γ, and these three are random variables. Thus, $\mu(x)$ has a distribution. In particular, if we find the distribution of $\mu(x)$ using the posterior distributions of α, β, and γ, then we'll get the posterior distribution $(\mu(x) \mid \text{data})$. At any particular x value, we can plot a histogram of the posterior of $\mu(x)$, and read off statistics such as the median or a 95% confidence interval, as illustrated below. The plot in the margin shows these three statistics (the top and bottom of the ribbon showing the 95% confidence interval, the white line in the middle showing the median), computed at a regular grid of x values.



Spaghetti plot



Another way to depict uncertainty is with a *spaghetti plot*. Take a sample of say 50 parameters $\theta_1, \ldots, \theta_{50}$ from the posterior distribution $(\Theta \mid \text{data})$. Each $\theta_i = (\alpha_i, \beta_i, \gamma_i, \sigma_i)$ corresponds to a particular response function $\mu_i(x) = \alpha_i + \beta_i x + \gamma_i x^2$. The plot in the margin shows 50 such response functions. Another way to think of this is that the *entire function* $\mu(\cdot)$ is a function-valued random variable, and we're plotting a sample $\mu_1(\cdot), \ldots, \mu_{50}(\cdot)$ drawn from $\mu(\cdot)$.

COMPUTATION

For the plots shown here, I have followed the computational method for finding Bayesian posteriors. First, propose a prior distribution Θ for the unknown parameters $(\alpha, \beta, \gamma, \sigma)$ and take m samples, θ_i for $i = 1, \ldots, m$. Each sample θ_i corresponds to a different version of the response function, call it $\mu_i(\cdot)$. For each θ_i compute a posterior weight w_i.

For the ribbon plot: for each x on a regular grid, compute a sample of m predicted values, $y_i = \mu_i(x)$ for $i = 1, \ldots, m$. Using the weights w_i, compute the median and a 95% confidence interval as described in section 8.4.

For the spaghetti plot: pick say 50 of the $\mu_i(\cdot)$ at random, where the chance of picking $i \in \{1, \ldots, m\}$ is w_i, and just plot them. Alternatively, plot all m of them, giving linewidth w_i to $\mu_i(\cdot)$.

8.6. Posterior prediction

According to the underpants theory of modelling, the parameters of a model and indeed the entire model itself are just fictions in the head of the modeller; so rather than report parameters we should report quantities that are meaningful in the real world. In this section we will see how we can report likely *outcomes* of future observation. For example,

- *The generative case.* Consider a possibly biased coin, in which we're modelling coin tosses as independent $\text{Bin}(1, \Theta)$. How might we tell our audience about the likely outcome of a new coin toss, informed by the coin tosses we've previously observed?

  Following section 8.4 we could say "The new coin toss will be heads with probability θ and my confidence interval for θ is such-and-such"; but that's showing underpants. Can we give a direct answer about the outcome, not an indirect answer about the model parameter?

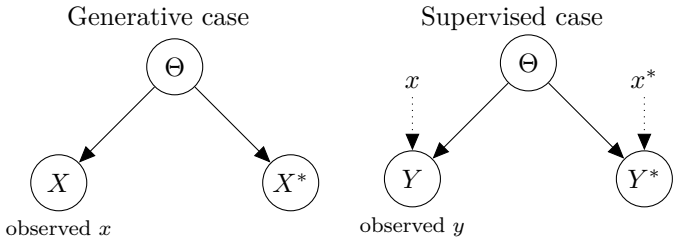
- *The supervised case.* In section 8.5 we considered a regression model $Y \sim \mu(x) + N(0, \sigma^2)$. How might we tell our audience about the outcome of a future observation at some new x^*, informed by the (x, y) data we've already observed?

  We could simply report that the new outcome is sampled from $N(\hat{\mu}(x^*), \hat{\sigma}^2)$, where $\hat{\mu}$ and $\hat{\sigma}$ are maximum likelihood estimates from the previously observed data. But it's naive to report estimates from noisy data without giving any indication about our confidence in those estimates. We saw in section 8.5 how to report our confidence about the expected outcome $\mu(x^*)$; but what the actual outcome, which has extra randomness thanks to the $N(0, \sigma^2)$ term?

There's a very simple answer, and a complicated answer. The simple answer is this: (1) Use the previously observed data to get the posterior distribution of the unknown parameter or parameters, and take a single sample θ from this distribution; then using θ generate a new outcome. (2) Repeat this many times, to get lots of samples of the new outcome. (3) Report the histogram.

This histogram tells us what the new outcome is likely to be, and it incorporates both our uncertainty about the unknown parameters and also the noise inherent in a new observation. These two types of uncertainty are called *epistemic* and *aleatoric* respectively. The word 'epistemic' means 'of or relating to knowledge', and it refers to how much we know about the parameters of our probability model. The more data we have collected, the more we know about Θ. The word 'aleatoric' means 'dependent on uncertain contingencies'. Even if we knew Θ perfectly, there'd still be irreducible randomness involved in the new coin toss.

Now for the complicated answer, using maths notation. The point of this complicated answer is twofold: it makes it clear why the simple answer works; and it gives us further options beyond brute-force simulation and plotting histograms.



In the generative case, x denotes the entire dataset we've observed, and X^* is the future outcome we're interested in. X and X^* don't need to be the same type: for example, in a gambling game, X might be a detailed dataset of all past outcomes, and X^* might be a particular winning event for a future gamble. The supervised case is much the same; there's just some extra notation to keep track of the predictor variables.

The diagram declares that we're modelling X and X^* as related *only* through Θ. In words, once we know Θ, the further details of X are irrelevant for the purpose of predicting X^*. In maths, we say that X and X^* are *conditionally independent* given Θ. In likelihood notation: for any x and x^* and θ,

$$\Pr_{X^*}(x^* \mid \Theta = \theta, X = x) = \Pr_{X^*}(x^* \mid \Theta = \theta)$$

or equivalently the joint likelihood can be written as a product of two factors,

$$\Pr_{X,X^*}(x, x^* \mid \Theta = \theta) = \Pr_X(x \mid \Theta = \theta)\ \Pr_{X^*}(x^* \mid \Theta = \theta).$$

We can reason about the posterior distribution of X^*, using the law of total probability and conditional independence:

$$\begin{aligned}\Pr_{X^*}(x^* \mid X = x) &= \int_\theta \Pr_{X^*}(x^* \mid \Theta = \theta, X = x) \Pr_\Theta(\theta \mid X = x)\, d\theta \quad \text{by LoTP} \\ &= \int_\theta \Pr_{X^*}(x^* \mid \Theta = \theta) \Pr_\Theta(\theta \mid X = x)\, d\theta \quad \text{by cond. ind.}\end{aligned}$$

This can be written rather densely as

$$= \mathbb{E}\left[\Pr_{X^*}(x^* \mid \Theta) \;\middle|\; X = x\right].$$

The inner term $\Pr_{X^*}(x^* \mid \Theta)$ is random because it's a function of Θ, which is a random variable. The outer expectation is an average over Θ given the observed data, i.e. it's an average over the posterior distribution of Θ.

Example 8.6.1 (Posterior predictive probability).
We have a biased coin with unknown probability of heads θ. Consider the two scientists tossing coins on page 131, and suppose they each have a uniform prior distribution for θ. Given their observed data, what is the probability that the next coin will be heads? What is the probability that the next two coins will be heads?

These are called *posterior predictive probabilities*. They're posterior because they're conditional on the observed data. They're predictive because they're asking about a future observation.

using indicator functions to estimate probabilities: page 103

For the first question: Let X^* be the indicator that the next coin toss is heads. The question is asking us for $\mathbb{P}(X^* = 1 \mid X = x)$, where x is the observed data, in this case the number of heads in 10 coin tosses. The general formula tells us

$$\begin{aligned}\mathbb{P}(X^* = 1 \mid X = x) &= \int_\theta \mathbb{P}(X^* = 1 \mid \Theta = \theta)\ \Pr_\Theta(\theta \mid X = x)\, d\theta \\ &= \int_\theta \theta\ \Pr_\Theta(\theta \mid X = x)\, dx \\ &= \mathbb{E}(\Theta \mid X = x) \quad \textit{by definition of expectation.}\end{aligned}$$

We learnt the computational method for answering this in section 6.2. Writing out the method in terms of the variables we have here,

$$\mathbb{E}(\Theta \mid X = x) \approx \sum_{i=1}^m w_i \theta_i$$

where $(\theta_1, \ldots, \theta_m)$ is a sample from the prior distribution and w_i are the associated posterior weights. We computed these in exercise 8.3.1. All that's left to is compute the sum:

$$\sum_{i=1}^m w_i \theta_i = \texttt{np.sum}\big(\texttt{w * } \theta\texttt{\_samp}\big).$$

For the second question: let $X^\dagger$ be the indicator that the next two coin tosses are heads. The question is asking us for $\mathbb{P}(X^\dagger = 1 \mid X = x)$. Using exactly the same reasoning as before, this is

$$\mathbb{P}(X^\dagger = 1 \mid X = x) = \mathbb{E}(\Theta^2 \mid X = x) \approx \sum_{i=1}^m w_i \theta_i^2 = \texttt{np.sum}\big(\texttt{w * } (\theta\texttt{\_samp ** 2})\big).$$

□

Example 8.6.2 (Laplace smoothing).
I have lived for $n = 10,585$ days, and on every one of these days the sun rose in the morning. What is the probability it will rise tomorrow morning?

The calculation to answer this is easy, it's the formulation that's hard. Here's how Laplace formulated it. Let X_i be the indicator that the sun rose on day i, $X_i \sim \text{Bin}(1, \Theta)$, and let Θ be the probability of the sun rising. Treat Θ as a random variable with prior distribution $U[0, 1]$. Let X^* be the indicator that the sun will rise tomorrow. Calculate the posterior predictive probability that $X^* = 1$, i.e. $\mathbb{P}(X^* = 1 \mid \text{data})$.

The prior is

$$\Pr_\Theta(\theta) = 1.$$

The likelihood of a single observation X is

$$\Pr_X(x \mid \Theta = \theta) = \begin{cases} \theta & \text{if } x = 1 \\ 1 - \theta & \text{if } x = 0 \end{cases}$$

and the likelihood of the entire dataset consisting of n ones is

$$\Pr(\textit{data} \mid \Theta = \theta) = \theta^n.$$

Thus the posterior is

$$\Pr_\Theta(\theta \mid \textit{data}) = \kappa\, \theta^n \quad \text{for some constant } \kappa$$

which we recognize as the density of a Beta distribution,

$$(\Theta \mid \textit{data}) \sim \text{Beta}(n + 1, 1).$$

The question asks us for $\mathbb{P}(X^* = 1 \mid \textit{data})$. This is

$$\begin{aligned} \mathbb{P}(X^* = 1 \mid \textit{data}) &= \int_\theta \mathbb{P}(X^* = 1 \mid \Theta = \theta) \Pr_\Theta(\theta \mid \textit{data})\, d\theta \quad \textit{by general formula above} \\ &= \int_\theta \theta \Pr_\Theta(\theta \mid \textit{data})\, d\theta \quad \textit{as } X^* \sim \text{Bin}(1, \Theta) \\ &= \textit{posterior mean, by definition of mean} \\ &= \frac{n+1}{n+2} \quad \textit{by looking up the Beta distribution} \\ &= 99.99055\%. \end{aligned}$$

□

Example 8.6.3 (Decomposition of uncertainty).
Consider the regression model from section 8.5 page 149,

$$Y \sim \alpha + \beta x + \gamma x^2 + N(0, \sigma^2).$$

Suppose we have observed a collection of datapoints (x_i, y_i). Let Y^* be the outcome at some new coordinate x^*. Find a 95% confidence interval for Y^*, given the observed data.

We saw in section 8.5 how to find a 95% confidence interval for the mean response $\alpha + \beta x^* + \gamma (x^*)^2$. Putting these two together, we get a *decomposition of uncertainty*: there is some uncertainty because we don't know what the *mean* of Y^* will be, and there's further uncertainty from taking into account the randomness of Y^*.

We want an interval $[\mathsf{lo}, \mathsf{hi}]$ *such that*

$$\mathbb{P}\big(Y^* \in [\mathsf{lo}, \mathsf{hi}] \mid \textit{data}\big) = 95\%.$$

Let's try to find a symmetric confidence interval, i.e.

$$\mathbb{P}(Y^* \leq \mathsf{hi} \mid \textit{data}) = 97.5\%$$

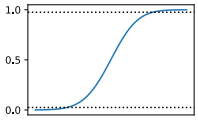
and $\mathbb{P}(Y^* < \mathsf{lo} \mid data) = 2.5\%$. *Using the law of total probability and conditional independence, and writing* Θ *for the tuple of unknown parameters* $(\alpha, \beta, \gamma, \sigma)$,

$$\begin{aligned}
\mathbb{P}(Y^* \le \mathsf{hi} \mid data) &= \mathbb{E}\big[\mathbb{P}(Y^* \le \mathsf{hi} \mid \Theta) \mid data\big] \\
&= \mathbb{E}\big[\mathbb{P}\big(N(\mu(x^*), \sigma^2) \le \mathsf{hi}\big) \mid data\big] \quad \text{where } \mu(x) = \alpha + \beta x + \gamma x^2 \\
&= \mathbb{E}\big[\Phi_{\mu(x^*),\sigma}(\mathsf{hi}) \mid data\big] \quad \text{where } \Phi_{\mu,\sigma} \text{ is the cdf for } N(\mu, \sigma^2) \\
&\approx \sum_i w_i \Phi_{\mu_i(x^*),\sigma_i}(\mathsf{hi})
\end{aligned}$$

where in the last line we are working with a sample $(\alpha_i, \beta_i, \gamma_i, \sigma_i)$ *from the prior distribution, each sample with an associated posterior weight* w_i, *and where* $\mu_i(x) = \alpha_i + \beta_i x + \gamma_i x^2$. *We want to choose* hi *to make this expression equal to 97.5%. There's no exact formula for* hi, *but it's easy to find numerically by inverting the function* $\mathbb{P}(Y^* \le y \mid data)$, *seen as a function of* y.

```
α_,β_,γ_,σ_ = ...
w_ = ...
Φ = scipy.stats.norm.cdf
xstar = 5
μ_ = α_ + β_*xstar + γ_*(xstar**2)
ygrid = np.linspace(0,15,100)
pgrid = np.average(Φ(ygrid[:,None], loc=μ_[None,:], scale=σ_[None,:]),
                   weights=w_, axis=1)
lo,hi = np.interp([.025,.975], pgrid, ygrid)
```

$\mathbb{P}(Y^* \le y \mid \text{data})$ as a function of y:



This is vectorized code. It computes a matrix of $\Phi_{\mu_i(x^*),\sigma_i}(y_j)$ *for every parameter-tuple* i *and for every sample point* y_j *on a regular grid, and then it averages over* i *to get* $p_j = \mathbb{P}(Y^* \le y_j \mid data)$. *This gives us a set of* (y_j, p_j) *pairs. Treating* p *as the predictor and* y *as the response, we can interpolate to get the approximate* y*-values such that* $p = 0.025$ *and* $p = 0.975$.

□

8.7. Model choice

Your PhD supervisor has achieved eminence for the data they have collected and the model they formulated to describe that data. A competing lab has just published a different model. Your supervisor asks you to figure which of the two models is a better fit for the data.

"That's a meaningless question" you reply sanctimoniously. "Speaking as a Bayesian, 'which model is better?' is uncertain, therefore like any other uncertainty it should be represented as a random variable. I can find a posterior distribution, if you tell me the prior."

"Just tell me which is better," your supervisor says. "You'll never make a scientific career if you can't draw conclusions."

A true Bayesian won't draw a conclusion about which model is better. But there is a nice way to write down the posterior distribution for the 'which model is better?' random variable, which is nearly as good.

Let the two models be $\mathcal{A}$ and $\mathcal{B}$, and let $m \in \{\mathcal{A}, \mathcal{B}\}$ denote which of the two models is correct. Treat m as a possible outcome of a random variable M: let $\mathbb{P}(M = \mathcal{A})$ be the prior probability that $\mathcal{A}$ is correct, $\mathbb{P}(M = \mathcal{B})$ the same for $\mathcal{B}$. In likelihood notation, the probabilities are $\Pr_M(\mathcal{A})$ and $\Pr_M(\mathcal{B}) = 1 - \Pr_M(\mathcal{A})$.

The two models may have unknown parameters. Write θ for the unknown parameters in model $\mathcal{A}$, and ϕ for the unknown parameters in $\mathcal{B}$, and assume we have set down prior distributions for them.

The first step of applying Bayes's rule is to write out the prior distribution. There are three unknowns here, M, Θ, and Φ, so we have to specify a prior joint distribution. We'll take an independent prior,

$$\Pr(m, \theta, \phi) = \Pr_M(m) \Pr_\Theta(\theta) \Pr_\Phi(\phi).$$

The second step is to write out the data density. Each of the two probability models, $\mathcal{A}$ and $\mathcal{B}$, proposes its own data density,

$$\Pr^{\mathcal{A}}(\text{data} \mid \theta) \quad \text{and} \quad \Pr^{\mathcal{B}}(\text{data} \mid \phi).$$

Let's combine these into one formula by

$$\Pr(\text{data} \mid \theta, \phi, m) = \begin{cases} \Pr^{\mathcal{A}}(\text{data} \mid \theta) & \text{if } m = \mathcal{A} \\ \Pr^{\mathcal{B}}(\text{data} \mid \phi) & \text{if } m = \mathcal{B}. \end{cases}$$

Now we can apply Bayes's rule:

$$\Pr_{M,\Theta,\Phi}(m, \theta, \phi \mid \text{data}) = \kappa \Pr_M(m) \Pr_\Theta(\theta) \Pr_\Phi(\phi) \Pr(\text{data} \mid \theta, \phi, m).$$

If we're trying to decide which model is better, we care about the posterior distribution of M, and Θ and Φ are nuisance parameters. So let's marginalize them out:

$$\Pr_M(m \mid \text{data}) = \int_{\theta,\phi} \Pr_{M,\Theta,\Phi}(m, \theta, \phi \mid \text{data}) \, d\theta \, d\phi$$

The random variable M takes only two values, $\mathcal{A}$ or $\mathcal{B}$, so let's write out $\Pr_M(m \mid \text{data})$ explicitly for these two cases:

$$\begin{aligned} \Pr_M(\mathcal{A} \mid \text{data}) &= \int_{\theta,\phi} \kappa \Pr_M(\mathcal{A}) \Pr_\Theta(\theta) \Pr_\Phi(\phi) \Pr^{\mathcal{A}}(\text{data}|\theta) \, d\theta \, d\phi \\ &= \kappa \Pr_M(\mathcal{A}) \int_\theta \Pr_\Theta(\theta) \Pr^{\mathcal{A}}(\text{data}|\theta) \, d\theta \\ \Pr_M(\mathcal{B} \mid X = x) &= \kappa \Pr_M(\mathcal{B}) \int_\phi \Pr_\Phi(\phi) \Pr^{\mathcal{B}}(\text{data}|\phi) \, d\phi \quad \text{likewise.} \end{aligned}$$

To simplify the notation, let's define what's called the *model evidence* for each of the two models,

$$\text{ev}(\mathcal{A} \mid \text{data}) = \int_\theta \Pr_\Theta(\theta) \Pr^{\mathcal{A}}(\text{data}|\theta) \, d\theta, \quad \text{ev}(\mathcal{B} \mid \text{data}) = \int_\phi \Pr_\Phi(\phi) \Pr^{\mathcal{B}}(\text{data}|\phi) \, d\phi.$$

Model evidence is just the marginal likelihood of the data, obtained by integrating out the model's parameters.

The equations are intimidating, but there's nothing more to this than simple binary classification, which we've seen several times already. We can find κ using the "densitites sum to one" rule, summing over two values, $\Pr_M(\mathcal{A} \mid \text{data}) + \Pr_M(\mathcal{B} \mid \text{data}) = 1$. This gives

Bayesian classification: exercise 5.2.3 page 92, and again exercise 8.3.3 page 145

$$\Pr_M(\mathcal{A} \mid \text{data}) = \frac{\Pr_M(\mathcal{A})\ \text{ev}(\mathcal{A} \mid \text{data})}{\Pr_M(\mathcal{A})\ \text{ev}(\mathcal{A} \mid \text{data}) + \Pr_M(\mathcal{B})\ \text{ev}(\mathcal{B} \mid \text{data})}$$

and similarly for $\Pr_M(\mathcal{B})$.

In words, we start with a prior belief $\Pr_M(m)$ about which model is correct, and we update that belief in the light of the data, according to the evidence for each model. Evidence is a scaling factor, used to reweight your prior belief about which model is correct. Don't try too hard to interpret the absolute value of evidence.

Don't use Bayes factors. The ratio

$$K = \frac{\Pr_M(\mathcal{A} \mid \text{data})}{\Pr_M(\mathcal{B} \mid \text{data})}$$

is called the *Bayes factor*. It's a simple rewrite of the posterior probabilities,

$$\Pr_M(\mathcal{A} \mid \text{data}) = \frac{K}{1+K}, \quad \Pr_M(\mathcal{B} \mid \text{data}) = \frac{1}{1+K}.$$

Some people use the Bayes factor as grounds for deciding between the two models, for example saying "If $K > 10$ then there is strong evidence in favour of model $\mathcal{A}$." This sort of language is weaselly. It gives the impression of making an inductive claim about model choice, but without having the guts to actually make that claim explicit.

A true Bayesianist would never[47] say "Use Model $\mathcal{A}$ rather than Model $\mathcal{B}$", they would only say "In the light of the data, and given prior beliefs, here are the updated weights to use for each of the models." Their predictions about new observations would be posterior predictions, as in section 8.6, averaging over all unknown parameters including M. This is called *Bayesian model averaging*.

Don't use BIC. There is an asymptotic expression for model evidence, called the *Bayesian Information Criterion* or BIC. In the limit where the number of observations n is large and the number of model parameters k is fixed, the evidence for a model is

$$\text{ev}(\text{model} \mid \text{data}) \approx e^{-\text{BIC}/2} \quad \text{where} \quad \text{BIC} = k \log n - 2 \log \Pr(\text{data} \mid \hat{\theta})$$

and $\Pr(\text{data} \mid \hat{\theta})$ is the maximized likelihood of the data under that model's parameters.

What's lovely about BIC is that it shows very clearly the impact of adding extra parameters. We can always make a model fit better (in the sense of getting a larger likelihood) by adding more parameters, but the BIC formula shows that adding more parameters also has a negative impact on model evidence.

Some people use BIC as a tool for model choice: given a set of alternative models, they selet the model with lowest BIC. This is inappropriate, for the same reason that model selection using Bayes factors is inappropriate. True Bayesianists do not choose between models.

[47] There is one exception. If one has a loss function over models, as in secction 8.4, then it is reasonable to select a single model so as to minimize posterior loss. I have never come across a data science situation where there is a plausible loss function over models.

9. Frequentism

Whenever we look at a dataset, we should also imagine the 'counterfactual multiverse' of all the possible ways the dataset might have turned out but didn't. This is what a probability model *is*. There is a school of inference, called the frequentists, who express what they have learnt from data by reference to this multiverse.

Frequentists like the Ancient One and Dr Strange don't just see the world in front of them (the observed data x), they also see the multiverse of all that might have been (the sample space of the random variable X).

Bayesians like Tony Stark and Princess Shuri, the consummate engineers, see the data that's given (x) and consider various explanations ($\Theta \mid X = x$).

To be concrete, let x be the dataset we observe, and let X be a random variable representing all the ways that the dataset might have turned out. Imagine an infinite collection of parallel universes[49], each with its own dataset drawn from distribution X. Perhaps we're interested in some parameter, and we computed a maximum likelihood estimate using the dataset we have—then in each parallel universe there will be a parallel data scientist who has computed their own estimate; and who is to say that ours is better than theirs? If all the estimates end up very close to each other then we should be confident in the result, and if there's a wide spread then we should not be confident.

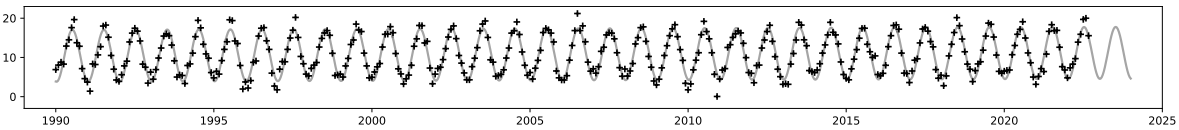
The difficulty, of course, is that we're stuck inside our own universe and we can't actually see the outcomes in all the parallel universes. As frequentists, we therefore need tools that allow us to use the data we have available in our universe, in order to reason about the multiverse. The Ancient One really could see the true distribution of X, but the humble frequentist has to find work-arounds to simulate it.

* * *

Both Bayesians and frequentists work with parametric models $\Pr_X(x\,;\,\theta)$, where θ denotes the unknown parameters. Bayesians deal with model uncertainty by treating the unknowns as a random variable Θ, and reasoning about their degree-of-belief $\Pr_\Theta(\theta)$. They see inference as a system of rules for manipulating subjective beliefs. Frequentists on the other hand believe in objective truth. They say that the true value of θ is fixed and non-random—Nature knows it but keeps it hidden close to her chest—and instead they reason about model uncertainty by sampling the range of opinions of data scientists across the multiverse.

[49]The word 'frequentism' comes from interpreting $\mathbb{P}(X = x)$ as the frequency of outcome x across all these parallel universes.

9.1. Parametric resampling



We've seen this climate dataset many times, and we've fitted the model

$$\mathsf{Temp}_i \sim \alpha \sin\big(2\pi(t_i + \phi)\big) + c + \gamma t_i + \text{Normal}(0, \sigma^2), \quad i = 1, \ldots, n$$

and obtained the estimate $\hat{\gamma} = 0.0315°\text{C/year}$. But how confident should we be about this estimate? Is it more like $0.0315 \pm 0.001°\text{C/year}$, or more like $0.0315 \pm 0.4°\text{C/year}$?

The frequentist philosophy is that we should simulate a multiverse of all the ways that the dataset *might* have turned out, ask what $\hat{\gamma}$ a parallel data scientist would have computed in each of them, and report the spread. And it's easy to simulate how the dataset might have turned out: we can just plug in the parameters $(\hat{\alpha}, \hat{\phi}, \hat{c}, \hat{\gamma}, \hat{\sigma})$ that we estimated from the data in *our* universe, and sample the dataset $(\mathsf{Temp}_i : i = 1, \ldots, n)$ according to our fitted probability model.

Let's state the procedure more formally:

Parametric resampling. Let our dataset be x, typically consisting of multiple observations $(x_1, \ldots, x_n)$. Suppose we have a probability model $\Pr_X(x\,;\,\theta)$ for the dataset, where θ denotes all the unknown parameters.

1. Decide on a readout function $t(X)$, to measure the quantity we're interested in. This might be a maximum likelihood estimator for a parameter of interest, or it might be any other data summary we like. It must be computable from a dataset, and it mustn't depend on the unknown parameters. Such a readout function is known as a *statistic*.
2. Fit the model, i.e. find the maximum likelihood estimator $\hat{\theta}$. Next, let X^* denote a synthetic dataset, generated from $\Pr_X(\cdot\,;\,\hat{\theta})$. Simulate many synthetic datasets. This is how we're simulating the multiverse, and it's called *parametric resampling*.
3. Compute t for each of the synthetic datasets we generated, and report the spread of values, for example with a histogram. This tells us what readouts our parallel-universe data scientists would have computed.

It's more honest to report the spread of $t(X^*)$ rather than just the observed $t(x)$. The only difference between our universe and all the other parallel universes is random chance, and that's not a basis for sound scientific inferences. We could for example simply plot a histogram of $t(X^*)$. Another standard report is a 95% confidence interval, described in the next section.

see also non-parametric resampling in section 9.6

The resampling method described here requires a leap of faith. First, it requires us to choose a parametric probability distribution in the first place. Second, the use of maximum likelihood estimators for resampling isn't always a good idea, as you'll see in example sheet 3. There's no universal rule for the best way to simulate the multiverse. How could there be?—how could we deduce *what might have happened* when all we know is *what actually happened*? You should think about where the data came from, and how it was collected, and decide for yourself the best approach.

9.2. Confidence intervals for statistics

You're writing up a news story about knife crime. You write "A 95% confidence for the mean number of attacks is $[51, 120]$*". Your colleagues look at you as if you're crazy. One of them, a self-styled data wizard, runs an database query* `SELECT AVG(num_attacks) FROM offences WHERE offence="knife"` *and gets the answer 85, and wonders why you can't give a straight answer to a straight question.*

Suppose we want to report our uncertainty about some observed statistic $t(x)$, and we've chosen a resampling distribution X^* that we think x is drawn from. A standard report is a *confidence interval* for $t(X^*)$. For example, we could report a two-sided interval $[\mathsf{lo}, \mathsf{hi}]$ where lo and hi are chosen such that

$$\mathbb{P}\big(t(X^*) \in [\mathsf{lo}, \mathsf{hi}]\big) = 0.95$$

and furthermore $\mathbb{P}\big(t(X^*) < \mathsf{lo}\big) = 0.025$ and $\mathbb{P}\big(t(X^*) < \mathsf{hi}\big) = 0.975$. In code,

```
t_sample = [t(rx*()) for _ in range(10000)]
lo, hi = np.quantile(t_sample, [0.025, 0.975])}
```

Other choices for the confidence interval are just as legitimate. For example if our statistic $t(X)$ is always ≥ 0 and we hope it's small, then we might report a one-sided interval $[0, \mathsf{hi}]$ with hi chosen so that $\mathbb{P}(t(X^*) < \mathsf{hi}) = 0.95$.

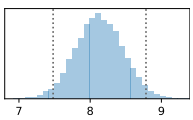
Example 9.2.1 (Confidence interval for mle).
We are given a dataset $7.2, 7.3, 7.8, 8.2, 8.8, 9.5$ which we believe is drawn from the Normal(μ, σ^2) distribution, where μ and σ are unknown. The maximum likelihood estimator $\hat{\mu}$ is just the sample mean, 8.13 in this case. Find a 95% confidence interval for $\hat{\mu}$.

Write the dataset as $x = (x_1, \ldots, x_n)$*, and write* $\hat{\mu}$ *as* $\hat{\mu}(x)$ *to emphasize that it's a statistic computed from the data* x*. We found formulae for the maximum likelihood estimators* $\hat{\mu} = \hat{\mu}(x)$ *and* $\hat{\sigma}$ *in section 1.7 page 28:*

$$\hat{\mu} = \frac{1}{n}\sum_{i=1}^{n} x_i\,, \quad \hat{\sigma} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \hat{\mu})^2}\,.$$

We can synthesize a new dataset X^* *by simply calling a* Normal$(\hat{\mu}, \hat{\sigma}^2)$ *random number generator* n *times. Simulate 10,000 such datasets, and get 10,000 samples of* $\hat{\mu}(X^*)$*.*

```
x = [7.2, 7.3, 7.8, 8.2, 8.8, 9.5]
n = len(x)

# 1. Define the readout statistic
def μ̂(x): return n.mean(x)

# 2. To generate a synthetic dataset ...
σhat = np.sqrt(np.mean((x-μ̂(x))**2))
def rx_star(): return np.random.normal(size=n, loc=μ̂(x), scale=σhat)

# 3. Sample the readout statistic, and report its spread
μ̂_ = [μ̂(rx_star()) for _ in range(10000)]
lo, hi = np.quantile(μ̂_, [.025, .975])
plt.hist(μ̂_, bins=30, alpha=.4)
plt.axvline(lo, linestyle='dotted', color='0.4')
plt.axvline(hi, linestyle='dotted', color='0.4')
plt.axvline(μ̂(x), linestyle='dashed', color='black')
```

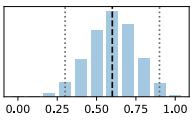


□

Example 9.2.2 (Confidence interval for mle).
I toss $n = 10$ coins and get $x = 6$ heads. Using the probability model $X \sim \text{Bin}(n, p)$ where X is the number of heads, and $n = 10$ and p is unknown, I estimate that the probability of heads is $\hat{p} = x/n = 0.6$. Find a 95% confidence interval for $\hat{p}$.

Write the maximum likelihood estimator as $\hat{p}(x)$, to emphasize that it's a statistic computed from the data x. We can synthesize a new outcome X^* by generating a $\text{Bin}\big(n, \hat{p}(x)\big)$ random variable.

```
x,n = (6,10)

# 1. Define the readout statistic
def p̂(x): return x/n

# 2. To generate a synthetic dataset ...
def rx_star(): return np.random.binomial(n=n, p=p̂(x))

# 3. Sample the readout statistic, and report its spread
p̂_ = np.array([p̂(rx_star()) for _ in range(10000)])
lo,hi = np.quantile(p̂_, [.025, .975])
```




```
p,c = np.unique(np.array(p̂_), return_counts=True)
plt.bar(p,c, width=0.07, alpha=.4)
plt.axvline(lo, linestyle='dotted', color='0.4')
plt.axvline(hi, linestyle='dotted', color='0.4')
plt.axvline(p̂(x), linestyle='dashed', color='black')
```

□

Example 9.2.3 (Comparing groups).
We are given a dataset $x = (4.3, 5.1, 6.1, 6.8, 7.4, 8.8, 9.9)$ which we believe is drawn from the Normal(μ, σ^2) distribution, and $y = (8.3, 8.5, 8.9)$ which we believe is drawn from Normal(ν, σ^2). Find a 95% confidence interval for $\hat{\nu} - \hat{\mu}$, the difference between the maximum likelihood estimators for the group means.

```
x = np.array([4.3, 5.1, 6.1, 6.8, 7.4, 8.8, 9.9])
y = np.array([8.3, 8.5, 8.9])
m,n = len(x), len(y)
```

The question tells us to use a readout statistic based on maximum likelihood estimators, and the resampling procedure requires them, so let's calculate them. When we write out the likelihood, the rule is that we include all the data and all the unknown parameters. In this question, "all the data" means "all the x_i and all the y_i", and the unknown parameters are μ, ν, and σ. So the likelihood function is

$$\begin{aligned}&\Pr(x_1, \ldots, x_m, y_1, \ldots, y_n \,;\, \mu, \nu, \sigma) \\ &\quad = \left(\prod_{i=1}^{m} \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x_i-\mu)^2/2\sigma^2}\right)\left(\prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(y_i-\nu)^2/2\sigma^2}\right)\end{aligned}$$

assuming all observations are independent. Taking logarithms, to make the algebra easier,

$$\log \text{lik} = -\frac{m+n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\left\{\sum_{i=1}^{m}(x_i-\mu)^2 + \sum_{i=1}^{n}(y_i-\nu)^2\right\}.$$

To find the maximum likelihood estimators, we need to solve three simultaneous equations:

$$\frac{\partial}{\partial\mu}\log\text{lik} = 0, \qquad \frac{\partial}{\partial\delta}\log\text{lik} = 0, \qquad \frac{\partial}{\partial\sigma}\log\text{lik} = 0.$$

Solving the first two we get $\hat{\mu} = \bar{x}$ and $\hat{\nu} = \bar{y}$ where $\bar{x}$ denotes the sample mean $\sum_i x_i/m$ and

$\bar{y} = \sum_i y_i/n$. *Solving the third equation gives*

$$\hat{\sigma} = \sqrt{\frac{1}{m+n}\left\{\sum_{i=1}^{m}(x_i - \hat{\mu})^2 + \sum_{i=1}^{n}(y_i - \hat{\nu})^2\right\}}.$$

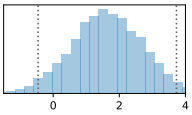
(You may spot that this is a special case of linear regression. We could have skipped all this algebra and gone straight for the computational solution using sklearn.linear_model *to find $\hat{\mu}$ and $\hat{\nu}$ as on page 36, and then used the general formula for $\hat{\sigma}$ from page 42.)*

```
# Step 1. Define a readout statistic
def t(x,y): return np.mean(y) - np.mean(x)

# Step 2. To generate a synthetic dataset ...
μ̂, ν̂ = np.mean(x), np.mean(y)
σ̂ = np.sqrt((np.sum((x - μ̂)**2) + np.sum((y - ν̂)**2))/(m+n))
```

Now we can use resampling to synthesize new datasets (X^*, Y^*). Remember, the goal is to create a synthetic dataset that matches the shape of the data we're given, and in this question we were given two separate lists, so that's what this function returns.

Python unpacking syntax: if z is a list or tuple and f is a function, then f(*z) denotes f(z[0],z[1],...).

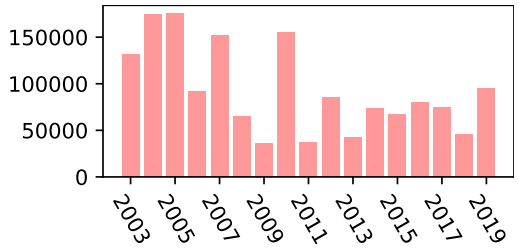
```
def rxy_star():
    return (np.random.normal(loc=μ̂, scale=σ̂, size=m),
            np.random.normal(loc=ν̂, scale=σ̂, size=n))

# 3. Sample the readout statistic, and report its spread
t_ = [t(*rxy_star()) for _ in range(10000)]
lo,hi = numpy.quantile(t_, [.025, .975])    # [-0.458, 3.729]
plt.hist(t_, bins=30, alpha=.4)
plt.axvline(lo, linestyle='dotted', color='0.4')
plt.axvline(hi, linestyle='dotted', color='0.4')
plt.axvline(t(x,y), linestyle='dashed', color='black')
```



□

Exercise 9.2.4 ().

The fires in the Amazon in summer 2019 were widely reported. According to satellite data from MODIS[50], the number of fires detected from the beginning of the year to 29 August 2019 was 95092, more than double that seen in the same period the previous year. The chart below shows the corresponding numbers over a span of years.



Is the 2019 value notably higher than the values since 2011, when Dilma Rousseff took office as president of Brazil?

Model the 2011–2018 values as Normal(μ, σ^2), and the 2019 value as Normal$(\mu + \delta, \sigma^2)$. Find a 95% confidence interval for the maximum likelihood estimator $\hat{\delta}$.

The thinking behind this model is: we want to determine whether there's a notable difference between 2011–2018 versus 2019, so let's propose a probability model which allows there to be a difference. We could have chosen a model with more parameters, e.g. different rates in each of the years 2011–2018, but the subtext of the question is that those years are homogeneous, so what's what we'll put in our model.

[50] Data from https://www.globalfiredata.org/forecast.html, csv at https://www.cl.cam.ac.uk/teaching/current/DataSci/data/modis.csv

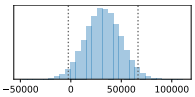
Why use μ and $\mu+\delta$ for the means? Why not μ and ν, and read off $\nu-\mu$ rather than δ? These are just two different parameterizations of exactly the same model, so it doesn't matter which we use. See example 2.6.2 page 47.

First, here is the code behind the plot. The full dataset includes cumulative number of fires on each day of the year; day 240 is 29 August.

```
modis = pandas.read_csv('https://www.cl.cam.ac.uk/teaching/current/DataSci/data/modis.csv')
df = modis[modis.day_of_year==240]
plt.bar(df.year.values, df.num_fires, width=0.8, alpha=.4)
plt.xticks(numpy.arange(2003,2020,2), rotation=-60)
```

As noted above, it makes no difference whether we find mles for $\hat\mu$ and $\hat\delta$, or whether we fit a model with two separate group means μ and ν as in excercise 9.2.3 and then let $\hat\delta=\hat\nu-\hat\mu$. For the sake of efficiency, we should reuse code from that exercise—but for the sake of variety we'll do it differently, linear-model style.

```
df = modis[(modis.day_of_year==240) & (modis.year>=2011) & (modis.year<=2019)].sort_values('year')
year,fires = df.year.values, df.num_fires.values

# 1. Define the readout statistic
def δ̂(x): return x[-1] - np.mean(x[:-1])

# 2. To generate a synthetic dataset ...
μ̂ = np.mean(fires[:-1])
pred = np.where(year<=2018, μ̂, μ̂+δ̂(fires))
σ̂ = np.sqrt(np.mean((fires-pred)**2))
def rx_star(): return np.random.normal(loc=pred, scale=σ̂)
```




```
# 3. Sample the readout statistic, and report its spread
δ̂_ = np.array([δ̂(rx_star()) for _ in range(100000)])
lo,hi = np.quantile(δ̂_, [.025, .975])  # [-2547, 66463]
plt.hist(δ̂_)
plt.axvline(lo, linestyle='dotted', color='0.4')
plt.axvline(hi, linestyle='dotted', color='0.4')
plt.axvline(δ̂(fires), linestyle='dashed', color='black')
```

In the end, the 95% confidence interval includes zero. This does not support the conclusion that 2019 is notably different to preceeding years.

□

9.3. Hypothesis testing and p-values

In Karl Popper's philosophy, a scientific hypothesis is a claim about the world that can in principle be proved false by data. For example, the hypothesis "all swans are white" is proved false if we observe a black swan.

But what about hypotheses that can't be logically disproved? For example, consider the hypothesis "My friend can't taste the difference between tea prepared with the milk first and with the tea first." Suppose I do an experiment and give this friend cups of tea, some made one way some made the other, and ask them to guess which is which. Even if I made 100,000 cups of tea and my friend correctly guessed every single one of them, I still wouldn't be able to definitively rule out the hypothesis — because there's still a possibility, however faint, that this observed data arose from random guessing.

It's nonetheless possible to test such hypotheses, using a procedure that was laid out by Ronald Fisher in 1925.[51] We'll first state the generic procedure, and then flesh it out as Fisher did with a concrete illustration, the tea-tasting hypothesis.[52]

Fisher's hypothesis testing procedure. First, propose a hypothesis that we wish to test (and possibly reject). This is called the *null hypothesis*, written H_0. Typical null hypotheses are "this fancy new machine learning algorithm is no better than the old one", or "the new drug that a biotech has developed is actually ineffective". Then,

1. Let x be our dataset. Choose a function $t : x \to \mathbb{R}$, called the *test statistic*.
2. Define a random synthetic dataset X^* using resampling based on the null hypothesis H_0.
3. Plot a histogram or draw up a table of the distribution of $t(X^*)$ and mark on it $t(x)$. Read off the probability of seeing a result as extreme or more extreme than $t(x)$. This probability is called the p-value, and a low p-value is a sign that H_0 should be rejected.[53]

This is a 'meta-procedure' since it doesn't specify *how* to perform each of these steps. It's up to us to be creative in picking a test statistic, resampling, and interpreting 'more extreme' — and indeed in choosing a useful hypothesis to test in the first place.

Example 9.3.1 (Fisher's tea-tasting test).
A lady of Fisher's acquaintance claimed to be able to taste the difference between tea prepared with the milk first and with the tea first. To test this, Fisher prepared eight otherwise identical cups, four with the tea first, four with the milk first, and shuffled them. He told her that there were four of each, and asked her to identify which was which. She identified all eight correctly. Explain how to test the hypothesis that she can't taste the difference, and find the p-value.

Let H_0 be the hypothesis "she can't taste the difference".

1. *Let x be a table listing the truth and her assigned label for each cup, and let $t(x)$ be the number of correctly assigned labels.*
2. *If H_0 were true, then her labels would be a random permutation of {tea, tea, tea, tea, milk, milk, milk, milk}.*
3. *The chance of labelling at k cups correctly,* if H_0 were true and her labelling were random, *can be found by simulation or by cunning combinatorial calculation. I want to*

[51] R. A. Fisher. *Statistical methods for research workers*. 1925. URL: http://www.haghish.com/resources/materials/Statistical_Methods_for_Research_Workers.pdf.

[52] Ronald Fisher. *The Design of Experiments*. Oliver and Boyd, 1935. URL: https://archive.org/download/in.ernet.dli.2015.502684/2015.502684.The-Design_text.pdf.

[53] Fisher suggested "If p is between .1 and .9 there is certainly no reason to suspect the hypothesis tested. If it is below .02 it is strongly indicated that the hypothesis fails to account for the whole of the facts. We shall not often be astray if we draw a conventional line at .05 and consider that [lower values of p] indicate a real discrepancy." He meant this as just a guide for interpretation, but less sophisticated thinkers than Fisher often take it as a hard-and-fast rule, "if $p < 0.05$ then conclude that H_0 is false".

emphasize the general procedure not the problem-specific calculations, so I'll just report the numbers: the chance of labelling k cups correctly is

k	0	2	4	6	8
$\mathbb{P}(\#correct = k)$	0.014	0.229	0.514	0.229	0.014

She in fact labelled all eight cups correctly, i.e. $t(x) = 8$. The probability of seeing a result as extreme or more extreme than this is $\mathbb{P}(\#correct \geq 8)$, and it's impossible to have > 8, so the p-value is 1.41%.

Fisher considers 1.41% too small for the null hypothesis to be credible, so he rejected it. □

Tests on parameters. Many hypotheses can be phrased as a question about the parameters of a model. For example, suppose we want to test whether a drug is effective. We might propose a general model that says patient outcomes are Normal(μ, σ^2) on treatment and Normal(ν, σ^2) on placebo, and propose the hypothesis "H_0: $\mu = \nu$". For such cases there's a natural way to set up the hypothesis test. Here's the general procedure, followed by illustrations.

1. *For the test statistic:* find the mle for the general model, then invent a test statistic that measures how far these estimated parameters are from meeting the constraint. For example, $t = (\hat{\mu} - \hat{\nu})^2$, or $t = \hat{\mu} - \hat{\nu}$.
2. *For the resampling:* find the mle under H_0. In other words, solve a constrained optimization problem — find the parameters that maximize the likelihood of the data, subject to the H_0 constraint. Define a random synthetic dataset X^* by resampling with these parameters.
3. *For the p-value:* if H_0 is true then the test statistic should be ≈ 0, since the estimated parameters should be close to meeting the constraint. Larger values are evidence against H_0, so they're what counts as "as extreme or more extreme than $t(x)$". Depending on the test statistic we've chosen, it might be large-positive values that are evidence against H_0, or both large-positive and large-negative values. The former is called a one-sided test, and the latter a two-sided test.

Example 9.3.2 (Equality of means for two samples).
We are given a dataset with readings from two groups, $x = (x_1, \ldots, x_m)$ and $y = (y_1, \ldots, y_m)$. Do the two groups have different means? Describe a hypothesis test of $H_0 : \mu = \nu$ for the general parametric model

$$X_i \sim N(\mu, \sigma^2), \qquad Y_i \sim N(\nu, \sigma^2).$$

The first step in Fisher's procedure is to define a test statistic. The general idea of the test statistic is that it should somehow relate to what we'd expect to see if H_0 were true. A natural choice here is to use $t = \hat{\mu} - \hat{\nu}$, the maximum likelihood estimator for δ; if H_0 is true then we'd expect $\hat{\delta}$ to be closer to 0, and otherwise we'd expect it to be further from 0, either positive or negative.

```
# 1. Define the test statistic
# For the general parametric model, the mle is μ̂ = x̄, ν̂ = ȳ
def t(x,y): return np.mean(y) − np.mean(x)
```

The second step is to define a procedure for generating a random synthetic dataset ***under the assumption that H_0 is true****. We'll first fit the H_0 model, in other words we'll fit*

$$X_i \sim N(\mu, \sigma^2), \qquad Y_i \sim N(\nu, \sigma^2)$$

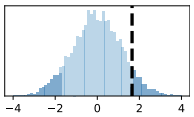
under the constraint $\mu = \nu$. This is a trivial constraint, because it means that X_i and Y_i are all Normal(η, σ^2) *where η is the common value, and we know how to fit this. We then sample values from the model using our fitted $\hat{\mu}$ and $\hat{\nu}$ and $\hat{\sigma}$.*

```
# 2. Generate a synthetic dataset, assuming H0
x = [4.3, 5.1, 6.1, 6.8, 7.4, 8.8, 9.9]  # example data
y = [8.3, 8.5, 8.9]                      # example data
```

```
η̂ = np.mean(np.concatenate([x,y]))
σ̂ = np.sqrt(np.mean((np.concatenate([x,y])-η̂)**2))
def rxy_star():
    return (np.random.normal(loc=η̂, scale=σ̂, size=len(x)),
            np.random.normal(loc=η̂, scale=σ̂, size=len(y)))
```

The third step is to plot the distribution of the test statistic assuming H_0 is true, and mark on the actual observed value of the test statistic, and compute the probability of seeing a value as extreme or more extreme than this observed value. In this case, if H_0 weren't *true, in other words if $\mu \neq \nu$, we'd expect to see either a smaller $\hat{\mu} - \hat{\nu}$ or a larger $\hat{\mu} - n\hat{u}$ depending on the sign of the difference. Thus, for this question, 'extreme' should be interpreted as 'extreme on either side of the distribution'. This is called a 'two-sided test'.*

Python unpacking syntax: if z is a list or tuple and f is a function, then f(*z) denotes f(z[0],z[1],...).




```
# 3. Sample the test statistic, compare to what was observed, and find p-value
t_ = np.array([t(*rxy_star()) for _ in range(10000)])
plt.hist(t_, bins=60, alpha=.3)
plt.axvline(x=t(x,y), linestyle='dashed', color='black')

p1 = np.mean(t_ >= t(x,y))
p2 = np.mean(t_ <= t(x,y))
p = 2*min(p1,p2)
```

The only fiddly bit of the answer is working out the p-value for a two-sided test. To understand what the code does, imagine a plot of all the synthetic $t(X^*, Y^*)$ points we've sampled:

Number them, in order of how extreme they are—one from the left, one from the right, one from the left, one from the right, and so on. Keep going until we hit $t(x,y)$.

We've marked all the sample points that are more extreme than $t(x,y)$. Thus the fraction of sample points we've marked is the p-value. There's a nifty formula for it:

$$p_1 = \frac{1}{n}\Big\{\#samples > t(x,y)\Big\}, \quad p_2 = \frac{1}{n}\Big\{\#samples < t(x,y)\Big\}, \quad p = 2\min(p_1, p_2).$$

To see why this formula works, consider the two different cases: either we hit $t(x,y)$ coming from the right hand side, in which case the answer we want is $2p_1$ and $p_1 \leq p_2$; or we hit it coming from the left and the answer we want is $2p_2$ and $p_2 \leq p_1$; in both cases the answer is $p = 2\min(p_1, p_2)$.
□

Exercise 9.3.3 (Equality of group means).
We are given three groups of observations from three different systems,

$$x = [7.2, 7.3, 7.8, 8.2, 8.8, 9.5]$$
$$y = [8.3, 8.5, 9.2]$$
$$z = [7.4, 8.5, 9.0],$$

and we wish to know whether they all come from the same distribution, or whether there are three different distributions. Start with a general probability model in which they could potentially come from three different distributions,

$$X_i \sim \text{Normal}(a, \sigma^2), \quad Y_i \sim \text{Normal}(b, \sigma^2), \quad Z_i \sim \text{Normal}(c, \sigma^2)$$

Let H_0 be that the three distributions are identical i.e. that $a = b = c$, or equivalently

$$X_i \sim Y_i \sim Z_i \sim \text{Normal}(\mu, \sigma^2).$$

Consider the test statistic

$$t = (\hat{a} - \hat{\mu})^2 + (\hat{b} - \hat{\mu})^2 + (\hat{c} - \hat{\mu})^2$$

where hats denote maximum likelihood estimators. If H_0 were true, we'd expect $t(x)$ to be small.

Find the value of the test statistic for the data given. What is the probability of seeing a value this large or larger, if H_0 is true?

```
x = [7.2, 7.3, 7.8, 8.2, 8.8, 9.5]
y = [8.3, 8.5, 9.2]
z = [7.4, 8.5, 9.0]
```

First, the test statistic. We've seen the normal distribution enough times by now that we can just write down the maximum likelihood estimators:

```
# 1. Define test statistic
def t(x,y,z):
    μ' = np.mean(np.concatenate([x,y,z]))
    a',b',c' = [np.mean(v) for v in [x,y,z]]
    return (a' - μ')**2 + (b' - μ')**2 + (c' - μ')**2
```

Next, work out how to resample the dataset. The general rule is "use a resampling scheme that fits with the null hypothesis." Here, the null hypothesis says that all three datasets are drawn from $\text{Normal}(\mu, \sigma^2)$*, so we'll fit* μ *and* σ *to the entire collection of observations, and resample the three datasets from those common values. Note that resampling must create a full dataset mirroring all our data, i.e. it should create* (X^*, Y^*, Z^*)*.*

fitting a Gaussian: exercise 1.7.3 page 28

```
# 2. To generate a synthetic dataset, assuming H₀:
data = np.concatenate([x,y,z])
μ̂ = np.mean(data)
σ̂ = np.sqrt(np.mean((data-μ̂)**2))

def rxyz_star():
    return (np.random.normal(size=len(x), loc=μ̂, scale=σ̂),
            np.random.normal(size=len(y), loc=μ̂, scale=σ̂),
            np.random.normal(size=len(z), loc=μ̂, scale=σ̂))
```

Third, plot the distribution of the test statistic assuming H_0 *to be true, and compute the p-value. The question tells us to use a one-sided p-value.*

```
# 3. Sample the test statistic under H₀, find the p-value
t_sample = np.array([t(*rxyz_star()) for _ in range(10000)])
p = np.mean(t_sample >= t(x,y,z))    # answer: p = 0.592
```

□

Exercise 9.3.4 (Arbuthnot's hypothesis test).
In the 18th century John Arbuthnot, a physician and member of the Royal Society, used statistical reasoning to prove the existence of God.[54] He reasoned thus: (i) it is natural to assume that male births are as likely as female births, (ii) in his study of 82 years worth of records of registered births in London, he found that in every single year there were more male births than female, (iii) it's very unlikely we'd see this by chance, (iv) it must therefore be that "provident Nature, by the Disposal of its wise Creator, brings forth more Males than Females; and that in almost a constant proportion", since "the external Accidents to which Males are subject (who must seek their Food with danger) make a great havock of them".

Setting aside Arbuthnot's assumptions, describe his reasoning in Fisher's language, and calculate the p-value.

Let H_0 be the hypothesis "male births are as likely as female births".

1. *Let the dataset x record, for each of these 82 years, the numbers of male and female births. Let $t(x)$ be the number of years in which male births exceed female.*
2. *If H_0 were true, and supposing the total number of births b in a year is given, we can simulate the number of female births as $\mathrm{Bin}(b, 1/2)$, and the number of male births is whatever is left.*
3. *The chance that male births exceed female in a given year,* if H_0 were true *is 0.5, give or take a small error due to ties. The p-value is the chance of seeing ≥ 82 such years, which is $0.5^{82} \approx 2.5 \times 10^{-25}$. (There are only 82 years of data, so the chance of seeing ≥ 82 such years is equal to the chance of exactly 82 such years.)*

If we like, we can frame the test parametrically. Let b_i be the total number of births in year i, and consider the general model for the number of male births $X_i \sim \mathrm{Bin}(b_i, p)$, and consider the null hypothesis $H_0 : p = 1/2$. It doesn't change our answer, but it may be easier to think through the second step.
□

Exercise 9.3.5 (Amazon fires).
For the Amazonian fire data on page 163, let x_i be the number of fires recorded for year i up to 29 August of each year. The media reported x_{2019}/x_{2018}, the year-on-year increase in number of fires, and claimed that this value was worryingly large, compared to figures from 2011 onwards. Was it?

The question leaves it to us to invent the null hypothesis. Let's propose the model that says $x_{2011}, \ldots, x_{2019}$ are all drawn independently from the same distribution, say

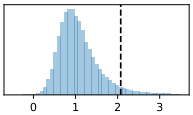
$$\mathrm{Normal}(\mu, \sigma^2), \qquad \mu, \sigma \text{ unknown.}$$

(Whether or not a Normal distribution is appropriate can be answered either by a domain expert, or by a data scientist with much more data.) The question asks us whether x_{2019}/x_{2018} is extreme, so let's take that to be the test statistic. The question asks whether it's worryingly large, so we'll take 'extreme value of the test statistic' to mean 'large positive value'.

```
# Load the dataset, and extract the x_i we want as a numpy vector
url = 'https://www.cl.cam.ac.uk/teaching/current/DataSci/data/modis.csv'
modis = pandas.read_csv(url)
df = modis[modis.day_of_year==240] # i.e. look at fires up to 29 August.
fires = df.loc[df.year>=2011].sort_values('year').num_fires.values

# 1. Define the test statistic
def t(x): return x[-1] / x[-2]

#2. To generate a synthetic dataset, assuming H_0:
μ̂ = np.mean(fires)
```

[54] John Arbuthnot. "An argument for divine providence, taken from the constant regularity observed in the births of both sexes". In: *Philosophical Transactions of the Royal Society* (1710). DOI: https://doi.org/10.1098/rstl.1710.0011

```
σ̂ = np.sqrt(np.mean((fires-μ̂)**2))
def rx_star(): return np.random.normal(size=len(fires), loc=μ̂, scale=σ̂)

# 3. Sample the test statistic under H_0, compute p-value
t_sample = np.array([t(rx_star()) for _ in range(10000)])
p = np.mean(t_sample>=t(fires))  # 0.0511, not grounds to reject H_0

plt.hist(t_sample, bins=np.linspace(-.5,3.5,50), alpha=.4)
plt.axvline(t(fires), linestyle='dashed', color='black')
```



□

Exercise 9.3.6 (Racial bias in police stop-and-search).
For the police stop-and-search data, page 49, we wish to know if there's evidence of any racial bias in Cambridgeshire policing. The numbers of stops that led to something being found, versus nothing being found, are

	Asian	Black	Other	White
find	116	170	28	1060
nothing	76	100	9	680

Let n_k be the total number of stops for people of ethnicity k, and let x_k be the number where something was found. Consider the general model $X_k \sim \text{Bin}(n_k, \beta_k)$, for which the maximum likelihood estimator is simply $\hat{\beta}_k = x_k/n_k$. Define the overall bias score to be

$$d = \max_{k,k'} |\hat{\beta}_k - \hat{\beta}_{k'}|.$$

Larger values correspond to more bias.

Compute d for the data given. Let the null hypothesis be that there is no bias, i.e. that the β_k are all equal. Under this hypothesis, what's the probability of seeing a value greater than or equal to the actual d for the data given?

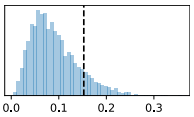
```
# It's a big file, so retrieve it and store locally for future use
import os.path
if os.path.exists('stop-and-search.csv'):
    print("file already downloaded")
else:
    !wget "https://www.cl.cam.ac.uk/teaching/current/DataSci/data/stop-and-search.csv"
police = pandas.read_csv('stop-and-search.csv')

# Preprocess the data
df = police.loc[~pandas.isnull(police['officer_defined_ethnicity']) &
                ~pandas.isnull(police['outcome']) &
                (police['force']=='cambridgeshire')].copy()
df['eth'] = df['officer_defined_ethnicity']
df['y'] = np.where(df['outcome'] != 'False', 'find', 'nothing')

# Quick tabulation, to check what data we're working with
tab = df.groupby(['y','eth']).apply(len).unstack()
```

The general rule is "use a resampling scheme that fits with the null hypothesis". Here, the null hypothesis is that all groups share a common parameter, $X_k \sim \text{Bin}(n_k, \theta)$, and it's easy to check that the maximum likelihood estimator for θ is $\hat{\theta} = (\sum_k x_k)/(\sum_k n_k)$.

```
x = tab.loc["find"]
n = tab.loc["find"] + tab.loc["nothing"]
θ̂ = sum(x) / sum(n)
def rx_star(): return [np.random.binomial(nk, θ̂) for nk in n]

# The test statistic
def d(x):
    β = np.array(x) / n # a vector of length 4
```

```
    res = 0
    for e1 in range(len(n)):
        for e2 in range(len(n)):
            res = max(res, abs(beta[e1] - beta[e2]))
    return res
d_actual = d(x)
d_samples = np.array([d(rx_star()) for _ in range(10000)])

# What we'd expect to see under H_0
plt.hist(d_samples, bins=50, alpha=.4)
plt.axvline(d_actual, color='black', linestyle='dashed')
p = np.mean(d_samples > d_actual)  # returns 0.1177
```

The probability of seeing a bias score this high or larger, if H_0 were true, is 11.8%. *This data doesn't provide compelling evidence of bias*[55].



□

Example 9.3.7 (Sign test).

I have a dataset consisting of 500 records, each record a pair (x_i, y_i) where x_i is the predictor variable and y_i is the true label. For example, x_i might be the text of a movie review, and $y_i \in \{☺, 😐, ☹\}$ might be the sentiment.

I have built two classification algorithms, A and B, and I wish to know if one is better than the other. I run them both on each record to get predicted classes $A(x_i)$ and $B(x_i)$. I compare these answers to the true label y_i, and I obtain a grade

$$w_i = \begin{cases} \text{“A” if } A \text{ is correct, } B \text{ not} \\ \text{“B” if } B \text{ is correct, } A \text{ not} \\ \text{“eq” if either both are correct, or both incorrect.} \end{cases}$$

I count up the total number for each grade, to get $n = (n_A, n_B, n_{\text{eq}}) = (70, 50, 380)$.

Consider the general probability model for the grades

$$\mathbb{P}(W = \text{“A”}) = p_A, \quad \mathbb{P}(W = \text{“B”}) = p_B, \quad \mathbb{P}(W = \text{“eq”}) = p_{\text{eq}}.$$

where we require $p_A + p_B + p_{\text{eq}} = 1$. Let the null hypothesis H_0 be “the two algorithms are equally as good”, i.e. $p_A = p_B$, and let the alternative hypothesis be the general model, i.e. “the two algorithms might or might not be as good”.

Using the test statistic $t = n_A + n_{\text{eq}}/2$, decide whether to use a one-sided or two-sided test, and find the p-value.

Let's implement a general purpose resampler, which can be used for the p parameters under H_0, or for any other p parameters.

```
# The data
n = (nA,nB,neq) = (70,50,380)

# Resample based on an arbitrary p, and compute test statistic
def rn_star(p): return np.random.multinomial(n=sum(n), pvals=p)
def t(nA,nB,neq): return nA + neq/2
```

The maximum likelihood estimators under H_0 can be found using the same method as in example 1.3.5 page 15. The result is

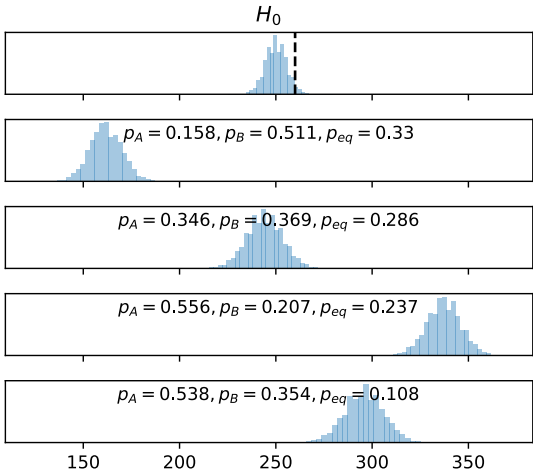
$$\hat{p}_A = \hat{p}_B = \frac{(n_A + n_B)/2}{n_A + n_B + n_{eq}}, \qquad \hat{p}_{eq} = \frac{n_{eq}}{n_A + n_B + n_{eq}}.$$

Python unpacking syntax: t(*rn_star(p)) means: call rn_star(p), get back a tuple (nA,nB,neq), and ‘unpack’ the call, i.e. call t(nA,nB,neq).

```
# Resampled values of the test statistic, under H_0
```

[55] When this course was taught in 2018, the police data at the time reported a value of $p = 0.969$ (using data up to 2018-11-01 and excluding ethnicity level "Other" on grounds of insufficient data). Fisher pointed out that very large values for p are grounds for suspicion; $p = 0.969$ indicates that the $\hat{\beta}_k$ coefficients are much closer together than one would expect by chance. An indication of the police numbers being fudged?

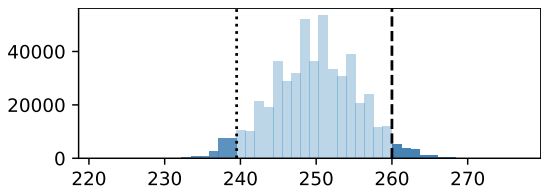
```
p₀ = [(nA+nB)/2/sum(n), (nA+nB)/2/sum(n), neq/sum(n)]
t_samples = np.array([t(*rn_star(p₀)) for _ in range(10000)])

fig,axs = plt.subplots(5,1, sharex=True, figsize=(4.5,4))

# Plot the histogram of resampled t values under H₀
axs[0].hist(t_samples, bins=30, alpha=.4)
axs[0].axvline(t(*n), linestyle='dashed', color='black')
axs[0].title('H₀')

# Plot more alternative histograms for randomly chosen H₁ parameters
for ax in axs[1:]:
    p = np.random.uniform(size=3)
    p = p / sum(p)
    ax.hist([t(*rn_star(p)) for _ in range(10000)], bins=30, alpha=.4)
```



If an alternative hypothesis is true, the distribution of t might be shifted to the right, or it might be shifted to the left. Either is possible, depending on the alternative. Thus an extreme value of $t(x)$, lying in either tail of the distribution of $t(X^*)$ under H_0, is evidence for rejecting H_0 in favour of the alternative.

Below is another plot, showing the histogram of $t(X^*)$ under H_0. It also marks out the two-sided tails, as described in example 9.3.2 page 166. The p-value, i.e. the total probability of lying in this two-sided region, is $\approx 5.8\%$. According to Fisher, a p-value in the range 5%–10% is grounds for suspicion that the two classifiers might be different, but not clear evidence. Best gather more data.

```
p = 2 * min(np.mean(t_samples > t(*n)), np.mean(t_samples < t(*n)))  # 0.0614
```



□

* * *

There's a lot of discretion involved in hypothesis testing. We have to decide on the null hypothesis, the test statistic, the resampling method, and the interpretation of 'more extreme'. The point of going through all these examples is to give a sense of how to exercise this discretion. There are some people who just want to be told "Here is exactly the test you use, and here's code for it, now go away and run it." And if you look up tests on Wikipedia, you'll find a laundry list of famous tests — Student's t-test for unpaired samples, his t-test for paired

samples, the F-ratio test, the χ^2 test, and then specialized tests such as Kolmogorov-Smirnov or Kruskal-Wallis.

I think we should avoid recipes. The trouble with recipes is that behind each recipe there's an assumption about the model, and often the model behind the recipe isn't the model that's right for our data. If we're working in probabilistic machine learning, then building probability models is our trade, and we have to be free to invent our own custom models. I've seen far too many cases of students and researchers who blindly use a hypothesis testing recipe because it's there in the textbook, and they don't stop and think about whether it's appropriate for their particular model.

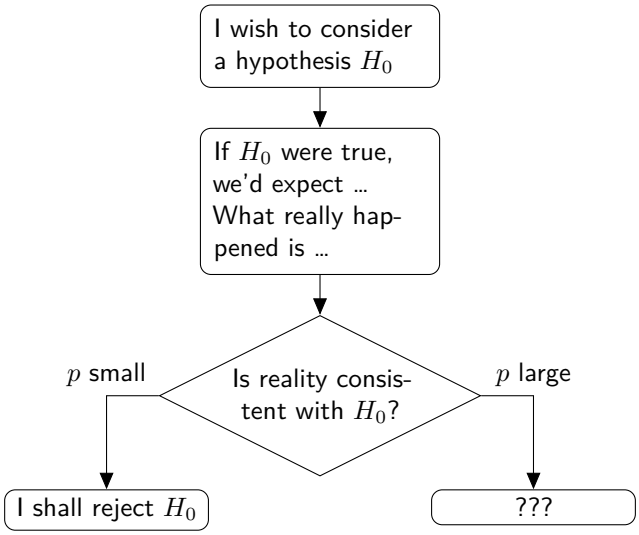
In the days of limited computing power, statisticians had to be clever and design cunning test statistics whose distributions could be computed without too much work, which is why there is a laundry list of elegant tests. Today, abundant computer power means we can invent whatever test statistic we like, and whatever probability model and null hypothesis, and just go ahead and implement them. The computational approach is what I have chosen to emphasize. It requires more thought on the part of the data scientist about what exactly a test *is*—and this is a good thing! Furthermore the computational approach is infinitely more flexible.

The classic way to teach hypothesis testing is with parameter-free test statistics. For example, the standard test for equality of means of two samples (example 9.3.2) is based on a test statistic whose distribution can be shown to not depend on the unknown parameters μ, δ, and σ, *under the assumption of normality.* I can't see the logic in looking at the data and deciding to use a Normal distribution but insisting on complete ignorance about its parameters.

9.4. Model choice

Fisher's procedure for hypothesis testing is subtly brilliant, and commonly misunderstood. Let's look into it carefully, paying particular attention to what it can tell us about choosing between models.

The most important thing to remember is that Fisher's procedure, like Karl Popper's idea of science, is concerned solely with model falsification. We start with a model — the null hypothesis H_0, the default — and ask whether the data suggests this model should be rejected.



It's easy to misinterpret what the procedure tells us. Here are some common misinterpretations:

- *I tested H_0 and found $p \geq$ MAGIC_CONST, therefore I should accept H_0.* — No. Fisher's framework, like Karl Popper's philosophy of science, is only concerned with falsifiability. If I have a tiny dataset I might get a large p-value even when H_0 is false. Best describe H_0 as "not yet rejected." That's why the right-hand outcome in the flowchart is labelled ???
- *The probability that H_0 is true is p.* — This is stupid! There is no such thing as "probability that a hypothesis is true" in frequentist statistics. A hypothesis is a claim about nature, and it's either true or not true.

Bayesianists have a different take: section 8.7 page 155

- *I want to chose between models H_0 and H_1, so I tested H_0, and I rejected it thus I must accept H_1.* — No. Fisher's procedure doesn't even mention an alternative. The p-value tells us that H_0 is a bad model for the data, not that any particular alternative model is good.
- *Since $p <$ MAGIC_CONST we must reject H_0.* — No! It's not logic that dictates we reject H_0, it's a policy decision.
- *Since $p <$ MAGIC_CONST I shall reject H_0.* — This is a valid statement, but it's still describing a policy choice, and I think it's better to report the outcome in neutral language, "The chance of seeing data as extreme as what we actually saw, assuming H_0, is p."

LIKELIHOOD RATIO AND TEST STATISTICS

Earlier, in section 4.1, I argued that likelihood is the natural way to measure how well a model fits the data. Why can't we just use likelihood, rather than Fisher's complicated definition of a p-value based on a test statistic?

Here's a thought experiment. I've tossed a coin 20 times, and I've gotten the sequence of results

$$0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0.$$

I'm interested in the hypothesis H_0: each coin toss is independent $\text{Bin}(1, 1/2)$, counting 0 as a tail and 1 as a head. Eyeballing, this hypothesis seems unlikely, since there are only 5

heads. But the likelihood of this sequence under H_0 is $1/2^{20}$, *exactly the same likelihood as any other sequence*. Since the likelihood is the same whatever the data, the likelihood is not a suitable measure of whether the data supports the hypothesis.

Likelihood is a stonking good idea, but it's only good for *comparing models* because it has no intrinsic baseline. Is a likelihood of $1/2^{20}$ good or bad? It depends what the alternative is! A convenient way to measure this is to propose an alternative model H_1 and measure the *likelihood ratio*,

$$\text{LR} = \frac{\Pr\big(\text{data}\,;\ \hat{H}_1 \text{ is true}\big)}{\Pr\big(\text{data}\,;\ \hat{H}_0 \text{ is true}\big)}$$

The hat notation here ($\hat{H}_0$ and $\hat{H}_1$) means that we should first fit the models, i.e. estimate any unknown parameters using maximum likelihood estimation, and then compute the likelihood according to the fitted models.

In the coin-toss example, consider the alternative model H_1 that says the tosses are independent $\text{Bin}(1, p)$, with p unknown. After a little algebra, it turns out that the likelihood ratio is an increasing function of $|x/n - 1/2|$, where x is the number of heads and n the number of coin tosses. In other words, both large values of x and small values of x indicate that H_0 is a poor explanation of the data compared to the alternative H_1.

The likelihood ratio as a function of x/n:

This is a formal justification of our hunch that the number of heads is relevant for judging whether H_0 fits the data. In Fisher's language, it corresponds to choosing the test statistic t = number of heads, and deeming it extreme if it is either large or small. (Or we could equivalently just use the likelihood ratio itself as a test statistic and use a one-sided test. It's a little harder to compute and to understand, but it'll give exactly the same p-value.)

The test statistic implicitly declares what sort of alternative we're considering. I think of it as a 'direction' in the space of possible datasets, grouping together a set of datasets that all indicate the same sort of alternative. In the coin example, our test statistic groups together datasets with a surprisingly small or large number of heads, and says that they all indicate the alternative $\text{Bin}(1, p)$ model rather than our null hypothesis H_0.

An hypothesis test looks for evidence against H_0, *but only in a single direction.* In the coin example, a test based on the number of heads is looking in the direction of "models where heads and tails aren't equally likely". A second glance at the data suggests another way that H_0 might be false — there are rather long runs of 0s and 1s, longer than we might expect from a model in which coin tosses are independent — and if we want to test this aspect of H_0, we'd need to invent a test statistic that points in the direction of models with non-independent coin tosses. Perhaps count the number of repeats?

Fisher's subtle brilliance was to gently separate the test statistic from the likelihood ratio. First, his p-value lets us report a solid interpretable probability, rather than a unitless measure. It's hard to know what exactly to do with the likelihood ratio *per se*: how big does it have to be for us to reject H_0? Second, his idea of a test statistic lets us talk loosely about a 'direction of evidence against H_0' without committing to a specific alternative hypothesis. The likelihood ratio is defined in terms of a specific alternative, and often we have only a vague sense of what the alternative might be.

CHOOSING BETWEEN MODELS

Given the above discussion, it should be no surprise that, in my opinion, the best way to choose between two models is simply to pick the one with the higher likelihood.

There is another approach to model choice, the Neyman–Pearson framework. A crude summary is that it gives a solid interpretable probabilistic meaning the problem of choosing between models. In my opinion it has led to more confusion than Fisher's approach, so I will not discuss it here.[56]

[56] For an interesting discussion of the two approaches, and of how many publications in science and social science get it wrong, see Jose D. Perezgonzalez. "Fisher, Neyman-Pearson or NHST? A tutorial for teaching data testing". In: *Frontiers in Psychology* (2015). DOI: 10.3389/fpsyg.2015.00223.

9.6. Non-parametric resampling

The heart of the frequentist approach is finding a way to simulate the multiverse of all possible datasets we might have seen.

All the examples we've looked at so far have involved parametric resampling, where we (1) propose a parametric probability model $\Pr_X(x\,;\,\theta)$ for the dataset, (2) find the maximum likelihood estimator $\hat{\theta}$, (3) simulate synthetic datasets from $\Pr_X(x\,;\,\hat{\theta})$.

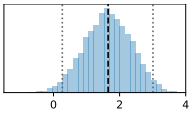
There is another way to resample that doesn't require an arbitrary choice of model. All models are wrong—so why use a model when you don't have to? In section 7.3 we concluded that best possible fit for a dataset is the dataset itself, i.e. the random variable that has the dataset's empirical distribution. Here are some examples that use sampling from the empirical distribution, also known as *non-parametric resampling*.

> Example 9.6.1 (Confidence interval for group difference).
> We are given datasets $x = (4.3, 5.1, 6.1, 6.8, 7.4, 8.8, 9.9)$ and $y = (8.3, 8.5, 8.9)$. The difference between the two sample means is $\bar{y} - \bar{x} = 0.53$. Find a 95% confidence interval using non-parametric resampling.
>
> *Remember, $\bar{y} - \bar{x}$ is a number, not a random variable, so it doesn't really* have *a confidence interval. The question is really asking for a 95% confidence interval for $\overline{X^*} - \overline{Y^*}$, where (X^*, Y^*) is a random dataset from the multiverse.*

To sample a single value from the empirical distribution of list x, we just pick a single item at random using np.random.choice(x). *For frequentist methods we actually need to create an entire synthetic dataset, i.e. we need to sample $n =$ len(x) values from the empirical distribution to get X^*, and likewise for Y^*. We can do this with* np.random.choice(x, size=n). *Lines 9 and 10, are where the magic happens.*

sampling commands: section 1.2 page 8

```
1  x = [4.3, 5.1, 6.1, 6.8, 7.4, 8.8, 9.9]
2  y = [8.3, 8.5, 8.9]
3
4  # 1. Define a readout statistic
5  def t(x,y): return np.mean(y) - np.mean(x)
6
7  # 2. To generate a synthetic dataset ...
8  def rxy_star():
9      return (np.random.choice(x, size=len(x)),
10             np.random.choice(y, size=len(y)))
```




```
11 # 3. Sample the readout statistic, and report its spread
12 t_ = [t(*rxy_star()) for _ in range(10000)]
13 lo,hi = np.quantile(t_, [.025, .975])  # [0.224, 3.038]
```

Why is the answer here different to our parametric answer for the same problem, exercise 9.2.3 on page 162? There we found a 95% confidence interval $[-0.438, 3.751]$, here we found a confidence interval $[0.224, 3.038]$. The two answers don't agree about whether "there is no difference" is within the realm of what's likely.

Note that the y datapoints are clustered tightly together, more tightly than the x datapoints. If we resample X^* and Y^* separately, as we did here, then we'll end up seeing very little variation in $\overline{Y^*}$. If we use a parametric model like that of exercise 9.2.3, in which the two datasets are assumed to share a common variance, then the wider spread of x values will bleed through into higher variation in $\overline{Y^*}$, leading to higher variation in $\overline{Y^*} - \overline{X^*}$.

Which is correct? That's something that we can't answer from the data, we can only answer from the context. Do we have reason to believe that the variability should *be the same in both cases? Anyway, resampling from a dataset of size $n = 3$ is sure to cause problems of interpretation—there's just too little information to make sound inferences.*

□

> Example 9.6.2 (Test of group difference).
> For the same datasets as above, test whether the two groups have the same distribution.

The null hypothesis H_0 is that x and y are sampled from a common distribution. If this is true, then the best fit for the common distribution is the empirical distribution of all the data we have. Lines 6 and 7 sample a synthetic dataset from this distribution.

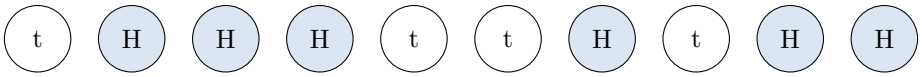
```
1  # 1. Define a test statistic
2  def t(x,y): return np.mean(y) - np.mean(x)
3
4  # 2. To generate a synthetic dataset, assuming H0 ...
5  def rxy_star():
6      return (np.random.choice(np.concatenate([x,y]), size=len(x)),
7              np.random.choice(np.concatenate([x,y]), size=len(y)))
8
9  # 3. Sample the readout statistic, find the p-value
10 t_ = [t(*rxy_star()) for _ in range(10000)]
11 p = 2 * min(np.mean(t_ >= t(x,y)), np.mean(t_ <= t(x,y)))
```

There is another way to test this null hypothesis. Consider taking all 10 datapoints, randomly picking 7 of them without replacement *to be X^*, and leaving the remaining 3 to be Y^*. If H_0 is true, then the assignment we observe in the actual dataset is just a sample of $t(X^*, Y^*)$, and we find the p-value in the usual way. This version of the test is called 'randomization over assignment', and it's an application of the idea behind Fisher's tea test.*
□

Fisher's tea test, section 9.3 page 165

Example 9.6.3 ().
I toss $n = 10$ coins and get $x = 6$ heads. I estimate that the probability of heads is $x/n = 0.6$. Find a 95% confidence interval for this.

Non-parametric resampling means "pick a value at random from the empirical distribution". All we're given in this question is a single observation $x = 6$, so the empirical distribution is too simple—every time we resample from it we get the same answer, $X^* = 6$. But there's another way to look at the observed data: look at it not as a count, but as a full-length dataset with 10 records describing the outcomes of 10 coin tosses.



To resample a single coin toss, we pick one of these 10 outcomes at random, and get heads with probability 6/10. To resample n coin tosses, we pick n of these at random, with replacement. The number of heads from n resampled coin tosses is $\text{Bin}(n, 6/10)$, which is in fact exactly the same answer we got from parametric resampling in example 9.2.2.
□

Exercise 9.6.4 ().
An exam question was attempted by 68 students from the Computer Science (CS) course, and 40 students from the Natural Sciences (NST) course. The question was marked out of 20. The number of students who attained each mark were

mark	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
CS	0	0	0	2	2	2	1	0	3	2	4	8	6	2	8	8	4	4	6	4	2
NST	1	0	0	1	1	0	1	4	2	0	2	2	8	6	4	3	1	2	0	1	1

The mean mark for CS students is 13.0, and that for NST students is 11.6, a difference of 1.5. Find a 95% confidence interval for the difference.

It's not immediately obvious what probability distribution we might use for this data, so a non-parametric approach is ideal.

The data given in the question is pre-aggregated (i.e. it shows the number of students for each mark), but it's perhaps easier to work with unaggregated data. In the code below, cs_star() *generates 68 unaggregated marks for CS students, drawn from the empirical distribution, and* nst_star() *generates 40 unaggregated marks for NST students.*

```
1  csn = numpy.array([0,0,0,2,2,2,1,0,3,2,4,8,6,2,8,8,4,4,6,4,2])
2  nstn = numpy.array([1,0,0,1,1,0,1,4,2,0,2,2,8,6,4,3,1,2,0,1,1])
```

```
m = numpy.arange(0,21)
def cs_star(): return numpy.random.choice(m, p=csn/numpy.sum(csn), size=numpy.sum(csn))
def nst_star(): return numpy.random.choice(m, p=nstn/numpy.sum(nstn), size=numpy.sum(nstn))
```

Now we simply resample the marks, and for each resampled dataset we record the mean difference. The 95% confidence interval turns out to be $[-0.17, 3.15]$*. This suggests that CS students might be cleverer, but the evidence isn't compelling since the confidence interval includes 0.*

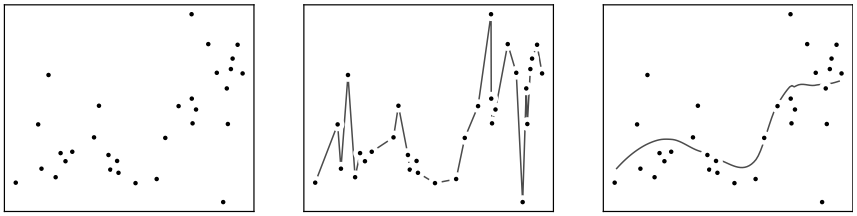
```
diff_sample = [numpy.mean(cs_star()) - numpy.mean(nst_star()) for _ in range(10000)]
lo,hi = numpy.quantile(diff_sample, [.025, .975])  # returns (-0.17, 3.15)
```

□

10. Empiricism

> *Every genuine scientific theory must be falsifiable. It is easy to obtain evidence in support of virtually any theory; the evidence only counts if it is the positive result of a genuinely risky prediction.*
>
> Karl Popper

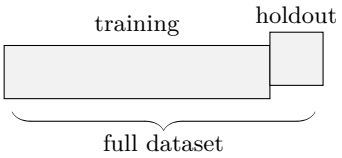
Suppose we have a collection of (x, y) points, where x is a predictor variable and y is the response. The left hand plot shows the points, and the other two plots show fitted curves. If our goal is "fitting data", then the curve from the middle plot is unimpeachable—it's a perfect fit—and yet somehow it feels wrong …



The problem is that it doesn't lead to generalizable conclusions. The curve in the middle plot slavishly follows the data—we say it's *overfitting*—and it's probably useless for predicting y for a new unseen data point x.

We want our models to work well on new unseen data.

This is what Karl Popper is getting at when he talks about prediction. The cornerstone of machine learning is evaluating models by the predictions they make on new data. We split the dataset into two parts, a training set and a holdout test set; we fit our model using the training set; and we evaluate it by its accuracy on the holdout test set.



This is what we might call the empiricist approach. It's empirical in the general sense that it's based on experiment, the experiment being to test how well our predictions work on the holdout test set. It's also empirical in the mathematical sense of being based on empirical distribution. Suppose the true unknown distribution of unseen data is p, and let the empirical distribution of the holdout set be $\hat{p}$. Assume that the holdout set is drawn from p, and that it is large enough; then $\hat{p} \approx p$. Thus, the accuracy of our model on $\hat{p}$ is approximately the accuracy of our model on p. This leap from $\hat{p}$ to p, from data to the world that produced the data, is the fundamental inductive claim behind empiricism.

Empirical distribution: section 7.3 page 128

Section 10.1 goes into more detail about how to evaluate a model on new datapoints. Prediction accuracy—i.e. read off what the model predicts, and compare it to the true outcome—is the obvious metric. But when we're working with probability models then it's more sensible to evaluate the log likelihood of the holdout test set; this is a more general metric, and it applies equally well to generative modelling as to supervised learning. Section 10.2 describes cross validation, the training mechanism for getting good holdout log likelihood.

Sections 10.3–10.5 take a harder look at the inductive claim behind holdout set evaluation. In short, my recommendation is to take the empiricist approach to choosing between models, the frequentist approach to exploring models, and frequentist or Bayesian approaches to reporting predictions.

10.1. Evaluation by holdout log likelihood

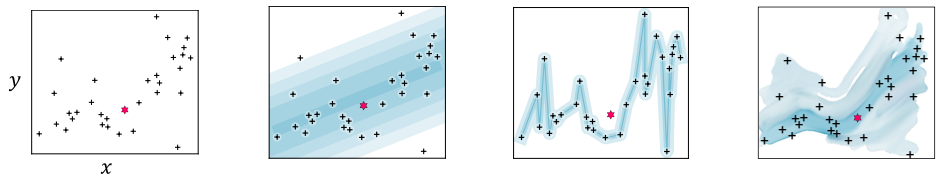
In this section we'll discuss why it's sensible to **evaluate probability models by measuring the log likelihood they assign to the holdout dataset.** In section 4.1 we saw why log likelihood is a sensible way to measure goodness of fit; what's new in this section is a discussion of log likelihood *of the holdout set* and how it deals with the problem of overfitting.

We won't try to *prove* that holdout log likelihood is the only correct evaluation – it's hard to even imagine a setting in which the choice of evaluation is a matter of proof – but we will provide compelling circumstantial evidence that it's sensible.

It's worth making a big deal of this! In probability modelling, there is one simple metric, log likelihood, that we can use for training and for evaluation, and that works for both supervised and generative modelling. There's no cleverness needed, no special treatment. In the 'prediction accuracy' mindset, by contrast, there's no obvious way to evaluate a generative model[57] — it doesn't make sense to measure its prediction accuracy on a holdout set, since generative models don't make predictions.

INTUITION, FOR THE CASE OF SUPERVISED LEARNING

Suppose our dataset consists of (x, y) pairs, where y is the label, and we're proposing a probability model $\Pr_Y(y\ ;\ x)$. These pictures illustrate why log likelihood of holdout datapoints is a good evaluation metric:



\+ training data
holdout data

underfit

overfit

Goldilocks fit

Underfitting (low log likelihood on training and holdout)

> If $\Pr_Y$ is very simple, for example a straight line fit plus noise, then training will push the noise term to be large so as to put at least some probability mass on the training datapoints, and so our model's predictions will be very 'imprecise'. In other words, at every x, there will be positive likelihood on a wide band of y values. Since likelihood must integrate to 1, the likelihood of any particular datapoint y_i will also be low, both for training and for holdout data.

Overfitting (high log likelihood on training, low on holdout)

> If $\Pr_Y$ tracks the data very precisely and concentrates all its probability mass to perfectly fit the datapoints, then our model makes very precise predictions, and it is very accurate (i.e. high log likelihood) on the training datapoints. But a holdout datapoint might not fall in this region of high probability mass, and then the likelihood of the holdout data will be low — our model makes inaccurate predictions.

Goldilocks (high-ish log likelihood on training and holdout)

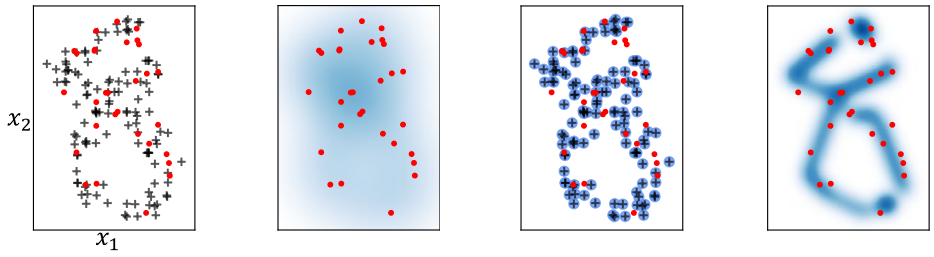
> The perfect fit concentrates its probability mass in regions where there is data, but not so much that it's surprised by the holdout datapoints. This means it makes reasonable predictions for unseen data, without claiming more precision than is warranted.

On top of this appeal to intuition, there is one more piece of circumstantial evidence in support of using holdout log likelihood for evaluation: log likelihood corresponds to standard loss functions for simple probability models (mean square error for the Normal distribution, cross entropy for the Categorical distribution), and these loss functions are widely accepted for evaluation.

[57] We could conceivably ask humans to evaluate the outputs it generates, to judge whether they are realistic. Or, better, take some machine-generated outputs and some holdout datapoints, and see if a human can tell them apart. But this doesn't scale—it's hardly *machine* learning if it needs a human in the loop. Instead, we could build a 'discriminator' neural network and measure how well it can tell the difference. This is a powerful idea—the basis for Generative Adversarial Networks—but it's hard to get right. Did our generator do well because it's a good generator, or because the discriminator we trained was inadequate?

INTUITION, FOR THE CASE OF GENERATIVE MODELLING

Suppose our dataset consists of $x = (x_1, x_2) \in \mathbb{R}^2$ datapoints, and we're proposing a generative probability model $\Pr_X(x)$. We want our model to be able to generate novel values, similar but not identical to what's in the dataset. These pictures illustrate why log likelihood of holdout datapoints is a good evaluation metric[58]. (They're very similar to the pictures for supervised learning! The difference is that for supervised learning the pictures show for every x a probability distribution over y, whereas for generative modelling they show a single probability distribution over the entire space.)



\+ training data
holdout data

underfit

overfit

Goldilocks fit

Underfitting (low log likelihood on training and holdout)
: We might choose a very simple $\Pr_X$ distribution. The joint density will then be smeared out over a large region of space, meaning that the log likelihood of the training dataset and of the holdout set are both low. And when we generate novel values (picking them according to the joint density), we'll get values all over this large region of space. In other words, this model does a bad job of mimicking the dataset.

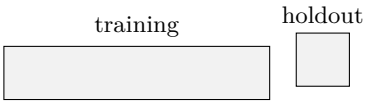
Overfitting (high log likelihood on training, low on holdout)
: We might choose a $\Pr_X$ distribution that tracks the shape of the training dataset very tightly, for example the empirical distribution of the dataset itself. This puts lots of probability mass in areas where there are datapoints, and very little elsewhere, so the log likelihood of the training dataset will be high and that of the holdout dataset will be low. And when we generate novel values, we'll just regurgitate the training datapoints. This generative model mimics the dataset, but it has no novelty.

Goldilocks (high-ish log likelihood on training and holdout)
: The perfect fit concentrates its probability mass in regions where there is data, but not so much that it's surprised by the holdout datapoints. The novel values we generate will mimic the shape of the dataset, but they won't regurgitate it.

[58]There are dissenting views: It can happen that a generative model produces bad-looking samples, even if its log likelihood seems high. See Lucas Theis, Aäron van den Oord, and Matthias Bethge. "A note on the evaluation of generative models". In: *ICLR*. 2016. URL: https://arxiv.org/abs/1511.01844v1; Ferenc Huszár. *How (not) to train your generative model*. 2015. URL: https://arxiv.org/abs/1511.05101. In my opinion the best way to deal with bad samples is to fix the deficiencies of the model, not to move the goalposts. For example, great gains in recurrent neural network performance were achieved by realizing that a fully-connected neural network wasn't good enough at preserving long-term history, and inventing LSTM networks to fix this problem. No need to ditch the log likelihood evaluation idea.

10.2. Cross validation

We want our model to work well on new unseen data, and we've seen that holdout log likelihood is a sensible way to measure how well it works. But what about training: how should we train a model so that it gets a good holdout log likelihood?



The challenge is overfitting. If we simply maximize our model's log likelihood on the training dataset, we'll end up with a model that fits the training dataset perfectly, but may well fail to generalize. (Indeed, in the case of generative modelling, we already know a perfect fit to the training dataset, namely the empirical distribution, which is nothing more than regurgitation of training dataset. It most definitely will not generalize.)

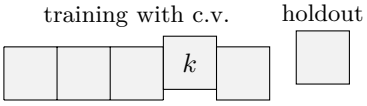
To avoid overfitting, therefore, we shouldn't simply seek the best-fitting model to the training dataset. There are several common strategies:

- We can limit ourselves to simple models, for example an explicit linear regression or a neural network with a small number of layers.
- We can add in a regularizer of some sort. Bayesianism gives a nice example of this. When we seek the maximum a posteriori (MAP) estimate, we're maximizing $\log \Pr(x;\theta) + \log \Pr_\Theta(\theta)$ where θ is the parameters we're optimizing over and x is the training dataset. The first term is the log likelihood of the training dataset. The second term is our prior for θ, and it's in effect a regularization term that nudges us away from pure maximum likelihood fitting.
- Another form of regularization that has been found to work well is randomized dropout in which, every iteration of the optimization, we randomly pick a set of parameters and set them to zero. This sort of random jitter seems to prevent a neural network from learning unnecessarily elaborate rules.

MAP estimate: section 8.4 page 147

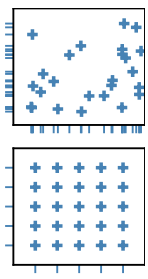
The simple MNIST classifier (section 3.3.3 page 67) shows how to use dropout in practice.

Each of these strategies carries with it a choice. (How simple should I make my model? What regularizer term should I add? How much dropout should I use?) Any such choice a called a *hyperparameter*. This simply means any aspect of the model that we can't choose purely by fitting the model to a training dataset.



There's a simple way to choose hyperparameters, called cross validation. Split the training dataset into K parts, e.g. $K = 20$. For $k \in \{1, \ldots, K\}$, pick out part k (called a *validation set*), train on the remaining $K - 1$ parts, then measure the log likelihood score on part k. Do this for every $k \in \{1, \ldots, K\}$ (or if that's too time consuming then do it for a few), and average the scores. Run this whole procedure for several values of the hyperparameter, and pick whichever gives the best average score.

FOLK WISDOM FOR HYPERPARAMETER TUNING



- When we need to optimize over several hyperparameters, it's often better to choose them randomly rather than gridded. Imagine that one of the parameters ends up being useless: then random search will give us better coverage of the other useful parameter than would gridded search.
- When we have no clue how big a hyperparameter should be, whether it should be closer to 10^{-5} or to 10^5, try choosing it randomly not as $U[10^{-5}, 10^5]$ but instead as $10^{U[-5,5]}$.

NO PEEKING

The cardinal rule is NO PEEKING. If we peek at holdout datapoints when fitting our model or choosing hyperparameters, and if we allow this peeking to influence our choices,

then we're liable to overfit. Be scrupulous in keeping the holdout set and the training set separate.

It's sometimes suggested that we'd get a better estimate of holdout log likelihood if we used 'cross-holdout evaluation'. In other words, split the dataset into chunks and use each chunk in turn as a holdout set, then average the results, since an average should have less noise than a single reading. Logically, there is nothing wrong with this, *as long as we choose absolutely everything about our model in advance*. If we make any architectural choices at all after seeing the data, or if we choose any hyperparameters, then it's peeking.

A subtle corollary of the no peeking rule is that it's important to pay attention to the *unit of prediction*. For example, suppose we're trying to predict the trips that a person will make, and we have a dataset of trips. Do we want to predict "new trips by new users"? If so, make sure that a given person's trips are either all in the training subset or all in the validation set—otherwise the fitting procedure might sneakily learn to identify a person based on side characteristics, and use this to deduce likely trips. Or do we want to predict "new trips by the current userbase"? If so, put some trips by every user in both the training subset and the validation set.

10.3. Model choice

All models are wrong but some are useful.

George Box[59]

The empiricist approach to model selection is utterly simple: when faced with a choice between two models, pick whichever model fits the holdout dataset better. For measuring fit, we suggested that holdout log likelihood is a sensible metric. The inductive claim is "going forwards, my chosen model will make better predictions than the other model." This answer is so simple and so patently reasonable that it's worth thinking harder about why anyone might ever think different.

log likelihood as a measure of fit: section 10.1 page 182

Model refutation versus selection. In science, we may discover that a theory is wrong, without having a replacement theory at hand. Likewise, the frequentist approach to induction asks "could this model have generated my dataset?" and lets us draw the conclusion that it couldn't, with no reference at all to an alternative model.

Here's an example to illustrate. Suppose our dataset consists of binary sequences of length 12,

`000111111111000`, `111100000111110`, ...

and we've proposed the model "each sequence consists of independent $\text{Bin}(1, 1/2)$ values". To test if this is a satisfactory model, we can propose a test statistic such as "number of transitions from 0 to 1 or from 1 to 0, averaged over all sequences". Eyeballing the two items in this dataset, it seems that the number of transitions is rather low for our model. If the dataset is big enough, and if our eyeballing impression is sustained, we'll end up with a low p-value.

By contrast, the empiricist and the Bayesianist approaches only make a comparison between a given set of alternative models. The empiricist asks which of the models fits best, as measured (if they so choose) by likelihood. The Bayesianist also finds the likelihood of the data under each model (the so-called "model evidence"), and then weights it by their prior belief in each model.

model evidence: section 8.7 page 155

In maximum likelihood estimation also, we're effectively making a choice between a set of alternative models. When we propose a parametric likelihood function $\Pr(x \cdot \theta)$, each particular value of θ specifies a particular probability distribution i.e. model, and maximum likelihood parameter estimation means selecting which of these models has the highest likelihood.

Test statistics and model exploration. Why do frequentists use arbitrary test statistics for deciding if their model fits the data, when likelihood is a clean and simple way to measure how well a model fits?

George Box famously said, "all models are wrong". There's no point in asking "is this model true?", because the conclusion is foregone: no model that we frail modellers propose is every *true*. There'll always be some little wrinkle in the data that our model fails to describe. In modelling, most of the time we're engaged in an iterative process of model exploration: we propose a model, we fit it, we look for the ways that it falls short in describing our data, and we use this to guide our next tentative model.

What we need, therefore, is tools that measure *how* a model falls short. Test statistics and p-values are useful, because each test statistic we choose corresponds to a particular direction in which the model might or might not need improvement. In the example above, of binary sequences, the test statistic "average number of transitions per sequence" is aimed in the direction of models with correlation. Another statistic, "average number of 1s per sequence", is aimed in another direction. Typically we'll try several different test statistics, to see different aspects of how our model falls short.

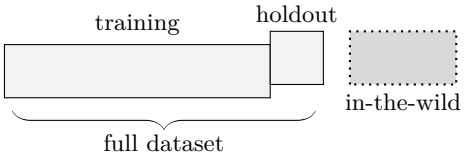
It's interesting to note that likelihood may well be useless for model exploration. In the example above, our proposed model "each sequence consists of independent $\text{Bin}(1, 1/2)$ values" means that every possible sequence has exactly the same likelihood, namely $1/2^{12}$. Likelihood is fine for comparing how two different models fit the same datapoints, but it's problematic for comparing how well a single model fits different datapoints, and it's the latter that is needed for model refutation and exploration.

[59] G. E. P. Box. "Robustness in the Strategy of Scientific Model Building". In: *Robustness in Statistics*. Vol. 1. May 1979, p. 40. URL: https://archive.org/details/DTIC_ADA070213.

10.4. Confidence

It's common in machine learning to produce a meretricious table with columns for several different models, rows for different dataset challenges, and big bold numbers highlighting the fact that the researcher's favourite model does best. (Sir Ernest Rutherford would call this stamp collecting.) But what scientific conclusions are we meant to draw from such a table?

Rutherford on stamp collecting, page 131



If we report "my classification algorithm achieves 93.7% accuracy on the holdout set", what many people will interpret is "for a new unseen in-the-wild datapoint, the probability we classify it correctly is 93.7%". This is NOT correct.

The correct interpretation is "The *average* accuracy *across a bunch of in-the-wild datapoints* is 93.7%." In other words, the holdout set accuracy indicates the average accuracy across a range of possible inputs, and it must NOT be taken as indicating the accuracy on an *individual* in-the-wild datapoint. Here's a thought experiment:

Suppose we've trained a climate model, which makes predictions about future temperatures. We evaluate its accuracy on a holdout set, and find the mean square error is 2.2°C. What is the accuracy of its prediction for January 2050?

It's impossible to say, of course. The holdout set accuracy is an aggregate over the holdout set we used for evaluation. Our model might for example be more accurate in winter months and less in summer months, or vice versa; the aggregate accuracy doesn't tell us about the accuracy in Januaries. And given that the climate is changing, there might not even be anything in the holdout set that permits a like-for-like comparison.

What if instead we train a probabilistic prediction model, which produces not only a prediction for January 2050 but also a standard deviation? Doesn't that standard deviation tell us the accuracy of its prediction?

It does, in a sense, but not in the sense of scientific induction. It only counts as induction if we make some sort of justified claim about in-the-wild data. The empiricist claim is that our model's fit on a holdout set is roughly its fit on in-the-wild data, on the basis that that the holdout set can stand in as a proxy. In the setting of probabilistic models, we'd measure our model's holdout log likelihood score. But again this score is an aggregate over the holdout dataset, and it doesn't tell us how well our probability model might fit on January 2050.

Bayesianist and frequentist schools of inference have more refined answers. They can make predictions like "The temperature in January 2050 will be 5.1°C, plus a noise term that we've modelled as Normal$(0, 1.4^2)$°C, plus or minus around 0.8°C representing a 95% confidence interval for parameter uncertainty". The confidence interval measures the strength of the inductive evidence for this prediction.

It's easy to train neural networks that model the noise term, as described in section 3.2; for Gaussian regression problems they can estimate the standard deviation of the response, and for classification problems they can estimate the probability associated with each class. But no one has found a satisfactory way to train them to report confidence intervals.

10.5. Generalizability / domain shift

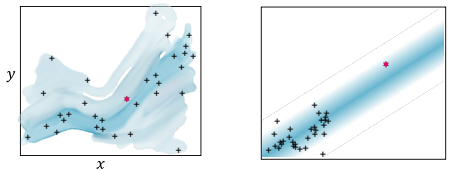
> *There is a notion of success which has developed [...] in recent years which I think is novel in the history of science. It interprets success as approximating unanalyzed data.*
>
> Noam Chomsky[60]

> *[I]t is more important to have beauty in one's equations than to have them fit experiment.*
>
> Paul Dirac[61]

> *Everything should be made as simple as possible, but not simpler.*
>
> Einstein[62]

Karl Popper, page 181

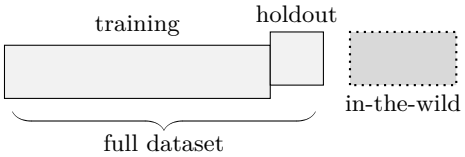
Scientists want to find 'laws', **generalizable conclusions that apply to new domains**. Karl Popper said that scientific theories should be tested by making risky predictions, by which he means predictions on new domains. Scientists love simple equations because they instinctively feel that such equations are more likely to generalize to new domains than would a deep learning model.



holdout data similar to training data

holdout data from different domain

If our model only works on a random subset of our initial dataset, it's a rubbish model. And yet it's common practice in machine learning (and even promoted in the sklearn documentation[63]) to pick a random subset of the data as the holdout set. If we report "my classification algorithm achieves 93.7% accuracy on the holdout set", what many people will interpret is "for a new unseen in-the-wild datapoint, the probability we classify it correctly is 93.7%". This is NOT correct. The correct interpretation is "… when averaged across a bunch of in-the-wild datapoints *that match the composition of my holdout set*," where typically the holdout set matches the composition of the training set. Chomsky damns this as "novel in the history of science" (by which he means "novel and foolish", not "novel and exciting"!)



* * *

[60] Remarks from *The Golden Age — A look at the original roots of artificial intelligence, cognitive science, and neuroscience*. Panel discussion at an MIT Symposium on Brains, Minds and Machines. May 2011. URL: http://mit150.mit.edu/symposia/brains-minds-machines.html. These remarks at timepoint 2:02:23.

[61] P. A. M. Dirac. "The evolution of the Physicist's Picture of Nature". In: *Scientific American* (May 1963). URL: https://blogs.scientificamerican.com/guest-blog/the-evolution-of-the-physicists-picture-of-nature/.

[62] Actual not-so-simple quotation: "It can scarcely be denied that the supreme goal of all theory is to make the irreducible basic elements as simple and as few as possible without having to surrender the adequate representation of a single datum of experience." Albert Einstein. "On the Method of Theoretical Physics". In: *Philosophy of Science* 1.2 (1934), pp. 163–169. URL: http://www.jstor.org/stable/184387

[63] https://scikit-learn.org/stable/modules/cross_validation.html

A practical way to get just a little bit closer to the scientist's mindset is to try to find a few validation sets that are different to our training set, and to tune our model's hyperparameters so that it does well on these validation sets. This way, when our model is presented with in-the-wild data from a new domain, it shouldn't be flummoxed because we've tuned it to respond well in new domains.

To make such a validation sets we could for example take the most extreme 10% of the datapoints, assuming we can find a way to measure extreme. For example, we might be working with a time series, in which case we could use the most recent 10% as a validation set. Or there might be some other parameter that governs the scenario, e.g. if we're modelling global climate data then we could take the period with the most extreme CO_2 levels as a validation set.

The ultimate answer, in the author's opinion, will be found in the field of Causal Machine Learning. Causal learning has been studied by econometricians and others for decades, and it is just beginning to reach machine learning. The most useful approaches involve training datasets consisting of grouped datapoints, each group coming from a different domain, and they propose new methods for model fitting that respect the grouped structure of the data.